

IDENTIFYING DECISION BOUNDARIES TO EXPLAIN BLACK-BOX MACHINE LEARNING PREDICTIONS

By
Adriaan Rowan

Dissertation presented for the degree of
Doctor of Philosophy
in the Faculty of Economics and Management Sciences
at Stellenbosch University

Supervisor: Prof. Sugnet Lubbe

March 2026

DECLARATIONS

By submitting this dissertation electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date 2 March 2026

Signature 

Copyright © 2026 Stellenbosch University
All rights reserved

ABSTRACT

The increasing deployment of machine learning models in high-stakes decision making has intensified the demand for interpretability methods that enable both practitioners and stakeholders to understand, trust and challenge model predictions. Existing approaches are limited; many are computationally intensive, restricted to either local or global explanations, or lack model-agnostic applicability.

This study developed a new post hoc explanation methodology that integrates principal component and canonical variate analysis biplot techniques with counterfactual reasoning to provide visual interpretations that are plausible, feasible and actionable. The framework uses biplots to generate two-dimensional visualisations that approximate multi-dimensional distances between data points. An inverse projection function enables variable values to be extracted from the two-dimensional surface while preserving the geometric relationships of the original high-dimensional data. This allows decision boundaries to be approximated in reduced space and the nearest counterfactuals identified with respect to those boundaries.

By mapping observed data points to their nearest counterfactuals, the method reveals not only which variables drive classification outcomes, but also the relative change required in each variable to alter those outcomes. Decomposing these counterfactual shifts provides local interpretability at the individual level and, when aggregated across the dataset, yields a global approximation of variable importance. This offers a computationally efficient alternative to Shapley-based approaches while retaining model-agnostic applicability.

Counterfactual identification also allows the exact contribution of each variable to a prediction to be quantified, answering the fundamental interpretability question: “What minimal change in the input would have led to a different outcome?” Furthermore, the method links interpretability to robustness: Models with counterfactuals lying close to the decision boundary are more sensitive to small perturbations, while those with more distant counterfactuals exhibit greater stability.

The uniqueness of this research lies in bridging geometric data visualisation with modern explainable AI (artificial intelligence) techniques. By re-purposing statistical tools traditionally used for dimension reduction, the study introduces an interpretable, visual and computationally efficient approach to understanding machine learning models. This contributes a novel perspective to the field of explainable AI and demonstrates its practical value in actuarial and risk-modelling contexts, where transparency, stability and accountability are paramount.

ABSTRAK

Die toenemende gebruik van masjienleermodelle vir belangrike besluitneming in die finansiële en mediese industrieë het die behoefte aan interpreteerbaarheidsmetodes sterk laat toeneem. Praktisyns en belanghebbendes wil nie net weet wát ’n model voorspel nie, maar ook hoekom. Baie bestaande metodes het egter beperkings: sommige is berekeningsintensief, ander fokus net op plaaslike of globale verklarings, en baie werk slegs vir spesifieke modeltipes en is dus nie model-onafhanklik nie.

Hierdie studie stel ’n nuwe *post hoc*-verklaringsmetodologie voor wat hoofkomponent- en kanoniese-variansie-analise- bistippingstegnieke kombineer met teenfeitlike redenering. Die doel is om visuele verklarings te verskaf wat geloofwaardig, uitvoerbaar en prakties bruikbaar is. Die raamwerk gebruik bistippings om tweedimensionele voorstellings te skep wat die multidimensionele afstande tussen datapunte benader. ’n Omgekeerde projeksiefunksie maak dit moontlik om veranderlike-waardes vanaf die tweedimensionele oppervlak terug te bereken, terwyl die meetkundige struktuur van die oorspronklike hoëdimensionele data behoue bly. Dit stel die navorser in staat om besluitgrense in ’n laerdimensionele ruimte te benader en die naaste teenfeitlikes relatief tot daardie grense te identifiseer.

Deur elke datapunt met sy naaste teenfeitlike te vergelyk, wys die metode watter veranderlikes ’n klassifikasie-uitkoms dryf en hoeveel elke veranderlike sou moes verander om die uitkoms te beïnvloed. Die ontleding van hierdie teenfeitlike verskuiwings gee plaaslike interpreteerbaarheid op individuele vlak. Wanneer die resultate oor die hele datastel saamgevoeg word, verskaf dit ook ’n globale aanduiding van veranderlike-belangrikheid. Dit bied ’n meer berekenings doeltreffende alternatief tot Shapley-gebaseerde benaderings, terwyl dit steeds model-onafhanklik bly.

Die identifisering van teenfeitlikes maak dit ook moontlik om die presiese bydrae van elke veranderlike tot ’n voorspelling te meet. Dit beantwoord die kernvraag van interpreteerbaarheid: “Watter minimale verandering in die inset sou tot ’n ander uitkoms gelei het?” Die metode toon verder ’n duidelike verband tussen interpreteerbaarheid en robuustheid: Modelle waarvan teenfeitlikes baie naby aan die besluitgrens lê, is meer sensitief vir klein versteurings, terwyl modelle met verder geleë teenfeitlikes meer stabiel is.

Die kernbydrae van hierdie navorsing is dat dit meetkundige datavisualisering met moderne verklaarbare KI-tegnieke kombineer. Deur statistiese gereedskap vir dimensievermindering op ’n nuwe manier te gebruik, bied die studie ’n interpreteerbare, visuele en berekenings doeltreffende manier om masjienleermodelle beter te verstaan. Dit wys hoe verklaarbare KI praktiese waarde kan toevoeg in aktuariële en risikomodelleringsomgewings, waar deursigtigheid, stabiliteit en aanspreeklikheid noodsaaklik is.

ACKNOWLEDGEMENTS

“If I have seen further, it is by standing on the shoulders of Giants” – Sir Isaac Newton

In my case, those giants are my parents, who inspired me to begin this journey; my supervisor, whose guidance helped shape this work; and my loving wife, whose patience and support carried me through.

To my sons: May you always have the curiosity and courage to ask why events unfold as they do, and the wisdom to discern what must change to shape a better outcome.

I would also like to express my heartfelt gratitude to all programmers who contribute their time, skills and creativity to developing open-source software for the world to use. Your generosity and dedication not only enable innovation and discovery, but also renew my faith in the spirit of collaboration and in humankind itself.

ERKENNINGS

“If I have seen further, it is by standing on the shoulders of Giants” – Sir Isaac Newton

In my geval is daardie reuse my ouers, wat my geïnspireer het om hierdie reis aan te pak; my studieleier, wie se leiding hierdie werk help vorm het; en my geliefde vrou, wie se geduld en ondersteuning my deurgedra het.

Aan my seuns: Mag julle altyd die nuuskierigheid en die moed hê om te vra waarom gebeure ontvou soos dit doen, en die wysheid om te onderskei wat moet verander om ’n beter uitkoms te bewerkstellig.

Ek wil ook my opregte dank en waardering uitspreek teenoor alle programmeerders wat hul tyd, kundigheid en kreatiwiteit beskikbaar stel om oopbron-sagteware vir wêreldwye gebruik te ontwikkel. Julle vrygewigheid en toewyding maak nie net innovasie en ontdekking moontlik nie, maar versterk ook my geloof in die gees van samewerking en in die mensdom.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 THE NEED FOR INTERPRETABILITY 1

1.2 OBJECTIVES OF THE STUDY 2

1.3 LAYOUT OF THE DISSERTATION 4

CHAPTER 2 INTERPRETABLE MACHINE LEARNING 5

2.1 INTRODUCTION 5

2.1.1 Post hoc model-agnostic vs. model-specific interpretation 5

2.1.2 Global and local interpretations 6

2.1.3 Goals and questions to be answered 6

2.1.4 Metrics of interpretability 7

2.2 DEFINITIONS, QUESTIONS AND METRICS USED IN THIS STUDY 7

2.3 POST HOC MODEL-AGNOSTIC VS. MODEL-SPECIFIC EXPLANATIONS 10

2.4 GLOBAL AND LOCAL INTERPRETATIONS 12

2.5 GOAL AND QUESTIONS TO BE ANSWERED 12

2.5.1 Goal of an interpretation 13

2.5.2 Audience of the explanation 14

2.5.3 Questions to answer 14

2.6 METRICS OF INTERPRETABILITY 16

2.6.1 Global interpretation metrics 16

2.6.2 Local interpretation metrics 17

2.7 ASPECTS TO CONSIDER WHEN INTERPRETING MACHINE LEARNING MODELS 18

2.7.1 Uncertainty and confidence in predictions 19

2.7.2 Impact of model complexity and number of features 21

2.7.3 Rashomon effect 22

2.8 INTERPRETABLE MACHINE LEARNING METHODS 22

2.8.1 Decision trees 22

2.8.2 Rule-based learners 23

2.8.3 Linear/Logistic regression 23

2.8.4 Generalised additive models 24

2.8.5 Bayesian models 25

2.8.6 Case-based reasoning 25

2.9 EXPLAINABLE ARTIFICIAL INTELLIGENCE METHODS 26

2.9.1 Explanations using a surrogate explanation model 26

2.9.2 Feature importance explanations 29
2.9.3 Model visualisation 30
2.10 COUNTERFACTUALS 31
2.10.1 Defining and finding a counterfactual 32
2.10.2 Constraints on counterfactual solutions 34
2.10.3 Metrics of the accuracy of the counterfactual 36
2.11 AREAS DEVELOPED IN THIS STUDY 37
CHAPTER 3 METHODS TO IDENTIFY COUNTERFACTUALS 38
3.1 INTRODUCTION 38
3.2 EUCLIDEAN AND MAHALANOBIS DISTANCE MEASURES 38
3.3 PRINCIPAL COMPONENT ANALYSIS BIPLOT 39
3.3.1 Constructing a biplot 40
3.3.2 Estimating the similarity and correlations within X 42
3.3.3 Predictivity and variable importance 44
3.3.4 Calibrated variable axes 46
3.4 CANONICAL VARIATE ANALYSIS BIPLOT 47
3.4.1 Axis predictivity and variable importance 50
3.4.2 Two-group CVA biplot 51
3.5 OPTIMAL ROTATION OF THE BIPLOT 52
3.5.1 Example of the rotated component biplot 56
3.5.2 Rotation of a CVA biplot 61
3.6 BINARY CLASSIFICATION DECISION BOUNDARIES 61
3.6.1 Decision boundary intervals 62
3.6.2 Counterfactuals and local decision boundaries 64
3.7 CURRENT METHODS FOR FINDING DECISION BOUNDARIES 67
3.7.1 Analytical solution 67
3.7.2 Heuristic search 70
3.7.3 Grid search 70
3.8 CALCULATING COUNTERFACTUALS USING A BIPLOT 71
3.8.1 Visualising the machine learning model $\boldsymbol{f}(\boldsymbol{X})$ on the biplot plane 76
3.8.2 Interpreting the biplot and estimated counterfactual 81
3.8.3 PCA vs. CVA biplots 82
3.9 ACCURACY, REQUIREMENTS AND BENEFITS OF USING THE BIPLOT 84
3.9.1 Accuracy of the biplot representation 85
3.9.2 Constraints for a biplot-generated counterfactual value 88
3.9.3 Metrics of the counterfactuals generated using biplots 91

3.10 USING BIPLOTS TO EXPLAIN MACHINE LEARNING MODELS 92

CHAPTER 4 GLOBAL AND LOCAL INTERPRETATIONS USING A BIPLOT 94

4.1 INTRODUCTION 94

4.2 GLOBAL INTERPRETATIONS USING A BIPLOT 94

4.2.1 Biplot representation and interpretation 96

4.2.2 Variable importance based on distance to decision boundary 101

4.2.3 Biplot measure of the model robustness 105

4.2.4 Biplot as a surrogate model with global fidelity 108

4.2.5 Global questions answered 111

4.3 LOCAL INTERPRETATIONS USING A BIPLOT 112

4.3.1 Identifying nearest counterfactual values 113

4.3.2 Shapley values 119

4.3.3 Local questions answered 122

4.4 CONCLUSION TO THE BIPLOT EXPLANATIONS 123

CHAPTER 5 APPLICATION FOR INTERPRETING LOAN DATA 126

5.1 INTRODUCTION 126

5.2 GLOBAL INTERPRETATION OF THE GBM MODEL 127

5.3 LOCAL SHAPLEY VALUE INTERPRETATION 134

5.3.1 Interpreting the true positive prediction 136

5.3.2 Interpreting the true negative prediction 137

5.3.3 Interpreting the false positive prediction 139

5.3.4 Interpreting the false negative prediction 141

5.4 CONCLUSION TO THE BIPLOT EXPLANATIONS 142

CHAPTER 6 FUTURE AREAS OF RESEARCH 143

6.1 INTRODUCTION 143

6.2 GLOBAL SHAPLEY VALUE INTERPRETATION USING ROTATED LOCAL DISTANCE CALCULATIONS 143

6.3 GRAND TOUR OF DATA INSTEAD OF COMPONENT SELECTION 144

6.4 DISTANCE TO DECISION BOUNDARIES AS APPROXIMATION OF PREDICTION PROBABILITIES 144

6.4.1 Uncertainty and confidence in predictions 145

6.4.2 Developing an accurate probabilistic prediction model 146

6.5 VISUALISATION AND CLUSTERING OF THE DATA 146

6.6 GENERALISATION OF THE METHODOLOGY 149

REFERENCE LIST 151

APPENDIX A: CODE 158
A1: Shapley value calculation 172
A2: Optimal Rotation of a Biplot 179
A3: Biplot Boundary Search 181
A4: Biplot Search of local decision boundaries 187
A5: Global Interpretations of Robustness and Variable Importance 198
A6: Surrogate Models 201
A7: Biplot Boundary Search with different Eigenvalues 207
A8: Local Interpretations with Shapley values 214

LIST OF TABLES

TABLE 3.1: RESULT OF THE COUNTERFACTUAL CALCULATION FOR THE IRIS DATASET TARGET DATA POINT 80

TABLE 4.1: VARIABLES IN THE PIMA DATASET 96

TABLE 4.2: TARGET DATA POINT VARIABLE VALUES 114

TABLE 4.3: THE DATA POINT AND THE VALUE OF ITS NEAREST UNCONSTRAINT COUNTERFACTUAL 116

TABLE 4.4: TARGET DATA POINT VARIABLE AND UNCONSTRAINT AND CONSTRAINT COUNTERFACTUAL VALUE SOLUTIONS 119

TABLE 5.1: VARIABLES IN THE LOAN DEFAULT DATASET 126

TABLE 5.2: FOUR EXAMPLE DATA POINTS THAT REPRESENT THE FOUR DIFFERENT TYPES OF PREDICTION OUTCOMES 135

LIST OF EQUATIONS

EQUATION 1: SVD OF X 40
EQUATION 2: CALCULATION OF R-DIMENSION REPRESENTATION OF X 41
EQUATION 3: BEST R-DIMENSIONAL REPRESENTATION OF X 41
EQUATION 4: TWO-DIMENSIONAL REPRESENTATION OF X 41
EQUATION 5: DECOMPOSITION A 42
EQUATION 6: DECOMPOSITION B 42
EQUATION 7: PARTITIONING OF THE TOTAL SSP MATRIX 48
EQUATION 8: SHAPLEY VALUES FOR CHANGE FROM DATA POINT TO COUNTERFACTUAL 120

LIST OF FIGURES

FIGURE 3.1: COMPARISON OF THE PCA BIPLOT USING DECOMPOSITIONS A AND B, WHERE DECOMPOSITION A ACCURATELY APPROXIMATES THE SIMILARITY BETWEEN THE DATA POINTS AND DECOMPOSITION B APPROXIMATES THE CORRELATION BETWEEN THE VARIABLES 43

FIGURE 3.2: ILLUSTRATION OF A CORRELATION MONOPLOT AND A UNIT CIRCLE, WHERE THE LENGTH RELATIVE TO ONE UNIT MEASURES THE $\boldsymbol{axis\ predictivity}$ 46

FIGURE 3.3: THE ROTATION AND DILATION REQUIRED FOR THE PROJECTED POINT TO CORRESPOND TO THE DATA POINT IN THE FULL TWO-DIMENSIONAL PCA DIMENSION SPACE, RESULTING IN THE ROTATED BIPLOT PLANE PASSING THROUGH THE DATA POINT 55

FIGURE 3.4: A THREE-DIMENSIONAL AND TWO-DIMENSIONAL BIPLOT OF THE THREE-VARIABLE IRIS DATA ILLUSTRATING THE ACCURACY OF THE DIMENSION REDUCTION AND PROJECTION OF A DATA POINT ONTO THE TWO-DIMENSIONAL BIPLOT PLANE 58

FIGURE 3.5: A THREE-DIMENSIONAL AND TWO-DIMENSIONAL RC BIPLOT OF THE THREE-VARIABLE IRIS DATA AFTER ROTATING THE PRINCIPAL COMPONENTS TO ENSURE THAT THE DATA POINT IS ON THE PLANE, REQUIRING FOR NO PROJECTION ONTO THE PLANE FOR THE GREEN DATA POINT 60

FIGURE 3.6: VISUALISATION OF THE DECISION BOUNDARY SET $\boldsymbol{XDB}$ WITH $\boldsymbol{XDB|g = 0}$ IN BLUE AND $\boldsymbol{XDB|g = 1}$ IN RED 63

FIGURE 3.7: COMPARING DECISION BOUNDARY SUBSETS OF DIFFERENT ML MODELS AND THE RESULTING NEED FOR DIFFERENT VALUES OF $\boldsymbol{\varepsilon}$ NECESSARY TO UNIQUELY IDENTIFY LOCAL DECISION BOUNDARIES 64

FIGURE 3.8: IDENTIFYING THE NEAREST COUNTERFACTUAL $\boldsymbol{xcf}$ FOR $\boldsymbol{xi}$ AND THE LOCAL DECISION BOUNDARY $\boldsymbol{XLDBi}$ THAT CONTAINS $\boldsymbol{xcf}$ 66

FIGURE 3.9: IDENTIFYING THE LOCAL DECISION BOUNDARY $\boldsymbol{XLDBi}$ FOR DATA POINT $\boldsymbol{xi}$ AND THE COMMON LOCAL DECISION BOUNDARY $\boldsymbol{XLDBkj}$ FOR DATA POINTS $\boldsymbol{xk}$ AND $\boldsymbol{xj}$, WHERE $\boldsymbol{\varepsilon = 0.01}$ 66

FIGURE 3.10: THE DATA POINT VECTOR, REPRESENTED BY $\boldsymbol{a}$, IS PROJECTED ONTO THE DECISION BOUNDARY TO CALCULATE THE COUNTERFACTUAL VALUE. GRAPH A INDICATES THE ORIGINAL TWO-VARIABLE DATA. GRAPH B INDICATES THE ADJUSTED BOUNDARY TO ENSURE THAT THE BOUNDARY PASSES THROUGH THE ORIGIN. 68

FIGURE 3.11: PROCESS MAP OF GENERATING COUNTERFACTUALS USING A PCA BIPLOT FOR THE MODEL $\boldsymbol{f(X)}$ 72

FIGURE 3.12: RC BIPLOT WITH CALIBRATED AXES AND DATA POINTS COLOURED BASED ON THEIR CLASS, AND THE TARGET DATA POINT IN GREEN 77
FIGURE 3.13: RC BIPLOT WITH THE TEST DATA POINT AND ITS COUNTERFACTUAL INDICATED IN LIGHT AND DARK GREEN ON THE DECISION SURFACE. THREE DATA POINTS ARE OUTSIDE THE CONVEX HULL OF THE TRAINING DATA. ONLY THE CONTOUR LINES WITHIN THE CONVEX HULL WILL BE USED TO FIND COUNTERFACTUALS. 79
FIGURE 4.1: PCA BIPLOTS OF THE PIMA DATASET. THE DATA IS COLOURED TO REFLECT THE DIABETIC STATUS. THE SECOND BIPLOT ONLY PLOTS THE TWO VARIABLES BEING INVESTIGATED FOR POTENTIAL DATA ERRORS. 97
FIGURE 4.2: PCA BIPLOTS OF THE PIMA DATASET WITH INSULIN AND SKINTHICKNESS VARIABLES THAT HAVE VALUES EQUAL TO ZERO REMOVED FROM THE DATASET. A TOTAL OF 394 RECORDS REMAIN, WITH 315 IN THE TRAINING DATASET. THE CORRELATION STRUCTURE OF THE DATA CAN BE INVESTIGATED WITH THE BIPLOT. 98
FIGURE 4.3: CVA BIPLOT OF THE PIMA TEST DATA, WITH THE PLANE APPROXIMATING THE GBM MODEL DECISION SURFACE WITHIN THE CONVEX HULL OF THE TRAINING DATA, THE LOCAL DECISION BOUNDARIES IN BLACK CONTOUR LINES AND COUNTERFACTUALS IN DARK GREEN. A POTENTIAL OUT-OF-DISTRIBUTION DATA POINT IS IDENTIFIED TO THE RIGHT. A LOCAL DECISION BOUNDARY IS IDENTIFIED TO THE TOP LEFT 100
FIGURE 4.4: ILLUSTRATION THE COUNTERFACTUAL CHANGE VECTOR $\boldsymbol{\delta i}$ DECOMPOSED ALONG THE VARIABLE AXES 102
FIGURE 4.5: DISTANCE-BASED VARIABLE IMPORTANCE PLOTS USING THE DECOMPOSITION OF THE VECTOR $(\boldsymbol{zcf} - \boldsymbol{zi})\boldsymbol{M2}$ −, ACROSS ALL TRAINING DATA POINTS, WITH EACH VECTOR STANDARDISED BY ITS STANDARD DEVIATION 104
FIGURE 4.6: COMPARISON OF A TRADITIONAL FEATURE IMPORTANCE PLOT VS. THE DISTANCE-BASED IMPORTANCE PLOT 105
FIGURE 4.7: COMPARISON OF THE LOCAL DECISION BOUNDARIES AND ROBUSTNESS OF TWO GBMS WITH 1 000 (ABOVE) AND 5 000 (BELOW) TREES SELECTED FOR THE HYPERPARAMETER FOR NUMBER OF TREES 107
FIGURE 4.8: THE BIPLOT OF THE GBM MODEL COMPARED TO THE SURROGATE MODEL OF THE GBM MODEL. THE SURROGATE MODEL PROVIDES HIGH FIDELITY AND INTUITIVE INTERPRETATIONS BY MAKING PREDICTIONS USING ONLY THE NEAREST LOCAL DECISION BOUNDARIES. 110
FIGURE 4.9: CVA BIPLOT OF THE UPDATED GBM MODEL WITHOUT THE BLOODPRESSURE VARIABLE AND THE CONVEX HULL OF THE TRAINING DATA. THE TARGET DATA

POINT IS HIGHLIGHTED IN GREEN. AN OUT-OF-DISTRIBUTION DATA POINT IS IDENTIFIED IN THE TOP LEFT. 114
FIGURE 4.10: TWO-CLASS RC BIPLOT FOR PIMA DATA FIT WITH A CVA BIPLOT TO A GBM MODEL. THE TARGET DATA POINT IS INDICATED IN GREEN AND ITS NEAREST UNCONSTRAINT COUNTERFACTUAL IN DARK GREEN. 116
FIGURE 4.11: RC BIPLOT GENERATED BY ROTATING THE FIRST AND SEVENTH COMPONENTS OF THE CVA BIPLOT TO PASS THROUGH THE DATA POINT AND IDENTIFYING THE NEAREST CONSTRAINT COUNTERFACTUAL. THE COUNTERFACTUAL IS WITHIN THE CONVEX HULL OF THE TRAINING DATA. A SPARSER (AND THEREFORE NEARER) SOLUTION IS INDICATED IN GREY. 118
FIGURE 4.12: SHAPLEY VALUES FOR THE TARGET DATA INDICATE THE CONTRIBUTION OF EACH VARIABLE TO THE PREDICTION OUTCOME 121
FIGURE 5.1: PCA BIPLOT OF THE LOAN DEFAULT TRAINING DATA, WITH THE TRAINING DATA INDICATED IN RED AND BLUE, FOR DEFAULT AND NO DEFAULT. IT IDENTIFIES OUTLIERS IN THE DATA THAT COULD BE REMOVED TO IMPROVE THE ACCURACY OF THE MODEL CALIBRATION. 128
FIGURE 5.2: INITIAL CVA BIPLOT OF THE TRAINING DATA WITH THE DATA POINTS COLOURED BASED ON THE VALUE OF THE TARGET VARIABLE. IT VISUALISES THE IMPACT OF THE PREVIOUS_DEFAULTS VARIABLE ON THE PREDICTION MODEL OUTCOME. 129
FIGURE 5.3: CVA BIPLOT OF THE TRAINING DATA, WITH THE CONTOUR LINES REPRESENTING THE LOCAL DECISION BOUNDARIES USED TO GENERATE THE GLOBAL COUNTERFACTUAL VALUES 130
FIGURE 5.4: GLOBAL VARIABLE IMPORTANCE PLOT USING THE DISTANCE TO THE BOUNDARY ALONG EACH VARIABLE AXIS. THE FIVE AXES REPRESENTING THE SHORTEST DISTANCE TO THE DECISION BOUNDARY WILL BE REMOVED. 131
FIGURE 5.5: CVA BIPLOT OF THE TEST DATA, WITH BOTH THE BIPLOT AND THE GBM MODEL CALIBRATED ON THE TRAINING DATA. THE COLOUR OF THE DATA POINTS REFLECTS THEIR PREDICTION OUTCOME. 132
FIGURE 5.6: THE GLOBAL SURROGATE MODEL OF THE GBM MODEL PROVIDES AN INTUITIVE INTERPRETATION OF HOW THE PREDICTOR VARIABLES INFLUENCE THE CLASSIFICATION OUTCOME 133
FIGURE 5.7: UNROTATED CVA BIPLOT BASED ON THE GLOBAL OPTIMAL GBM MODEL. THE FOUR DATA POINTS ARE COLOURED ACCORDING TO PREDICTION OUTCOME, WITH THEIR NEAREST COUNTERFACTUALS INDICATED IN DARK GREEN. 135
FIGURE 5.8: BIPLOT AND SHAPLEY PLOT LOCAL INTERPRETATION FOR THE RED – TRUE POSITIVE PREDICTION 137

FIGURE 5.9: BIPLOT AND SHAPLEY PLOT LOCAL INTERPRETATION FOR THE BLUE – TRUE NEGATIVE PREDICTION 138
FIGURE 5.10: BIPLOT AND SHAPLEY PLOT LOCAL INTERPRETATION FOR THE ORANGE – FALSE NEGATIVE PREDICTION 140
FIGURE 5.11: BIPLOT AND SHAPLEY PLOT LOCAL INTERPRETATION FOR THE PURPLE – FALSE POSITIVE PREDICTION 141
FIGURE 6.1: COMPARISON OF THE MDS WITH NEAREST NEIGHBOUR CLUSTERING VS. PCA BIPLOTS WITH VARIOUS MODEL-BASED CLUSTERING 148

CHAPTER 1 INTRODUCTION

1.1 THE NEED FOR INTERPRETABILITY

The last decade has witnessed the rise of sophisticated machine learning (ML) models that make accurate predictions for individual data points. Credit scores, insurance risk and health status are examples of fields where ML models are now commonly used to make predictions.

However, the value of an ML model is not determined by predictive accuracy alone. It must also be able to provide reasons for individual predictions and answer questions such as “Why was my loan application rejected?” or “What do I need to change for the bank to approve my loan?”

Some predictive models are transparent by design, with clearly defined formulations that allow users to interpret how inputs influence outputs. Such models can readily provide explanations for their predictions. Others are black-box models, which make decisions using formulas that are either too complicated for any human to understand or proprietary and therefore not publicly disclosed. Either way, one cannot understand its inner workings, and they do not provide a way of answering questions on individual predictions (Rudin *et al.*, 2021).

This lack of transparency has raised ethical and legal concerns, leading to the introduction of regulatory frameworks such as the South African government’s Protection of Personal Information Act (POPI Act) (Republic of South Africa, 2013) and the European Union’s General Data Protection Regulation (European Union, 2016). These govern automated decision-making models, and the POPI Act “require[s] a responsible party to provide a data subject with sufficient information about the underlying logic of the automated processing of the information relating to him or her” (Republic of South Africa, 2013:76).

Therefore, there is an increasing need to enhance the interpretability of ML models used to make predictions about individuals. In recent years, many published papers have discussed methods to improve the interpretability of predictive, or ML, models. The number of total publications whose title, abstract and/or keywords refer to the field of either interpretable or explainable artificial intelligence (AI) has increased almost 20-fold between 2018 and 2023, according to research by Sharma, Sharma, Sharma, Sethi and Chen (2024). This indicates a clear shift in the focus of recent research. These publications also highlight a difference

between interpretable and explainable AI. This study did not investigate the differences and similarities between AI and ML and treated them as equivalent terms.

The field of ML can be divided into supervised and unsupervised ML. In this study, 'machine learning' refers to supervised ML, where the target variable is known and identified as either in either class 0 or class 1.

The objective of this study was to introduce a novel method for interpreting ML models by identifying decision boundaries. This approach aimed to provide meaningful global insights into how a model functions and to deliver intuitive, case-specific explanations for individual predictions.

1.2 OBJECTIVES OF THE STUDY

The objectives of this study arose from the gap in current interpretability methods, which often lack a fast, visual and model-agnostic approach capable of providing both global and local explanations of ML models. Existing approaches are frequently limited in scope: Some are model-specific, others are computationally expensive, and many fail to provide actionable, feasible and plausible insights at the level of individual predictions.

Against this backdrop, the study pursued two overarching aims:

1. To develop a novel method for identifying and visualising local decision boundaries and counterfactuals using biplots (Chapter 3)

2. To extend this method to create global and local interpretation techniques that improve transparency, trust and robustness in ML models (Chapter 4).

These aims were operationalised through four specific research objectives:

1. Visualisation of the data structure, model output and its decision boundaries:

   The first objective is to develop a method for visualising the data structure, model output, decision boundaries and counterfactuals using a biplot. This objective sought to provide insights into the distribution of the data, the correlations between variables and the confidence of predictions, while also helping to identify outliers and the variables most relevant to changing prediction outcomes. A secondary objective was to construct a high-fidelity surrogate model that can be used for prediction in contexts where interpretability is critical.

2. Global feature importance identification method:

   The second objective was to develop a novel global feature importance measure grounded in counterfactual geometry. Feature importance was defined as the average distance required along each variable axis for an observation to reach its nearest counterfactual. This approach directly quantifies the extent to which a variable must change in order to alter the model's prediction, thereby linking importance to the structure of the decision boundary rather than to variance decomposition or permutation effects. The resulting metric enables variables to be ranked according to their structural influence on prediction outcomes and provides a principled basis for identifying redundant or weakly contributing features. In doing so, it supports model simplification and potential improvements in generalisation without sacrificing predictive performance.

3. Robustness evaluation method:

   The third objective was to develop a global robustness metric defined as the Euclidean distance between observations and their nearest counterfactuals. This measure captures the geometric stability of model predictions by quantifying how resistant the decision boundary is to small perturbations in the input space. Unlike traditional performance metrics that focus solely on predictive accuracy, this robustness measure evaluates the structural resilience of the model. It therefore provides an additional criterion for model comparison and selection, particularly in regulated or high-stakes environments where stability, reliability and consistency of predictions are as important as accuracy.

4. Local interpretation of individual predictions:

   The fourth objective was to develop a post hoc, model-agnostic framework for the local interpretation of individual predictions through the combined use of counterfactual explanations and Shapley-value-based attribution. Counterfactuals were generated subject to constraints of sparsity, plausibility and actionability, ensuring that proposed changes remain consistent with the multivariate distribution of the data. These counterfactual explanations identify the minimal changes required for a prediction to cross the decision boundary, thereby answering what must change for the predicted class to switch. Shapley values were used in parallel to decompose the prediction into additive contributions of each variable, thereby identifying which features contributed most to the observed outcome. The objective was to answer two critical questions: (i)

Which values contributed most to the prediction outcome? and (ii) What changes are needed for the prediction to switch categories?

The study focused on providing explanations for predictions in two-class classification problems using tabular data. The method described can be extended to multi-class and regression predictions.

1.3 LAYOUT OF THE DISSERTATION

This dissertation consists of six chapters, including this introduction. Chapter 2 summarises various definitions of interpretability and explainable ML methods proposed in the literature, including the goals these methods aim to achieve. This chapter explains the foundational terminology by defining different model interpretation methods, including current techniques such as LIME and SHAP. It presents an assessment of their effectiveness and limitations in interpreting ML models. It concludes with an introduction to and definition of counterfactual values. This was the key concept used in developing interpretation methods in this study.

Chapter 3 gives the definitions of biplots and how they can be used to identify decision boundaries and counterfactual values. It concludes by explaining the constraints that apply to counterfactuals and how they can be overcome.

Chapter 4 reports on the use of counterfactuals to develop global and local interpretation methods of the underlying ML model and its predictions.

Chapter 5 gives a practical example on credit data using the methods developed in chapters 3 and 4. This chapter shows how biplots answer the initial questions of “Why was my loan application rejected?” or “What do I need to change for the bank to approve my loan?”

Finally, Chapter 6 concludes with a summary of the potential directions for future research to expand on the methodology developed here.

CHAPTER 2 INTERPRETABLE MACHINE LEARNING

2.1 INTRODUCTION

There are many aspects to the terms ‘interpretation’ or ‘explanation’ of an ML model, and many ways to classify the various methods for interpreting and explaining ML models. Therefore, the definitions of these terms need to be determined before the appropriateness of a method can be analysed or a new method proposed.

For example, certain explanation methods can rank the global importance of various input features of gradient-based models. This method can meet the goals of causality and informativeness, providing insight into the inner workings of a model. This is useful for model owners and domain experts to answer the question of which variables do not contribute to predictions and should be removed to improve out-of-sample prediction accuracy. The metric for measuring the effectiveness of the method in providing explanations is the resulting improvement of the out-of-sample prediction accuracy. However, this method may not be able to assist in achieving the goal of improving trust in the model for the individual to whom it is applied, as it cannot help explain individual predictions. The method can also only be used with gradient-based models, which limits its usefulness in specific situations. Therefore, the method can provide a good interpretation of the model according to one definition, but not another.

The remainder of this section gives a summary of the taxonomy of the four main concepts that determine what we mean when we say a model is interpretable. This chapter then gives the preferred definition of interpretability used in the study, before going into detail to explain all the potential definitions and methods that can be used.

2.1.1 Post hoc model-agnostic vs. model-specific interpretation

There is no clear or single accepted definition of interpretable ML and explainable AI, or what the difference is, with “a vast and confusing literature on some combination of interpretability and explainability” (Rudin *et al*., 2021:2). However, a general understanding of the two concepts is explained below.

Interpretable machine learning (IML) methods refer to models that are interpretable by design and are often described as *transparent models*. Examples include decision tree models and generalised linear models (GLMs). Interpretable models are defined by Rudin *et al*. (2021:3) as “a model that obeys a domain-specific set of constraints to allow it to be more easily understood by humans. These constraints can differ dramatically depending on the domain”.

Transparent models are defined by Vilone and Longo (2021) as models that meet the requirements of simulatability, decomposability or algorithmic transparency. These are discussed in section 2.3, but the central concept is that the user can understand the structure of the model and the impact of the input data and parameters on the output predicted.

In contrast, certain types of ML models are not interpretable or transparent and are called *black-box models*. Examples include feedforward neural networks or gradient boosting models (GBMs). These types of models require explainable artificial intelligence (XAI) methods that provide post hoc explanations. Post hoc explanations convert the result of non-interpretable models into an explainable result, without directly explaining the model itself – only its outcome. Methods that provide post hoc explanations are separated into model-agnostic and model-specific methods. Model-agnostic methods can provide insight into any type of model, including models that are traditionally seen as transparent. An example of a model-specific method is one that applies gradient-based analysis. These are only applicable to gradient-based ML models that make use of a function that is differentiable.

2.1.2 Global and local interpretations

The definition of an interpretation can be separated into global and local interpretations. Global interpretability implies explaining what patterns are present in general, such as the impact of the features and the model design on how the model makes decisions (Molnar, 2020). Preferably, this includes providing algorithmic transparency into the model. However, this is seldom the case.

Local interpretability implies knowing the reasons for a specific decision (such as why a particular loan application was rejected) and is needed when one needs a justification for a specific decision (Doshi-Velez & Kim, 2017).

2.1.3 Goals and questions to be answered

The interpretation method must also meet certain goals and address the needs of specific audiences. The goal and the audience, in turn, determine the type of questions that may be asked of a model and its predictions. There is a range of possible options for the goals,

audience and the questions considered in the literature. These will determine the definition of the term ‘interpretation’ or ‘explanation’, and whether a global and a local definition are required. Therefore, they must be agreed to before selecting an interpretation method.

2.1.4 Metrics of interpretability

In addition to defining an interpretation, a clear metric is required to evaluate the performance of the method. Just because an ‘interpretable’ model is used does not mean that the interpretations provided measure well according to the metrics described.

Selecting a clear metric will enable a meaningful comparison of the interpretations provided by different models, allowing an explanation to be selected when two methods provide contradictory interpretations. It can also help determine whether an explanation method is truly model-agnostic or whether it only works for specific ML models (Barredo Arrieta *et al.*, 2020).

2.2 DEFINITIONS, QUESTIONS AND METRICS USED IN THIS STUDY

This section summarises the definition of an interpretation of an ML model that applies to the methods developed in this study. The terms ‘interpretation’ and ‘explanation’ are used interchangeably in this dissertation, while using the separate definitions for IML and XAI provided above. IML refers to models that are transparent by design, while XAI refers to methods that provide transparency even if the underlying model design is not transparent. However, a better use of the terms would be to link them to global and local interpretations, where explanability refers to global interpretations and interpretability refers to local interpretations.

The proposed method is an XAI method that provides post hoc and model-agnostic interpretations; therefore, it can be applied to any ML model. The method identifies counterfactual values for each data point and uses these to provide both global and local interpretations. Therefore, the method allows any type of ML model to become transparent.

A ‘counterfactual’ can be defined as a close neighbour of an actual data point, but with the desired different prediction outcome. Wachter, Mittelstadt and Russell (2017:5, 6) describe a counterfactual of a prediction as the “closest possible world” or the data point that requires “the smallest change to the feature values to change the prediction to a predefined output”. Therefore, the counterfactual is an example of how the input features need to be changed to change the prediction of a data point.

The method provides a global interpretation of the ML model by identifying and visualising local decision boundaries using a two-dimensional approximation of the decision surface of the model. This provides *algorithmic transparency* of the ML model, with the shape of the local decision boundaries and the interaction with the variable axes giving insight into the way in which the underlying algorithm makes predictions.

The method also provides a global feature importance metric using the distance of the data points to the decision boundary. These distances help identify and remove the least important variables and reduce the complexity of the final model that requires an interpretation.

The method provides a local interpretation for an individual model prediction by calculating its nearest counterfactual values and using Shapley values to determine the contribution of each variable to changing the prediction to the desired class. This provides *decomposability transparency* on the contribution of each variable to the prediction of an individual data point.

The following audience, goals and questions must be considered when interpreting the results of the models:

- The proposed audience is both domain experts and product owners or managers. The method may still be too complex for interaction by the public. However, a suitably trained person will be able to interpret the results of the model to gain a better understanding of how various features impact the predictions of the model. It will also enable the person to formulate an answer for a member of the public regarding the reasons behind a specific prediction and how the prediction outcome can be changed.
- The goals of the explanations provided are causality and informativeness. Causality is the ability to discover new ways that different variables interact and their impact on real-world problems. Informativeness refers to the ability to extract information about the inner workings of a model and understand the causes of the interactions it discovers.
- Examples of global questions that the model will try to answer are as follows:
    - Which variables contribute most to the predictive performance of the model, and which could be removed without degrading, or potentially even improving, accuracy?
    - Which data points exert the most influence on model calibration, and which are least accurately predicted?

  - Does the model behave reasonably and consistently across different regions of the feature space and on a case-by-case basis?
  - Why are certain instances misclassified, and what changes would be required for their correct classification?

  These questions have been addressed in the past using various methods, as described later in section 2.8 and 2.9. Using counterfactuals provides global interpretations that focus on what is required to change the prediction outcomes.

- Examples of local questions that the model will try to answer are as follows:
  - How much should the input variable values change to give an alternative prediction, with a related 'if' where the input state is constrained in some way or if a different prediction is possible at all? Therefore, what combination of available changes in inputs will result in a change in the prediction? This focuses on actionable features.
  - How much did each feature contribute to the individual prediction?
  - Which factor needs to change the least to change the prediction?

  Counterfactuals aim to make the explanation understandable to the individuals to whom the model is applied. These are examples of questions it can answer:

  - What do I need to change for the bank to approve my loan?
  - What is the most likely cause of my medical diagnoses?
  - What features make one flower type different from another?
- It therefore does not attempt to explain the internal logic, but provides a set of simple 'if-then' statements that will result in the change in the prediction that is concise, transparent and easily assessable (Wachter *et al.*, 2017).

The interpretations aim to improve the global prediction accuracy and robustness of the model by removing features identified as less important in making predictions. The global interpretations should also be stable and avoid explanation vectors that point in the wrong direction in some areas of the feature space (Laugel, Lesot, Marsala, Renard & Detyniecki, 2019). On a local level, the explanation provided for an individual data point should meet the identity, stability and separability requirements proposed by Honegger (2018).

The proposed interpretation method aims to provide answers to the questions above and meet the specified requirements. However, it will not address all possible definitions, metrics,

audiences, goals and questions related to interpretability. Alternative definitions are summarised in this chapter, explaining aspects to consider when developing an interpretation method.

2.3 POST HOC MODEL-AGNOSTIC VS. MODEL-SPECIFIC EXPLANATIONS

Certain types of ML models are interpretable by design and therefore do not require additional methods to ensure that they are understood. Interpretable models are often referred to as 'transparent models'. A transparent model is defined by Vilone and Longo (2021) as one that meets the requirements below:

- Simulatability: the capacity of a model to allow a user to step through every calculation required to produce a prediction and understand its structure and functioning entirely
- Decomposability: the degree to which a model can be decomposed into its individual components (input, parameters and output) and the impact of each component on the predictions can be intuitively explained
- Algorithmic transparency: the degree of confidence in a learning algorithm behaving 'sensibly' with previously unseen data. This aims to explain both how the model is calibrated to the data and how it generates predictions given a set of feature values.

The quality of individual prediction explanations can also be measured in terms of more human-friendly features. Miller (2017) defines the following human-friendly characteristics of explanations:

- Contractiveness: Humans usually do not ask why a specific prediction was made, but rather why this prediction was made instead of another prediction. In other words, there is a tendency for people to think in terms of counterfactual cases. They are also not specifically interested in all the factors that led to the prediction, but instead in the factors that need to change (in the input) so that the prediction (output) would also change.
- Selectivity: Explanations do not need to cover the complete list of causes of the prediction. Instead, people prefer selecting one or two leading causes from a variety of possible causes as the explanation. As a result, the explanation model should be able to list the most important features that contribute to the prediction. Even better would be to allow users to select which features they want explanations for, focusing on those that are actionable for the user.

- Focus on the abnormal: This links to selectivity and novelty. People focus more on abnormal causes to explain events. Therefore, if an input feature is abnormal, it should be included in the list of features of the explanation.

In contrast, ML models that are not interpretable or transparent are called 'black-box models'. These types of models require XAI methods to 1) produce explainable models while maintaining a high level of learning performance (e.g. prediction accuracy) and 2) enable humans to appropriately trust the results of ML models (Barredo Arrieta *et al.*, 2020). Barredo Arrieta *et al*. (2020) expand on this, stating that XAI is linked to *post hoc explainability*, which aims at communicating understandable information about how an already developed model produces its predictions for any given input. Post hoc explainability can be separated into two main groups: those that apply to any kind of ML model, called *model-agnostic methods*, and those that are only applicable to a specific type of ML model.

It is often said that black-box models combined with XAI methods provide better prediction accuracy but worse interpretability than IML methods. This notion is refuted by Rudin *et al*. (2021), who state that black boxes are generally unnecessary, given that their accuracy is typically not better than that of a well-designed, interpretable model. Therefore, explanations that seem reasonable can undermine efforts to find an interpretable model of the same level of accuracy as the black box. Rudin *et al*. (2021:5) continue with the following principle:

> It is important not to assume that one needs to make a sacrifice in accuracy in order to gain interpretability. In fact, interpretability often begets accuracy, and not the reverse. Interpretability versus accuracy is, in general, a false dichotomy in machine learning.

Even though IML is transparent, model-agnostic post hoc XAI methods can still be used to better understand and interpret the results of IML models, especially for non-domain-expert audiences. Therefore, whether a model is transparent or not, improved interpretability can lead to more accurate predictions. This is related to the idea that having human-in-the-loop ML processes that build models interactively will result in improved outcomes when models are deployed in real-world situations (Krause, Perer & Ng, 2016).

Good ML practice involves an iterative approach in which the interpretations of the model help improve the selection of the model, features and hyperparameters. Barredo Arrieta *et al*. (2020) state that interpretability can improve the accuracy of the predictions in the following ways:

- Detecting and consequently correcting for bias in the training dataset, including identifying and removing out-of-distribution data points

- Facilitating the provision of robustness by highlighting potential adversarial perturbations that could change the prediction
- Ensuring that only meaningful variables infer the output, i.e. guaranteeing that an underlying truthful causality exists in the model reasoning.

To conclude, the choice between using either an interpretable or a black-box ML model depends on the problems being modelled and how the model will be applied. Just because an 'interpretable' model is used does not mean that the interpretations provided measure well according to the metrics described, that the audience will understand them, or that they will achieve the goals of interpretability. In addition, a black-box model may not always outperform an interpretable model, as the interpretation process can help improve the accuracy of the model. Each model and explanation method should be considered individually and selected based on the specific problem being investigated and the metrics used to assess performance in terms of both accuracy and interpretability.

2.4 GLOBAL AND LOCAL INTERPRETATIONS

The explanation provided by the IML model or the post hoc XAI method can focus on the model as a whole, helping to understand how the model makes decisions and the impact of the features and the model design. Alternatively, an explanation can focus solely on explaining individual predictions or a group of related predictions, regardless of how the model generates these predictions. These are also referred to as 'global and local interpretations'.

Locally important features are not always globally important, and vice versa: Methods that attempt to provide global interpretations may perform worse on a metric of local interpretability (Molnar, 2020). While global fidelity of an interpretation method would imply local fidelity, identifying explanations that are globally faithful remains a challenge for complex models. Hence, many methods only focus on providing local interpretations for the predictions (Ribeiro, Singh & Guestrin, 2016). Therefore, local interpretations are sometimes aggregated to provide a global interpretation.

2.5 GOAL AND QUESTIONS TO BE ANSWERED

There are many aspects to the term 'interpretation'. The goal of the interpretation, along with the audience and their questions, will determine whether a global or local interpretation is required and, therefore, which model and method to use. This is because the type of answers that different prediction models and interpretation methods can provide varies.

2.5.1 Goal of an interpretation

There are many possible goals or reasons why the results of these models must be interpreted and explained. These have been reviewed in the literature by Barredo Arrieta *et al*. (2020), Vilone and Longo (2021), Rudin *et al*. (2021), Doshi-Velez and Kim (2017), Alvarez-Melis and Jaakkola (2018), Guidotti *et al.* (2018) and Lipton (2018).

The main goals discussed are summarised below.

- Trustworthiness: This might be considered as the confidence in whether a model will act as intended when facing a given problem. However, the definition of trust is not clear. Some define it as simply confidence that a model will perform well. If so, a sufficiently accurate model should be demonstrably trustworthy, and interpretability would serve no purpose. Alternatively, trust depends on the type of examples it gets right or wrong. If the model makes the same type of mistakes or errors that humans do, it is likely to be considered trustworthy in the sense that there is no expected cost of relinquishing control. However, if it gives different predictions than humans, then there may always be an advantage to maintaining human supervision of the algorithms. Accurate predictions alone are therefore not sufficient to generate trust in ML models. The level of accuracy required from a model to ensure trustworthiness is consequently also higher than that required from humans and examples are required where the model was correct, but the human was incorrect.
- Confidence: Trust can also be increased if the level of confidence in, or an estimate of the likelihood of, a prediction can be highlighted. This will help mitigate any loss in trust that will occur when incorrect predictions are made. Confidence is linked to the metric of certainty in a prediction discussed in section 2.6.2.
- Causality: Another common goal of interpreting a model is that of finding causality among data variables. This should enable researchers to uncover new ways in which different variables interact and their impact on real-world problems. This can then lead to deeper research into understanding why these interactions occur.
- Informativeness: Related to causality is the need to extract information about the inner relations of a model and understand the causes of the interactions discovered by the model. Therefore, the explanations should support the extraction and transfer of novel knowledge to human decision makers, allowing domain experts to understand the structure and patterns within the different data classes.

- Fairness: From a social standpoint, explainability can be considered as the capacity to guarantee that decisions conform to ethical standards or can help highlight any bias in the data the model was exposed to.
- Interactivity or usability: Explainability in a model can also be seen as the ability of a model to be interactive, or to assist a user in accomplishing a task with awareness.

The goal helps define the terms 'interpretation' or 'explanation' and whether global or local explanations are required, and must be agreed to before selecting an interpretation method.

2.5.2 Audience of the explanation

The goal of the interpretation, and therefore the type of explanation required and the need for interpretable models, depend on the audience. Model owners and domain experts are likely to be more interested in understanding the model, whereas users affected are only interested in an explanation of the prediction impacting them. Barredo Arrieta *et al.* (2020:4) state as follows: "Given a certain audience, explainability refers to the details and reasons a model gives to make its functioning clear or easy to understand."

The different groups of possible audiences include domain experts, managers, executive board members, regulatory entities/agencies, product owners, managers and users affected by model decisions. For example, domain experts may prefer a larger and more sophisticated model over a smaller and sometimes more opaque one (Guidotti *et al*., 2018).

2.5.3 Questions to answer

The goal and the audience determine the type of questions that may be asked from a model and its predictions. The model and explanation method used can vary depending on which of these goals or questions are the most important to achieve. Examples of questions that have been proposed are summarised below.

General questions proposed by Lim (2012) for users affected by the model are as follows:

- What certainty is there that a prediction will be true?
- What happens if I change a feature?
- Why did the model (not) give the specific predictions, and how can a prediction be contested (related to the goal of causality)?
- How much should the input variable values change to give an alternative prediction, with a related 'if' where the input state is constrained in some way? Therefore, what

combination of available changes in inputs will result in a change in the prediction, and is a different prediction possible at all? This focuses on actionable features.

Krause *et al*. (2016) suggest the following questions by domain experts:

- Which data point has the most influence on the calibration of the model, and which are the least well predicted?
- Does the model behave correctly on a case-by-case basis?
- Why are certain data points not being accurately predicted?
- What are the most important features for a given case?

  This last question could be rephrased as:

  - How much did each feature contribute to the individual prediction?
  - Which factor needs to change the least to change the prediction?

Ustun, Spangher and Liu (2019:1) define recourse "as the ability of a person to change the decision of the model through actionable input variables". They list the following questions that counterfactuals can also help answer:

- Does a model provide recourse to all individuals who are subject to its predictions?
- How does the difficulty of recourse vary across individuals in a target population?
- Are there disparities in recourse between subgroups in the target population?
- What changes can an individual make to obtain a desired prediction from the model?

Doshi-Velez and Kim (2017) recommend the following broad situations:

- Forward simulation: Presented with an explanation and an input, the interpretation must allow a user to correctly simulate the output of the model, regardless of the true output.
- Counterfactual simulation: Presented with an explanation, an input and an output, the interpretation must allow a user to ask what must be changed to change the prediction of the method to a desired output (and related variants).

Any proposed XAI solution must be able to answer these types of questions. However, a single XAI solution may not be able to answer all these questions, and the method should specify which of these it is trying to address.

2.6 METRICS OF INTERPRETABILITY

Even models seen as interpretable do not provide clear answers to the questions mentioned, and both interpretable models and post hoc methods lack clear metrics for evaluating the performance of the explanation provided. Clear metrics should allow for a meaningful comparison of how well a model fits the definition of interpretability and determine whether an explanation method is truly model-agnostic or whether it only works for specific ML models (Barredo Arrieta *et al.*, 2020).

The following are possible metrics for interpretation. These can be separated into global interpretation and local interpretations. Any proposed XAI solution should be measured against these definitions and metrics to determine whether it provides a better interpretation than existing methods.

2.6.1 Global interpretation metrics

Vilone and Longo (2021) propose the following metrics for measuring the accuracy of global post hoc interpretations:

- Prediction accuracy: Global interpretation methods that investigate the importance of the input features can be measured according to the increase in prediction accuracy of including or excluding a feature based on the recommendation of the method.
- Fidelity: This measures the performance of a surrogate model in providing global interpretations of the ML model. Guidotti *et al*. (2018) define 'fidelity' as the extent to which a surrogate model can accurately imitate the predictions made by a black-box model. Surrogate models are simplified but interpretable models used to mimic the function of a more complex black-box model. These models are created by training a model on the prediction of the more complex models. The interpretation of the surrogate model is then used to explain the more complex model. Global fidelity implies that the predictions by the surrogate are as accurate as those of the black-box model, measured across all data points.
- Explanation completeness: This refers to the extent to which the interpretation gives insight into the underlying model. This involves capturing the maximum number of features from the input that affect the decision-making process of the model, including the impact of the interaction between the features and how they vary across all data points.

- Robustness or sensitivity to input perturbation: This refers to the ability of a model to give consistent predictions for data points that are close in the input space. A robust model is less likely to be fooled by adversarial attacks, which carefully perturb inputs to fool a model into incorrect predictions.

  While both adversarial and counterfactual examples involve perturbing input data to alter model predictions, their purposes differ: Adversarial examples are crafted to exploit or undermine the model, whereas counterfactual examples are designed to improve understanding of the behaviour of the model.

  Robustness can be measured by the ability of the model to maintain accurate predictions even when some features of the input dataset or model parameters are removed, masked or replaced with random values. Robustness also ensures that counterfactual explanations are meaningful and realistic.

  Another robustness measure is the Counterfactual Explanation-based Robustness Score (CERScore) by Sharma, Henderson and Ghosh (2019). This measures the expected distance between the input instances and their corresponding counterfactuals. Given two ML models, if the counterfactuals are further away from the input instances on average for one model compared to the other, the model will have a higher CERScore, and the interpretations using their counterfactuals should be more robust.

2.6.2 Local interpretation metrics

The following metrics can be used for measuring the accuracy of individual prediction explanations:

- Prediction accuracy: If the prediction model is not very accurate, then the explanations for the individual predictions are unlikely to be accurate or insightful.
- Local fidelity: This refers to the accuracy of an individual prediction made with a surrogate model when a surrogate model is used to interpret the predictions of the black-box model. The lower the fidelity, the less reliable the local explanation will be. Local fidelity implies that the surrogate model is accurate in the area around the data point or prediction being investigated.
- Certainty: A good explanation model should also give a measure of the confidence in the correctness of any individual prediction. As discussed in section 2.7.1, the prediction score does not indicate the likelihood of the prediction being correct.

Providing this likelihood, along with confidence intervals for the prediction, will increase trust in the prediction.

- Stability: This refers to consistency between the explanation given to two data points. An explanation model should give similar explanations to two data points with similar features. This feature can be summarised by the following axioms proposed by Honegger (2018):
    - Identity: Identical objects must have identical explanations.
    - Separability: Non-identical objects cannot have identical explanations.
    - Stability: Similar objects must have similar explanations.

    These three axioms are necessary but not sufficient requirements for interpreting individual data points; however, the quality of an individual interpretation can be measured by using these three axioms as metrics.

- Consistency: This refers to consistency between the explanation given to the prediction by two different models trained on the same data. Consistency is not always achievable due to the Rashomon effect, where different models assign varying weights to different features in the data. These will result in different interpretations even when the models give the same prediction outcome.
- Novelty: The explanation should also describe whether the data point being explained comes from a region in the feature space that is far away from the distribution of the training data. In such cases, the model may be inaccurate, and the explanation may be useless. One way of providing this information is to locate the data instance to be explained in the distribution of the training data. Furthermore, the concept of novelty is related to the concept of certainty: Higher novelty should result in lower certainty due to a lack of reliable data.

2.7 ASPECTS TO CONSIDER WHEN INTERPRETING MACHINE LEARNING MODELS

Any proposed XAI method must address the specified goals of the interpretation, be appropriate for the target audience, answer specific questions and perform well on the selected metrics. However, the nature of the underlying model and the number of features included impact the ability of any interpretation to meet these goals. Therefore, when providing an interpretation, the following limitations should be considered.

2.7.1 Uncertainty and confidence in predictions

One of the definitions discussed is the ability to explain the confidence in a prediction. Even though classification models give a prediction value between 0 and 1, this does not necessarily equate to the confidence in the prediction or the likelihood of the data point being in class 0 or class 1. The ability to give prediction likelihoods depends on the model used.

Let $\boldsymbol{Y}$ be the output of the unknown function $f(\boldsymbol{x})$ for every input $\boldsymbol{x}$. The goal of supervised ML (Vapnik, 1999; Vapnik & Izmailov, 2020) is to estimate the function $\hat{f}(\boldsymbol{X})$, which best approximates Y. When an assumption is made on the joint distribution $P(\boldsymbol{x}, y)$, statistical decision theory can be used to approximate $f(\boldsymbol{x})$ with $P(y|\boldsymbol{x})$, by conditioning on $P(\boldsymbol{x})$. Alternatively, a function-fitting paradigm (learning by example) or functional approximation and estimation method can be used.

The methods for determining the classifier $f(y|\boldsymbol{x})$ can be segmented into two groups: generative and discriminative classifiers (Argouarc'h, Desbouvries, Kawasaki & Barat, 2024):

- Generative, or informative, classifiers learn a model of the joint class densities or probability $p(\boldsymbol{x}, y)$ (Ng & Jordan, 2001) and then make predictions using Bayes' rule to calculate

$$p(y|\boldsymbol{x}) = p(\boldsymbol{x}, y)/\, p(\boldsymbol{x}).$$

Alternatively, the class densities $p(\boldsymbol{X}|y = k)$ and priors $P(y)$ are estimated. If we assume $P(y) = \pi_k$ is the prior probability of class k and $f_k(x) = p(\boldsymbol{X}|y = k)$ is the class-conditional density of $\boldsymbol{X}$ in class $G = k$, then

$$p(\boldsymbol{x}) = \sum_k p(\boldsymbol{X}|y = k)p(y).$$

In comparing the two classes k and l, it is sufficient to look at the log ratio of

$$log(p(y = k|\boldsymbol{x}) \,/\, p(y = l|\boldsymbol{x})) = \log(\, f_k(\boldsymbol{x})\pi_k \,/\, f_l(\boldsymbol{x})\pi_l).$$

The decision boundary is then induced, and the full log likelihood can be determined from (Rubinstein & Hastie, 1997):

$$\log p(\boldsymbol{x}, y) = \log p(y|\boldsymbol{x})\, p(\boldsymbol{x}).$$

Examples of these methods include linear discriminant and naïve Bayes models.

- Discriminative classifiers model the posterior $p(y|\boldsymbol{x})$, and hence the discriminants, directly with a focus on modelling the class decision boundaries or class membership probabilities. Compared to the generative approach, the discriminative classifier throws away the information in the marginal density $p(\boldsymbol{x})$. These include linear regression models.

Models that use no probability model are also sometimes referred to as 'discriminative classifiers'. These model $f(\boldsymbol{x})$ directly and are solved using the function-fitting paradigm. Examples include neural networks and support vector machine models.

The main drawback of the function-fitting approach is that it does not give a posterior class probability $P(y = k|\boldsymbol{x})$. Without this estimate, the confidence in the prediction cannot be specified. Even methods that do provide the posterior probabilities do not always provide accurate estimates, with naïve Bayes in general pushing predicted values towards 0 and 1 (Niculescu-Mizil & Caruana, 2005). Decision trees assign a prediction of $p = k/n$ as the score of any example assigned to a leaf that contains k positive training examples and n total training examples (Zadrozny & Elkan, 2001).

To maximise the predictions for many practical prediction and decision-making tasks, it is crucial to develop an accurate probabilistic prediction model from data. ML models that are not optimised for obtaining accurate prediction probabilities are referred to as miscalibrated. In general, a set of predictions of a binary outcome is well calibrated if the outcomes predicted to occur with probability p do occur about p fraction of the time, for each probability p that is predicted (Naeini, Cooper & Hauskrecht, 2015).

The less certainty there is about the accuracy of a prediction, the less reliable the interpretation of the prediction will be. This, in turn, will reduce trust in the model and make the interpretations less helpful in understanding the interactions between the features and the output. To increase trust in the model, the interpretation should quantify the uncertainty associated with a prediction.

Two common methods for mapping model predictions to posterior probabilities are Platt calibration and isotonic regression. Unfortunately, these methods are designed for binary classification, and it is not trivial to extend them to multiclass problems (Caruana, 2005).

The probabilities of support vector machines are approximated by a sigmoid function using Platt scaling, where:

$$P(y = 1|x) = P_{A,B}(f) = 1/(1 + exp(Af(x) + B).$$

Here $f(\boldsymbol{x})$ is the support vector decision function and A and B are two scalar parameters that are estimated using a maximum likelihood method on the same training set as that used for the original classifier f.

The confidence of a prediction can be further enhanced by adding confidence intervals to a prediction. This helps quantify the certainty that a sample belongs to a certain class. A 95% confidence interval will mean that for 100 such cases; 95 will be in the specified class. Statistical models such as GLMs can give the confidence interval of a prediction because the link function used assumes a certain distribution for the mean and variance of the prediction. Similar statistics are not available in other black-box models. Such explanations, even if no individual reasons for the prediction are provided, will increase the understanding and interpretability of ML models.

2.7.2 Impact of model complexity and number of features

A measure of the complexity of a model is the number of prediction regions it creates, i.e. regions separated by different decision boundaries. Model complexity depends on both algorithmic complexity – the design of the model – and the number and type of input features. Complexity can be defined in many ways, including the Vapnik–Chervonenkis dimension of the classifier (Vapnik & Chervonenkis, 1971).

The more complex a model, the more difficult it is to create global interpretations (Guidotti *et al.*, 2018). Complexity is often used as the antonym to interpretability: The more complex a model, the less likely it is that the explanations provided will be the true underlying reason for a prediction or will explain the inner workings of the model.

A more complex model requires a longer calculation period for some interpretation methods, and more simplifying assumptions to provide answers within a reasonable calculation period. If these assumptions are invalid, the interpretation provided will not be reliable. These limitations to any interpretation should be understood when assessing whether one method of interpretation is better than another.

The ability to generate local and global interpretations increases if the model gives monotonic predictions, i.e. if the predictions of the model only increase or decrease as an input variable increases or decreases. The type of explanations and the method used depend on the type of input data. The interpretability of tabular data improves with the sparsity of the number of features because humans can handle only roughly seven cognitive entities at the same time (Rudin *et al.*, 2021).

Lastly, individual input variables can be continuous, ordinal or nominal data types. For a method to be truly interpretable, it must be able to accommodate all types of data. An alternative is to create dummy variables to convert categorical values into numeric values or cut the continuous variable into intervals to create ordinal values. However, this may dramatically increase the search space, which can make both the prediction and the interpretations less accurate. The distance measurement used in the explanation method should also be able to measure distances across different data types.

2.7.3 Rashomon effect

The primary issue with interpreting ML models is that different models utilise transformations of the features to achieve the same predictions or minimum error rate. Consequently, the interpretation of the reasons behind these predictions will vary. This phenomenon was first described by Breiman (2001b) as the 'Rashomon effect'.

The Rashomon effect is closely connected to the instability that occurs when multiple models are crowded together, each having approximately the same training or test set error. The two models are close to each other in terms of error, but can differ significantly in terms of the form of the model. A slight perturbation of the data or in the model construction will cause a different change from one model to another, and can change which model is selected as the final prediction model. This limits the ability of the model and its interpretations to achieve the goal of giving accurate insight into the real-world causality of the targeted outcome, as they will give different and possibly contradictory reasons for the effect.

2.8 INTERPRETABLE MACHINE LEARNING METHODS

The literature covers six broad groupings of transparent and IML models that meet the definition of transparency as simulatability, decomposability and algorithmic transparency outlined in section 2.3. However, just because these methods are called transparent does not mean that these models always provide both global and local interpretations, and they do not answer all the types is questions listed in section 2.5.

2.8.1 Decision trees

Decision trees predict results using a series of nodes that represent feature values and a leaf that represent a class label or regression value. Therefore, it represents a series of embedded 'if-then-else' decision rules. This makes both the local and the global interpretation very

intuitive. The method can repeat the use of a feature as a node, with the relative number of times a feature is used approximating the influence of a feature.

Single tree approximation methods are also a common method for developing model-agnostic XAI methods, as explained by Guidotti *et al*. (2018). According to Hastie, Tibshirani and Friedman (2009), decision trees are the only classification method that provides individual (local) explanations. These models are often extended to an ensemble of trees, which increases the prediction accuracy of the model. However, this results in the model losing its intuitive interpretability.

2.8.2 Rule-based learners

A generalisation of decision trees is 'rule-based learners' or 'decision sets'. These create a series of if-then statements, but without the hierarchical or ordered nature of decision trees. It therefore does not always require the 'else' component. It can also have multiple conditions at each node, where decision trees typically do not. These m-of-n rules have n conditions, and the result is true if m of them are met. They can also make use of fuzzy rule-based systems that are designed for a broader scope of action, allowing for the definition of verbally formulated rules over imprecise domains (Barredo Arrieta *et al.*, 2020).

Both decision trees and rule-based learners are highly intuitive and enable the accurate simulation of the decision-making process of the model. It provides a step-by-step explanation of how the decision is made, what will happen when certain inputs are changed and what needs to change to alter the output. The more complex the rules, the less transparent the model will be. In addition, it is difficult to determine the most influential feature on a global and local level.

2.8.3 Linear/Logistic regression

A linear (for regression) or logistic (for binary classification) model assumes a linear dependence between the input variables and the predicted variable. This constraint results in a very transparent model at the expense of prediction accuracy.

The model can be generalised by transforming the result with a link function or incorporating interaction terms. These are referred to as generalised linear models. Lasso and ridge regression can improve the selection of variables and calibration of the coefficients through appropriate regularisation. Lasso regression helps remove individual features, making the model sparser and thereby further improving its interpretability.

The following aspects of the coefficients make regression models interpretable:

- The intercept represents the predicted outcome of an instance when all features are at their mean value.
- The sign (+ or -) of the coefficient determines the direction (increase or decrease) in which the variable value must change to change the outcome.
- The size of the coefficient – the bigger the coefficient, the smaller the change in variable value needed to achieve the same change in prediction, and therefore the more influential the variable. This does not take into account the relative size of the variable unit. To overcome this, the variable unit size can be normalised. The coefficient will then represent the impact of a one-standard-deviation change.
- The model output often includes the z-score of each variable, which tests the null hypothesis that the coefficient value is zero. The large z-score will lead to the rejection of the null hypothesis. Therefore, the model output provides a natural way of variable selection, with small z-score variables interpreted as being insignificant and removed from the model.

The model has many built-in outputs that can be used to interpret each variable and what needs to change to achieve a desired prediction outcome. Regression models are also algorithmically transparent, with the model internals explained by the beta coefficients provided in the summary output. However, the coefficient values are not readily understandable to a non-expert audience, and the model does not explain which combination of features needs to change to minimise the required adjustments to achieve the desired outcome.

2.8.4 Generalised additive models

Generalised additive models (GAMs) extend the flexibility of GLMs by replacing individual input variables with a smooth spline function of the input variable. This allows the model to capture the non-linear impact of individual variables, while retaining the independence between variables and ensuring algorithmic transparency. This can increase the model prediction accuracy.

GAMs are often presented as the gold standard for intelligibility when only univariate terms are considered (Guidotti *et al.*, 2018). The spline function can be plotted to illustrate how the prediction changes as the input variable value changes, providing a visual representation of the impact of linear and non-linear features on the model predictions. This can also help

assess the importance of each feature. However, the GAM coefficient size and sign values are not readily interpretable, unlike those in a GLM.

2.8.5 Bayesian models

Bayesian models can also take the form of naïve Bayes models. These predict a class based on the product of the individual class-conditional marginal densities of each input variable (Hastie *et al.*, 2009). The resulting form is similar to that of GAMs. Like GAMs, these models can also give a clear visual representation of the relationships between features and the target.

The main advantage of a Bayesian model is the ability to fully determine the distribution of the $f(\boldsymbol{x}, y)$. This allows the model prediction to also give the confidence in the prediction.

2.8.6 Case-based reasoning

Rudin *et al*. (2021) explain case-based reasoning as a paradigm that involves solving a new problem using known solutions to similar past problems. This is transparent, as it predicts in a manner similar to how humans would attempt the problem.

There are two types of case-based reasoning techniques: (1) nearest neighbour-based techniques and (2) prototype-based techniques. Nearest neighbours predict the value of a new data point by looking at the class of known data points that it most closely resembles – those with the smallest 'distance' or the largest 'similarity'. The classic method is k-nearest neighbours, which takes the average prediction of the k closest known data points. In these methods, the accuracy can be improved using different measures or transformations of distance, such as Mahalanobis distance. However, this may reduce the transparency of the model. The selection of too large a value for k may also reduce transparency. It also requires a substantial number of distance calculations to find the nearest neighbours, which can impede the ability to implement the method.

This has led to the development of prototype-based methods. These create a set of prototypes for each class. Therefore, for a new data point, similarity must only be calculated relative to these data points. The class of the closest prototype is allocated to the new data point. This removes the need for an excessive number of distance calculations and the need to select k. The prototype will also serve as a good representative of the class, making the interpretation easier to trust. The prototype can be either an actual data point or a calculated representative point.

Various methods have been developed to select the most appropriate prototypes. Kim and Austin (2016) proposed Maximum Mean Discrepancy critic, which selects prototypes so that the distribution of the prototypes is close to the data distribution, and Kaufman and Rousseeuw (1987) proposed calculating k-medoids instead of k data points as prototypes.

Providing individual examples of classifications helps build trust in the model and explains how it works. These methods can also be used for any model and input type. However, these methods do not explain which variables are the most influential and do not give intuitive answers to questions such as how much the input variable values should change to give an alternative prediction.

2.9 EXPLAINABLE ARTIFICIAL INTELLIGENCE METHODS

This section summarises different post hoc model-agnostic XAI methods found in the literature. These methods provide either global or local interpretations. This list of methods is not exhaustive, with more methods explained in various articles and books, including Holzinger *et al.* (2020) and Barredo Arrieta *et al*. (2020). Model-specific XAI methods are not covered in this study.

2.9.1 Explanations using a surrogate explanation model

A surrogate model explains a black-box model by training an explainable model that mimics or approximates the predictions made by the more complex black-box model. The explanations provided by the surrogate models are then assumed to apply to the predictions of the complex model. The surrogate model is therefore often one of the six transparent models discussed in section 2.8. For example, a decision tree can be used to approximate the results of a neural network model.

Surrogates can give either global or local explanations, and their accuracy is measured by their local or global fidelity. Global surrogates aim to replicate and explain all the predictions of the black-box models. Local surrogates provide an explanation for an individual data point. If the surrogate has high fidelity, then this method gives very reliable explanations. The steps to create a surrogate model can be explained as follows:

1. Let f be the original prediction model to be explained and g the explanation model. Let the focus be on explaining a prediction $f(\boldsymbol{x})$ for a single input $\boldsymbol{x}$.
2. The input $\boldsymbol{x}$ is simplified to create $\boldsymbol{x}'$, with the map between the two created by $\boldsymbol{x} = h(\boldsymbol{x}')$. This allows certain variables or levels to be dropped to simplify the

explanations provided by the surrogate model, or continuous variables to be binned into ranges that have the same explanation (i.e. continuous variables are binned or categorical variables are one-hot encoded, to allow variables to be represented by 0's or 1's.)

3. Local methods then try to ensure that $g(\boldsymbol{z}') = f\big(h(\boldsymbol{z}')\big)$ whenever $\boldsymbol{z}' = \boldsymbol{x}'$, with $\boldsymbol{z}'$ the individual data point of interest.
4. The prediction model $f(\boldsymbol{x})$ has j features, which can be reduced to i features in the explanation model.

Surrogate models often take the form of an additive feature attribution method, which is a linear function of binary variables. A linear model is interpretable, with the coefficients providing an explanation for an individual data point $\boldsymbol{z}'$. The simplified model is defined as:

$$g(\boldsymbol{z}') = \theta_0 + \sum_{i=1}^{M} \theta_i \boldsymbol{z}_{\boldsymbol{i}}'$$

where $\boldsymbol{z}' \in \{0,1\}^M$, M is the number of simplified input features and $\theta \in R$.

Two methods that use an additive feature attribution are LIME (Ribeiro *et al*., 2016) and SHAP (Lundberg & Lee, 2017a). LIME (Local Interpretable Model-agnostic Explanations) fits a model by first creating m permutations of the data point $\boldsymbol{x}$ and predicting the outcome of each point with the black-box model. The permutations are created by drawing non-zero elements of $\boldsymbol{x}'$ uniformly at random – with $\boldsymbol{x}'$ the one-hot encoded version of $\boldsymbol{x}$ such that $\boldsymbol{x} = h(\boldsymbol{x}')$.

The explanation model is then fitted to this new dataset, with the new data points weighted by their similarity score and the insignificant features removed (causing the reduction in features from j to i). The remaining features then provide local explanations.

SHAP (SHapley Additive explanation) values calculate Shapley values to attribute to each feature the change in the expected model prediction when conditioning on that feature. They explain how to get from the base value $E[f(\boldsymbol{z})]$, that would be predicted if we did not know any features, to the output value of $f(\boldsymbol{x})$. SHAP values arise from averaging the θ_i values across all possible orderings of a conditional expectation function of the original model. Therefore, SHAP answers the question of why this prediction is different from the average prediction and not how the prediction for this individual can result in a different outcome.

Shapley values is a method originally invented for assigning pay-outs to players depending on their contribution towards the total pay-out (Shapley, 1952). In the explanation setting, the

features are the players and the individual prediction is the total pay-out (Aas, Jullum & Løland, 2021). The Shapley value is the fair allocation of the contribution of each of the M features in the pay-out function v and is calculated as:

$$\theta_j(v) = \sum_S \frac{|S|!(M-|S|-1)!}{M!}\big(v(S \cup \{j\}) - v(S)\big), j = 1 \dots M,$$

i.e. a weighted mean over all subsets S of players not containing player j.

With SHAP, the contributions to a single prediction $f(x)$ can be summarised as follows:

$$f(\boldsymbol{x}) = \theta_0 + \sum_j^M \theta_j \text{ where } \theta_0 = E[f(\boldsymbol{x})],$$

i.e. the SHAP explains the difference between the prediction $y = f(\boldsymbol{x})$ and the global average prediction. The difference between SHAP and LIME for an individual data point is summarised by Aas *et al*. (2021) as follows:

- The sum of the Shapley values for all features is equal to the difference between the prediction $f(\boldsymbol{x})$ and the mean probability based on all the data points.
- The sum of the LIME values is also equal to the difference between the prediction $f(\boldsymbol{x})$ and a mean probability, but here the mean is taken over all data points similar to data point $\boldsymbol{x}$.

Calculating Shapley values is computationally very demanding. This has led to the development of efficient approximations, such as Shapley regression values, by Strumbelj and Kononenko (2010), Lundberg and Lee (2017a; 2017b) and Lipovetsky and Conklin (2001). The primary benefit of using Shapley values is that the impact on the prediction is perfectly distributed among the input features, whereas LIME does not guarantee this.

For additive feature attribution methods to give valid interpretations, they should aim to meet three desirable properties, as defined by Lundberg and Lee (2017):

1. Property 1: Local accuracy: When approximating the original model f for a specific input $\boldsymbol{x}$, local accuracy requires the explanation model to at least match the output of f for the simplified input $\boldsymbol{x}'$ (which corresponds to the original input $\boldsymbol{x}$).
2. Property 2: Missingness: If the simplified inputs represent feature presence, then missingness requires features missing in the original input to have no impact.

3. Property 3: Consistency: This states that if a model changes so that the contribution of some simplified input increases or stays the same regardless of the other inputs, attribution of that input should not decrease.

LIME does not meet property 3, but Shapley values do. Shapley values and SHAP therefore provide more reliable explanations than LIME.

2.9.2 Feature importance explanations

Feature importance methods aim to answer the question of how much each feature contributes to the individual predictions. Various feature importance explanation methods have been developed, including salient or gradient methods, sensitivity analysis and permutation-based methods, which highlight either the local or the global importance of a feature.

The original method proposed by Breiman (2001a) defines global feature importance as the decrease in prediction accuracy (increase in error rate) when a covariate is permuted (made uninformative). Local feature importance has been subsequently developed, including the use of SHAP for a single prediction or the combined effect over all the predictions (Lundberg *et al.*, 2019).

Feature importance can be used to improve model accuracy and calibration by assisting with feature selection. It identifies the least important feature to delete in each iteration of fitting the model via backward selection. Feature importance methods can measure one of the following:

- How important a variable is to a specific prediction (local importance). This is determined by how much including or excluding the feature changes the prediction made for the individual data point.
- How important a variable is in the overall prediction (global importance), which helps with the feature selection process for the calibration of a model. This is determined by how much including or excluding the feature improves the global prediction accuracy of the model.
- How important a variable is independently of a model, which helps with the goals of causality and informativeness.

2.9.3 Model visualisation

The use of visualisations to provide model interpretations can be split into two broad approaches. The first gives a graphical representation of otherwise numeric results. This can be seen in partial dependence, individual conditional expectations, accumulated local effects and sensitivity analysis plots, where the visual representation allows for easy inspection of the results of these post hoc model-agnostic interpretation methods (Apley & Zhu, 2020; Friedman, 2001; Goldstein, Kapelner, Bleich & Pitkin, 2014).

The second approach applies dimension reduction techniques to create two-dimensional maps of the data. The most common method is multi-dimensional scaling (MDS), which allows users to investigate the plot and gain insight into the relationships within the data. MDS attempts to find points in a two-dimensional Euclidean space that generate distances that approximate distances given in an ($n \times n$) symmetric distance matrix.

MDS can also be used as a form of clustering by visualising the data and creating clusters based on the proximity of the data in the two-dimensional space. This can be extended by converting each pixel on the two-dimensional map to the original dimension space using an inverse projection method. This allows decision zones to be created on the map that show the areas with the same classification. The decision boundaries are then visualised as the border where the classification changes from one colour to another.

Two state-of-the-art non-linear dimensionality reduction techniques are t-SNE and UMAP. Rodrigues, Espadoto, Hirata and Telea *et al.* (2019) point out that non-linear dimension reductions typically do not feature an inverse projection function to allow the variable values to be extracted from the two-dimensional surface. Where the inverse projection function exists, it is often time-consuming to calculate and apply to the data, and it can result in inverse projection errors. Therefore, these maps are not efficient in determining the decision boundaries on the two-dimensional surface. In addition, the distances to the visible decision boundary on the two-dimensional surface do not always correspond to the distances to the actual decision boundary in the original feature space.

Sohns, Garth and Leitte (2023) expand the dimension reduction to a principal component analysis (PCA) biplot, where the variables are plotted as axes to indicate the relationship between data points and variables and map the decision boundary and counterfactuals. The PCA biplot provides an efficient way to create the inverse projection function, ensuring that the visual distances of the data points to the local decision boundary represent a monotonic relationship to the distances to the actual decision boundary in the original feature space.

As the complete dataset is unlikely to contain only linear correlations, Sohns *et al*. (2023) recommend restricting the PCA training samples to a subset of nearest neighbours to the target data point, where the linear relationship is more likely to hold. The subset should contain a balance of both the current and the counterfactual classes to ensure that the hyperplane contains the decision boundary. The plane is shifted to ensure that the target data point is on the hyperplane. Variables can be removed from the plot by fixing their values when selecting the subset.

The nearest counterfactual is selected by measuring the $L1$-distance in the feature space. If the decision boundary visualisation shows unexpected patterns, it could indicate that the model contains an artefact from overfitting to a biased training set and raise suspicions concerning the robustness of the ML model. However, it may be due to distortions caused when mapping high-dimensional data onto a two-dimensional space.

2.10 COUNTERFACTUALS

The last type of post hoc XAI method discussed makes use of counterfactuals. A counterfactual can be defined as a close neighbour of an actual data point, but with its predictions giving the desired different outcome. LIME and SHAP values explain a prediction by determining how much each feature contributed to it. Counterfactual explanations differ from feature attributions, because they generate data points with a different, desired prediction instead of attributing a prediction to the feature (Dandl, Molnar, Binder & Bischl, 2020). Neither LIME nor SHAP provide counterfactuals (White & D'Avila Garcez, 2020).

Counterfactuals traditionally give local explanations (Wielopolski, Furman, Stefanowski & Zięba, 2024). On their own, counterfactuals do not provide insight into the global behaviour of the model. However, the idea can be expanded to global counterfactual explanations, which generate a single representative vector for the entire dataset.

There is a range of model-agnostic counterfactual estimation methods, with the main difference being the optimisation method used to determine the closest counterfactual. Certain methods of generating counterfactuals are model-specific. These are mostly applied to differentiable classification models to allow gradient-based optimisation of the distance function.

Investigating the counterfactuals can address various goals of IML, such as causality or fairness. Causality can be investigated by determining which variables have the most significant impact on different data points. Fairness can be investigated by understanding how

difficult it is for two data points that are similar except for a specific feature (e.g. gender) to reach their counterfactual and change their predictions. This is linked to the idea of recourse, and whether the difficulty of recourse varies across individuals in a target population or whether there are disparities in recourse between subgroups in the target population.

Wachter *et al*. (2017) state that counterfactuals are the only method that can truly address an individual's right to obtain an explanation. Depending on the method used to find the counterfactual, they are also less likely to infringe on trade secrets or the rights and freedoms of others (e.g. privacy), as many methods require no disclosure of the training data or detailed information about the workings of the targeted ML model.

A counterfactual explanation can give an interpretation of the prediction, regardless of whether the classifier is wrong or the counterfactual generated is absurd (Laugel, Lesot, Marsala, Renard & Detyniecki, 2018). Therefore, the counterfactual generation method requires a metric to evaluate the counterfactual or explanation produced, allowing for comparison of which counterfactual yields the better explanations.

2.10.1 Defining and finding a counterfactual

There are many proposed methods of generating counterfactuals. Most define a counterfactual as the solution $\boldsymbol{\delta}$ found by optimising the following objective function for a data point $\boldsymbol{x}_0 \in R^p$:

$$\boldsymbol{\delta}: \min f(\boldsymbol{x}_0, \boldsymbol{\delta}) + g_{dist}(\boldsymbol{\delta})$$

where x_0 is the current individual data point and is part of the p-dimensional feature space, f is the calibrated prediction model, $y \in \{0,1\}$ is the target variable and $\boldsymbol{\delta}$ is the vector representing the change made to the current value $\boldsymbol{x}_0$ to reach the counterfactual value $\boldsymbol{x}_{cf}$. $f(\boldsymbol{x}_0, \boldsymbol{\delta})$ requires the predicted class of $\boldsymbol{x}_{cf}$ to be different from $\boldsymbol{x}_0$. Then $g_{dist}(\boldsymbol{\delta})$ is the cost function to be optimised (Ustun *et al*., 2019; Van Looveren & Klaise, 2019).

Therefore, a counterfactual is the solution to an optimisation function that balances two requirements:

- Given a data point $\boldsymbol{x}_0 \in R^p$, $y \in \{0,1\}$, a prediction model f calibrated on $\{\boldsymbol{x}, y\}$ and $\boldsymbol{x}_0$ predicted to be in class $\hat{y} = f(\boldsymbol{x}_0) = 0$, create a new data point $\boldsymbol{x}_{cf}$ such that $\hat{y} = f(\boldsymbol{x}_{cf}) = 1$.

- $\boldsymbol{x}_{cf} = \boldsymbol{x}_0 + \boldsymbol{\delta} \in R^p$ must be the closest possible point to $\boldsymbol{x}_0$ given some distance metric g_{dist}, with the vector $\boldsymbol{\delta}$ the change/perturbances required to cause a change in the prediction of the target data point.

The cost function can be expanded to include one of the following:

- A range of different distance metrics, including $L0$, $L1$ and $L2$, a combination similar to elastic net regularisation (Zou & Hastie, 2005) or distance measures that can accommodate categorical variables, such as Gower or Jaccard distances. The selection of a weight applied to the different distance measures becomes another hyperparameter that must be solved.
- Multiple non-distance terms (Dandl *et al.,* 2020; Van Looveren & Klaise, 2019) or probability weighing terms (Dhurandhar *et al.*, 2019; Ustun *et al.*, 2019; Wachter *et al.*, 2017).
- Additional constraints to ensure that the results meet requirements for justifiability, feasibility, sparsity or actionability (Poyiadzi, Sokol, Santos-Rodriguez, De Bie & Flach, 2020; White & D'Avila Garcez, 2020).

Finding the solution requires the use of some optimisation or search process. These vary across the different methods and are analytical, heuristic or grid search solutions. The optimisation method must be able to escape local minima to find the counterfactual value. It must complete the calculation in a reasonable time period, taking into account that computation time will increase as the number of counterfactuals or the size of the feature space increases (Van Looveren & Klaise, 2019).

The proposed solutions vary in their approach. Some methods are model-agnostic, while others work only for specific model designs, such as decision trees or differentiable models. They also vary in their ability to accommodate categorical data and continuous variables, or in their ability to generate a new data point as the solution or only select a point from the training dataset. Some methods can only find a counterfactual if they have access to the underlying internals of the model and/or the training data.

Lastly, certain constraints may be put on the counterfactual to improve its ability to give realistic solutions to questions on why a prediction was made or how the model works (Mazzine & Martens, 2021). These constraints are discussed in the next section.

2.10.2 Constraints on counterfactual solutions

The ability of counterfactuals to provide a valid interpretation of a model or prediction depends on their ability to generate solutions that meet the following constraints. If the proposed method cannot meet these constraints, it will be less practically applicable for solving real-world problems.

- Actionable: A counterfactual should only include changes to actionable features, such as income level. Non-actionable features fall into three categories: immutable features, which cannot change (e.g. gender), conditionally immutable features, which can change but only in one direction (e.g. age or academic qualifications) and mutable but non-actionable features, which may change only as a consequence of changes in other variables, rather than through direct intervention. Therefore, the counterfactual generation method should limit the features that can be changed to those that are actionable. The closer the counterfactual is to the original data point, the smaller the changes needed to achieve the different prediction. This makes the explanation more actionable, and users can more easily understand and implement the necessary changes.
- Feasible: A feasible feature implies that a change in the prediction class is possible, given the distribution of $\boldsymbol{x} \in R^p$. Therefore, $\boldsymbol{x}_{cf}$ must be within the range of valid values for each feature for the outcome to be feasible. The counterfactual generation method should specify the upper and lower bounds possible for each actionable feature to ensure that the change is feasible. This is also explored by Poyiadzi *et al*. (2020), who propose generating counterfactual explanations that respect the underlying data distribution and are connected via high-density paths (based on the shortest path distances defined via density-weighted metrics) to the explained instance to ensure that the counterfactual is actionable and feasible. Feasible values can be identified by selecting only counterfactuals from the existing training data examples. However, training samples are not always available when performing post hoc evaluation or explaining a model.
- Plausible: The notion of plausibility in counterfactual explanations refers to the likelihood that suggested changes occur within the realistic subset of the training data manifold (Wielopolski *et al.*, 2024). A plausible feature considers the probability of $\boldsymbol{\delta}$ occurring, allowing for the conditional distribution of the other features. Often, the assumption of independent variables is used, but this may not be appropriate. Plausibility and feasibility are related terms, with feasibility referring to individual

features and plausibility to the overall distribution of the data. The solution must therefore not be trapped in a local minimum in which only feasible, but not plausible, solutions are provided. To be plausible, the results should not be an out-of-distribution value. This requirement can be met by requiring the counterfactual to be selected from a list of actual training data points, or by choosing the nearest prototype or 1-nearest neighbour to the calculated counterfactual. Solutions can be made more plausible by standardising each feature. This will ensure that the change not just depends on the scale of the variable, but also considers the multiple of the units of standard deviation from the feature mean (Tolomei, Silvestri, Haines & Lalmas, 2017). Plausible also refers to the existence of a continuous path between $\boldsymbol{x}_{cf}$ and a training data point $\boldsymbol{x}_a$ in the same class as the counterfactual, such that no decision boundary of f is met. This is also referred to as a justified counterfactual, ensuring that the counterfactual is grounded in an actual data point. Solutions far from the decision boundary are less likely to be justified (Laugel *et al.*, 2019). Alternatively, a counterfactual is called ε-justified if there is a chain of individual variable changes, each smaller than ε, where with each change the classification remains the same. A justified solution ensures that the counterfactuals are linked to ground truth data and not generated because of an artefact in the ML model. To measure this, access to the original training data is required. ML models such as logistic regression or 1-NN classifier have, by design, no unjustified counterfactuals. Therefore, the link between connectedness and overfitting is not immediately apparent and requires further investigation.

- Sparse: The solution should also aim to be as sparse as possible by limiting the number of non-zero features in $\boldsymbol{\delta}$. Multiple small changes may result in a smaller distance function, but a sparse solution will be easier to explain and implement for the user. One method of generating sparse solutions is by using a $L0$ or $L1$ distance function when solving for $\boldsymbol{\delta}$. However, there is a trade-off between sparsity and plausibility. If only a single feature is changed to reach the counterfactual value, then the counterfactual value may be an out-of-distribution value and not plausible given the data distribution. This was developed by Laugel *et al*. (2018), who propose identifying the nearest counterfactual using the $L2$ distance measure. The solution is made sparser by removing features using a naïve heuristic that the smallest coordinates of the vector $\boldsymbol{x}_{cf} - \boldsymbol{x}_o$ is the least relevant locally and can therefore be ignored, assuming that by not changing that variable, the counterfactual remains in a different class than the target data point.

- Diversity: The solution should preferably be able to provide multiple and a diverse range of solutions (Mothilal, Sharma & Tan, 2020). Methods that provide a range of possible solutions will allow the user to select the most appropriate and actionable way to change the outcome. The range of answers will also give insight into how the model makes predictions. A 'flipset' is a set of actionable and feasible counterfactuals, which will therefore provide recourse, ranked by the measure of distance and/or sparsity. This requires a range of solutions to be provided by the selected method.

The requirements for the features to be actionable, feasible and/or plausible are related recourse. The potential lack of recourse is often used to motivate calls for increased model transparency and explainability. However, transparency and explainability do not guarantee recourse, as a linear model, which is transparent and explainable, will also not necessarily give recourse.

2.10.3 Metrics of the accuracy of the counterfactual

There are numerous proposed metrics for evaluating and comparing different counterfactual generation methods. These metrics are summarised by Mazzine and Martens (2021) and Williams, Katakkar, Heidari and Kolter (2024) as follows:

- Distance to counterfactual: The distance between the proposed counterfactual point and the data point, measured using various distance measures including Euclidean, Mahalanobis or the MAD (the $L1$ distance divided by the median absolute deviation distance). Counterfactuals aim to minimise the selected distance measure. Therefore, the best method is the one that can generate the nearest counterfactual value.
- Calculation time: The average time required for the method to generate a single counterfactual. The longer it takes to find a solution, the less useful it will be in generating local and global explanations.
- Coverage: The number of data points for which a method was able to find a counterfactual in a different class. Not all methods guarantee that a counterfactual can be found in the time available to run the model.
- Redundancy: The number of features in a counterfactual that can be reverted to their original (reference) values without altering the predicted class. In other words, it reflects the number of unnecessary changes included in the counterfactual. Redundancy can also be interpreted as a measure of the scarcity or efficiency of the solution.

- Stability: The ability to generate similar counterfactuals in two different runs of the counterfactual generation model while using the same input parameters. It may contrast with the diversity metric. Methods that struggle to replicate the results of the optimisation process will give diverse results, but this will have a detrimental effect on the stability metric.

These can be used to determine whether the method proposed in this study improves on the existing methods.

2.11 AREAS DEVELOPED IN THIS STUDY

This chapter provided an overview of key considerations when evaluating the interpretability of an ML model or the post hoc explanations derived from it. These include the definition of interpretability, whether the interpretation method is model-specific or model-agnostic, whether it offers local or global explanations, and the types of questions or objectives it seeks to address.

The chapter also reviewed existing interpretation techniques, distinguishing between models that are inherently transparent and model-agnostic post hoc methods, which can be applied to any type of ML model to provide either global or local explanations.

It concluded with an introduction to the concept of counterfactuals, emphasising their role in answering questions related to model behaviour and predictions. The discussion included the key constraints that counterfactuals must meet in order to provide meaningful and useful interpretations.

Chapter 3 presents the methodology proposed for generating counterfactuals that satisfy the interpretability requirements outlined in this chapter.

CHAPTER 3 METHODS TO IDENTIFY COUNTERFACTUALS

3.1 INTRODUCTION

This chapter explains the key methods and definitions required to identify and visualise the counterfactual values of target data points. It covers the construction of PCA and canonical variate analysis (CVA) biplots, the rotation of the biplot plane and the identification of local decision boundaries. All computations and figures were generated in R version 4.4.3 (R Core Team, 2025). Biplots were constructed with the biplotEZ package by Lubbe *et al.* (2025) and additional code is provided in Appendix A.

The chapter concludes by explaining how the biplot and the counterfactuals should be interpreted, and how the method should be implemented to address potential shortcomings. These counterfactuals and decision boundaries will then be used in subsequent chapters to provide post hoc, model-agnostic interpretations of ML models.

3.2 EUCLIDEAN AND MAHALANOBIS DISTANCE MEASURES

Euclidean distance between two points is the length of the line between the points. Originally calculated from Cartesian coordinate points using the Pythagorean theorem, it is sometimes referred to as the 'Pythagorean distance'.

The length of a vector $\boldsymbol{x} = [x_1, \dots, x_p]$ in the p-dimensional real vector space $\boldsymbol{R}^p$ can be given by the Euclidean $L2$ norm:

$$\|\boldsymbol{x}\|_2 = (x_1^2 + x_2^2 + \cdots + x_p^2)^{1/2}.$$

The Euclidean distance between two points is the straight line with length:

$$\|\boldsymbol{x} - \boldsymbol{y}\|_2 = ((x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_p - y_p)^2)^{1/2}.$$

Euclidean distance is scale-dependent, with the contribution of each dimension dependent on the measurement size of the dimension. If the data is first normalised with each variable having a mean of zero and a standard deviation of one, then the distance measure ignores the correlation between variables and each variable contributes equally to the distance measure.

An alternative distance measure is the Mahalanobis distance. This measure scales each dimension to ensure unit variance along the principal components of the data. Therefore, where $\boldsymbol{T}$ is a positive-definite covariance matrix of the data, the Mahalanobis distance between two points is given by:

$$d(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{T})^2 = ((\boldsymbol{x} - \boldsymbol{y})'\boldsymbol{T}^{-1}(\boldsymbol{x} - \boldsymbol{y})).$$

Mahalanobis distance is therefore unitless and scale-invariant, and takes into account the correlations between the variables in the dataset.

Since $\boldsymbol{T}$ is positive-definite, so is $\boldsymbol{T}^{-1}$. Therefore, the square root matrix is always defined, with:

$$\boldsymbol{T}^{-1} = \boldsymbol{L}\boldsymbol{L}'.$$

$\boldsymbol{L}$ is guaranteed by the spectral theorem, but it is not unique. Different choices have different theoretical and practical advantages (Kessy, Lewin & Strimmer, 2018).

3.3 PRINCIPAL COMPONENT ANALYSIS BIPLOT

Biplots can be understood as a type of MDS and are a means of giving an optimal (in a least-squares sense) two-dimensional representation of the multi-dimensional dataset $\boldsymbol{X}$: $n \times p$. This enables the Euclidean distances between points in the two-dimensional space to reflect the similarity of their values in the original high-dimensional space. Points with more similar multi-dimensional values appear closer together on the biplot.

Biplots extend MDS by representing the variables of $\boldsymbol{X}$ as axes within the same two-dimensional space. This allows the approximate values of each data point in each dimension to be inferred by projecting the points onto these axes. The ‘bi’ in the term ‘biplot’ therefore refers to the simultaneous representation of both the observations (data points) and the variables (as calibrated axes).

The construction of biplots is summarised below and is explained in more detail by Gower, Lubbe and Le Roux (2011).

3.3.1 Constructing a biplot

The basis for the construction of a biplot is the singular value decomposition (SVD) of the data matrix $\boldsymbol{X}: n \times p$, with n the number of rows of data and p the number of variables.

The SVD of any matrix $\boldsymbol{X}: n \times p$, assuming $n \geq p$, with $\boldsymbol{X}$ assumed to be centred without loss of generality, can be expressed as:

$$\boldsymbol{X} = \boldsymbol{U}^*\boldsymbol{\Sigma}^*\boldsymbol{V}^{*\prime}, \text{ with}$$

$$\boldsymbol{U}^*: n \times n \text{ a matrix with orthogonal columns, with } \boldsymbol{U}^{*\prime}\boldsymbol{U}^* = \boldsymbol{I}_n$$
$$\boldsymbol{V}^*: p \times p \text{ a matrix with orthogonal columns, with } \boldsymbol{V}^{*\prime}\boldsymbol{V}^* = \boldsymbol{I}_p$$

$$\boldsymbol{\Sigma}^*: n \times p \text{ a matrix of the form } \boldsymbol{\Sigma}^* = \begin{bmatrix} \boldsymbol{\Sigma} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}$$

Equation 1: SVD of X

Let $Rank(\boldsymbol{X}) = k$, then the diagonal matrix $\boldsymbol{\Sigma}^* : n \times p$ contains the singular values $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_k > 0$ and $\lambda_{k+1} = \lambda_{k+2} = \ldots = \lambda_p = 0$, while $\boldsymbol{\Sigma}$ is a $k \times k: k \leq p$ diagonal matrix, with diagonal elements the non-zero singular values of $\boldsymbol{X}$ in non-increasing order.

Then $\boldsymbol{X}$ can be written as $\boldsymbol{X}: n \times p = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}'$, where $\boldsymbol{U}: n \times k$ and $\boldsymbol{V}: p \times k$ consists of the first k columns of $\boldsymbol{U}^*$ and $\boldsymbol{V}^*$, respectively, and both are orthonormal. This then implies that $\boldsymbol{XV} = \boldsymbol{U\Sigma}$. $\boldsymbol{U}$ is referred to as the matrix of left singular vectors. $\boldsymbol{V}$ is referred to as the matrix of right singular vectors, or the projection matrix.

This then extends to a key result by Hotelling (1933) that, for $\boldsymbol{Y} = \boldsymbol{XV}$, the columns of $\boldsymbol{Y}$ are uncorrelated (the off-diagonal elements of $\boldsymbol{Y}'\boldsymbol{Y}$ are equal to zero), and the variance of each column of $\boldsymbol{Y}$ is given by the eigenvalues $\boldsymbol{\Sigma}^2$ that correspond to each eigenvector $\boldsymbol{V}$ of $\boldsymbol{X}'\boldsymbol{X}$. This is represented by $\boldsymbol{cov}(\boldsymbol{Y}) = \boldsymbol{V}'\boldsymbol{cov}(\boldsymbol{X})\boldsymbol{V} = \boldsymbol{V}'\boldsymbol{V}\boldsymbol{\Sigma}^2\boldsymbol{V}'\boldsymbol{V} = \boldsymbol{\Sigma}^2$. $\boldsymbol{\Lambda}$ is used to represent $\boldsymbol{\Sigma}^2$, such that $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^2$.

An r-dimensional approximation of $\boldsymbol{X}$ is given by:

$$\widehat{\boldsymbol{X}}_{[r]} = \boldsymbol{U\Sigma JV'} = \boldsymbol{UJ\Sigma V'} = \boldsymbol{UJ\Sigma JV'}, \text{ where}$$

$$\boldsymbol{J}: p \times p = \begin{bmatrix} \boldsymbol{I}_r & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} \end{bmatrix} \text{ and } r \leq p,$$

with the $p - r$ smallest diagonal values of $\boldsymbol{\Sigma}$ replaced by 0.

Equation 2: Calculation of r-dimension representation of X

As proved by Eckart and Young (1936), $\widehat{\boldsymbol{X}}_{[r]}$ is an optimal r-dimensional approximation of $\boldsymbol{X}$ in the least-squares sense that

$$\left\|\boldsymbol{X} - \boldsymbol{X}_{[r]}\right\| = tr\left\{(\boldsymbol{X} - \boldsymbol{X}_{[r]})(\boldsymbol{X} - \boldsymbol{X}_{[r]})'\right\}$$

Equation 3: Best r-dimensional representation of X

is minimised for all matrices $\widehat{\boldsymbol{X}}_{[r]}$ of rank not larger than r. This is the same as minimising the sum of the squares between the original data and the data projected onto the plane spanned (or generated) by the columns of $\boldsymbol{V}_r = \boldsymbol{VJ}$. Therefore, the distance from the multi-dimensional data points in $\boldsymbol{X}$ to the r-dimensional data points in $\widehat{\boldsymbol{X}}_{[r]}$ is, on average, a minimum.

The columns of $\boldsymbol{VJ}$ are often termed the 'principal components', or the 'principal component loadings'. The principal component scores are obtained from $\boldsymbol{Y} = \boldsymbol{XVJ}$ and the biplot created using the matrix $\boldsymbol{V}$ is called a PCA biplot.

In the case where $r = 2$, the orthogonal projection of all the rows of $\boldsymbol{X}$ into two dimensions is given by the first two columns of $\boldsymbol{V}$, with:

$$\widehat{\boldsymbol{X}}_{[2]} = \boldsymbol{UJ\Sigma J^*V}_2' = \boldsymbol{XV}_2\boldsymbol{V}_2'$$

and

$$\boldsymbol{V}_2 = \boldsymbol{VJ}^*, \text{with } \boldsymbol{J}^*: p \times r = \begin{bmatrix} \boldsymbol{I}_r \\ \boldsymbol{0} \end{bmatrix}.$$

Equation 4: Two-dimensional representation of X

The biplot plane represents the two-dimensional approximation of the centred $\boldsymbol{X}$**.** The first two columns of $\boldsymbol{XV}_2$ provide the coordinates of the data points on the biplot plane, corresponding to the orthogonal projection of the rows of $\boldsymbol{X}$ onto the biplot plane spanned by the vectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$. The new x- and y-axes of the biplot are referred to as the 'scaffolding axes'.

The variable matrix $\boldsymbol{V}$ is projected onto the plane as $\boldsymbol{VJ}^*$. The two coordinates in the $j'th$ row in $\boldsymbol{V}_2$ give the direction and endpoints of the $j'th$ biplot variable axes, represented as vectors from the origin.

Using the notation introduced above, the coordinate $\boldsymbol{z}'$, resulting from the projection of any data point $\boldsymbol{x}$ on the two-dimensional biplot plane, is given by $\boldsymbol{z}' = \boldsymbol{x}'\boldsymbol{V}_2$. This implies that any new data point not included in the original dataset can be added to the biplot plane. The process of placing such new points onto the existing biplot is known as 'interpolation'.

The projected two-dimensional coordinate can also be converted back to the original p-dimensional value with $\widehat{\boldsymbol{x}}' = \boldsymbol{x}'\boldsymbol{V}_2\boldsymbol{V}_2' = \boldsymbol{z}'\boldsymbol{V}_2'$, where $\boldsymbol{V}_2' = \boldsymbol{J}^{*\prime}\boldsymbol{V}'$. The process of converting a point on the biplot plane to the original p-dimensional space is called 'prediction'. The matrix $\boldsymbol{V}_2'$ is referred to as the 'PCA prediction matrix'. As some of the information was lost when reducing the number of principal components to two, in general, $\widehat{\boldsymbol{x}}' \neq \boldsymbol{x}'$.

3.3.2 Estimating the similarity and correlations within X

The equation $\widehat{\boldsymbol{X}}_{[2]} = \boldsymbol{UJ\Sigma JV}_2'$ can also be represented as the product of two matrices:

$$\underset{n \times p}{\widehat{\boldsymbol{X}}_{[2]}} = \underset{n \times 2}{\boldsymbol{G}} \quad \underset{2 \times p}{\boldsymbol{H}'}.$$

Using the notation above, the PCA biplot is constructed from two possible decompositions:

Decomposition A:

$$\widehat{\boldsymbol{X}}_{[2]} = \begin{matrix} \boldsymbol{G} & \boldsymbol{H}' \\ \boldsymbol{U\Sigma J}^* & (\boldsymbol{VJ}^*)' \end{matrix}$$

Equation 5: Decomposition A

or *Decomposition B:*

$$\widehat{\boldsymbol{X}}_{[2]} = \begin{matrix} \boldsymbol{G} & \boldsymbol{H}' \\ \boldsymbol{UJ}^* & (\boldsymbol{V\Sigma J}^*)' \end{matrix}$$

Equation 6: Decomposition B

$\widehat{\boldsymbol{X}}_{[2]}\widehat{\boldsymbol{X}}_{[2]}'$ gives the approximation of the distances between the rows of $\boldsymbol{X}$, and therefore the distances between data points on the biplot surface provide an estimate of the similarity of the

data points. As some of the information was lost by reducing the number of principal components to two, the estimate of the distances is not completely accurate.

$\widehat{\boldsymbol{X}}'_{[2]}\widehat{\boldsymbol{X}}_{[2]}$ gives the approximation of the correlation between the columns of $\boldsymbol{X}$, and therefore the angles between the axes indicate the degree to which variables are correlated, with axes at or close to right angles indicating uncorrelated variables.

Notice that in Decomposition A, the principal component scores $\boldsymbol{Y} = \boldsymbol{XVJ}^* = \boldsymbol{U\Sigma J}^*$ are represented in matrix $\boldsymbol{G}$ and the loadings $\boldsymbol{VJ}^*$ are represented in matrix $\boldsymbol{H}$. Depending on how $\widehat{\boldsymbol{X}}_{[2]}$ is decomposed, the biplot can either more accurately approximate the similarity between the rows of $\boldsymbol{X}$ or approximate the correlation between the columns of $\boldsymbol{X}$.

The difference between the two decompositions is shown in Figure 3.1. The correlation between variables 1 and 2 is 0.8, which is reflected in the angle between variables 1 and 2 in the second plot of Decomposition B and is only approximated in the first plot of Decomposition A.

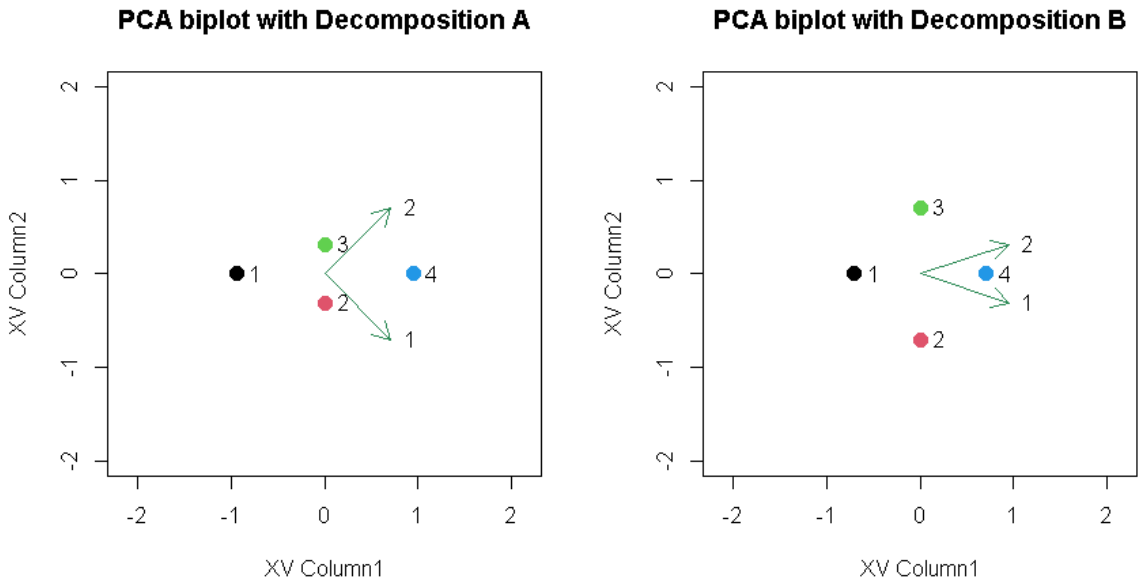


Figure 3.1: Comparison of the PCA biplot using decompositions A and B, where Decomposition A accurately approximates the similarity between the data points and Decomposition B approximates the correlation between the variables

The PCA biplot with Decomposition B plots the variable vectors as the rows of $(\boldsymbol{V\Sigma J}^*)$. The data points $\boldsymbol{UJ}$ can be added to the plot, and the inner product of $\boldsymbol{UJ}^*$ and $\boldsymbol{V\Sigma J}^*$ estimates $\boldsymbol{X}$. However, the distances between the pairs of rows do not approximate the distances between the rows of $\boldsymbol{X}$ directly.

In this case, the inner product of the variable axes $\frac{1}{n-1}\boldsymbol{V\Sigma J}^{*}(\boldsymbol{V\Sigma J}^{*})'$ gives an estimate of the covariance matrix of $\boldsymbol{X}$. Small angles between variable axes imply a strong correlation. Variables that increase in the same direction imply a correlation close to 1, and those that increase in different directions imply a correlation close to -1. Variables axes that are orthogonal imply there is no correlation between them. As some of the information was lost by reducing the number of principal components to two, the estimate of the correlation is not entirely accurate.

As an alternative, $\boldsymbol{U\Sigma J}^{*}$ and $\boldsymbol{V\Sigma J}^{*}$ can be plotted to retain their individual meanings of relative distance/similarity between data points and correlations between variables, but their inner product will not estimate $\boldsymbol{X}$, and the value of the data point cannot be measured via the projection onto the biplot axes. This plot is called a 'joint plot', while the term 'biplot' is reserved for a plot where the inner products of the points and axes are respected.

3.3.3 Predictivity and variable importance

The PCA projection reduces the number of dimensions from p to $r = 2$, causing information on each variable to be lost.

$\boldsymbol{\Lambda}$ is equal to the variance along each of the biplot plane axes. This is the same as the sum of squares of the original variables when the means are set equal to zero, with the sum of squares over the variables in $\boldsymbol{X}$ equal to the sum of the elements of $\boldsymbol{\Lambda}$.

The trace of $\boldsymbol{\Lambda J}$ represents the total variance captured by the biplot. Therefore, the quality of the plot can be measured by the reduction in the total variance shown, with:

$$Quality = \frac{tr(\boldsymbol{\Lambda J})}{tr(\boldsymbol{\Lambda})}.$$

Each variable, represented by the rows in $\boldsymbol{V}$, is orthogonal and has been unitised to reflect a length of one. After reducing the dimensions represented, the length of each variable will be less than one, with variables being shortened by the contribution of each of the principal components to its length, which were removed when reducing the number of dimensions. Variables close to the biplot plane will lose little length, as they are accurately represented on the plane, and the contribution to their length by the principal components removed is very low. The variables perpendicular to the plane will lose most of their length. The extent to which each variable reduces in length reflects the information lost from each variable. The percentage remaining reflects the adequacy with which each variable is represented, where 'adequacy' is defined as:

$$adequacy = diag(\boldsymbol{VJV}')/\boldsymbol{diag}(\boldsymbol{VV}'), \text{ with } \boldsymbol{diag}(\boldsymbol{VV}') = \boldsymbol{I}.$$

The unit circle represents perfect adequacy, and the shorter the variable vector compared to the circle, the poorer the adequacy of the variable. For a PCA biplot constructed from a standardised data matrix, this shortened vector will still reflect one standard deviation. However, the projection of a data point onto a variable vector will have lost information due to the reduced number of principal components shown. Therefore, the estimate of the data point onto the end of the variable vector is either more or less than one standard deviation, depending on the distance and direction of the point and the vector relative to the biplot plane.

As explained above with the quality measure, $\boldsymbol{\Lambda}$ reflects the total variance captured by each of the biplot axes in $\boldsymbol{V}$. A measure of the importance of each variable is how much of the original variance is retained by each variable after the dimension reduction and therefore the remaining 'variance accounted for' per variable.

The total sum of squares can be decomposed into:

$$\boldsymbol{X}'\boldsymbol{X} = \boldsymbol{V\Lambda V}' = \boldsymbol{V\Sigma}(\boldsymbol{V\Sigma})'.$$

Therefore, the length of each vector $(\boldsymbol{V\Sigma})_j$ (as shown in Decomposition B above) reflects the contribution to the total variance in the data by variable j. The reduction in the lengths of each variable in $\boldsymbol{V\Sigma J}^*$ when plotted on the biplot plane reflects the reduction in the variance accounted for or sum of squares for each variable after the dimension reduction. This reduction is a measure of the axis predictivity:

$$Axis\ predictivity: \boldsymbol{\Pi}: p \times p = diag\left(\widehat{\boldsymbol{X}}'\widehat{\boldsymbol{X}}\right)\ [diag(\boldsymbol{X}'\boldsymbol{X})]^{-1} = diag(\boldsymbol{V\Lambda JV}')\ [diag(\boldsymbol{V\Lambda V}')]^{-1}.$$

The diagonal elements of $\boldsymbol{\Pi}$ are the measures of the predictive powers of each variable and clearly cannot exceed unity. The closer to unity, the more of the original variance of each variable is represented on the biplot plane.

If $\boldsymbol{X}$ was first standardised and divided by the factor $(n-1)^{-1/2}$, then the variance along each axis will be standardised and the length of $\boldsymbol{V\Sigma}$ will be equal to 1. However, the correlation *between* the variables in the columns of $\boldsymbol{X}$ will still be reflected by the variance *within* each column of $\boldsymbol{Y}$, represented by $\boldsymbol{\Sigma}$**.** The length of each variable $\boldsymbol{V\Sigma J}^*$ will be within the unit circle, and the reduction of each is a measure of the $axis\ predictivity$ $\boldsymbol{\Pi}$. The angle between the variables will reflect an approximation of the correlation between the variables of $\boldsymbol{X}$. When plotting only these variable axes, the plot is called a 'correlation monoplot'. It is a variation of the PCA biplot Decomposition B, without the data points, but with the unit circle added. An

example with different data that contains four variables is shown in the graph in Figure 3.2. The variance of variables 2 and 4 is almost entirely accounted for in the plot.

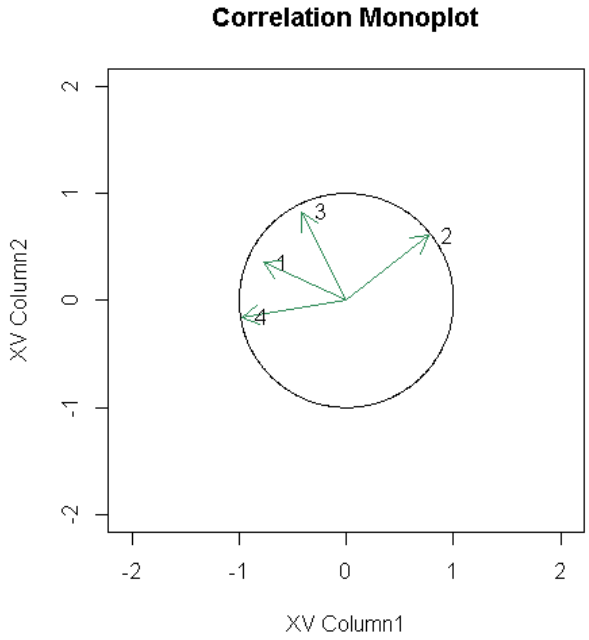


Figure 3.2: Illustration of a correlation monoplot and a unit circle, where the length relative to one unit measures the $\boldsymbol{axis\ predictivity}$

Similarly, the sample predictivity is the diagonal elements of the matrix:

$$Sample\ predictivity: \boldsymbol{\Psi}: p \times p = diag\left(\widehat{\boldsymbol{X}}\widehat{\boldsymbol{X}}'\right)\ [diag(\boldsymbol{XX}')]^{-1} = diag(\boldsymbol{U\Lambda JU}')\ [diag(\boldsymbol{U\Lambda U}')]^{-1}.$$

A value of unity (1) implies that the sample is in the plane of approximation, and a value of 0 when the direction of the vector representing the data point is orthogonal to the plane. A data point with low sample predictivity will not be accurately interpreted by the biplot explanation method.

3.3.4 Calibrated variable axes

In this dissertation, biplots are presented as multivariate analogues of scatter plots with calibrated axes. The original biplot, introduced by Gabriel (1971), represents variable axes using vectors drawn from the origin, with the direction and endpoint defined by the two coordinates of the $j'th$ row in $\boldsymbol{V}_2$.

A biplot allows the approximate value of a variable for an individual data point to be determined by orthogonally projecting that point onto the relevant variable axis and calculating the length

along the axis from the origin to the projected point. For example, the value of the first variable for the second data point in the data matrix $\boldsymbol{X}$ can be estimated using the scalar product of the first row of $\boldsymbol{V}'$ with the second row of $\boldsymbol{Y} = \boldsymbol{XV}$.

However, when variable axes are represented solely by the vectors in $\boldsymbol{V}_2$, it becomes difficult for the viewer to intuitively determine the lengths of the data point and variable vectors, making it challenging to estimate the original variable values from the biplot. To address this limitation in interpretation, Gower and Hand (1996) introduced biplot axes that are calibrated with markers.

Calibrating the variable axes requires the unit length of each row of $\boldsymbol{V}_2$. Each row is represented by the coordinate $\boldsymbol{b}_k$. The coordinate is calibrated to represent one unit length through the scaling process $\boldsymbol{b}_k / \| \boldsymbol{b}_k \|$.

Once the length of each unit of a variable on the biplot plane is determined, markers can be added to the variable axes that represent the variable values in the original unit of measurement. This allows the viewer to estimate a variable value by orthogonally projecting the data point onto the corresponding axis and interpolating between the nearest two calibration markers. This is equivalent to reading values from the Cartesian axes in ordinary scatter plots.

These calibrated axes are used throughout the remainder of this dissertation to represent two-dimensional biplots. For three-dimensional biplots, the variable vectors are used.

3.4 CANONICAL VARIATE ANALYSIS BIPLOT

An alternative to a PCA biplot is a CVA biplot. A PCA biplot optimally represents a dataset $\boldsymbol{X}$: $n \times p$ in two dimensions. When different classes of observations are present in the dataset, a CVA biplot optimally separates the classes in the biplot space, with the canonical transformation matrix used to add the individual data points to the biplot plane. Let:

$\overline{\boldsymbol{X}} : k \times p = (\boldsymbol{G}'\boldsymbol{G})^{-1}\boldsymbol{G}'\boldsymbol{X}$ be the matrix of group means, where

$\boldsymbol{X}$ is the centred data with p variables and n measurement, k the number of classes and n_k the number of entries in each class.

$\boldsymbol{G}$ is an $n \times k$ indicator matrix where each row consists of 0's except for a 1 in the column associated with that sample

$\boldsymbol{G'G} = \boldsymbol{N}$ = diag$(n_1, n_1, \ldots, n_k)$ and

$$\boldsymbol{Q}: n \times n = \boldsymbol{G}(\boldsymbol{G'G})^{-1}\boldsymbol{G'}.$$

The matrix of the total group sums of squares and products (SSP) ($\boldsymbol{T}$) can be partitioned into the matrix of between-groups SSP ($\boldsymbol{B}$) and within-groups SSP ($\boldsymbol{W}$):

$$\begin{array}{lcccc} \boldsymbol{T} & = & \boldsymbol{B} & + & \boldsymbol{W} \\ \boldsymbol{X'X} & = & \boldsymbol{X'G}(\boldsymbol{G'G})^{-1}\boldsymbol{G'X} & + & \boldsymbol{X'}[\boldsymbol{I} - \boldsymbol{G}(\boldsymbol{G'G})^{-1}\boldsymbol{G'}]\boldsymbol{X}. \end{array}$$

Equation 7: Partitioning of the total SSP matrix

A CVA biplot aims to transform the variables such that the Euclidean distance between the canonical means is the same as the Mahalanobis distance between the group means:

$$\delta_{\boldsymbol{kh}}{}^2 = (\bar{\boldsymbol{x}}_{\boldsymbol{k}} - \bar{\boldsymbol{x}}_{\boldsymbol{h}})'\boldsymbol{W}^{-1}(\bar{\boldsymbol{x}}_{\boldsymbol{k}} - \bar{\boldsymbol{x}}_{\boldsymbol{h}}).$$

This requires a non-singular transformation matrix $\boldsymbol{L}: p \times p$ such that $\boldsymbol{x'LL'x} = \boldsymbol{x'W}^{-1}\boldsymbol{x}$ or $\boldsymbol{LL'} = \boldsymbol{W}^{-1}$. Consider the eigenvector equation $\boldsymbol{WL} = \boldsymbol{L\Lambda}$, where we scale the eigenvectors to give $\boldsymbol{L'WL} = \boldsymbol{I}$, noting that although the eigenvectors are always orthogonal, their scaling is arbitrary. The choice of scaling in this dissertation gives $\boldsymbol{L'L} = \boldsymbol{\Lambda}^{-1}$ and $\boldsymbol{LL}^T\boldsymbol{WLL'} = \boldsymbol{LIL'}$ or $\boldsymbol{L'} = \boldsymbol{W}^{-1}$, as required.

In particular, the group means $\bar{\boldsymbol{X}}$ are transformed to $\bar{\boldsymbol{X}}\boldsymbol{L}$ in the canonical space and are called the 'canonical means' with $\bar{\boldsymbol{X}}\boldsymbol{LL'}\bar{\boldsymbol{X}}' = \bar{\boldsymbol{X}}\boldsymbol{W}^{-1}\bar{\boldsymbol{X}}'$, confirming the Mahalanobis distances between the means $\bar{\boldsymbol{X}}$**.** Therefore, we have a property that is of practical consequence: Mahalanobis distances between the means $\bar{\boldsymbol{X}}$ are represented in the canonical space as ordinary Pythagorean distances between the canonical means.

This canonical space can then be approximated in fewer dimensions through the SVD of $\bar{\boldsymbol{X}}\boldsymbol{L}$, which results in a matrix $\boldsymbol{V}$ such that $\boldsymbol{M}: p \times p = \boldsymbol{LV}$. This incorporates both the transformation to canonical variables and the PCA orthogonal transformation. The matrix $\boldsymbol{M}$ is referred to as the 'canonical transformation matrix'.

In the canonical space, the variables are uncorrelated with unit variance via the transformation with a common matrix $\boldsymbol{W}^{-1}$. Therefore, the CVA biplot is scale-invariant and removes correlations between the data within groups, assuming the within-group covariance matrix is the same for each group.

The rank of the matrix $\bar{\boldsymbol{X}}\boldsymbol{L}$ is the same as the rank of $\bar{\boldsymbol{X}}: k \times p$, and the weighted sum of any row of the centred matrix $\boldsymbol{X}$ can be determined from the sum of the group means of the remaining $k-1$ sum of the rows. Therefore:

$$Rank(\bar{\boldsymbol{X}}\boldsymbol{L}) \leq min(k-1, p) = m,$$

with the eigenvalues associated with the first m eigenvectors being greater than zero, and the remaining eigenvalues being zero. In the case where $k = 2$, there will be only one non-zero eigenvalue and $p - 1$ zero eigenvalues.

In the above, the diagonal matrix $\boldsymbol{\Lambda}: p \times p$ contains the eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_p \geq 0$, where $\lambda_k = \lambda_{k+1} = \ldots = \lambda_p = 0 \; if \; k < p + 1$.

Since the $Rank(\bar{\boldsymbol{X}}\boldsymbol{L}) \leq m$, a $k-1$ dimensional biplot will represent the k group means exactly. The CVA biplot requires the data to contain a minimum of three classes to be presented on a two-dimensional plane.

The sample points can then be interpolated onto the CVA biplot with the transformation

$$\boldsymbol{z}' = \boldsymbol{x}'\boldsymbol{M}\boldsymbol{J}^{*\prime}$$

and any point on the plane can be predicted in the original space by using the first two rows of the inverse matrix $\boldsymbol{M}^{-1}$ with the transformation.

$$\boldsymbol{x}' = \boldsymbol{z}'\boldsymbol{J}^{*}\boldsymbol{M}^{-1}.$$

From Equation 7 above, $\boldsymbol{B} = \boldsymbol{X}'\boldsymbol{G}(\boldsymbol{G}'\boldsymbol{G})^{-1}\boldsymbol{G}'\boldsymbol{X} = \boldsymbol{X}'\boldsymbol{G}'(\boldsymbol{G}'\boldsymbol{G})^{-1}(\boldsymbol{G}'\boldsymbol{G})(\boldsymbol{G}'\boldsymbol{G})^{-1}\boldsymbol{G}'\boldsymbol{X} = \bar{\boldsymbol{X}}'\boldsymbol{N}\bar{\boldsymbol{X}}$. In general, this can be written as $\bar{\boldsymbol{X}}'\boldsymbol{C}\bar{\boldsymbol{X}}$. The choice in $\boldsymbol{C}$ can result in the canonical plane going through the unweighted centroid of the group means ($\boldsymbol{C} = \boldsymbol{I} - k^{-1}\boldsymbol{1}\boldsymbol{1}'$), the weighted centroid of group means ($\boldsymbol{C} = \boldsymbol{N}$) or the centroid of all the samples ($\boldsymbol{C} = \boldsymbol{I}$). The weighted centroid of group means $\boldsymbol{C} = \boldsymbol{N}$ was used in this dissertation.

To create the CVA biplots, the matrices $\boldsymbol{M}_2 = \boldsymbol{M}\boldsymbol{J}^{*}$ and $\boldsymbol{M}_2^{-} = \boldsymbol{J}^{*\prime}\boldsymbol{M}^{-1}$ were used instead of $\boldsymbol{V}_2$ and $\boldsymbol{V}_2'$ that were used for PCA biplots. The matrix $\boldsymbol{M}_2^{-}$ is referred to as the 'CVA prediction matrix'.

In summary, the CVA first calculates the canonical space, where the Euclidean distance between the group means of the transformed variables is equivalent to the Mahalanobis distance in the original space. It then proceeds by applying PCA to these transformed variables. The two steps can be combined, and the transformation can be applied to the

original data to project it onto the CVA biplot plane. This gives an optimal separation of the group means in the data, with the biplot plane created by the group mean points. As the group means have many fewer points than the original data points (k vs n), this reduces the complexity of the SVD calculation required for large datasets.

3.4.1 Axis predictivity and variable importance

Similar to the PCA biplot, the accuracy of recovering both the group means and the within-group values can be calculated for a CVA biplot. Defining between-group predictivity requires a decomposition in terms of the PCA of the canonical means and the fitted values

$$\overline{\boldsymbol{X}}\boldsymbol{L} = \widehat{\overline{\boldsymbol{X}}}\boldsymbol{L} + (\overline{\boldsymbol{X}} - \widehat{\overline{\boldsymbol{X}}})\boldsymbol{L}, \text{ with } \widehat{\overline{\boldsymbol{X}}}\boldsymbol{M} = \overline{\boldsymbol{X}}\boldsymbol{M}\boldsymbol{J} \text{ and } \widehat{\overline{\boldsymbol{X}}} = \overline{\boldsymbol{X}}\boldsymbol{M}\boldsymbol{J}'\boldsymbol{J}\boldsymbol{M}^{-1}.$$

This is the equivalent in the PCA biplot for $r = 2$ of $\widehat{\boldsymbol{X}}_{[2]} = \boldsymbol{U}\boldsymbol{J}\boldsymbol{\Sigma}\boldsymbol{J}^{*}\boldsymbol{V}_2' = \boldsymbol{X}\boldsymbol{V}_2\boldsymbol{V}_2'$. The results are explained in Gower *et al*. (2011), with:

$$Overall\ quality = \frac{tr\left(\boldsymbol{L}'\widehat{\overline{\boldsymbol{X}}}'\boldsymbol{C}\widehat{\overline{\boldsymbol{X}}}\boldsymbol{L}\right)}{tr(\boldsymbol{L}'\overline{\boldsymbol{X}}'\boldsymbol{C}\overline{\boldsymbol{X}}\boldsymbol{L})} = \frac{tr(\boldsymbol{\Lambda}\boldsymbol{J})}{tr(\boldsymbol{\Lambda})},$$

where at most the first $m = min(k - 1, p)$ of the eigenvalues λ_i are nonzero.

Similar to PCA biplots, CVA biplots can be measured by their quality, axes and sample predictivity. A measure of the goodness of approximations of the class means can be defined as the total squared reconstruction error for means (TSREM):

$$TSREM = \boldsymbol{tr}\left\{\left(\overline{\boldsymbol{X}} - \widehat{\overline{\boldsymbol{X}}}\right)\left(\overline{\boldsymbol{X}} - \widehat{\overline{\boldsymbol{X}}}\right)'\right\}$$

A similar measure of the goodness of approximations of the samples can be defined as the total squared reconstruction error for samples (TSRES):

$$TSRES = \boldsymbol{tr}\left\{(\boldsymbol{X} - \widehat{\boldsymbol{X}})(\boldsymbol{X} - \widehat{\boldsymbol{X}})'\right\}$$

While the k group means are exactly represented in a $k - 1$* dimensional biplot, the samples are not exactly represented. Therefore $TSREM = \boldsymbol{0}$ and $TSRES > \boldsymbol{0}$.

It follows that the two-group means ($k = 2$) are optimally represented on a one-dimensional CVA biplot, irrespective of the number of variables p.[1]

3.4.2 Two-group CVA biplot

When $k = 2$, $Rank(\bar{\boldsymbol{X}}\boldsymbol{L}) = 1$. The first eigenvalue is then not equal to zero, and the remaining $p - 1$ eigenvalues are equal to zero. There is then no automatic way to optimally select the next eigenvector or linear combination of eigenvectors to include as the next spanning variable or scaffolding axis for a k-dimensional biplot (the second scaffolding axes needed to create a two-dimensional CVA biplot when there are two groups, or a third scaffolding axis needed for a three-dimensional biplot when there are two or three groups).

To find the second scaffolding axis, a linear combination of eigenvectors is selected such that it minimises the TSRES without sacrificing the exact predictions we already have for the class means in the first dimension.

The process for solving this optimisation problem is explained in Le Roux and Gardner-Lubbe (2024). Let

$$\boldsymbol{M} = [\boldsymbol{M}_1 : p \times (k-1) \quad \boldsymbol{M}_2 : p \times (p-k+1)], \text{ and}$$

$$\boldsymbol{M}^{-1} = \begin{bmatrix} \boldsymbol{M}^{(1)} : (k-1) \times p \\ \boldsymbol{M}^{(2)} : (p-k+1) \times p \end{bmatrix}.$$

Then

$$\boldsymbol{M}^{(2)}\boldsymbol{M}^{(2)\prime} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}', \text{ and}$$

$$\boldsymbol{M}_{opt} = [\boldsymbol{M}_1 \quad \boldsymbol{M}_2\boldsymbol{V}].$$

Therefore, it is possible to create a two-dimensional CVA biplot when $k = 2$, using the matrix $\boldsymbol{M}_{opt}\boldsymbol{J}^*$ to create the biplot axes and project the group means and samples onto the biplot plane.

The number of new eigenvectors, and therefore axes that can be created, is now increased to p, as the original eigenvector matrix $\boldsymbol{M}$ has p rows and columns. The additional $\boldsymbol{M}_j : j = k, k+1, \ldots, p$ scaffolding axes are added such that $\bar{\boldsymbol{X}}\boldsymbol{m}_j = \boldsymbol{0}$, with the TSRES minimised

[1] In the case of a $k - 2$ dimensional biplot, TSREM will also be greater than 0.

and without sacrificing the exact predictions we already have for the group means. The TSREM remains zero.

The additional scaffolding axis is optimal in the sense that no other scaffolding axis can be found that will improve TSRES while restricting TSREM to be zero.

3.5 OPTIMAL ROTATION OF THE BIPLOT

This section discusses the rotation of the projection matrix and the associated biplot data points and variable axes in such a way that the predicted value and original value of a target individual data point $\boldsymbol{x}_i$ is the same, i.e. to ensure that:

$$\widehat{\boldsymbol{x}}_i' = \boldsymbol{x}_i' \boldsymbol{V}_2 \boldsymbol{V}_2' = \boldsymbol{x}_i'$$

This is necessary because in general, projection and prediction of $\boldsymbol{x}_i$ are not equal to its original value. The rotation ensures that the biplot plane passes through the target data point and no information concerning the data point is lost due to the projection and prediction process.

Globally, the rotation results in a biplot that will give a less accurate approximation of the high dimensional data, but locally for the individual data point, the fit will be accurate. The new projection matrix will be indicated by $\boldsymbol{V}^{\text{new}}$ and the subsequent biplots generated by this matrix referred to as a 'rotated component biplot' (RC biplot).

The sample predictivity indicates the distances for each data point to the biplot plane. The greater the distance, the less accurately the biplot represents that data point. However, both the data points and the variable axes can be rotated in such a way that a specific data point lies exactly on the biplot plane. In this case, the projection of that point onto the plane preserves all information related to it.

The method for optimally rotating the data point onto the biplot plane is based on Procrustes analysis (Cox & Cox, 2008). This involves the optimal one-to-one transformation of points in one configuration to the points in another configuration. In this application, transformation focuses on a single data point $\boldsymbol{x}_{1\times p}$ in question, instead of the entire matrix $\boldsymbol{X}$: $n \times p$. The data point in the p-dimensional PCA space is *transformed* to be in the same position in the lower two-dimensional space as would be achieved through the *projection* onto the two-dimensional biplot space, as per the normal biplot calculation. Therefore, when dimension reduction occurs, no information is lost for the target data point, as the point is already on the biplot surface and will have perfect sample predictivity. This improves the local accuracy of the

representation of the target data point, at the cost of reduced overall fidelity in representing the dataset as a whole.

To calculate the rotation matrix, three vectors are constructed: $-\boldsymbol{x}_{p\times 1}$, the origin and $\boldsymbol{x}_{p\times 1}$. These three points can be represented in a matrix of three rows as a symmetric point around the origin:

$$\ddot{\boldsymbol{X}}_{3\times p} = \begin{bmatrix} -x_1 & \cdots & -x_p \\ 0 & \ddots & 0 \\ x_1 & \cdots & x_p \end{bmatrix}$$

Here $\boldsymbol{x}$ is already centred, and in the case of a PCA biplot, potentially scaled, with $\boldsymbol{V}$ the projection matrix calculated from the SVD of the original centred (and scaled) data matrix $\boldsymbol{X}_{n\times p}$. $\ddot{\boldsymbol{X}}_{3\times p}$ is then transformed into two matrices:

$$\boldsymbol{Y} = \ddot{\boldsymbol{X}}\boldsymbol{V} \text{ and } \boldsymbol{Z} = \ddot{\boldsymbol{X}}\boldsymbol{V}\boldsymbol{J}^* = \ddot{\boldsymbol{X}}\boldsymbol{V}_r.$$

$\boldsymbol{Y}$ remains in the p-dimensional PCA space and $\boldsymbol{Z}$ is reduced to the r-dimensional space used for the biplot.

If $p \neq r$, padding is used to ensure that the matrices are of the same size, e.g. $r = 1, p = 2$:

$$\boldsymbol{Z}_{3\times p} = \begin{bmatrix} -z_1 & 0 \\ 0 & 0 \\ z_1 & 0 \end{bmatrix}, \qquad \boldsymbol{Y}_{3\times p} = \begin{bmatrix} -y_1 & -y_2 \\ 0 & 0 \\ y_1 & y_2 \end{bmatrix}$$

The Procrustes analysis solution of the optimal rotation matrix $\boldsymbol{A}$ is calculated with

$$\boldsymbol{C} = \boldsymbol{Y}'\boldsymbol{Z},$$

the SVD of $\boldsymbol{C} = \boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{V}^{*\prime}$

and $\boldsymbol{A} = \boldsymbol{V}^*\boldsymbol{U}'$.

The optimal dilation ρ is calculated as:

$$\rho = \frac{tr(\boldsymbol{Z}\boldsymbol{A}\boldsymbol{Y}')}{tr(\boldsymbol{Z}'\boldsymbol{Z})}$$

The transformation ensures that the data point in the r-dimensional biplot space is equal to the value in the full PCA space. Therefore:

$$\ddot{\boldsymbol{X}}\boldsymbol{V} = \boldsymbol{Y} = \rho\boldsymbol{Z}\boldsymbol{A} = \rho\ddot{\boldsymbol{X}}\boldsymbol{V}\boldsymbol{J}^*\boldsymbol{A}, \text{ where } \boldsymbol{A}^{-1} = \boldsymbol{A}'.$$

Using the Procrustes method, a new projection matrix is created as $\boldsymbol{V}^{\text{new}} = \boldsymbol{V}\boldsymbol{A}'$. This rotation ensures that the biplot plane is rotated to align with the target data point, such that when the data is projected using the new matrix, the target point retains its original position, with

$$\ddot{\boldsymbol{x}}'\boldsymbol{V}\boldsymbol{A}' = \ddot{\boldsymbol{z}}'.$$

The dilation and inverse dilation $\left(\frac{1}{\rho}\right)$ is not required, as it would uniformly shift all data on the biplot plane. This adjustment would unnecessarily modify the biplot axes in an attempt to recover information lost in the original projection. This new projection matrix, and the RC biplot created with it, was used in the same way as discussed in the construction and evaluation of a PCA or CVA biplot. In the case of the CVA biplot, two components of the transformation matrix are rotated until it passes through the target data point.

Allowing the data to be predicted to the original space requires the new transposed prediction matrix $(\boldsymbol{V}^{\text{new}})' = \boldsymbol{A}\boldsymbol{V}'$ and the predicted point $\hat{\boldsymbol{x}}' = \boldsymbol{z}'\boldsymbol{A}\boldsymbol{V}'\boldsymbol{J}'$.

This is illustrated in Figure 3.3 with an example of five data points in two dimensions and their corresponding one-dimensional biplot. The code required to rotate the biplot can be found in Appendix A under A2: Optimal Rotation of a BiplotA2: Optimal Rotation of a Biplot.

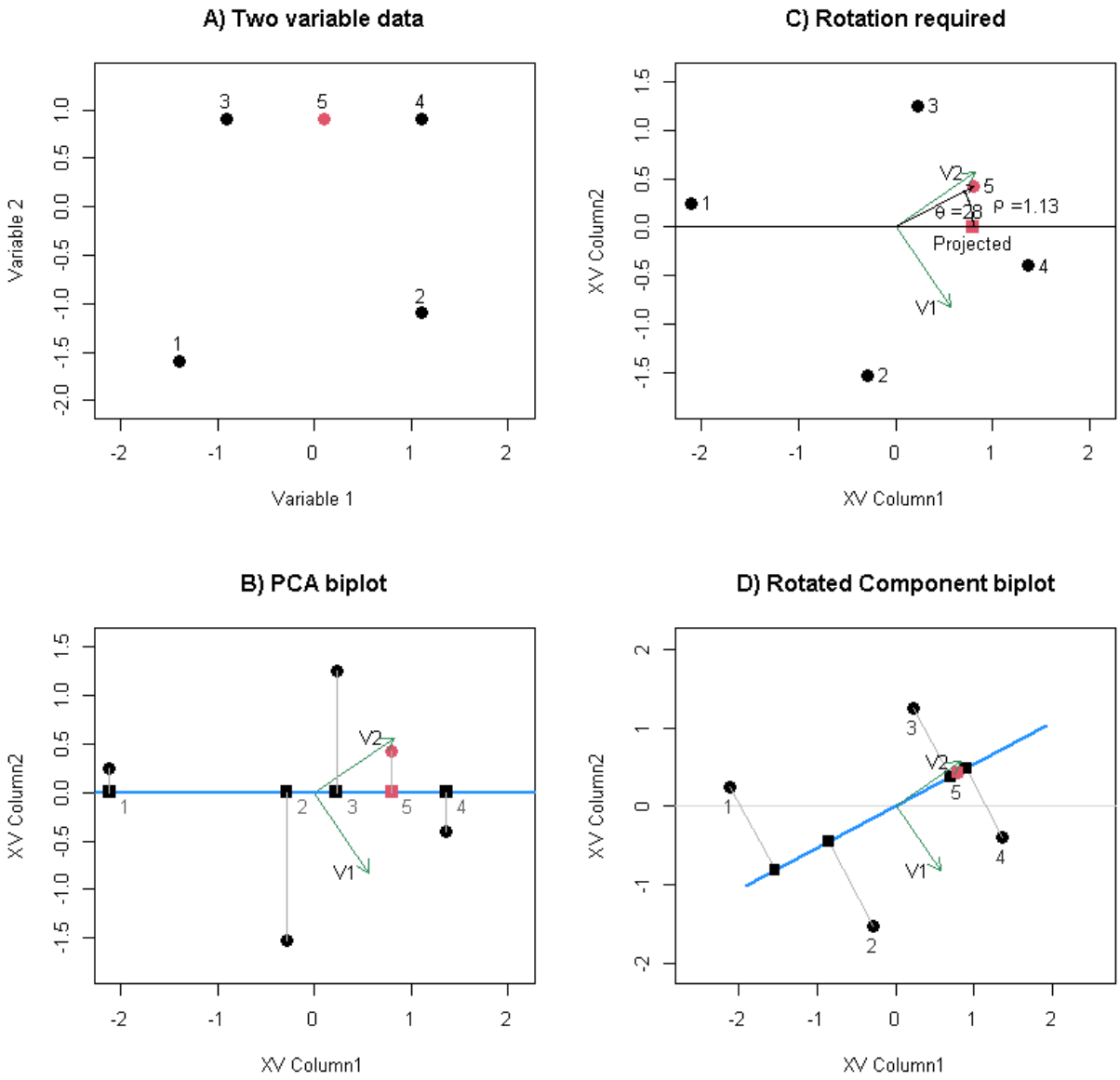


Figure 3.3: *The rotation and dilation required for the projected point to correspond to the data point in the full two-dimensional PCA dimension space, resulting in the rotated biplot plane passing through the data point*

Illustration A shows the original example data $\boldsymbol{X}$. Illustration B shows the PCA biplot $\boldsymbol{XV}$ in the original two dimensions, along with the projected value for a one-dimensional PCA biplot. The most variance is shown along the x-axis, which represents the first principal component, or the first column of $\boldsymbol{XV}$. The one-dimensional biplot projects the data onto the first principal component, causing information concerning the target data point number 5 to be lost.

Illustration C indicates the rotation required to ensure that data point number 5 in the full dimensional space (red circle) is optimally rotated to align with the one-dimensional projected value (red square). This will require a rotation of $28°$ and a dilation of $\rho = 1/1.13$.

Illustration D indicates the new RC biplot $\boldsymbol{XV}^{\text{new}}$, with $\boldsymbol{V}^{\text{new}} = \boldsymbol{VA}'$. Applying the dimension reduction $\boldsymbol{XV}^{\text{new}}\boldsymbol{J}^{*}$ will result in the data being projected onto the rotated one-dimensional biplot plane, indicated by the squared data points. In this case, the position of data point number 5 remains unchanged, as it already lies on this new one-dimensional plane. However, the relative positions of data points 1 to 4 have changed.

Variable vector number 2 is now almost parallel to the biplot plane, whereas variable vector number 1 is almost perpendicular. Therefore, the axis predictivity of variable number 2 improves to almost the unit value at the expense of variable number 1.

The sample predictivity for data point number 5 was 0.72. This increases to 1 in the rotated plot, indicating that no information is lost for this data point in the one-dimensional RC biplot. However, the sample predictivity of data points 1 and 4 reduced.

The overall accuracy of the estimates for all the data points decreases, and the sum of squares of the residuals increases when using the new $\boldsymbol{V}^{\text{new}}$, as calculated in Equation 3, with the sum of squares of the residuals equal to:

$$t\boldsymbol{r}(\boldsymbol{X}'\boldsymbol{X}) - \boldsymbol{tr}(\boldsymbol{X}'\boldsymbol{X}\boldsymbol{V}^{\text{new}}\boldsymbol{J}\boldsymbol{V}^{\text{new}'}).$$

The sum of squares of the residuals increases from 0.28 to 4, compared to the total sum of squares of 14.

3.5.1 Example of the rotated component biplot

The rotation correction resolves the inconsistency between the prediction of a target data point made by the model $\boldsymbol{f}(\boldsymbol{X})$ and the class implied by its projection onto the biplot plane. This issue is particularly evident when using a two-dimensional biplot to approximate a multi-dimensional decision surface. The concept is illustrated using the iris dataset in figure 3.4 and Figure 3.5 below for a PCA biplot.

In Figure 3.4, a three-dimensional PCA biplot was created using three flower features of the iris data. This full rank biplot provides an exact representation of the variable axes, data points and decision surface, with no dimensionality reduction. The two-dimensional biplot, however, reflects only the projection onto the first two principal components. The z-axis (third component), representing the direction of least variance, is omitted. When the dimensions are reduced to two, the data point is projected onto the surface, and the biplot plane becomes an approximation of the decision surface.

In this example, the green target point, which lies above the two-dimensional plane in the three-dimensional space, is projected orthogonally onto the biplot plane. While $f(\boldsymbol{X})$ correctly classifies the point into class 1 (the region in three-dimensional space shaded in red), its projection appears in the class 0 region (the region shaded in blue) of the two-dimensional biplot. This misalignment occurs because the biplot plane fails to capture the orientation of the decision surface near the point, which lies obliquely above the plane.

Three-dimensional and two-dimensional original PCA biplot

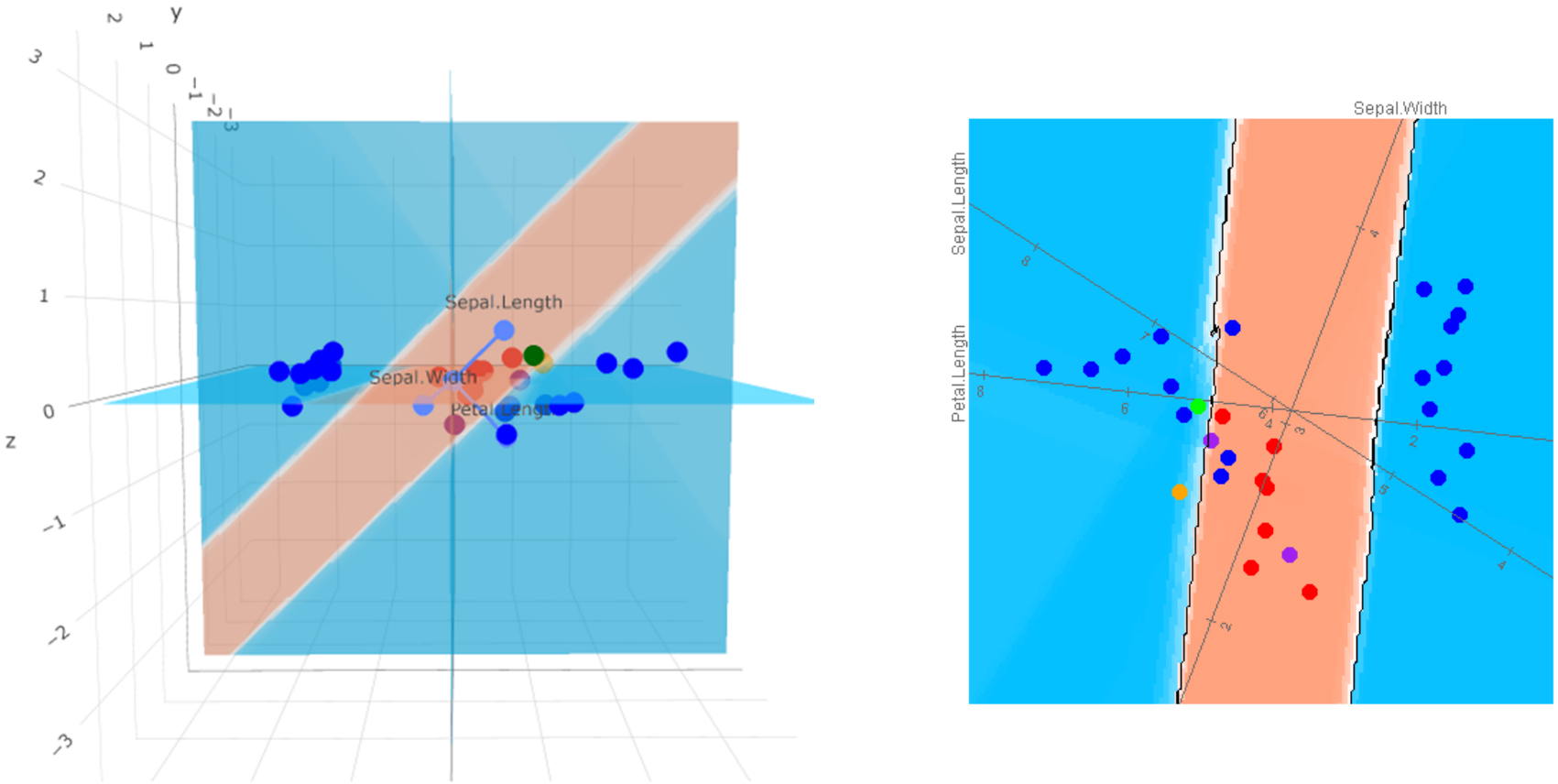


Figure 3.4: *A three-dimensional and two-dimensional biplot of the three-variable iris data illustrating the accuracy of the dimension reduction and projection of a data point onto the two-dimensional biplot plane*

To overcome this problem, the biplot is rotated to ensure that the data point in the full dimensional space is positioned on the biplot plane, and therefore that the decision surface is accurately approximated by the biplot plane in the vicinity of the target data point. This is illustrated in figure 3.5.

For the optimal rotation of the data point onto the biplot plane, the rotation matrix $\boldsymbol{A}$ is calculated for the target data point. This results in a new projection matrix $\boldsymbol{V}^{\text{new}} = \boldsymbol{VA}'$. When the target data point is interpolated onto the RC biplot surface and predicted back to the original p-dimensional space, no information is lost, with:

$$\widehat{\boldsymbol{x}}_i' = \boldsymbol{x}_i' \boldsymbol{V}_r^{\text{new}} (\boldsymbol{V}_r^{\text{new}})' = \boldsymbol{x}_i'$$

and the classification for both will be the same, with:

$$f(\widehat{\boldsymbol{x}}_i) = f(\boldsymbol{x_i}).$$

Therefore, the interpolated and projected values are identical, and the classification of the model is consistent with what is visualised on the biplot, with the red data point now in a red area of the biplot plane. The rotation ensures that the decision surface is accurately approximated at the location of the target point, making the counterfactual explanations and boundary visualisation locally valid.

Three-dimensional and two-dimensional rotated PCA biplot

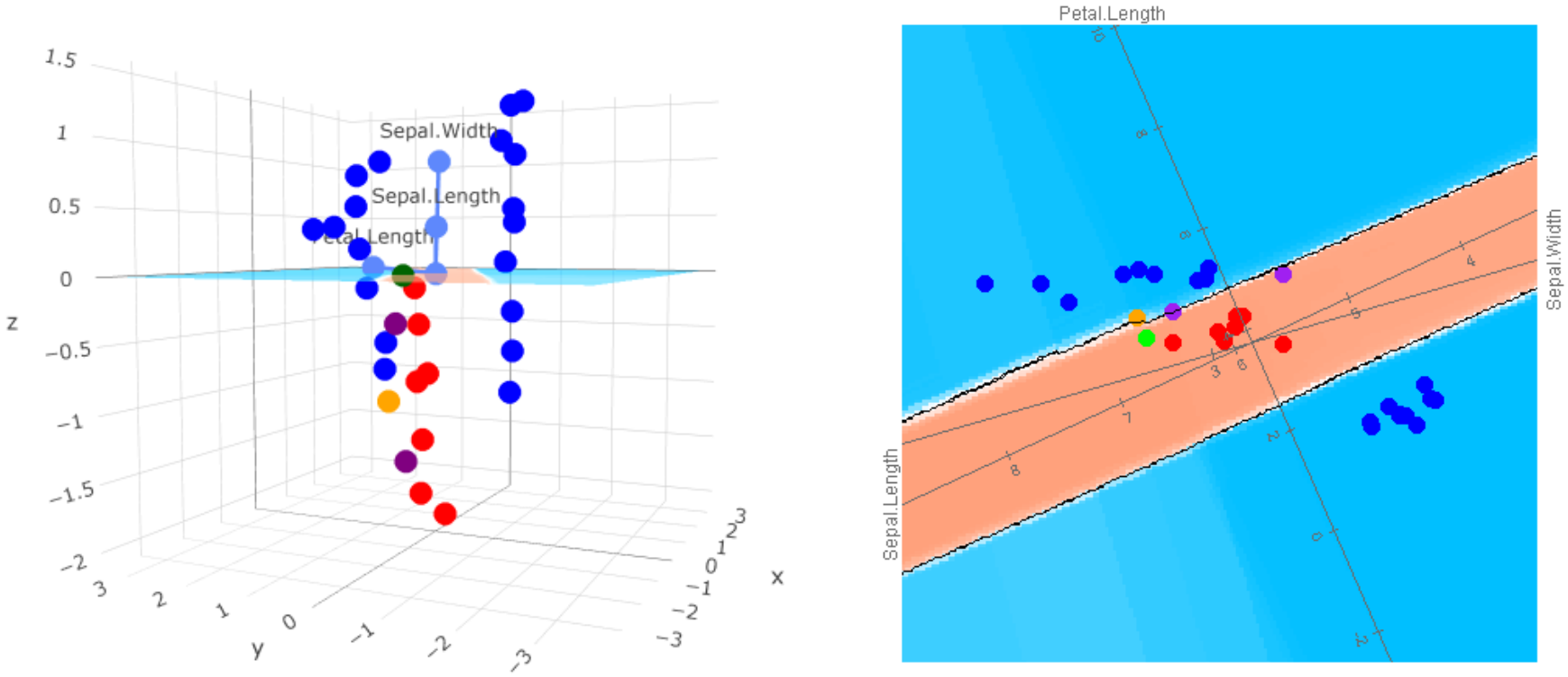


Figure 3.5: A three-dimensional and two-dimensional RC biplot of the three-variable iris data after rotating the principal components to ensure that the data point is on the plane, requiring for no projection onto the plane for the green data point

It is important to note that while the rotation resolves inconsistency for the target data point between the prediction of the model and the biplot-implied prediction, it does not guarantee that the nearest decision boundary lies on the rotated biplot plane. Therefore, although the rotated biplot gives a faithful local approximation, it may still omit closer boundaries outside the visualised plane. This limitation is especially relevant when multiple decision boundaries are present in the full multi-dimensional space.

3.5.2 Rotation of a CVA biplot

The optimal rotation matrix ensures that the biplot plane passes through the target data point and can be calculated in the same manner for the transformation matrix $\boldsymbol{M}$ and the two-group optimised transformation matrix $\boldsymbol{M}^{opt}$ as with projection matrix $\boldsymbol{V}$.

The new canonical transformation projection matrix is equal to $\boldsymbol{M}^{new} = \boldsymbol{M}^{opt}\boldsymbol{A}'$. The new prediction matrix used to create the RC biplot will be $(\boldsymbol{M}^{new})^{-} = \boldsymbol{A}(\boldsymbol{M}^{opt})^{-1}$.

This rotation ensures that the target data point is located on the biplot plane. However, it will result in a rotation of the group mean points away from the CVA biplot plane. The predictions of the class mean in the first $(k - 1)$ dimensions are not exact anymore.

In this section, the biplot space was rotated to ensure consistent classification of a specific data point to either of class 0 or 1. The next section explains the construction of local decision boundaries.

3.6 BINARY CLASSIFICATION DECISION BOUNDARIES

The goal of supervised ML is to create a prediction model $\widehat{\boldsymbol{Y}} = f(\boldsymbol{X})$, with the target variable denoted by $\boldsymbol{Y}$ and the input variables by $\boldsymbol{X}$**.** In the case of binary classification models, the values of $\boldsymbol{Y}$ will be either 0 or 1. The model $f(\boldsymbol{X})$ represents a p-dimensional decision surface spanned by the training input variables in $\boldsymbol{X}$. In the case of a linear ML model, $f(\boldsymbol{X})$ is a hyperplane.

The theoretical decision boundary value b is the set of data points on the decision surface $\boldsymbol{X}_{TDB}$ such that $f(\boldsymbol{X}_{TDB}) = b$. This value determines which class label $\hat{G} \in (0,1)$ the model assigns to the data, with

$$\hat{G} = 1 \text{ when } \hat{Y} \geq b \text{ and } \hat{G} = 0 \text{ when } \hat{Y} < b.$$

The decision boundary value b is selected to optimise the calibration of $f(\boldsymbol{X})$. The value will depend on the structure of $f(\boldsymbol{X})$ and the method used to determine the model parameters that minimise the selected performance metric or loss function.

One of the reasons why it is difficult to explain how $f(\boldsymbol{X})$ makes predictions is because the model does not always explicitly provide a way to understand the subregions of the decision surface and the shape of the decision boundary surrounding each region. The decision surface can create local minima and maxima in the subregions of $f(\boldsymbol{X})$, and the impact of a change in $\boldsymbol{X}$ is different depending on the current position of $\boldsymbol{X}$ relative to the decision surface. If the value of $\boldsymbol{X}_{TDB}$ such that $f(\boldsymbol{X}_{TDB}) = b$ cannot be determined analytically, then it must be determined numerically.

In traditional linear models, the decision surface is a hyperplane and $f(\boldsymbol{X})$ monotonically increases or decreases. The impact of each variable can be defined irrespective of the value of $\boldsymbol{X}$ and the combination of values $\boldsymbol{X}_{TDB}$ that will result in $\widehat{\boldsymbol{Y}} = b$ can be determined analytically. The decision boundary for a binomial GLM model using the logit transformation $log[p/(1-p)]$ is defined by the hyperplane $B_0 + \boldsymbol{B'x} = \boldsymbol{0}$, and the optimal decision boundary value is equal to $b = p = 0.5$.

3.6.1 Decision boundary intervals

When the values of $\boldsymbol{X}_{TDB}$ that represent $f(\boldsymbol{X}_{TDB}) = b$ cannot be found analytically, or do not exist, a weaker definition is used. The decision boundary is then defined as a 'decision boundary set' $\boldsymbol{X}_{DB}$, which contains counterfactual values on the other side of the theoretical decision boundary in an interval defined by ε. The decision boundary set values for $\boldsymbol{X}_{\hat{g}=0}$ and $\boldsymbol{X}_{\hat{g}=1}$ are the two subsets represented by:

$$\boldsymbol{X}_{DB}|_{\hat{g}=1} \text{ when } b \leq \boldsymbol{f}(\boldsymbol{X}) \leq b + \varepsilon \text{ and}$$

$$\boldsymbol{X}_{DB}|_{\hat{g}=0} \text{ when } b - \varepsilon \leq \boldsymbol{f}(\boldsymbol{X}) < b, \text{ where}$$

$$\boldsymbol{X}_{DB}|_{\hat{g}=1} \cup \boldsymbol{X}_{DB}|_{\hat{g}=0} = \boldsymbol{X}_{DB}.$$

The decision boundary set is represented by two contour lines on either side of the theoretical decision boundary in Figure 3.6, with the black line representing the theoretical decision boundary being estimated.

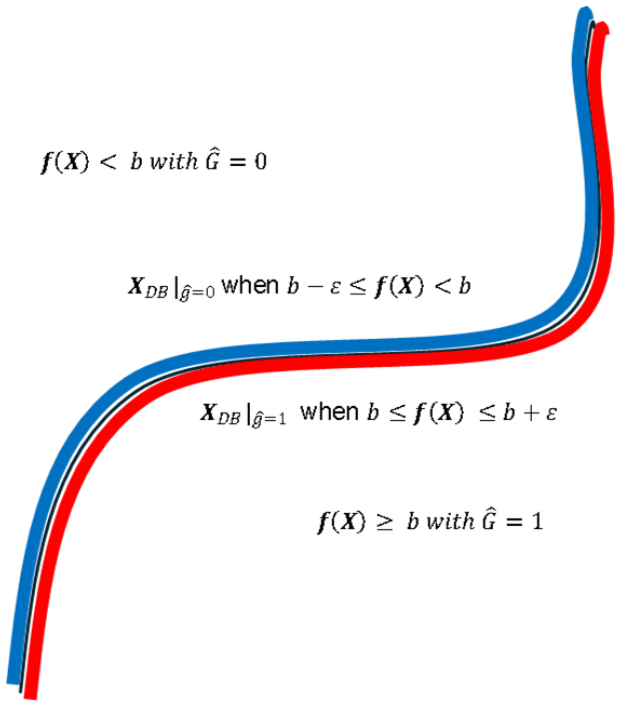


Figure 3.6: Visualisation of the decision boundary set $\boldsymbol{X_{DB}}$ with $\boldsymbol{X_{DB}|_{\hat{g}=0}}$ in blue and $\boldsymbol{X_{DB}|_{\hat{g}=1}}$ in red

The choice of ε will depend on the complexity of $f(\boldsymbol{X})$ and its decision boundary. By increasing the value of ε, this definition guarantees that the decision boundary set will contain a counterfactual on the other side of the decision boundary. However, larger values of ε will imply a less accurate estimation of the actual decision boundary and result in counterfactuals that are further away from the decision boundary than theoretically possible.

The reason for requiring different possible interval sizes ε can be visualised in the two illustrations in Figure 3.7 below.

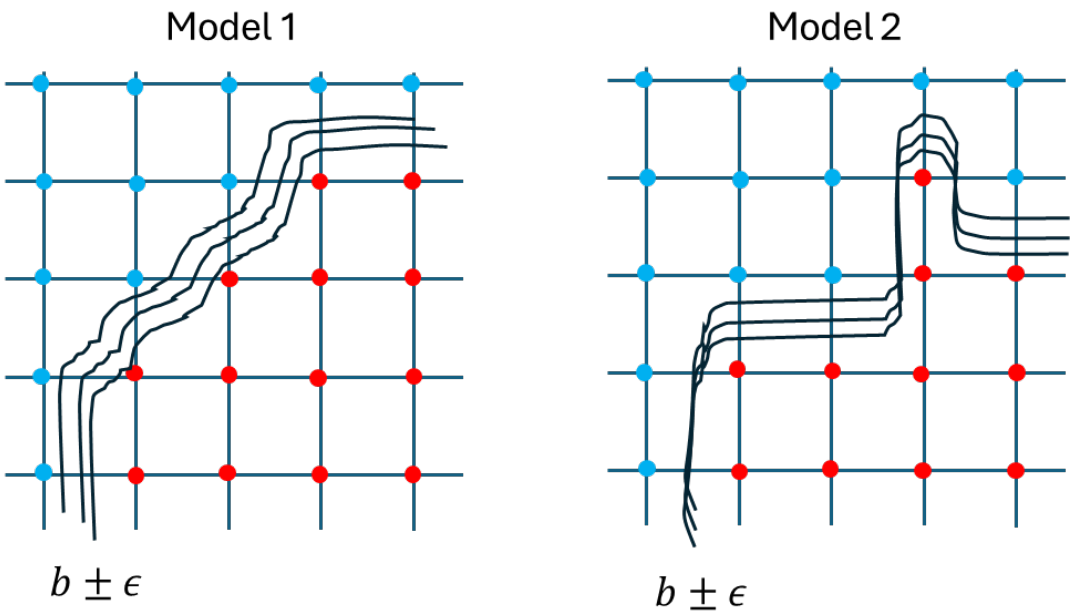


Figure 3.7: Comparing decision boundary subsets of different ML models and the resulting need for different values of ε necessary to uniquely identify local decision boundaries

The decision boundary set is the points that separate the blue and red, or class 0 and 1, grid points. The decision boundary for Model 2 has a more complex shape to accommodate the bigger variation in the predictions, with a faster increase or decrease in the prediction outcome away from boundary b. Therefore, the value of ε will have to increase to ensure that a unique decision counterfactual is contained within the interval on either side of the decision boundary in $\boldsymbol{X}_{DB}|_{\hat{g}=1}$ or $\boldsymbol{X}_{DB}|_{\hat{g}=0}$. These two subsets of data points are easier to find in two dimensions than in the p-dimensional space.

3.6.2 Counterfactuals and local decision boundaries

The methods investigated in the literature review focus on identifying counterfactuals, not the local decision boundary. The two concepts are related, but methods for identifying counterfactuals will not always identify the local decision boundaries.

Counterfactuals were defined in section 2.10 as follows:

- Let f be the prediction function calibrated on $\{\boldsymbol{x}, y\}$ with $\boldsymbol{x} \in \boldsymbol{R}^p$ the p-dimensional feature space and $y \in \{0,1\}$. For a data point $\boldsymbol{x}_i$ predicted to be in class $\hat{y} = f(\boldsymbol{x}_i) = 0$, create a new data point $\boldsymbol{x}_{cf}$ such that $\hat{y} = f(\boldsymbol{x}_{cf}) = 1$.

- $\boldsymbol{x}_{cf} = \boldsymbol{x}_i + \boldsymbol{\delta} \in \boldsymbol{R}^p$ must be the closest possible point to $\boldsymbol{x}_0$ given some distance metric, with the vector $\boldsymbol{\delta}$ the change/perturbances required to cause a change in the prediction of the target data point.

This can be refined to ensure that $\boldsymbol{x}_{cf}$ must be an actual data point within the test or training dataset, not just a simulated one.

The local decision boundary $\boldsymbol{X}^i_{LDB}$ is then defined as the subset of $\boldsymbol{X}_{DB}|_{\hat{g}=0}$ or $\boldsymbol{X}_{DB}|_{\hat{g}=1}$ that contains the counterfactual $\boldsymbol{x}_{cf}$ of $\boldsymbol{x}_i$, with $\boldsymbol{X}^i_{LDB} \subset \boldsymbol{X}_{DB}$. Instead of specifying the decision boundary, the set of local decision boundary points $\boldsymbol{X}^i_{LDB}$ and the counterfactual $\boldsymbol{x}_{cf} \in \boldsymbol{X}^i_{LDB}$ will be used to answer questions concerning the targeted data point $\boldsymbol{x}_i$. Therefore, the goal moves from finding $\boldsymbol{X}_{TDB}$ such that $f(\boldsymbol{X}_{TDB}) = b$ to finding $\boldsymbol{X}_{LDB}$ such that $f(\boldsymbol{X}_{LDB}) \in [b \pm \varepsilon]$. Then within $\boldsymbol{X}^i_{LDB}$ we find the estimated counterfactual $\boldsymbol{x}_{cf}$.

To simplify the discussion, the example in figure 3.8 illustrates a two-dimensional case where $\boldsymbol{X}: n \times 2$. Here, the decision boundary subset $\boldsymbol{X}_{DB}|_{\hat{g}=0}$ consists of two potential local decision boundaries, represented by the blue lines $\boldsymbol{X}^i_{LDB}$ and $\boldsymbol{X}^i_{LDB*}$. The counterfactual $\boldsymbol{x}_{cf}$ is identified as the nearest element in $\boldsymbol{X}_{DB}|_{\hat{g}=0}$ and the subset $\boldsymbol{X}^i_{LDB}$ is the local decision boundary that contains $\boldsymbol{x}_{cf}$. Another potential counterfactual, $\boldsymbol{x}^*_{cf}$, lies within the alternative subset $\boldsymbol{X}^i_{LDB*}$, but it is not used, as the Euclidean distance from the original data point $\boldsymbol{x}_i$ to $\boldsymbol{x}^*_{cf}$ is greater than to $\boldsymbol{x}_{cf}$. The length of the vector $\boldsymbol{\delta}$ represents the distance between the $\boldsymbol{x}_i$ and $\boldsymbol{x}_{cf}$.

Although $\boldsymbol{x}_{cf}$ is indicated as satisfying $f(\boldsymbol{x}_{cf}) = b - \varepsilon$, it may take any value within the interval $[b - \varepsilon, b)$. The specific value selected depends on the shape of the local decision boundary and the method used to identify it.

There are elements in $\boldsymbol{X}^i_{LDB}$ that are further away from $\boldsymbol{x}_{\mathrm{i}}$ than the elements that form part of the circular local decision boundary $\boldsymbol{X}^i_{LDB*}$. However, all the elements of $\boldsymbol{X}^i_{LDB}$ form part of the local decision boundary $\boldsymbol{x}_{\mathrm{i}}$.

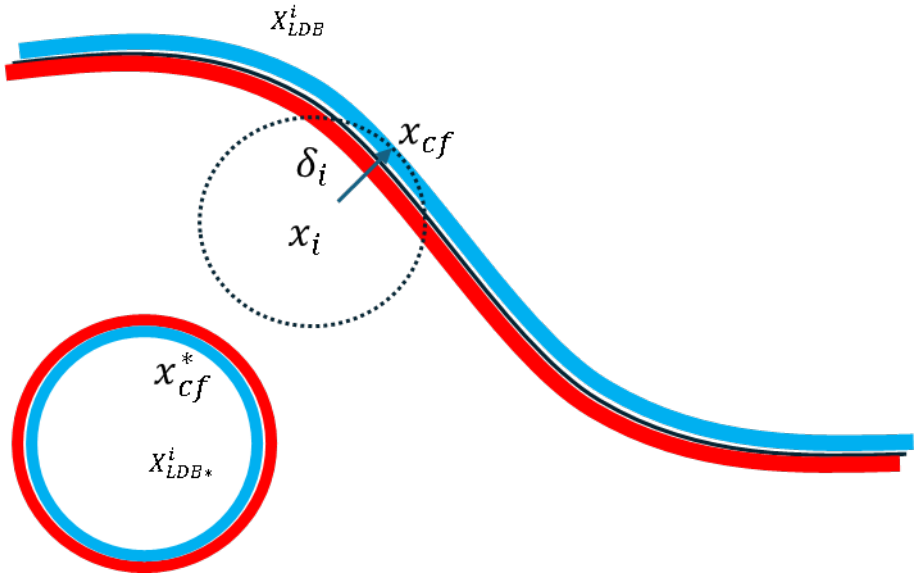


Figure 3.8: Identifying the nearest counterfactual $\boldsymbol{x_{cf}}$ for $\boldsymbol{x_i}$ and the local decision boundary $\boldsymbol{X^i_{LDB}}$ that contains $\boldsymbol{x_{cf}}$

Overall, there are four subsets of local decision boundaries, caused by two local minima in the data. Each subset occurs in a pair, representing prediction class 0 or 1. This is visualised in Figure 3.9.

Data points $\boldsymbol{x}_i$, $\boldsymbol{x}_j$ and $\boldsymbol{x}_k$ are all three classified by the model as $\hat{G} = 1$. The biplot visualises the two subsets of local decision boundaries $\boldsymbol{X}^i_{LDB}$ and $\boldsymbol{X}^{kj}_{LDB}$ that represent $\hat{G} = 0$.

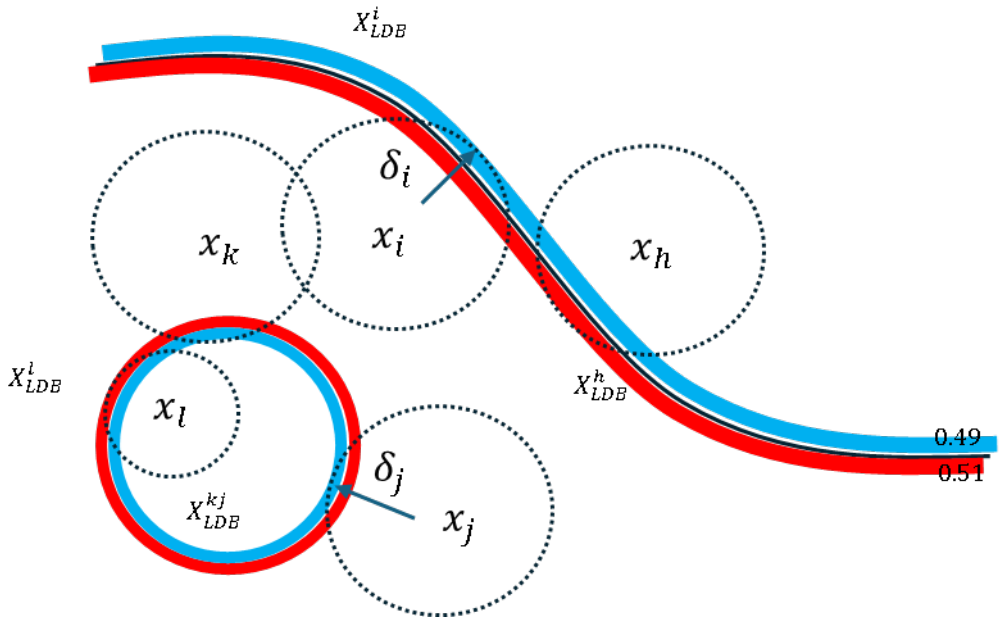


Figure 3.9: Identifying the local decision boundary $\boldsymbol{X^i_{LDB}}$ for data point $\boldsymbol{x_i}$ and the common local decision boundary $\boldsymbol{X^{kj}_{LDB}}$ for data points $\boldsymbol{x_k}$ and $\boldsymbol{x_j}$, where $\boldsymbol{\varepsilon = 0.01}$

The nearest counterfactuals for both $\boldsymbol{x}_j$ and $\boldsymbol{x}_k$ are both elements of $\boldsymbol{X}_{LDB}^{kj}$. Their counterfactuals are different, but they share the same local decision boundary. Their common local decision boundary allows the biplot to cluster $\boldsymbol{x}_j$ and $\boldsymbol{x}_l$ together separately from $\boldsymbol{x}_i$, even though $\boldsymbol{x}_i$ and $\boldsymbol{x}_k$ are closer to each other than to $\boldsymbol{x}_j$.

Similarly, $\boldsymbol{x}_l$ and $\boldsymbol{x}_h$ are both classified in class 0, but have different local decision boundaries. These boundaries form the pair to $\boldsymbol{X}_{LDB}^{kj}$ and $\boldsymbol{X}_{LDB}^{i}$ but represent $\hat{G} = 1$.

The example above assumes that the data contains two variable features. If the data includes more than two features, the counterfactuals and local decision boundaries will be estimated through reference to their estimated two-dimensional values $\boldsymbol{Z}$ on the biplot plane.

The code required to calculate the local decision boundary and counterfactual can be found in Appendix A under A3: Biplot Boundary Search.

3.7 CURRENT METHODS FOR FINDING DECISION BOUNDARIES

This section discusses three general methods of finding the decision boundary, before proceeding to the proposed method for finding the decision boundary and counterfactuals using biplots.

3.7.1 Analytical solution

The nearest boundary value for any sample point can be found analytically for a linear model as the orthogonal projection of the data point onto the hyperplane representing the decision boundary.

The normal vector $\boldsymbol{V}$ represents the hyperplane at the selected decision boundary b. Therefore, the nearest decision boundary value will be calculated as the projection of vector $\boldsymbol{a}$ (see Figure 3.10B) onto vector $\boldsymbol{v}$ and is solved with the following equation:

$$proj_{\boldsymbol{v}}\boldsymbol{a} = \frac{\boldsymbol{a}'\boldsymbol{v}}{\boldsymbol{v}'\boldsymbol{v}}\boldsymbol{v}.$$

In the case of a logistic regression model with two variables, the model is defined by the following equation:

$$\log\left(\frac{p}{1-p}\right) = b_0 + b_1x_1 + b_2x_2.$$

This can be illustrated in the two graphs in Figure 3.10.

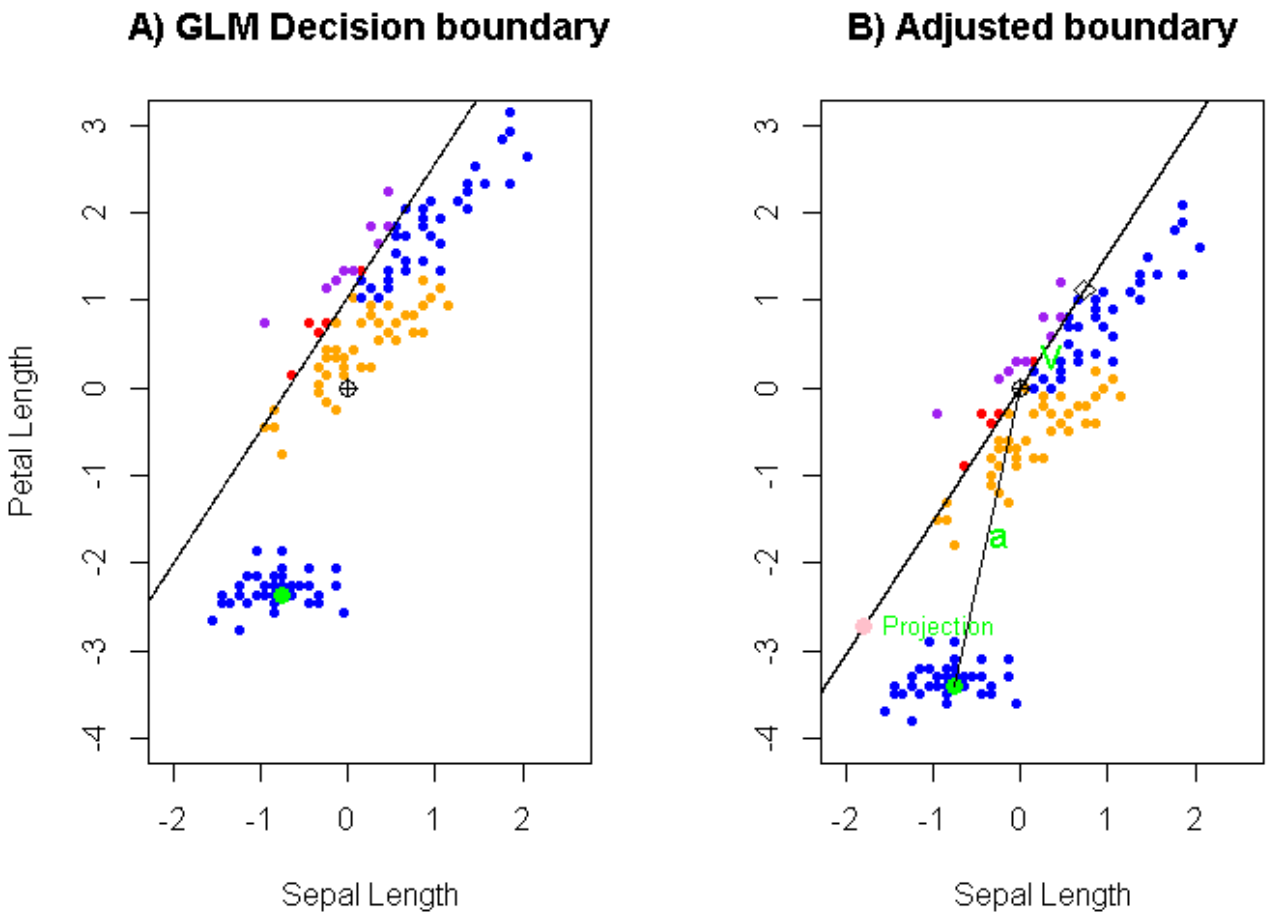


Figure 3.10: The data point vector, represented by $\boldsymbol{a}$, is projected onto the decision boundary to calculate the counterfactual value. Graph A indicates the original two-variable data. Graph B indicates the adjusted boundary to ensure that the boundary passes through the origin.

A binomial GLM model is fitted to the centred iris dataset using petal length and sepal length as input variables and the flower versicolor set as the target class 1. The data points that are correctly classified are coloured in blue (class 0) and red (class 1). Data points that are incorrectly classified as class 1 are coloured in yellow. Data points that are incorrectly incorrect classified as class 1 are coloured in purple. The decision boundary is indicated by the black line, with the coefficients of the model equal to:

$$Intercept: -0.76$$

$$Sepal\ length\ x_1: -1.12$$

$$Petal\ length\ x_2: 0.74.$$

The original data and classification results are shown in Figure 3.10A. However, the decision boundary does not pass through the origin. Therefore, the data and decision boundary should be shifted by deducting $\left(-\frac{b_0}{b_2}\right)$ from x_2, or downward by:

$$\frac{0.76}{0.74} = 1.04.$$

The projection of data point $a^*: (x_1 = -0.94;\ x_2 = -2.36)$ onto $x_2 = -\frac{-0.76}{0.74} - \frac{-1.12}{0.74}x_1$ is then solved with the adjusted $a: (x_1 = -0.94;\ x_2 = -3.40)$ and hyperplane $x_2 = -\frac{-1.12}{0.74}x_1$, with normal vector $\boldsymbol{v}: (-0.74;\ -1.12)$:

$$\boldsymbol{v}'\boldsymbol{v} = 1.81$$

$$\boldsymbol{a}'\boldsymbol{v} = (-0.94)(-0.74) + (-3.4)(-1.12) = -4.52$$

and

$$proj_{\boldsymbol{a}}v = -\frac{4.52}{1.81} * (-0.74;\ -1.12) = (-1.84;\ -2.81).$$

In terms of the origin, the decision boundary value is $(-1.84;\ -1.77)$.

This approach can be extended to the multivariate case and applied to GAMs, which are based on linear fits to an extended set of basis functions. The orthogonal projection onto the high-dimensional hyperplane defined by these extended basis vectors remains unique.

This method yields a unique, consistent solution that is computationally efficient. A diverse set of counterfactuals can be generated by projecting multiple data points onto the hyperplane. As the hyperplane is defined for the entire dataset, each resulting projection can serve as a potential counterfactual.

However, the resulting counterfactuals may not always be actionable, plausible or feasible, particularly if the projected point lies far from the observed data distribution. In such cases, the counterfactual may represent an outlier, reducing its interpretive usefulness. As illustrated in the example above, the projection point is distant from any of the training data points. This can be resolved by selecting the nearest justified solution to the projected counterfactual.

3.7.2 Heuristic search

An analytical solution is seldom available to find $\widehat{\boldsymbol{X}}_{DB}$ such that $\boldsymbol{f}(\widehat{\boldsymbol{X}}_{DB}) = 0.5$ for ML models $f(\boldsymbol{X})$. An alternative method is to use a suitable numerical optimisation method.

Optimisation methods such as Newton-Raphson are not suitable for problems where $f(\boldsymbol{X})$ create local minima and maxima that prevent the solution for $f(\widehat{\boldsymbol{X}}_{DB}) = 0.5$ to be found. Some approaches require that $f(\boldsymbol{X})$ be differentiable. Where these are not possible, heuristic optimisation solutions such as genetic algorithms can be used to estimate the decision boundary (Sharma *et al*., 2019). The benefit of a heuristic search is that it can be model-agnostic and allow for different data types or data ranges.

Heuristic methods do not guarantee that the decision boundary identified corresponds to a plausible value, as the generated solution often does not account for the probability of its occurrence under the multivariate distribution of the data. Incorporating this probability would require an accurate estimate of the underlying distribution, which is not always readily available.

Feasible and actionable counterfactuals can be generated by specifying which variables are allowed to change, as well as the permissible range for each variable during the optimisation process. However, this often comes at a computational cost. Solving for each local decision boundary can be time-consuming, particularly when the search space is large.

Moreover, the outcomes of these heuristic algorithms can vary between runs, unless the search range is tightly constrained. In some instances, the algorithm may fail to converge, resulting in no counterfactual being produced.

3.7.3 Grid search

An alternative solution is a grid search approach. In this case, all possible combinations of input variables are created and assessed in $f(\boldsymbol{X})$. If the grid is fine enough, then this approach ensures that all local minima are tested, and only optimal solutions are found.

However, continuous variables contain an infinite set of possible values. Therefore, a fixed grid search can only test specific points within the range of the continuous variables. The resulting combination of variables represented in the dataset $\widehat{\boldsymbol{X}}_{GS}$ can never test all possible combinations of input variables and may never include values for which $f(\widehat{\boldsymbol{X}}_{GS}) = 0.5$. The number of data points grows by $\prod_p m_i$, with m_i the number of grid points for each of the p variables. Therefore, the grid search must decide on the graduality of the search to balance

the resulting computational requirements against the need to find values for which $f(\widehat{\boldsymbol{X}}_{GS})\sim 0.5$.

Grid search methods can produce solutions that are highly improbable under the multivariate distribution of the training data. If the generated solutions do not reflect the underlying data distribution, the resulting decision boundaries will not represent plausible values. Furthermore, a grid search that explores implausible regions of the input space is computationally inefficient. When the multivariate distribution is unknown or poorly approximated, it becomes difficult to determine which grid search values are realistic.

These issues can be addressed by developing an efficient and representative multivariate sampling approach. The proposed solution is to employ either PCA or CVA biplots as a dimensional reduction technique. When the selected samples are projected back into the original feature space, they preserve the correlation structure of the original data, thereby improving plausibility while maintaining computational efficiency.

The main benefit of using biplots as the basis for the dimension reduction and grid search is their ability to provide a linear inverse projection function that allows the multi-dimensional variable values to be extracted from the two-dimensional surface. This allows the user to visualise the decision boundary on the two-dimensional surface of the biplot. In addition, the distance between the data points and the decision boundary correspond to the distances to the actual decision boundary in the original feature space, ensuring that the visualisation gives an intuitive and accurate interpretation of the data and ML model.

3.8 CALCULATING COUNTERFACTUALS USING A BIPLOT

This section proposes a novel approach for generating counterfactuals $\widehat{\boldsymbol{x}}_{cf}$ for the model $f(\boldsymbol{X})$. The biplot plane provides a two-dimensional approximation of the decision surface of $f(\boldsymbol{X})$ using two principal component axes. This enables the estimation of local decision boundaries and counterfactual values on the biplot plane, as described in section 3.6.

The PCA biplot is created with the projection matrix $\boldsymbol{V}_2$ and prediction matrix $\boldsymbol{V}_2'$. The equivalent CVA biplot uses the canonical transformation matrix $\boldsymbol{M}_2$ and prediction matrix $\boldsymbol{M}_2^-$. The process for creating a PCA biplot is visualised in Figure 3.11 below.

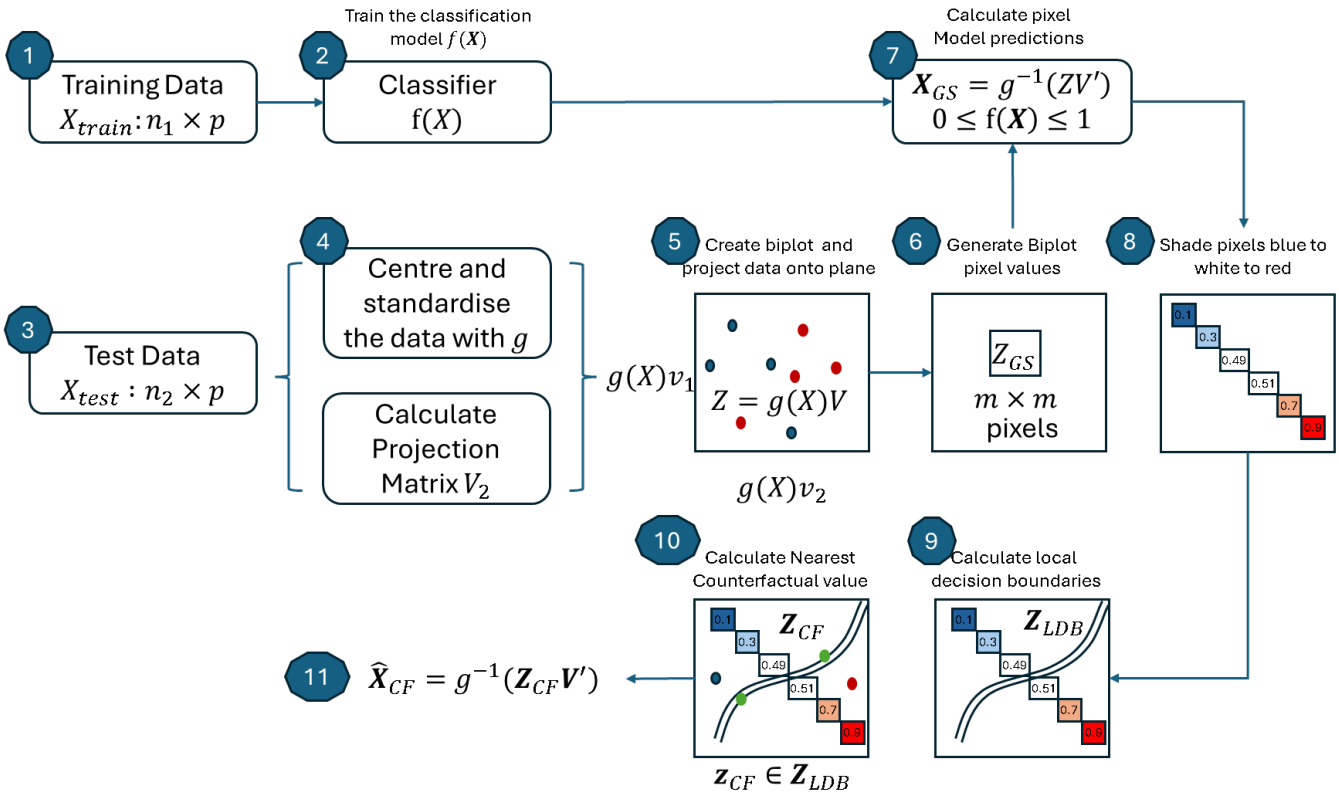


Figure 3.11: Process map of generating counterfactuals using a PCA biplot for the model $\boldsymbol{f(X)}$

1) The method uses the unstandardised training data $\boldsymbol{X}_{train}$, where $\boldsymbol{X}$: $n_1 \times p$, with n_1 the number of rows in the training data and p the number of explanatory variables to train a two-class classification model $f(\boldsymbol{X})$.

2) This method provides an interpretation for $f(\boldsymbol{X})$. In the case of a multi-class classification problem, the results need to be adjusted to two classes by specifying the target class as 1 and all the other classes as 0. The model output must be calibrated to be between 0 and 1.

3) The data being investigated and that requires interpretation is referred to as the 'unstandardised testing data' $\boldsymbol{X}_{test}$: $n_2 \times p$, with n_2 the number of rows in the test data. $\boldsymbol{X}_{test}$ can be the same dataset as $\boldsymbol{X}_{train}$, separate dataset sampled from the same population, or an individual data point requiring a local interpretation.

4) To construct the PCA biplot, the data must first be centred, and preferably standardised, before calculating the SVD of $\boldsymbol{X}$ resulting in the projection matrix $\boldsymbol{V}_2$. Centring vector $\boldsymbol{x}_{mean}$ and standard deviation vector $\boldsymbol{x}_{std}$ are recorded and used to define the standardisation function $g(\boldsymbol{X})$, where:

$$g(\boldsymbol{x}_i) = \frac{(\boldsymbol{x}_i - \boldsymbol{x}_{mean})}{\boldsymbol{x}_{std}}.$$

If the data is not standardised, or in the case of a CVA biplot, $\boldsymbol{x}_{std}: p \times 1 = \mathbf{1}$.

Ideally, the training data should be used to compute $\boldsymbol{V}_2$ or $\boldsymbol{M}_2$ and g. If $\boldsymbol{X}_{test}$ is representative of the distribution of $\boldsymbol{X}_{train}$ or if $\boldsymbol{X}_{train}$ is not accessible, then $\boldsymbol{X}_{test}$ can be used to calculate $\boldsymbol{V}_2$ or $\boldsymbol{M}_2$ and g instead.

For the CVA biplot, the equivalent process explained in section 3.4 is followed to create the transformation matrix $\boldsymbol{M}_2$ instead of $\boldsymbol{V}_2$. The two-dimensional CVA biplot requires the data to be grouped into three or more groups to allow the creation of the canonical transformation matrix $\boldsymbol{M}$. If the target value consists of three or more classes, these can be used for grouping the data. For two-class data, $f(\boldsymbol{X})$ can determine the groups.

If the actual class of the data is also known, the data points can be classified into four groups to reflect the true positive, true negative, false positive and false negative prediction outcomes.

If the class of the data point in the test data is not known, then only the positive or negative prediction class will be available. In this case, the optimised canonical transformation matrix $\boldsymbol{M}_{opt}$ will be used to generate the two-dimensional biplot.

The projection may also be rotated to ensure that the biplot plane passes through a specific target data point. This will result in the updated projection matrix $\boldsymbol{V}^{\text{new}}$ or $\boldsymbol{M}^{\text{new}}$. This improves the accuracy of the local interpretation of the data point. In this case, the biplot will be referred to as the RC biplot.

5) The biplot x- and y-scaffolding axes are constructed, and the centred (and, if applicable, standardised) test data points in $\boldsymbol{X}_{test}$ are projected onto the biplot plane as $\boldsymbol{Z} = g(\boldsymbol{X})\boldsymbol{V}_2$.

   A 1:1 aspect ratio is applied to the x- and y-axes to preserve the relative distances between data points, ensuring that the spatial relationships in the biplot are not distorted.

   Calibrated variable axes are plotted through the origin. These allow approximate variable values to be read directly from the biplot by projecting the data points onto the variable axes. The projected data points are coloured according to their class – either red (class 1) or blue (class 0). If this is not known, they can be coloured in black.

The distribution of the data on the biplot plane may indicate outliers as data far from the centre of the biplot. It can also show potential clustering of the data that indicate artefacts in the data that can be removed. This information can be used to update the training data used and potentially refit the model $f(\boldsymbol{X})$. In the case of the CVA biplot, this will result in an updated classification of the data and grouping, resulting in a new transformation matrix $\boldsymbol{M}$.

6) A grid search of the biplot plane generates the m^2 pixel values represented by $\boldsymbol{Z}_{GS}$. It is sensible to limit the grid to include only pixels within the convex hull of the data points on the biplot plane. This reduces the number of grid points that needs to be predicted by the model in Step (7).

7) The grid points are mapped back to the original p-dimensional space using the inverse projection matrix $\boldsymbol{V}_2'$. After this projection, the inverse standardisation function g^{-1} is applied to convert the points to their original scale, resulting in the matrix $\boldsymbol{X}_{GS} = \boldsymbol{Z}_{GS}\boldsymbol{V}_2'$. These values are then used as input into the prediction model. The model outputs $f(\boldsymbol{X}_{GS})$ will range from 0 to 0.5 to 1.

8) These predictions are used to generate a dense map on the biplot plane by shading the corresponding pixels in $\boldsymbol{Z}_{GS}$, with a colour gradient from blue (class 0) to white (boundary) to red (class 1). If the decision boundary threshold b is set to 0.5, the white regions on the biplot will represent the estimated location of the decision boundary. The dense map illustrates the local decision surface of the ML model and helps visualise how the model makes predictions and the impact of the variables on the decision outcome.

9) The contour lines on the biplot plane that connect the pixel value in $\boldsymbol{Z}_{GS}$ corresponding to decision boundary values $b \pm \varepsilon$ define the local decision boundaries $\boldsymbol{Z}_{LDB}$. Multiple sets of local decision boundaries may emerge, depending on the number of local minima and maxima represented on the biplot.

   As the complexity of the decision surface increases, it becomes more challenging to identify separate contour lines for $b \pm \varepsilon$. In such cases, a larger tolerance value ε is required to generate meaningful contour lines that approximate the local decision boundary value for $\boldsymbol{X}_{LDB}^{i}$ for the subsets representing $\boldsymbol{X}_{DB}|_{\hat{g}=0}$ and $\boldsymbol{X}_{DB}|_{\hat{g}=1}$.

   The creation of the contour lines was explained in section 3.6.

10) This step consists of three parts. First, the model $f(\boldsymbol{X})$ determines the colour of the data points. If the actual class of the data is also known, then the data points can be classified into four results using four corresponding colour classifications:

True negative = blue	False positive = purple
False negative = yellow	True positive = red

If the correct class of the data point is not known, the data will be coloured either blue when they are predicted to be in class 0 or red when in class 1. Second, the test data points are then added back to the biplot created in steps 5 to 9. Lastly, the counterfactual value $\boldsymbol{z}_{cf}$ for the data point $\boldsymbol{z}_i$ is identified by search for the element of the local decision boundaries $\boldsymbol{Z}_{LDB}$ that represents the opposite prediction class and is the shortest Euclidean distance on the biplot plane from $\boldsymbol{z}_i$. The specific subset of $\boldsymbol{Z}_{LDB}$ that contains $\boldsymbol{z}_{cf}$ is referred to as the 'local decision boundary' of $\boldsymbol{z}_i$, and is denoted as $\boldsymbol{Z}_{LDB}^{i}$.

By calculating this distance directly on the two-dimensional biplot, the direction and proximity between the original data point and its counterfactual can be intuitively visualised and interpreted. The counterfactual value is then added to the biplot and coloured in green.

11) The counterfactual point is estimated as $\hat{\boldsymbol{x}}_{cf} = g^{-1}(\boldsymbol{z}_{cf}\boldsymbol{V}_2')$ by converting the two-dimensional counterfactual back into the original p-dimensions. Similarly, the complete set of local decision boundary points is recovered using $\hat{\boldsymbol{X}}_{LDB} = g^{-1}(\boldsymbol{Z}_{LDB}\boldsymbol{V}_2')$. These can then be used to interpret the ML model $f(\boldsymbol{X})$.

The code required to calculate the biplot and the counterfactual values can be found in Appendix A under A4: Biplot Search of local decision boundaries. The process of creating the local decision boundary and counterfactuals using a PCA biplot is explained in more detail below.

3.8.1 Visualising the machine learning model $f(X)$ on the biplot plane

An example of the process of creating a PCA biplot is explained below. It follows the same steps described above. The goal is to generate a local interpretation for a target data point and to compare it to the interpretation provided by the coefficients of the fitted GLM model.

- The example is based on the 150 entries in the iris dataset (Fisher, 1936). The example only includes three of the four flower features. They are sepal length, sepal width and petal length. The training and test data are split 80/20. This results in a training dataset $\boldsymbol{X}_{train}$, where $\boldsymbol{X}$: 120×3.
- A binomial GLM model $f_{glm}(\boldsymbol{X})$ with a logit link function is fitted to the training data. As there are three flower classes, the flower type versicolor is selected as the target variable or class 1. The other flower types are classified as class 0.
- The testing dataset $\boldsymbol{X}_{test}$ is the remaining data, with $\boldsymbol{X}$: 30×3. This dataset is visualised on the biplot.
- As the test data consists of only 30 entries, the training data $\boldsymbol{X}_{train}$ will be used to estimate the projection matrix $\boldsymbol{V}_2$, the centring vector $\boldsymbol{x}_{mean}$ and the standard deviation vector $\boldsymbol{x}_{std}$. The vector $\boldsymbol{x}_{mean}$ and vector $\boldsymbol{x}_{std}$ are recorded and used to define the standardisation function g.
- $\boldsymbol{X}_{train}$ is centred and standardised before calculating the SVD of $\boldsymbol{X}$, resulting in the projection matrix $\boldsymbol{V}_2$.
- As the goal is to find a local interpretation for an individual data point, the biplot is rotated to ensure that the target point is on the biplot plane. This results in a new projection matrix $\boldsymbol{V}^{new}$ and $\boldsymbol{V}_2^{new}$.
- The new projection matrix is used to create the biplot x- and y-scaffolding axes and the rotated biplot plane, along with the calibrated variable axes. The test data points in $\boldsymbol{X}_{test}$ are projected onto the biplot plane as $\boldsymbol{Z} = g(\boldsymbol{X})\boldsymbol{V}_2^{\text{new}}$, with the data points coloured based on their class – red for class 1 and blue for class 0 and the target data point highlighted in green. This is shown in Figure 3.12.

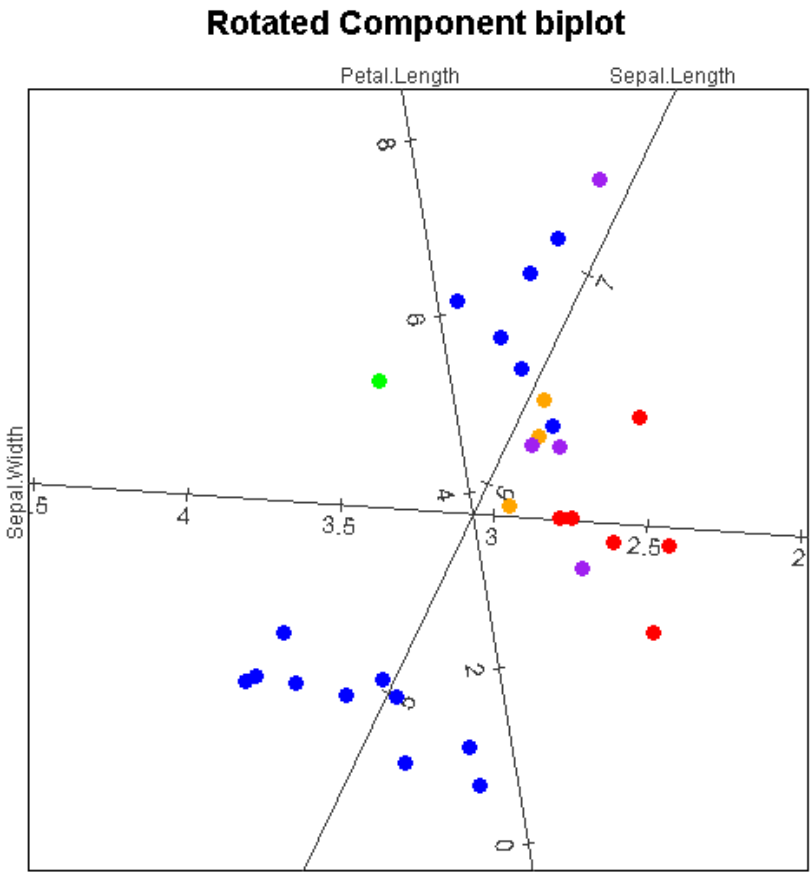


Figure 3.12: RC biplot with calibrated axes and data points coloured based on their class, and the target data point in green

- A grid search of the biplot plane generates the m^2 pixel values represented by $\boldsymbol{Z}_{GS}$. The grid only includes pixels within the convex hull of the training data points on the biplot plane, as the training data was used to estimate the projection matrix $\boldsymbol{V}_2$.
- The grid points are mapped back to the original p-dimensional space using the inverse projection matrix $\boldsymbol{V}_2'$. After this projection, the inverse standardisation function g^{-1} is applied to convert the points to their original scale, resulting in the matrix $\boldsymbol{X}_{GS}$, with:

$$\hat{\boldsymbol{x}}_i = g^{-1}(\boldsymbol{z}_i\boldsymbol{V}_2') = (\boldsymbol{z}_i\boldsymbol{V}_2') \cdot \boldsymbol{x}_{std} + \boldsymbol{x}_{mean}$$

$$\text{where } \boldsymbol{x}_i,\ \boldsymbol{x}_{std} \text{ and } \boldsymbol{x}_{mean}: 1 \times p$$

$$\boldsymbol{z}_i: 1 \times 2 \text{ and}$$

$$\boldsymbol{V}_2': 2 \times p.$$

- These values are then used as input into the prediction model $f(\boldsymbol{X}_{GS})$ and result in outputs values that range from 0 to 0.5 to 1.

- The predictions are used to generate a dense map on the biplot plane by shading the corresponding pixels in $\boldsymbol{Z}_{GS}$, with a colour gradient from blue (class 0) to white (boundary) to red (class 1). The dense map only reflects the data within the convex hull of the training data. The biplot plane now visualises the local decision surface and the impact of the variables on the prediction outcome.
- The biplot gives a way to estimate the local decision boundary $\boldsymbol{X}_{LDB}$. When $\boldsymbol{X}_{test}$ is in class $G = 0$, the local decision boundary is represented by $\boldsymbol{X}_{LDB}|_{\hat{g}=1}$ when $\boldsymbol{f}(\boldsymbol{X}_{GS}) \in [b, b + \varepsilon]$. In this case, $\varepsilon = 0.001$ and the data point is predicted to be in class 0. The contour line representing the counterfactuals of class 0 connects points that have a prediction value of 0.501. The contour line with a prediction value of 0.499 represents the local decision boundary for data predicted to belong to class 1.
- The test data points are then added back to the biplot. The model $f(\boldsymbol{X})$ determines the colour of the data points. As the actual class of the data is known, the data points are classified into four classes using four corresponding colour classifications. The red points represent the data with a true positive prediction outcome, the blue points the true negative data, the purple points the false positive and the orange points the false negative prediction outcome.
- The counterfactual of the target data point $\boldsymbol{x}_i$ is calculated as the nearest point from $\boldsymbol{z}_i$ to the elements of $\boldsymbol{Z}_{LDB}$ measured using Euclidean distance in two dimensions. In other words, it is the point in $\boldsymbol{Z}_{LDB}$ that is visually closest to $\boldsymbol{z}_i$ on the biplot plane.

The biplot is updated in Figure 3.13. The biplot indicates the incorporation of the decision boundary, with the target data point $\boldsymbol{z}_i$ in light green and its counterfactual $\boldsymbol{z}_{cf}$ – the nearest value in the local decision boundary – in dark green. The value of their variables can be read from the calibrated axes by projecting the points onto the axes and interpolating the value between the markers.

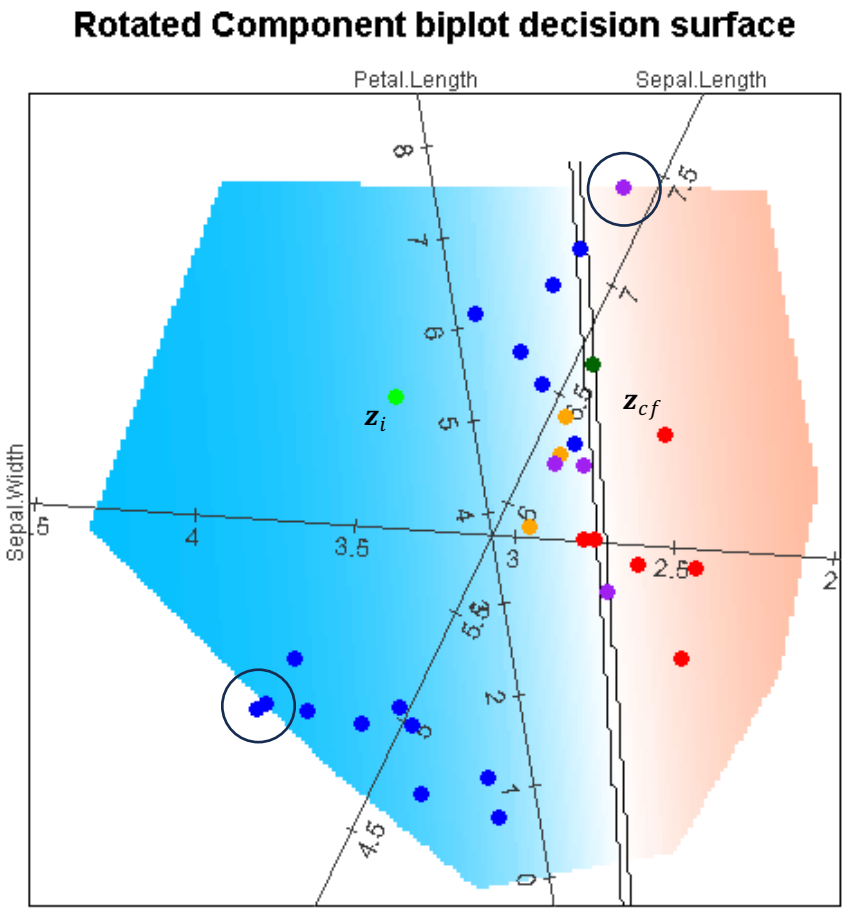


Figure 3.13: RC biplot with the test data point and its counterfactual indicated in light and dark green on the decision surface. Three data points are outside the convex hull of the training data. Only the contour lines within the convex hull will be used to find counterfactuals.

Three potential out-of-distribution data points can be identified as those that are located outside the convex hull of the training data. Care should be taken when making and interpreting predictions for these data points. Only the contour lines within the convex hull will be used to find counterfactuals.

- The counterfactual point is then transformed back to the original p-dimensional space as $\hat{\boldsymbol{x}}_{cf} = g^{-1}(\boldsymbol{z}_{cf}\boldsymbol{V}_2')$ using the inverse standardisation function and prediction matrix.

The counterfactual value calculated for the target data point, along with the value of the GLM model coefficients and the change required to reach the counterfactual, is summarised in Table 3.1 below.

Table 3.1: Result of the counterfactual calculation for the iris dataset target data point

Variable	Data point	Counterfactual	Model coefficient	Change required
Sepal length - x_1	6.2	6.66	-0.06	0.46
Sepal width - x_2	3.4	2.80	-2.77	-0.60
Petal length - x_3	5.4	5.38	0.13	-0.02

The most significant change required to reach the counterfactual requires a reduction along the sepal width variable. This is consistent with the biplot representation, which shows that the sepal width axis is approximately perpendicular to the decision boundary. Consequently, movement along this variable has the most significant influence on altering the prediction outcome.

The fitted GLM model coefficient for sepal width is a negative value. This implies that a reduction in the sepal width is required to increase the prediction score and change the classification from class 0 to class 1. This is aligned with the value of the sepal width variable decreasing along its axis for the test data point to reach the decision boundary, as visualised on the biplot.

The z-scores from the fitted model indicate that only the intercept and the sepal width variable are significant. This is consistent with the biplot interpretation of the contribution of each variable.

The sepal length coefficient is also negative. Therefore, based on the model coefficient, it is expected that the variable value should also decrease to reach the decision boundary. The sepal length and sepal width variables are negatively correlated (their variable axes increase in almost the opposite direction). Therefore, the counterfactual value for class 0 prediction will always require the sepal length to increase as the sepal width decreases to generate plausible counterfactual explanations.

The petal length is almost parallel to the decision boundary. Therefore, movement along this variable has the least influence on altering the prediction outcome. The value also changes the least when moving towards the counterfactual.

3.8.2 Interpreting the biplot and estimated counterfactual

The example above illustrates the process of constructing a single two-dimensional representation of the data points and variables, the decision surface and decision boundaries generated by $f(\boldsymbol{X})$, and the counterfactual values for each data point. This representation supports the following interpretations of both the data and the fitted model.

- The biplot provides a two-dimensional linear approximation of the data by illustrating the relative positions of data points and variables. The multivariate distributional structure of $\boldsymbol{X}$ is retained, with $\widehat{\boldsymbol{X}}_2'\widehat{\boldsymbol{X}}_2$ serving as an approximation of the correlation matrix between the columns of $\boldsymbol{X}$, provided that $\boldsymbol{X}$ is first standardised. The angles between the variable axes reflect the degree of correlation between variables, offering insights into the underlying multivariate distribution of the training data. Selecting grid points directly from the biplot allows for sampling that approximates the multivariate distribution of the original dataset. Therefore, the change required to reach the counterfactual values will also reflect the multivariate distributional structure and therefore plausible changes.

  The relative positions of the data points convey their similarity. Points that lie far from the centre of the biplot or that exhibit low sample predictivity can be interpreted as outliers or out-of-distribution data points. Model developers may consider excluding such out-of-distribution data points when training $f(\boldsymbol{X})$ to improve its overall predictive accuracy.

- The variable axes on the biplot indicate how the projected value of a data point changes as its position on the biplot shifts. Movement along a variable axis results in an increase or decrease in the corresponding projected value. In contrast, movement orthogonal to a variable axis results in little or no change in the projected value of that variable. These directional sensitivities reflect the correlation structure embedded in the biplot.

  The amount by which each variable must change for a data point to reach its local decision boundary is influenced by the orientation of the variable axes relative to the direction of movement for that point. Variables with axes that are perpendicular to the local decision boundary will experience greater changes, while those with more parallel axes will experience smaller changes.

As a result, variables of a data point whose axes run perpendicular to the local decision boundary are more likely to be influential in determining the classification by the model of a given data point, compared to those whose axes run parallel to the boundary.

- The shape of the local decision boundaries and the distances between data points and their counterfactuals, as illustrated by the biplot, offer valuable insight into the nature of the decision surface and boundaries produced learned by the ML model by $f(\boldsymbol{X})$.

  When the decision surface in the original p-dimensional space is a hyperplane, the corresponding decision boundaries on the biplot will appear as a straight line. Conversely, if the decision surface is a curved hypersurface, the decision boundaries will be represented by one or more curved lines. The more complex the hypersurface, the more intricate the shapes of the local decision boundary intervals $\boldsymbol{Z}_{LDB}$.

  When data points are, on average, closer to the decision boundary, it becomes easier to identify the nearest counterfactual examples (Giorgi, Veglianti, Silvestri & Tolomei, 2025). However, consistently short distances between data points and their counterfactuals may suggest that the model is susceptible to minor changes in input values. This sensitivity could be indicative of overfitting or poor generalisation and may also point to a lack of robustness against adversarial attacks.

  In the presence of strong overfitting, the decision surface of the model becomes highly convoluted, potentially to the extent that it perfectly separates every training sample. Overfitting may also be indicated on the biplot by local decision boundary regions that contain no training data.

These insights, on their own, do not provide metrics with which to select training data or choose between different models. However, they do help the modeller gain a better intuitive understanding of how the variables and data impact the model and why different models result in different predictions and prediction accuracy. This provides algorithmic transparency, even for otherwise black-box ML models, with the shape of the local decision boundaries and the interaction with the variable axes giving insight into how the underlying algorithm makes predictions.

3.8.3 PCA vs. CVA biplots

The creation of the CVA biplot follows the same process as that of the PCA biplot, but uses the canonical transformation matrix in place of the principal component projection matrix. The main advantages of the CVA biplot over the PCA biplot are as follows:

- Separation of group means: The CVA biplot achieves a greater degree of separation between group means than the PCA biplot. It maximises the ratio of between-group variance to within-group variance. As the objective of the model $f(\boldsymbol{X})$ is to distinguish between groups through decision boundaries, better separation on the biplot plane leads to a more accurate visual representation of the data points relative to these boundaries.

  A PCA biplot does not accurately represent the group means of each class, as the separation of the group means does not contribute to the calculation of the PCA biplot or the data visualised on it. Consequently, group means are not preserved or emphasised in the visualisation.

  However, the estimation of the covariance matrix used in CVA is sensitive to outliers, and the resulting CVA biplot can vary significantly depending on the data included in the estimation of the common within-group covariance matrix. The CVA method assumes that the within-group SSP matrices are equal across all groups. If this assumption is violated, then, although CVA is relatively robust, the theoretical optimality of the approximation of the group means in the CVA biplot may no longer be valid.

- Scaling of variables: The CVA biplot provides a scale-invariant representation of the data by adjusting for the covariance between variables. As a result, a unit movement in any direction on the biplot reflects an equal level of variance and likelihood, with each group's data assumed to be spherically distributed around its within-group mean.

  In contrast, the PCA biplot is scale-dependent. Variables with larger scales tend to dominate the overall variance, leading to a biplot that disproportionately emphasises separation along these dimensions. To address this issue, variables can be standardised by dividing each by its standard deviation prior to performing the PCA.

- Measurement of Mahalanobis distances between data points: The CVA biplot effectively measures Mahalanobis distances between a data point and its counterfactual by calculating Euclidean distances on a transformed biplot plane that accounts for the correlation structure in the data.

  In contrast, Euclidean distance between standardised data points, such as in a PCA biplot, ignores the correlation and all the features contribute equally to the total distance. This can lead to an overestimation of differences where variables are strongly correlated, as their joint variation is not accounted for.

Mahalanobis distance addresses this limitation by incorporating the correlation between variables and scaling distances using the common within-group covariance matrix $\boldsymbol{W}$. The CVA transformation adjusts the relative positions of the data points on the biplot plane and the corresponding decision boundaries and counterfactuals by contracting distances along directions with high variance and expanding those with low variance. Changes along variable axes with low variance result in larger Mahalanobis distances, reflecting lower likelihood, while changes along axes with high variance yield shorter distances, indicating greater plausibility.

Therefore, Mahalanobis distance provides a measure of statistical similarity between data points rather than purely geometric distance. When choosing between two potential counterfactuals, the one with the shorter Mahalanobis distance is more likely to represent a plausible, interpretable change.

Unlike the CVA biplot, the PCA biplot, even after standardising variables, assumes independence between features. As a result, it treats counterfactuals in different directions as equally plausible if their Euclidean distances are the same, ignoring the correlation structure of the original data.

This section discussed how biplots can identify the local decision boundaries and the nearest counterfactual values. The next section investigates the accuracy and benefits of this approach.

3.9 ACCURACY, REQUIREMENTS AND BENEFITS OF USING THE BIPLOT

Section 3.8 introduced the use of biplots to generate counterfactual values for the predictions of an ML model. The following sections evaluate these counterfactuals against established requirements and constraints for valid counterfactual solutions, as well as the metrics commonly used to compare different generation methods, explained in section 2.10. This evaluation aims to determine whether the biplot-based counterfactuals can be reliably used to interpret the underlying model $f(\boldsymbol{X})$.

Chapter 4 then explores how counterfactuals and the approximated decision boundaries can be used to interpret the behaviour and predictions of the model. Their quality will be assessed using the metrics identified in section 2.6.

3.9.1 Accuracy of the biplot representation

The accuracy of the approximation of the local decision boundaries and counterfactuals by the biplot will depend on the following factors:

- Shape of the multi-dimensional decision surface: The biplot plane only approximates a slice of the decision surface along two component axes. If the biplot plane is rotated, it will be along two combinations of the components. Consequently, decision boundaries present in other parts of the high-dimensional space, not represented on the biplot plane, will be omitted when calculating counterfactuals.

  Therefore, the shape of the multi-dimensional decision surface created by the ML model determines how well it is approximated by the biplot plane. If it contains many maxima and minima, the biplot plane will not be able to approximate all the decision boundaries and counterfactuals implied by these. This may result in missing counterfactuals that are closer to the target data point in the original feature space. Using these closer counterfactuals might lead to different interpretations of both the model and its predictions.

  The accuracy of the biplot approximation improves if the two components used to create the biplot capture a large proportion of the variance in the data and if the complexity of the decision surface decreases. Under these conditions, the risk of misinterpretation is reduced.

  To gain alternative perspectives on the decision boundary, different combinations of components can be selected to generate various biplot representations. These will result in different local decision boundaries and counterfactuals that can be evaluated to determine whether they are closer to the target data point than those generated by the first two components.

- Distance to the biplot plane: The accuracy of the model prediction, as approximated on the biplot plane, depends on the distance of the data point from the biplot plane. As data points are interpolated onto the biplot plane, the original point $\boldsymbol{x}$ is generally not equal to its projection $\hat{\boldsymbol{x}}$. Consequently, the prediction of the model $f(\boldsymbol{x})$ in the original p-dimensional space may differ from the prediction implied by the position of the projected point on the biplot plane, $f(\hat{\boldsymbol{x}})$. For example, it is possible to have $f(\boldsymbol{x}) < b$ while $f(\hat{\boldsymbol{x}}) > b$, or vice versa, and a data point correctly classified by the model as class 0 may be projected onto a region of the biplot corresponding to class 1.

This will be reflected as a difference between the colour of the data points and the colour gradient of the biplot plane of the region where the point is projected. The frequency with which the data point and the biplot region colour differ can indicate regions where the biplot poorly approximates the model $f(\boldsymbol{X})$. The greater the distance between $\boldsymbol{x}$ and its projection $\hat{\boldsymbol{x}}$, the larger this discrepancy is likely to be.

In such cases, the counterfactual estimated from the biplot will be generated from an inaccurate location, causing the explanation vector to point in the wrong direction. This must be avoided, as discussed in section 2.2, as it will lead to an invalid and misleading the interpretation derived from the counterfactual.

This risk can be mitigated by rotating the biplot plane using a transformed projection matrix $\boldsymbol{V}^{\text{new}} = \boldsymbol{VA}'$, where $\boldsymbol{A}$ is calculated individually for each data point. This ensures that the target data point lies on the biplot plane. The counterfactual estimated from the rotated biplot will then produce explanation vectors pointing in the correct direction, enabling a valid interpretation of the prediction. However, the problem cannot be resolved for all the data points at once.

- Distance between z_0 and $\mathrm{Z}_{\mathrm{LDB}}$ compared to between x_0 and $\widehat{\mathrm{X}}_{\mathrm{LDB}}$: The biplot method identifies the nearest counterfactual by finding the closest point between $\boldsymbol{z}_0$ and the local decision boundary set $\boldsymbol{Z}_{LDB}$ on the biplot plane, rather than between $\boldsymbol{x}$ and $\widehat{\boldsymbol{X}}_{LDB}$ in the original p-dimensional space. As a result, the estimated counterfactual $\hat{\boldsymbol{x}}_{cf}$ is not necessarily the closest point to $\boldsymbol{x}$ in the original feature space, as in general $\boldsymbol{x} \neq \hat{\boldsymbol{x}}$.

  Conversely, if the nearest counterfactual were determined by measuring distance directly in the original space between $\boldsymbol{x}$ and $\widehat{\boldsymbol{X}}_{LDB}$, the corresponding interpolated counterfactual $\boldsymbol{z}_{cf}$ will not correspond to the nearest point on the local decision boundary set $\boldsymbol{Z}_{LDB}$ in the biplot space. This would complicate the intuitive identification and interpretation of counterfactuals based on their relative positions on the biplot plane.

  Different models or different combinations of components lead to different biplot representations and resulting counterfactuals. When choosing between the i different possible counterfactuals of $\boldsymbol{x}$, the nearest counterfactual $\hat{\boldsymbol{x}}_{i,cf}$ is selected as the counterfactual that corresponds to the shortest change vector $\boldsymbol{d}_i$ measured in the full dimensional space using Euclidean distance for PCA biplots and Mahalanobis distance for CVA biplots.

Therefore, the change is measured in the biplot space and transformed into the original p-dimensional space as:

$$\boldsymbol{d}_i = (\boldsymbol{z}_{i,cf} - \boldsymbol{z}_{i,0})\boldsymbol{V}_2' \text{ or } (\boldsymbol{z}_{i,cf} - \boldsymbol{z}_{i,0})\boldsymbol{M}_2^-$$

where i refers to the biplot considered.

- The number of model features included in the biplot: As the number of features included in the model increases, or the less correlated the data is, the smaller the proportion of variance that will be captured by the first two components used to generate the biplot. This reduces the accuracy of the approximation by the biplot of the data and decision surface and increases the distance of data points from the decision surface.

  Including more variables also tends to lower the axis predictivity of individual variables and reduces the accuracy with which they are represented on the biplot. In addition, a biplot with many variable axes becomes cluttered, making visual interpretation and understanding of the impact of each variable less intuitive.

  The biplot can be simplified, and the accuracy of interpretation improved, by limiting the number of variables used to calibrate the model to seven or fewer. As explained in section 2.7.2, reducing the number of variables enhances the interpretability and comprehensibility of the explanations provided. Axis predictivity can assist in identifying which variables contribute most to accurate interpretations on the biplot. Variables with low axis predictivity can be hidden from the biplot when more than seven variables are present, thereby improving visual clarity. Alternatively, the ML model itself should be simplified to include no more than seven predictive variables.

- Restrictions: It is worth noting that counterfactuals generated using the biplot method currently only apply to binary classification models built on continuous variable data represented in a tabular format, with each row representing a unique data point. However, the technique can be extended to multi-class classification problems or regression models by setting the boundary threshold b equal to the target regression value.

  Binary categorical variable data can be incorporated into the biplot method by converting their values to 0 and 1 and treating it as continuous variable data. Integer data is included as a continuous variable data type.

However, ordinal and nominal categorical variables with more than two levels do not have an intrinsic scale, and representing these data types with an axis will result in a representation that is both misleading and inconsistent, implying an order and comparative scale that do not exist. Therefore, more work is needed to generate counterfactuals for categorical variable data.

3.9.2 Constraints for a biplot-generated counterfactual value

This section outlines how the biplot method should be implemented to ensure that the counterfactuals generated meet the requirements necessary to provide valid interpretations of the prediction by a model. Specifically, it explains how to create counterfactuals that are actionable, feasible, plausible, scarce and diverse.

- Actionable: To be considered actionable, only those variables of a data point that can change should be included when determining the local decision boundaries, and therefore the potential counterfactuals.

  One approach for identifying actionable features of individual data points is to construct the biplot using only the actionable and conditionally mutable variables, while fixing the inactionable variables at the value of the target data point. This ensures that counterfactuals are identified and interpreted under the assumption that immutable features remain unchanged. However, this method may lead to the generation of counterfactuals that are implausible, as it ignores the joint dependency structure of the complete set of features.

  An alternative option is to construct the biplot using the full dataset and applying the actionability constraints by filtering the elements of the local decision boundaries to include only those that meet these constraints. In this case, the counterfactual is selected from the nearest point within the remaining subset of the boundary values. This method preserves the data structure, but may lead to situations where no actionable decision boundaries are visible on the biplot plane.

- Feasible: Feasibility is addressed by ensuring that the distance to the decision boundary is calculated within the limits defined by the distribution of $\boldsymbol{x} \in R^p$. In other words, the counterfactual $\boldsymbol{x}_{cf}$ must fall within the valid range of values for each variable.

  This is achieved by filtering the elements of the local decision boundary to include only those values that lie within the feasible upper and lower bounds for each feature. These

bounds are typically derived from the observed training data distribution or expert-defined constraints.

The main limitation of the biplot method arises when no counterfactual is visible on the biplot plane within the feasible and actionable range of the variables, given the decision boundary threshold b used.

One solution is to construct alternative biplots from different combinations of two components to generate alternative local decision boundaries and potential counterfactual values. However, for some data points it may be the case that no possible biplot representation contains a feasible or actionable counterfactual. This many indicate that there is no valid counterfactual prediction outcome that meet the required constraints in any location on the multi-dimensional decision surface.

- Plausible: For counterfactual values to be plausible, the local decision boundaries must represent combinations of features that reflect the joint distribution of $\boldsymbol{x} \in R^p$. If the local decision boundaries do not reflect plausible combinations of features, the resulting counterfactuals will also be implausible and may fail to provide meaningful insights into the predictions made by the ML model.

  One of the limitations of the grid search approach is that it can generate decision boundaries, and corresponding counterfactual solutions, far from the centre of the biplot. These solutions are often unlikely when evaluated against the distribution of the training data. To address this, local decision boundaries should be restricted to include only those elements that lie within the convex hull of either the training or the test data, depending on which was used to generate the projection matrix $\boldsymbol{V}$. This ensures that the candidate counterfactuals remain within the realm of observed or likely feature combinations.

  In addition, any local decision boundary that encloses a region devoid of actual data points should be excluded from consideration. This aligns with the requirement that real, observed data should support counterfactuals. For example, as illustrated in Figure 3.9, the local decision boundary $\boldsymbol{X}_{LDB}^{kj}$ would have been excluded if the data point x_l had not been present in the test set. This safeguards the interpretability of counterfactuals by avoiding reliance on artefacts arising from model overfitting or poor generalisation.

  Finally, counterfactuals generated using the CVA biplot are generally more plausible than those generated with a PCA biplot, as they are based on the Mahalanobis

distance, which takes into account the covariance structure of the data. When comparing two candidate counterfactuals, the one with the shorter Mahalanobis distance to the original data point is more likely to reflect a realistic and plausible solution.

- Sparse: A sparse solution refers to a counterfactual that requires the fewest number of variables to change to reach the local decision boundary. Unlike some other methods, the biplot approach does not support the use of $L0$ or $L1$ distance measures to explicitly enforce sparsity.

  However, sparsity can be encouraged in the biplot method by removing variables that do not contribute meaningfully to the global interpretability or accuracy of the biplot, as described in section 4.2.2. Simplifying $f(\boldsymbol{X})$ by reducing the number of features it includes, particularly those that contribute little to the overall predictive accuracy or interpretability of the model, will result in sparser counterfactuals.

  Similarly, for individual-level explanations, variables that do not contribute to the prediction outcome can be excluded from the explanation, as shown in section 4.3.2. Restricting the analysis to actionable variables, as discussed earlier, also naturally results in sparser counterfactuals.

  It is important to note that enforcing sparsity may lead to a greater overall distance to the counterfactual and could yield less plausible solutions, particularly if key features are omitted.

- Diverse: The biplot method identifies multiple local decision boundaries, each representing a distinct set of potential counterfactual solutions. As a result, the method naturally supports the generation of diverse counterfactuals. This diversity ensures that, if the nearest counterfactual solution is not actionable or feasible, alternative counterfactuals located on other local decision boundaries may still be valid and interpretable.

  By visualising these multiple paths to class reversal on the biplot plane, the method provides a richer understanding of the decision surface. It facilitates the selection of counterfactuals that best meet these practical constraints, whether in terms of actionability, feasibility or plausibility.

3.9.3 Metrics of the counterfactuals generated using biplots

Different methods of generating counterfactuals can be evaluated using the metrics discussed in section 2.10.3. The following subsections describe how the proposed biplot method performs in relation to each of these metrics.

- Distance measure: In the PCA biplot, distances between data points on the two-dimensional plane approximate the Euclidean distances in the original p-dimensional space. This allows for intuitive interpretation of similarity and proximity between data points.

  In the CVA biplot, the canonical transformation matrix is used. As a result, the Euclidean distances on the biplot plane correspond to Mahalanobis distances in the original feature space. This makes the CVA biplot more reflective of the underlying data distribution, especially when feature correlations are present.

- Calculation time: The biplot method substantially reduces computation time by limiting the grid search to a two-dimensional space. If the granularity of the grid is set to m, only m^2 pixel locations need to be evaluated, compared to $\prod_p m_i$ points in the full p-dimensional space.

  The search space can be further reduced by restricting it to the convex hull of the available data, assuming the dataset is representative of the actual distribution. The calculation of the projection matrices $\boldsymbol{V}$ (for PCA) or $\boldsymbol{M}$ (for CVA) is computationally efficient, and distance measurements are performed in two dimensions. This allows the method to generate counterfactuals for a large number of data points with minimal computational cost.

- Coverage: As long as the local decision boundary is represented on the biplot plane, the method will produce counterfactuals. Because the search is visual and geometric, the method can identify at least one valid counterfactual for each point projected onto the biplot. In practice, this approach ensures complete coverage for all data points included in the biplot, enabling efficient batch generation of counterfactuals. However, these counterfactuals will not always be the absolute nearest counterfactuals in the full feature space, and they may also not meet the actionability and feasibility requirements.

- Redundancy: The redundancy of a counterfactual solution can be assessed using Shapley values. These indicate the marginal contribution of each variable to the change in prediction outcome. Variables that have no impact on class reversal can be

identified and excluded from the explanation. This approach is discussed further in section 4.3.2, and helps improve the interpretability and sparsity of counterfactuals.

- Stability: The biplot method is deterministic: It will always generate the same counterfactual solutions provided that the same transformation or projection matrices $\boldsymbol{V}$ or $\boldsymbol{M}$ are used. However, if these matrices are computed from different datasets (e.g. when the training data is unavailable) or in the case of CVA biplots with different calibrations of $f(\boldsymbol{X})$, the resulting biplots and therefore the decision boundaries and counterfactuals may differ. This introduces some dependency on the data used to generate the projection or transformation matrices.

3.10 USING BIPLOTS TO EXPLAIN MACHINE LEARNING MODELS

This chapter developed the methodology for using PCA and CVA two-dimensional biplots to visualise the approximated decision surface of the model $f(\boldsymbol{X})$ and identify the local decision boundaries and counterfactuals of target data points. The only change required between using either the CVA or the PCA biplots is replacing the matrices $\boldsymbol{V}$ and $\boldsymbol{V}'$ with $\boldsymbol{M}$ and $\boldsymbol{M}^{-1}$ (or $\boldsymbol{M}_{opt}$ and $\boldsymbol{M}_{opt}^{-1}$ in the case where the data contains only two classes).

To optimise the results and ensure that the constraints on counterfactuals are met, the following steps are followed when producing a counterfactual value.

1. A biplot can provide post hoc interpretations without reference to the training data. This requires that the sample of $\boldsymbol{X}_{test}$ is representative of the distribution of $\boldsymbol{X}_{train}$. If the sample is not representative, reference to the training data $\boldsymbol{X}_{train}$ will still be required. The rest of the dissertation does not differentiate between $\boldsymbol{X}_{test}$ and $\boldsymbol{X}_{train}$ and only refers to $\boldsymbol{X}$.
2. Initial data investigations will be performed using the PCA biplot, as this removes the need to first calibrate $f(\boldsymbol{X})$. This may result in the removal of outlier data points and variables that result in data leakage. The cleaned data can then be used to calibrate $f(\boldsymbol{X})$.
3. The CVA biplot is then used to generate local and global interpretations, as it ensures that the estimated counterfactual reflects the shortest Mahalanobis distance in the original feature space. As the CVA transformation requires pre-defined group classifications to separate group means, the grouping is based on the four prediction outcomes of the classification model: true positives, true negatives, false positives and false negatives. Consequently, the CVA biplot will vary depending on the type and

calibration of $f(\boldsymbol{X})$, as different models yield different prediction outcomes and group assignments. Additional outliers can be identified by the initial CVA biplot, which will then be removed from the training data before training the final $f(\boldsymbol{X})$.

4. The local decision boundaries are filtered to only contain feasible counterfactuals within the upper and lower bounds for each feature of the training data. Only local decision boundaries within the convex hull of the training data on the biplot plane will be considered as possible counterfactuals. In addition, any local decision boundary area that does not contain data points is discarded.
5. These local decision boundaries are then used to generate counterfactuals for each data point. The distance to the counterfactuals is used to generate a global variable importance measure, compare the robustness of different models and generate a surrogate prediction model.

For local explanations of individual data points, the following further steps are required:

6. The transformation matrix is rotated to ensure that the RC biplot plane passes through the target data point. The local decision boundaries $\boldsymbol{X}_{LDB}$ can then be filtered to only contain actionable, feasible and plausible counterfactuals.
7. To find the nearest possible counterfactual, the target data point is rotated onto all the combinations of the components of $\boldsymbol{V}$ or $\boldsymbol{M}$ to create $\boldsymbol{V}^{\text{new}}$ or $\boldsymbol{M}^{\text{new}}$. The first two components of the rotated component projection matrix are used to create different biplot representations and local decision boundaries, and the nearest counterfactual solution is selected. The nearest counterfactual is then used to explain the reasons why the prediction was made and what needs to change to change the prediction outcome. If this does not result in any solution, then it is assumed that no constraint counterfactual exists for this data point.

The methods to generate the interpretations from the counterfactuals are explained in Chapter 4. These will help answer the questions initially posed in Chapter 1.

CHAPTER 4 GLOBAL AND LOCAL INTERPRETATIONS USING A BIPLOT

4.1 INTRODUCTION

This chapter develops methods that generate both global and local interpretations of the predictions made by any ML model $f(\boldsymbol{X})$. These interpretations are made possible by identifying and analysing the local decision boundaries $\boldsymbol{X}_{LDB}$ and corresponding counterfactual values $\boldsymbol{X}_{CF}$ for each data point in $\boldsymbol{X}$, using the methods introduced in Chapter 3.

The objective was not to optimise prediction accuracy, but rather to use the biplot visualisation and counterfactual generation techniques to explain model behaviour. As stated in section 2.3, "interpretability often begets accuracy". By enhancing interpretability, specifically the algorithmic transparency of the model, the insights may in turn support improvements in model performance. The interpretations will make it possible to answer specific global and local questions concerning the model and predictions made for individual target data points.

The ultimate goals of the interpretations are causality and informativeness. Interpretations should reveal how different variables interact, how these interactions impact real-world outcomes and what relationships exist within the internal structure of the model. This can guide model selection, feature engineering and data selection, while supporting more informed, transparent and trustworthy ML practice.

4.2 GLOBAL INTERPRETATIONS USING A BIPLOT

This section discusses the development of post hoc model-agnostic global interpretation methods that leverage the local decision boundaries and counterfactuals generated through the biplot methodology. This visual representation supports the exploration of critical interpretability questions, such as the following:

- Which variables contribute most to the predictive performance of the model, and which could be removed without degrading, or potentially even improving, accuracy?
- Which data points exert the most influence on model calibration, and which are least accurately predicted?

- Does the model behave reasonably and consistently across different regions of the feature space and on a case-by-case basis?
- Why are certain instances misclassified, and what changes would be required for their correct classification?

By illustrating the position and shape of decision boundaries relative to the data and variable axes, the biplot enhances confidence in the soundness of the learning algorithm. It aids in identifying out-of-distribution observations and systematic misclassifications, while providing insight into the directional influence of individual variables on the model output.

Variable importance can be assessed by measuring the distance along each variable axis between a data point and its nearest counterfactual. Variables that consistently show significant changes across multiple data points are considered more influential in the prediction outcomes of the model. Conversely, variables with limited changes may be redundant and candidates for removal, thereby simplifying the model and potentially improving generalisation.

The distance to a counterfactual across all data points serves as a measure of prediction robustness. Larger distances suggest that the predictions of the model are more stable and resistant to small perturbations in input features. This robustness metric provides an alternative means of model evaluation beyond standard out-of-sample accuracy, supporting more nuanced model selection.

The biplot can be used to construct a surrogate model that approximates the behaviour of a more complex, black-box classifier. This surrogate is interpretable, as its predictions are directly aligned with the variable axes and visualised decision boundaries on the biplot. The fidelity of this surrogate model is measured by how closely its predictions match those of the original model. This indicates the reliability of the global interpretability provided.

The proposed interpretation methods are demonstrated using the Pima Indian Diabetes dataset (Schulz *et al.*, 2006). This dataset consists of 768 records of Pima Indian women aged 21 and older and is widely used as a benchmark for classification problems. It includes eight predictive variables and a binary target variable, as summarised in Table 4.1. The data is split 80:20 into a training and testing dataset.

Table 4.1: Variables in the Pima dataset

Feature	Description
Pregnancy	Number of times pregnant
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skin fold thickness (mm)
Insulin	2-hour serum insulin (lh/ml)
BMI	Body mass index
DiabetesPedigreeFunction	Diabetes pedigree function
Age	Age (years)
Outcome	Binary value indicating non-diabetic/diabetic

4.2.1 Biplot representation and interpretation

The initial analysis uses a PCA biplot to visualise the training data and calibrated variables axes. The PCA biplot is shown in Figure 4.1. The data points are coloured to reflect the class of the target variable in the training data, with blue representing non-diabetic and red diabetic. The biplot indicates no distortion by class and few outliers. However, the SkinThickness and Insulin variables contain many values equal to zero. This can be identified by the axes label 0 for these variables being near the centre of the biplot, thereby representing the variable value for many of the data points. This is easier to see when plotting only these two variables on the biplot, as seen in the second part of Figure 4.1.

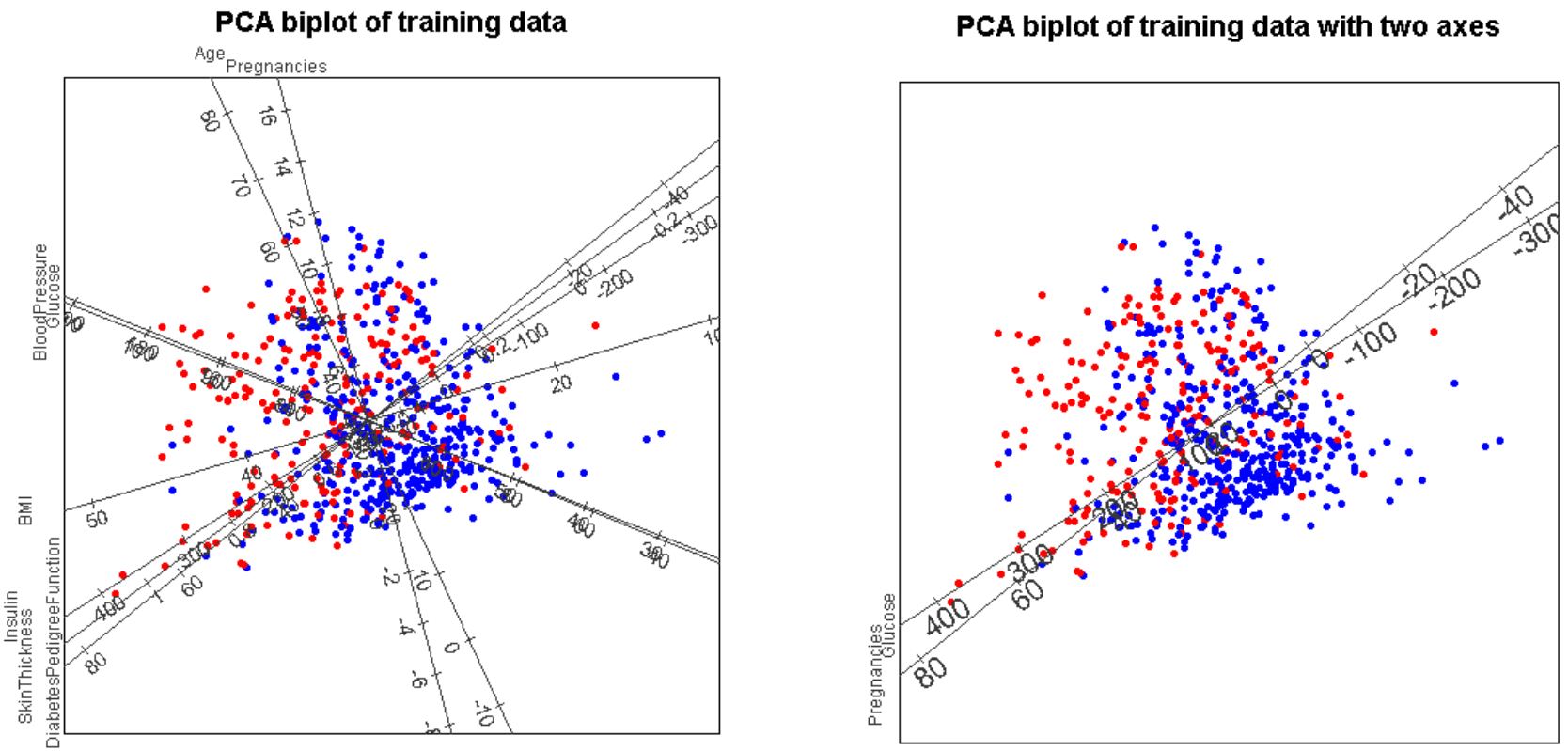


Figure 4.1: PCA biplots of the Pima dataset. The data is coloured to reflect the diabetic status. The second biplot only plots the two variables being investigated for potential data errors.

There are many methods of dealing with this data issue. To simplify the analysis in this dissertation, the records with Insulin and SkinThickness variables that have values equal to zero are removed from the dataset and 394 records remain. The training dataset now consists of 315 entries. This results in the updated PCA biplot shown in Figure 4.2.

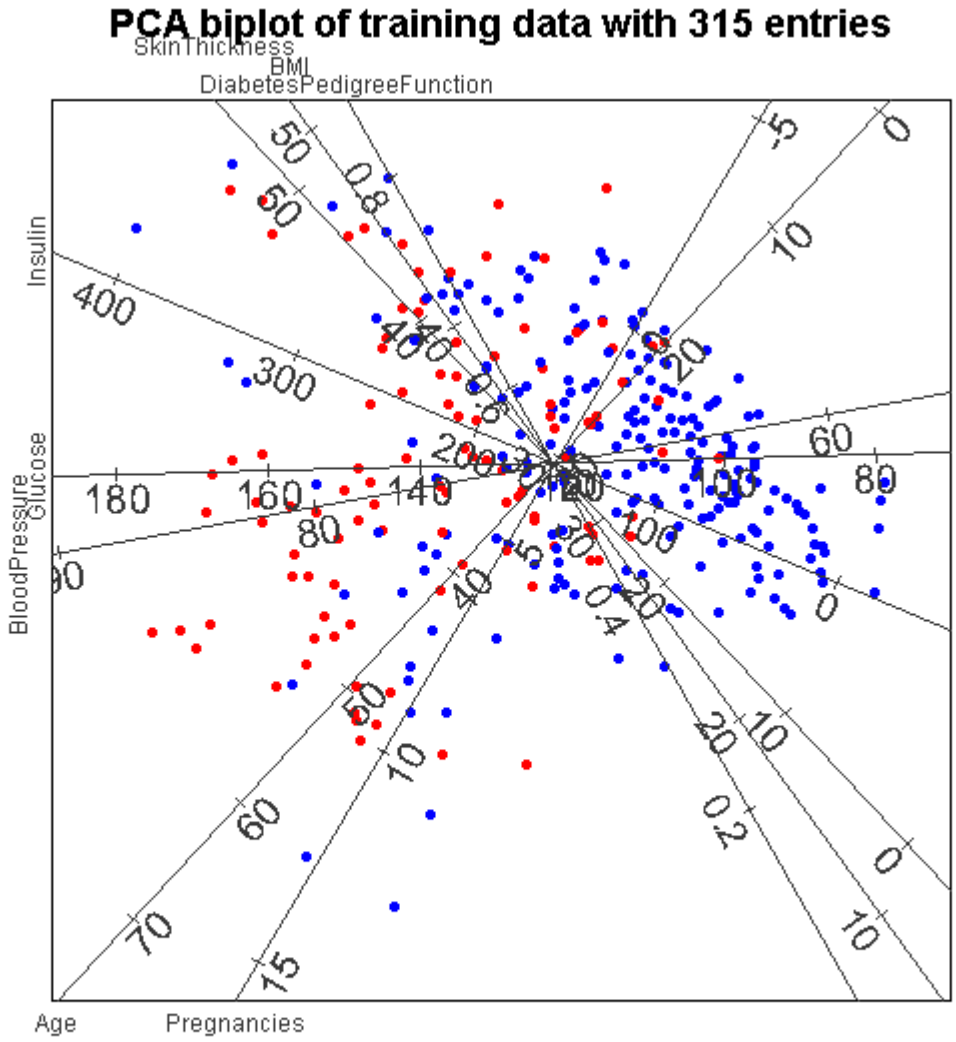


Figure 4.2: PCA biplots of the Pima dataset with Insulin and SkinThickness variables that have values equal to zero removed from the dataset. A total of 394 records remain, with 315 in the training dataset. The correlation structure of the data can be investigated with the biplot.

The variable axes in the PCA biplot illustrate the correlation structure among the input features. Notably, Pregnancy and Age exhibit strong correlation. Similarly, Glucose and BloodPressure are highly correlated. In contrast, BMI, DiabetesPedigreeFunction and SkinThickness form another correlated group that is largely uncorrelated with Pregnancy and Age.

These groupings offer early insight into the underlying multivariate structure of the dataset and suggest which variables tend to move together when $f(\boldsymbol{X})$ is calibrated and creates decision boundaries.

The Pima data is then modelled using a GBM. The training data calibrates the canonical transformation matrix $\boldsymbol{M}_2$ and creates the biplot x- and y-scaffolding axes and biplot plane. For the calculation of the CVA transformation matrix $\boldsymbol{M}$, the training data is grouped into four categories corresponding to the possible prediction outcomes of a binary classification model: true positives, true negatives, false positives and false negatives. The training data also defines the standardisation function g and the convex hull that determines which training and test data points are included on the biplot. No outlier data is removed.

The initial analysis focuses on the training data, which is illustrated using a CVA biplot shown in Figure 4.3. The training data is projected onto the biplot plane using $\boldsymbol{Z} = g(\boldsymbol{X})\boldsymbol{M}_2$. As the target variable class of the training data is known, the data points are colour-coded according to the prediction outcome, with red points representing a true positive prediction outcome, blue points representing true negatives, purple points representing false positives and orange points representing false negative prediction outcomes.

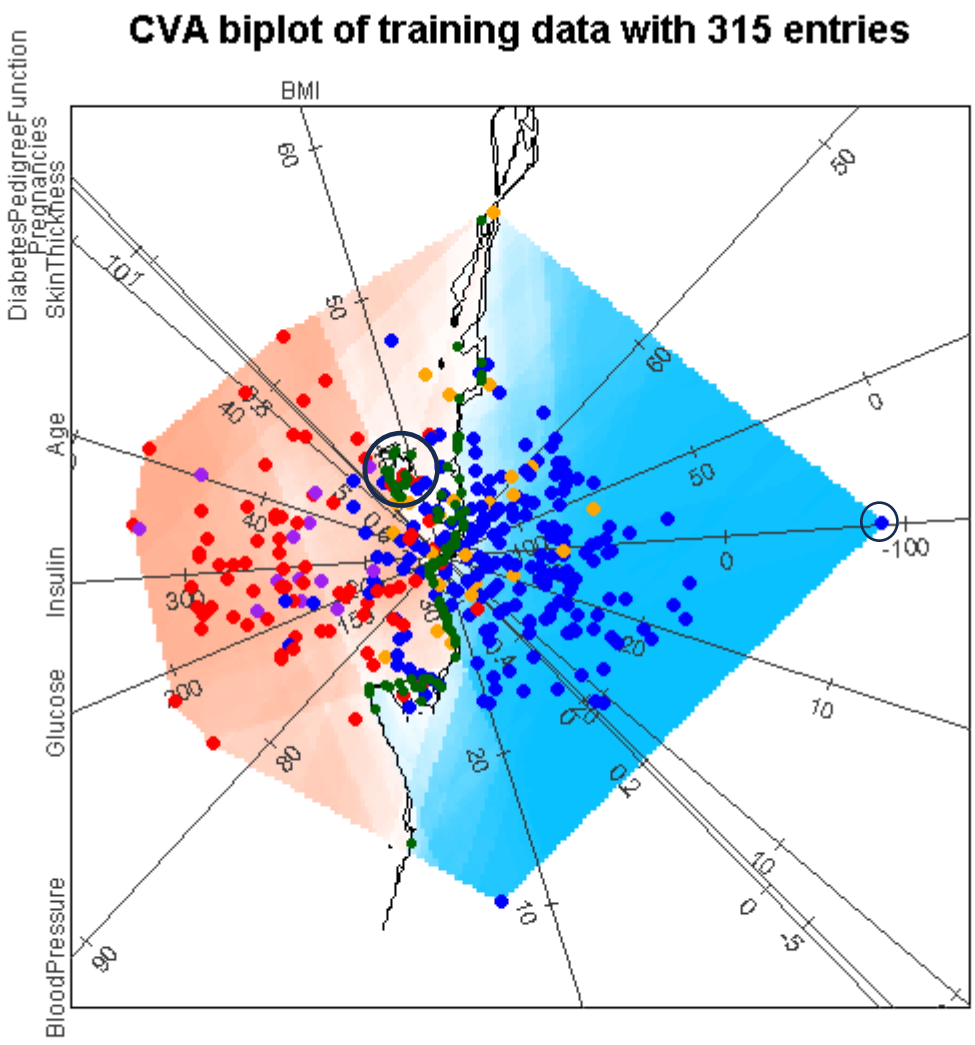


Figure 4.3: CVA biplot of the Pima test data, with the plane approximating the GBM model decision surface within the convex hull of the training data, the local decision boundaries in black contour lines and counterfactuals in dark green. A potential out-of-distribution data point is identified to the right. A local decision boundary is identified to the top left.

The CVA biplot plane provides an approximation of the decision surface of the two-class prediction model, shown as a dense colour gradient from blue to white to red. The local decision boundaries are overlaid as black contour lines.

The CVA biplot gives an optimal separation of the four group means. The relative angles of the CVA variable axes, shown in grey, reflect the correlations between the variable values of the group means and are therefore different from the PCA biplot. This is because the correlations between the group means are not necessarily the same as the correlations within the full training dataset.

Most of the data is clustered near the centre of the biplot, with relatively few outliers near the edges of the convex hull. An example of a potential out-of-distribution data point is identified to the right of the plot. This reflects a value with a glucose level of zero, where all the other values have a minimum of 56. This data point could be removed to improve the accuracy of the model calibration.

The positions of the red and blue points suggest that the two classes are well separated in the projected space. The misclassified points (in purple and orange) are distributed relatively evenly across the biplot, suggesting that the model is not missing any obvious systematic factor and is making errors in a relatively unbiased manner.

The counterfactual of the target data point is calculated as the nearest point from $\boldsymbol{z}_i$ to the elements of $\boldsymbol{Z}_{LDB}$ measured using Euclidean distance in two dimensions. In other words, it is the point in $\boldsymbol{Z}_{LDB}$ that is visually closest to $\boldsymbol{z}_i$ on the biplot plane. To maintain plausibility and avoid overfitting, only decision boundary regions that contain training data or fall within the convex hull of the data are retained. An example of such a region is identified to the top left. As this local decision boundary contains training data points, it is not considered an artefact and will be retained as a local decision boundary from which to identify counterfactual values

The counterfactual point is the estimated as:

$$\widehat{\boldsymbol{x}}_{cf} = g^{-1}(\boldsymbol{z}_{cf}\boldsymbol{M}_2^{-1}).$$

The amount by which each variable must change for a data point to reach its local decision boundary is influenced by the orientation of the variable axes relative to the direction of movement for that point. Variables with axes that are perpendicular to the local decision boundary will experience greater changes, while those with parallel axes will experience smaller changes. The distances between the data points and their counterfactuals along each axis can now be used to generate global interpretations of the model. This is explained in the following sections. Furthermore, the local decision boundaries can be used to construct a surrogate model that delivers global interpretations of the underlying ML model. This is also explained in the following sections.

4.2.2 Variable importance based on distance to decision boundary

The distances between the data points and their counterfactuals along each axis provide insights into how $f(\boldsymbol{X})$ makes predictions and determines which variables contribute most to its prediction accuracy. This can help identify fewer influential variables, allowing them to be

removed to produce sparser, more interpretable models. This may potentially improve the overall model prediction accuracy.

A common approach to global interpretation involves plotting the Shapley values for each data point across all variables, often in the form of a jitter plot. Variables with the highest Shapley values across the dataset are considered globally important. However, calculating exact Shapley values is computationally intensive and often inefficient for large-scale applications. SHAP, an approximation method for Shapley values, was discussed in detail in section 2.9.1. An alternative, less computationally intensive and therefore more commonly used method is the feature importance plot, as proposed by Breiman (2001a), outlined in section 2.9.2.

This section proposes a new method for assessing global variable importance: ranking variables based on the total standardised contribution to the change from each data point to its counterfactual, measured along each variable axis. The variable with the largest total contribution across all data points is considered the most globally influential.

The contribution from each variable is illustrated for a single data point in Figure 4.4.

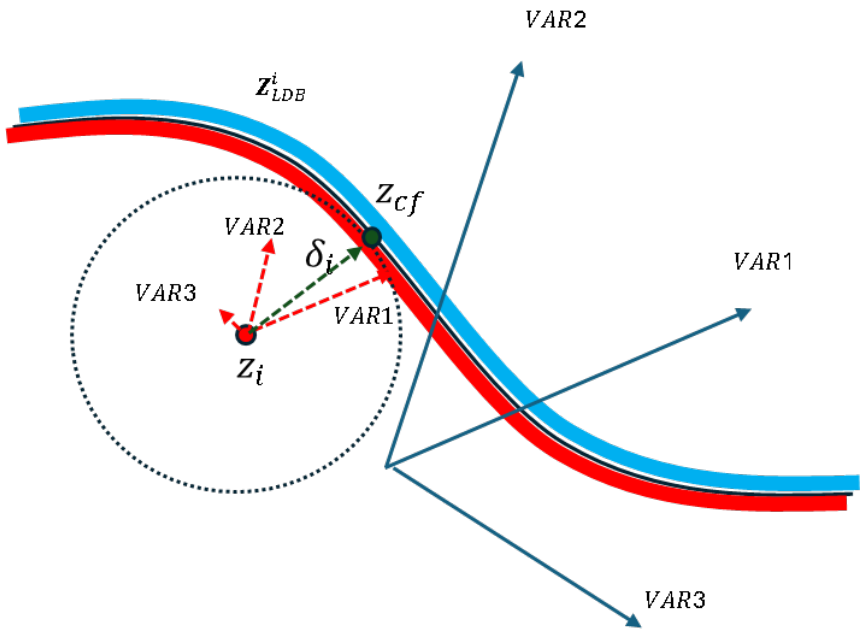


Figure 4.4: Illustration the counterfactual change vector $\boldsymbol{\delta_i}$ decomposed along the variable axes

This shows the change vector from an individual data point $\boldsymbol{z}_i$ to its nearest counterfactual $\boldsymbol{z}_{cf}$. The vector $\boldsymbol{\delta}_i = (\boldsymbol{z}_{cf} - \boldsymbol{z}_i)$ gives the direction and magnitude of the required change on the biplot plane and is orthogonal to the local decision boundary for each of the $i = 1\ to\ n$ training data points. This vector can be decomposed into the contribution of each variable to

the counterfactual shift by converting it to the original p-dimensional space using the projection $\boldsymbol{d}_{ij} = (\boldsymbol{z}_{cf} - \boldsymbol{z}_i)\boldsymbol{M}_2^-$, where $j = 1\ to\ p$. The projection and transformation matrices $\boldsymbol{V}$ and $\boldsymbol{M}$ can be used interchangeably, as the process is the same.

Each component of change vector $\boldsymbol{d}_{ij}$ along the p variable axes reflects how much the particular variable must change for the data point to cross the local decision boundary, which combined reflects the multi-dimensional change required. The longer the component along a given variable axis, the greater the contribution of that variable to the prediction change.

In this instance, variable number 1 has the longest component, which implies that changing variable number 1 alone would result in the greatest total shift towards the boundary. In this sense, variables with longer components are more 'efficient' levers for influencing predictions and therefore more globally important. Seen differently, the data point reaching the decision boundary along variable number 3 alone will result in a counterfactual that is much further away from data point $\boldsymbol{x}_i$ than the proposed counterfactual $\boldsymbol{x}_{cf}$.

To ensure comparability across variables with different scales, the change along each axis is standardised by dividing it by the overall standard deviation of that variable. This standardisation ensures that the relative contribution of each variable is not distorted by differences in measurement units or scale. However, the standard deviation reflects the distribution of the underlying data. The presence of outliers in the data can distort the standard deviation and therefore the standardisation, which may distort interpretation from the distance-based variable importance. An ML model that can still give good predictions even in the presence of outliers in the training data will give a different weighting to the importance of the variables.

This decomposition is repeated for all data points in the training dataset and the distance along each axis plotted by variable. The result is a jitter plot of the standardised axis-level contributions for each variable across all counterfactuals. Figure 4.5 presents such a plot of each $\boldsymbol{d}_{ij}/\boldsymbol{std}_j$. The colour of each point reflects the prediction outcome of $f(\boldsymbol{X})$.

The total distance $\boldsymbol{T}_j$ measure per variable j is the sum over all the data points of the absolute value of the standardised vector component $\boldsymbol{\delta}_{ij}$ of the variable. Therefore,

$$\boldsymbol{T}_j = \sum_{\boldsymbol{i}} \left| \frac{\boldsymbol{d}_{ij}}{\boldsymbol{std}_j} \right|, j: 1\ to\ p.$$

The variable with the shortest total distance $\boldsymbol{T_j}$ can be interpreted as the least important for the predictions made by $f(\boldsymbol{X})$. This is similar to the method proposed by Laugel *et al*. (2018), introduced in section 2.10.2, where sparser solutions are identified by removing features using a naïve heuristic that the variable with the smallest coordinates in the vector $\boldsymbol{x_{cf}} - \boldsymbol{x_i}$ is the least relevant locally.

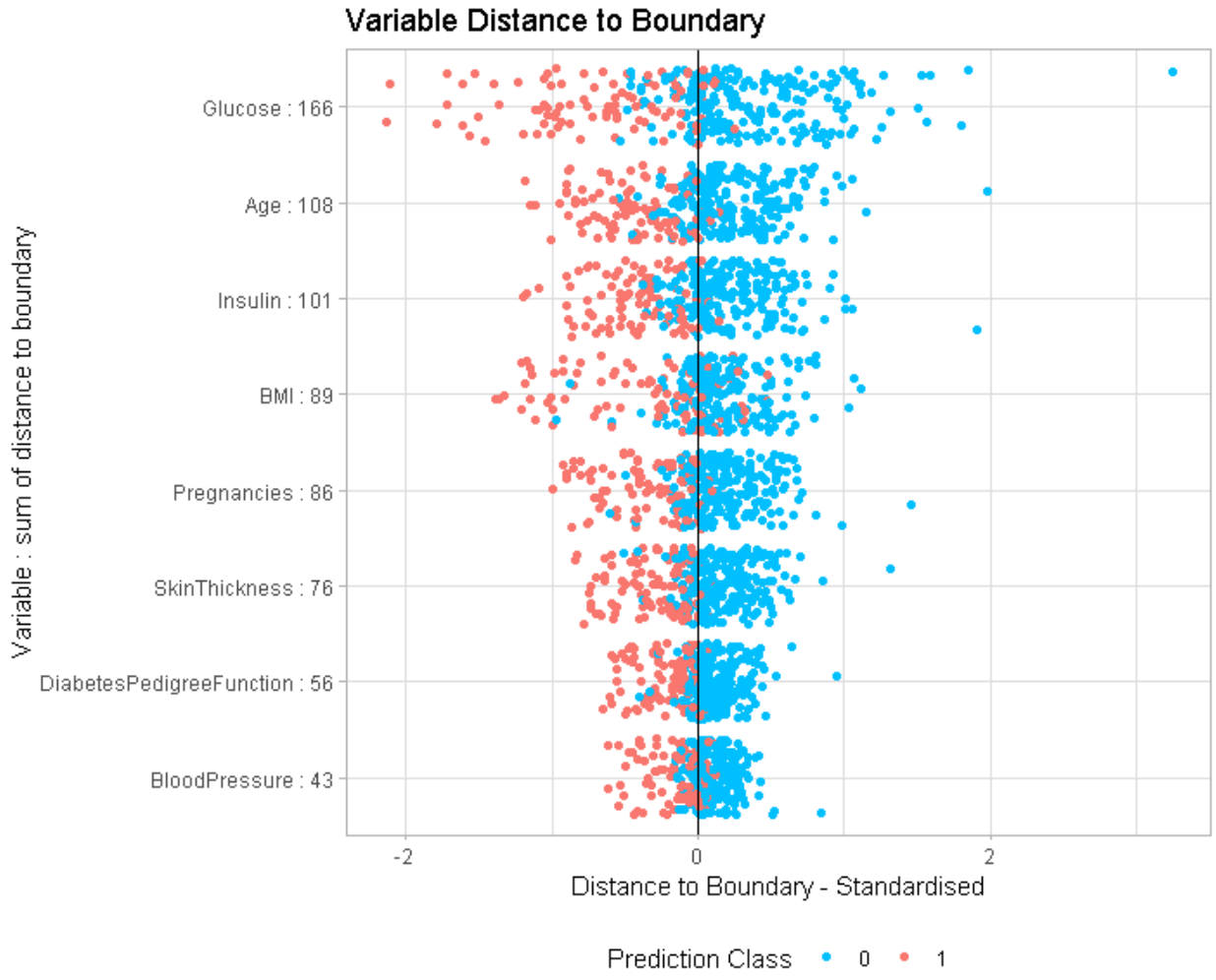


Figure 4.5: Distance-based variable importance plots using the decomposition of the vector $(\boldsymbol{z_{cf}} - \boldsymbol{z_i})\boldsymbol{M_2^-}$*, across all training data points, with each vector standardised by its standard deviation*

The results can be compared against the built-in feature importance plot of the model generated by the GBM model and shown in Figure 4.6.

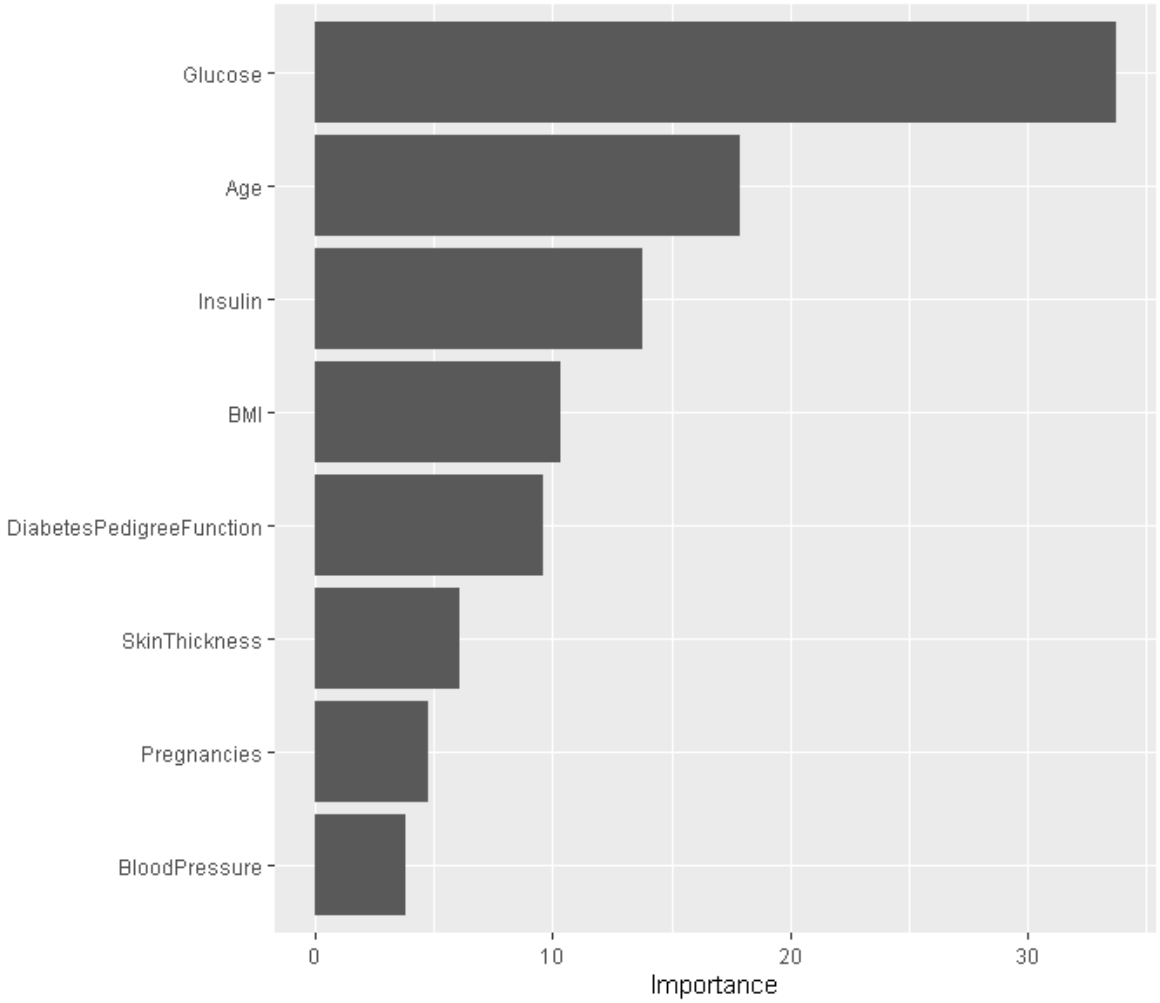


Figure 4.6: Comparison of a traditional feature importance plot vs. the distance-based importance plot

Variables with consistently minimal influence may be removed to simplify the model, producing counterfactuals that are not only sparser but also easier to interpret. Both the jitter plot and the feature importance plot identify BloodPressure as the least important variable.

The code required to calculate the variable importance and robustness score can be found in Appendix A under A5: Global Interpretations of Robustness and Variable Importance.

4.2.3 Biplot measure of the model robustness

Another global interpretation provided by the biplot is an assessment of the robustness of the model. As explained in section 2.6.1, the robustness is measured as the sum of the distance from the data points to the decision boundary. This is calculated as

$$R = \sum_{j} \boldsymbol{T}_j$$

and provides a single-value summary of the variable-based distances discussed in section 4.2.2. This robustness metric enables a meaningful comparison between two models calibrated on the same dataset. This can be either between two similar model calibrated with different hyperparameters or between two different model types.

The biplot representation in figure 4.7. shows the initial GBM model calibrated with 1 000 trees and another GBM calibrated with 5 000 trees, and their test data points and corresponding counterfactuals. The data and axes are slightly different between the two, as the different underlying ML models group the data differently into the four classes used in the CVA transformation matrix $\boldsymbol{M}$.

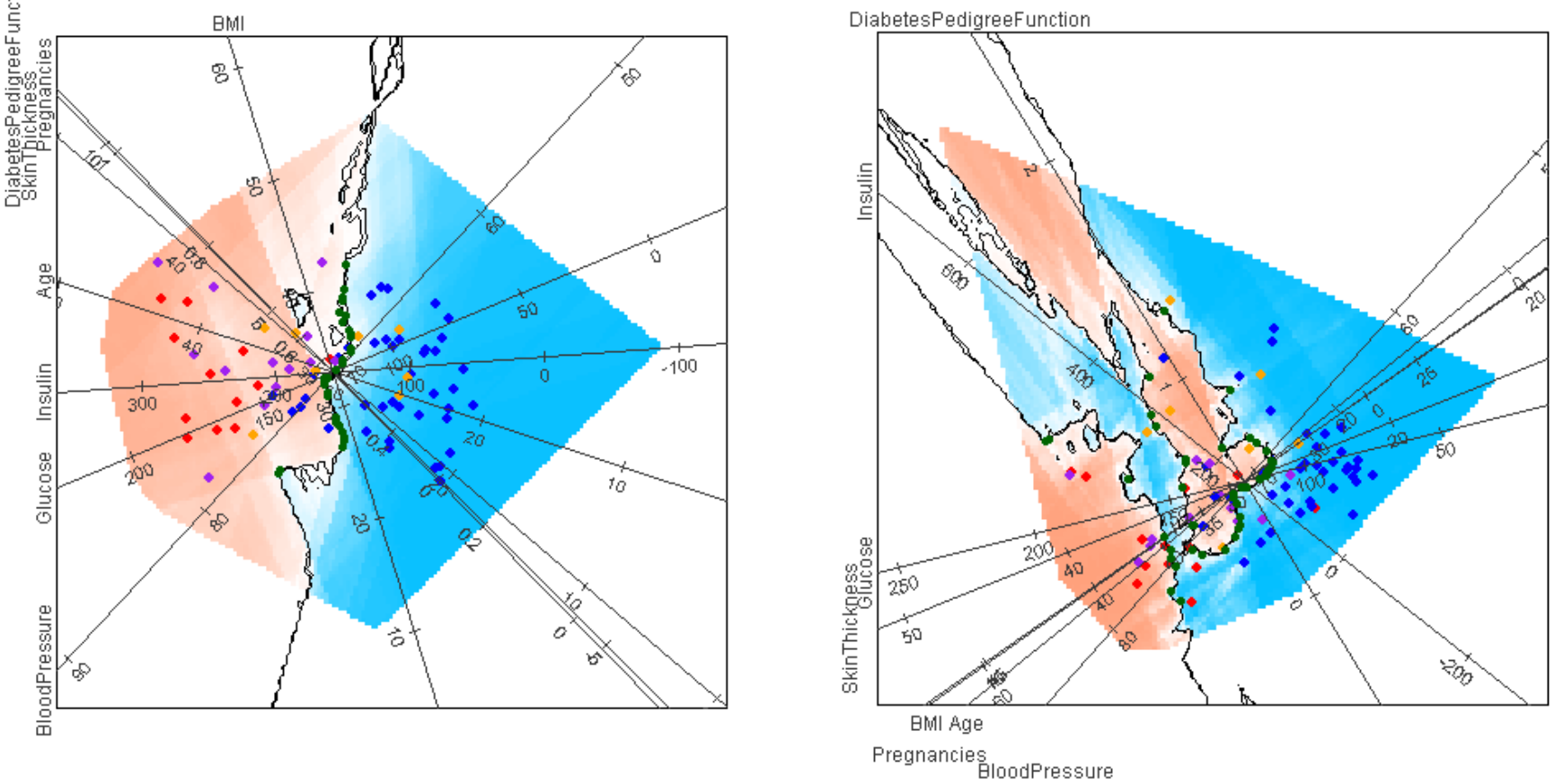


Figure 4.7: Comparison of the local decision boundaries and robustness of two GBMs with 1 000 (above) and 5 000 (below) trees selected for the hyperparameter for number of trees

The GBM model with 5 000 trees has a training accuracy of 99%, but a test data accuracy of only 72%. The total distance of the training data points to their counterfactuals is 450. The GBM model with only 1 000 trees has a training accuracy of 88%, but a test data accuracy of only 71%. The total distance of the training data points to their counterfactuals is 720.

A greater total distance indicates that the GBM model with 1 000 trees is more robust, as it requires larger changes in the input features to alter its predictions. This is reflected in the smoother and more stable shape of the decision boundaries in the biplot shown in Figure 4.7.

Although the training data prediction accuracy of the 5 000 trees model is very high and much better than the 1 000 trees model, the lower robustness score should be an indicator that the model is more prone to overfitting and less likely to generalise well to new data. This is consistent with the many sharp and irregular shapes of the local decision boundaries that suggest a degree of overfitting, indicating that a more regularised model may provide better generalisation to unseen data. Therefore, it is not unexpected that the performance on the test data will be much reduced. It ends up only slightly better than the 1 000 trees model. Based on these insights, the GAM model with 1 000 trees is recommended over the GBM with 5 000 trees for this application.

4.2.4 Biplot as a surrogate model with global fidelity

An alternative approach for interpreting the model using the biplot is to construct a surrogate prediction model based on the nearest local decision boundaries identified on the biplot plane. The contour lines on the biplot, which represent the estimated local decision boundaries (defined as $\boldsymbol{X}_{DB}|_{\hat{g}=0}$ and $\boldsymbol{X}_{DB}|_{\hat{g}=1}$ in section 3.6), delineate regions corresponding to each predicted class: blue for class 0 and red for class 1.

These boundaries can be leveraged to classify new data points based on their projected location on the biplot plane and their proximity to the nearest local decision contour. In the case of a data point located between two contour lines representing $[b \pm \varepsilon]$, the data point will be classified to reflect the class of the nearest contour line representing the nearest local decision boundary. All data points enclosed by the same contour are assigned the same predicted class, as they will be nearest to the same local decision boundary. The effect is to create a surrogate model that approximates the behaviour of the original ML model within the biplot space. Because the biplot provides the best two-dimensional representation of the training data, the surrogate model offers a faithful two-dimensional approximation of the original model. The code required to calculate the surrogate biplot model can be found in Appendix A under A6: Surrogate Models.

Figure 4.8 illustrates the surrogate model created for the GBM classifier, with the test data projected onto the biplot and classified based on the surrounding local decision boundaries. The original GBM model achieved a prediction accuracy of 71%. In comparison, the surrogate model achieved a global fidelity of 65%. The biplot-based surrogate closely replicates the behaviour of the original model in the reduced space. It can potentially improve on the performance of $\boldsymbol{f}(\boldsymbol{X})$ by simplifying the model and avoiding overfitting to the training data.

CVA Biplot based GBM model with its corresponding Surrogate model

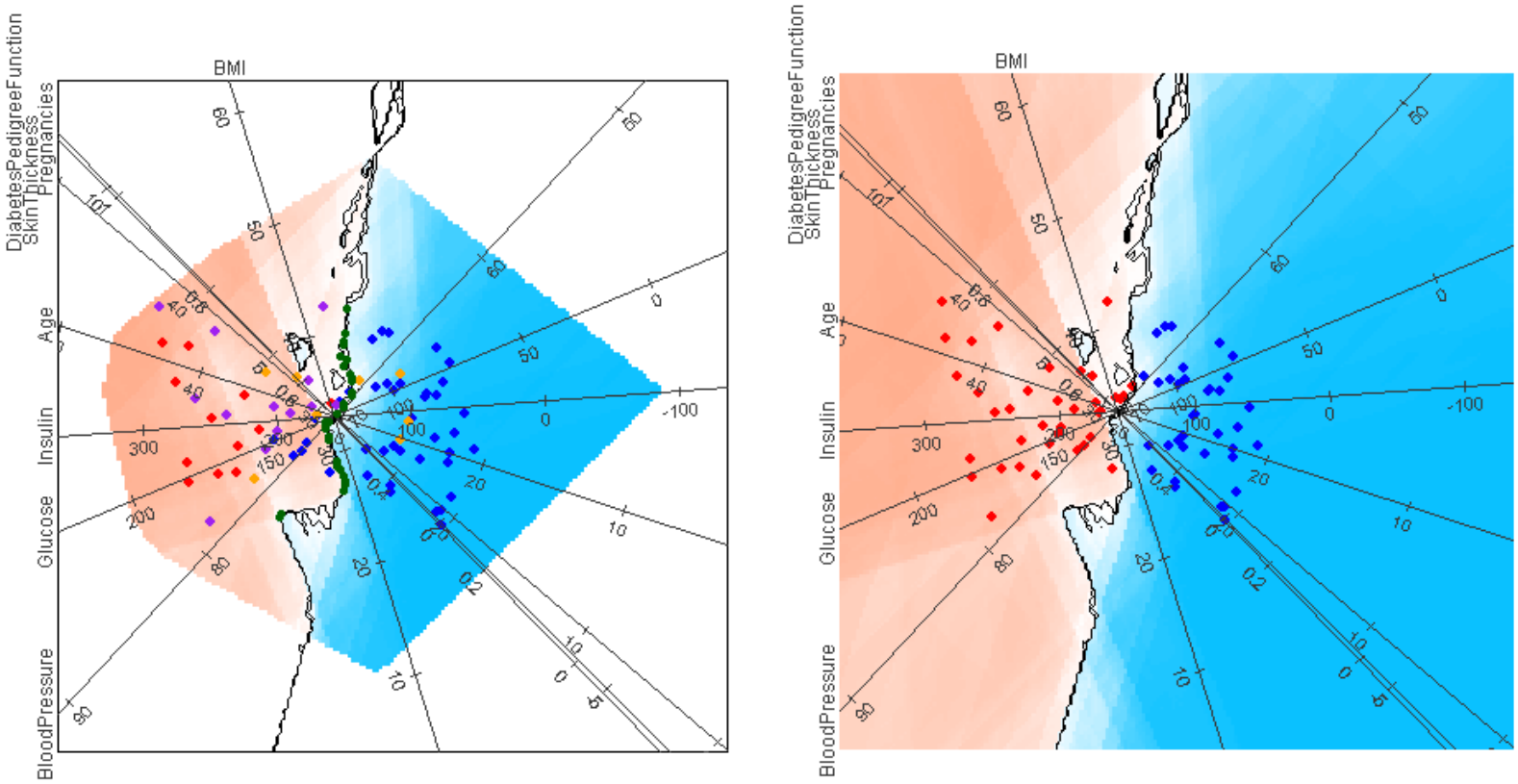


Figure 4.8: The biplot of the GBM model compared to the surrogate model of the GBM model. The surrogate model provides high fidelity and intuitive interpretations by making predictions using only the nearest local decision boundaries.

The resulting data classification aligns with the visualisation presented by the biplot, eliminating distortions that may arise when projecting data points into regions that suggest a different classification than that assigned by $f(\boldsymbol{X})$ in the full dimensional space. This consistency enhances the interpretability of the biplot, as the predictions of the model can be intuitively explained by referencing the variable axes and the direction towards the nearest local decision boundary.

For example, the surrogate model indicates how a reduction for all the variables will reduce the prediction and increase the likelihood of prediction of non-diabetic. Specifically, it shows that the variable axes of Glucose, Insulin and Age are the most perpendicular to the local decision boundary and therefore will have the biggest impact on the prediction for most of the data points. The decision boundary also bends at the centre bottom of the biplot. This impacts data points classified as diabetic whose prediction will change to non-diabetic if their BMI value reduces as they move down on the biplot along the BMI axis. The BloodPressure axis moves parallel to this bend, reducing the relative impact on the prediction outcome as the variable value reduces. This is consistent with the distance to boundary plot shown in Figure 4.5.

The surrogate model offers stable and coherent explanations, avoiding cases where variable axes appear to point in the wrong direction for specific data points. This limitation is sometimes seen in conventional biplot projections. Because it preserves the geometric and predictive structure of the original model in a visually interpretable two-dimensional form, the surrogate model can serve as a transparent alternative to the underlying model in settings where interpretability is crucial. It provides intuitive, localised explanations for each data point, regardless of the complexity or opacity of the original ML algorithm.

4.2.5 Global questions answered

The biplot method gives algorithmic transparency by providing an interpretable two-dimensional representation of both the data and the decision boundaries of the model. The shape of the local decision boundaries can explain potential reasons for overfitting to the training data, without having to rely on the numeric outcomes of cross-validation tests to indicate whether the model will generalise well to new data.

The biplot did not indicate which data points are most influential, but it did help identify potential data errors where the Insulin and SkinThickness variables had too many zero values. Alternative data investigation methods could also have identified these issues. However, the biplot gives a method to visualise the entire dataset that captures the correlations, multivariate

distribution and value of the data points. Further potential out-of-distribution data points were identified, but they were not removed from the dataset.

The biplot indicates that Glucose, Insulin and Age have the biggest impact on the prediction for most of the data points. It recommends the removal of the BloodPressure variable, which results in an improved prediction accuracy from 71% to 74%.

The biplot provided a way to select between different models or model calibrations based on their robustness to change. This provides a way to decide between different options in the case where the prediction accuracy is similar.

The spread and location of the misclassified data points and the shape of the decision boundary can also indicate regions where the model does not predict well. This may require data from this region to be excluded from the model, or require that the prediction model be improved, if data in that area is critical to the application of the model.

4.3 LOCAL INTERPRETATIONS USING A BIPLOT

Local interpretability implies understanding the reasons behind a specific prediction made by a model. This section discusses the use of the biplot to identify the nearest counterfactual value to an individual data point that would alter those predictions and calculate the Shapley values that determine the contribution of each variable to changing the prediction to the desired class. The explanation does not directly explain the internal logic of the model, but provides answers to the following questions:

- How much should the input variable values change to give an alternative prediction, with a related ‘if’ where the input state is constrained in some way or if a different prediction is possible at all? Therefore, what combination of available changes in inputs will result in a change in the prediction?
- How much did each feature contribute to the individual prediction?
- Which factor needs to change the least to change the prediction?

The answers provided by the biplot will reflect the underlying calibration of the ML model being interpreted. The method will always give an explanation, irrespective of the global accuracy of the model or the importance of different variables. However, different models may yield different reasons for the same prediction. As such, the real-world insight provided by this interpretation depends on how well the underlying model captures the actual variable

interactions and their causal impact. This variability is a manifestation of the Rashomon effect, where multiple equally accurate models may offer differing explanations.

To support a meaningful local interpretation, the biplot must identify the nearest valid counterfactual – a point that is not only close in terms of distance, but that also meets the constraints of actionability, feasibility and plausibility given the context of the data point specified in section 3.9.2.

This section explains how the biplot meets these requirements and how the explanation can be refined to produce sparser counterfactuals involving the smallest possible number of changes.

4.3.1 Identifying nearest counterfactual values

The example below demonstrates how the model explains a specific prediction: why a particular patient is classified as having diabetes, and what actionable changes could reduce their predicted risk. The prediction was generated using the GBM model with 1 000 trees.

As suggested in section 2.7.2, the explanation should attempt to limit the number of features under consideration to a maximum of seven to ensure that the interpretation remains both informative and cognitively manageable. The variable BloodPressure is removed, as suggested by the jitter plot created in section 4.2.2. This simplifies the model and its local explanation. By removing variables that do not contribute to the prediction, it avoids the risk of attributing the reason for an individual prediction to a variable that does have a real-world impact on the outcome. Test data points that are outside the convex hull of the training data can also be removed, as they are considered out-of-distribution data points. Care should be taken when making predictions and interpretation for these data points, as the model may be badly calibrated for them. One such point has been removed.

Removing BloodPressure and the one outlier results in a GBM model with a higher test data accuracy of 74%. The total distance of the training data points to their counterfactuals is 752, indicating an increase in the robustness of the model.

The variable values of the target data point that requires a local interpretation are indicated in Table 4.2 below.

Table 4.2: Target data point variable values

Variable	Target data point
Pregnancies	4
Glucose	173
SkinThickness	14
Insulin	168
BMI	29.7
DiabetesPedigreeFunction	0.361
Age	33

The CVA biplot of the updated model, with the target data point in light green, is shown in Figure 4.9. The outlier data point is identified in the top left.

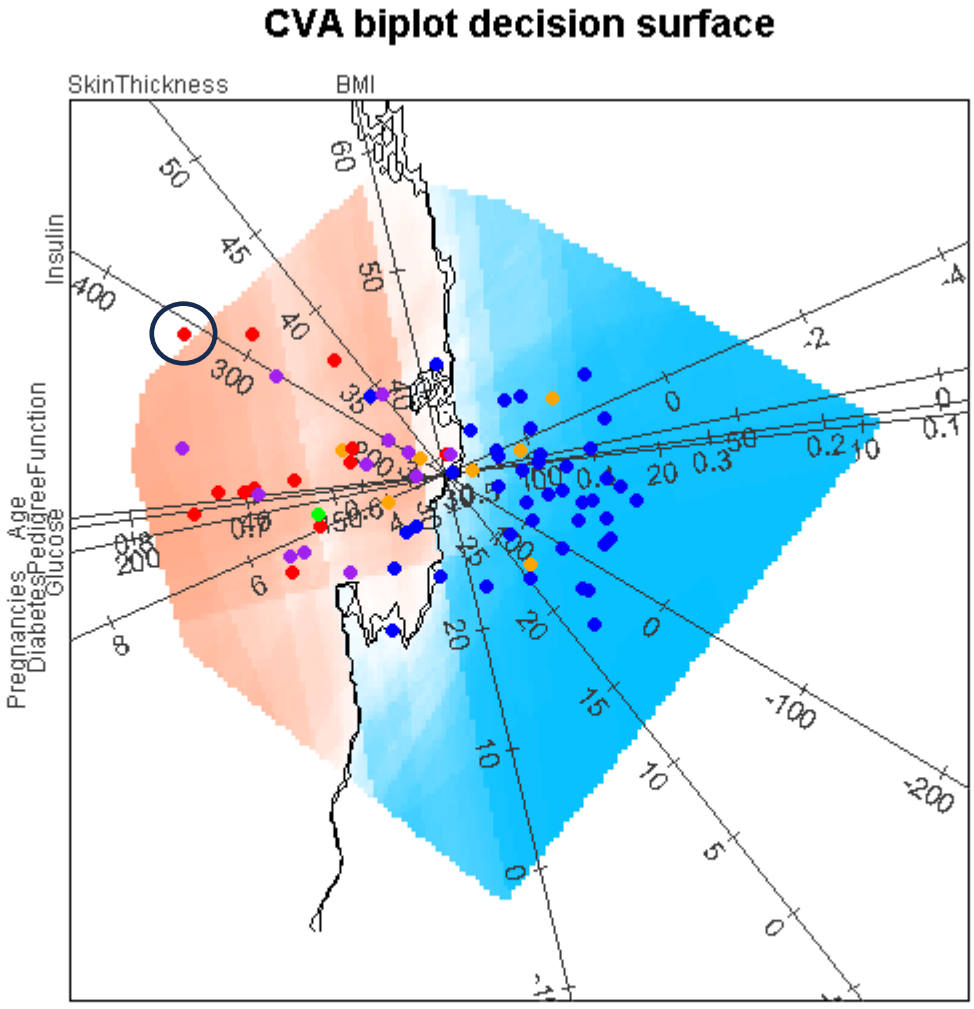


Figure 4.9: CVA biplot of the updated GBM model without the BloodPressure variable and the convex hull of the training data. The target data point is highlighted in green. An out-of-distribution data point is identified in the top left.

The model currently predicts that the target individual has diabetes – it is predicted to be in class 1 with a prediction score of 0.763. The counterfactual is the nearest data point on the biplot plane that is predicted to be in class 0, with a prediction score below 0.5.

To improve the accuracy and stability of the interpretation, the biplot is rotated to ensure that the target data point is on the biplot plane. The rotation of the biplot is explained in section 3.5.2. This results in a new transformation matrix:

$$\boldsymbol{M}^{\text{new}} = \boldsymbol{M}\boldsymbol{A}'.$$

The new RC biplot is created using the first two components of $\boldsymbol{M}^{\text{new}}$.

The nearest valid counterfactual to the target data point is indicated as a dark green circle in Figure 4.10. As the prediction in class 1 is a true positive, it is in the red area of the plane. The counterfactual value is calculated as $\hat{\boldsymbol{x}}_{cf} = \boldsymbol{z}_{cf}(\boldsymbol{M}_2^{\text{new}})^{-}$. If an unconstraint solution is acceptable, then the convex hull of the data should not be used to limit the search of a counterfactual. This will allow the biplot to identify the nearest counterfactual from any local decision boundary, even if those counterfactual values are outliers to the data used to calibrate the biplot.

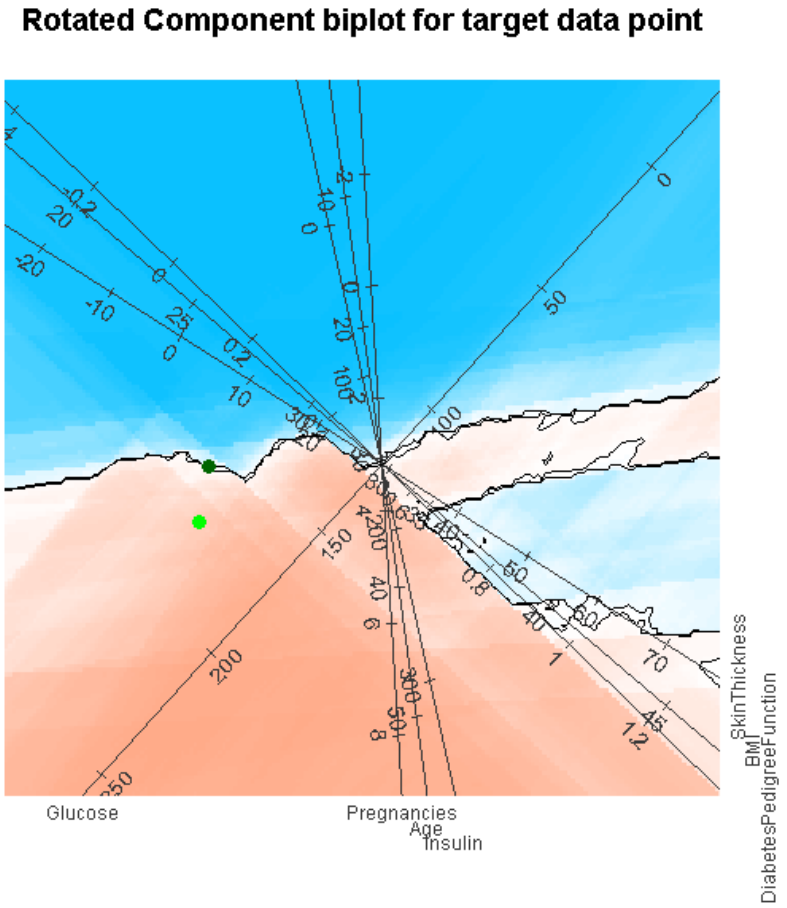


Figure 4.10: Two-class RC biplot for Pima data fit with a CVA biplot to a GBM model. The target data point is indicated in green and its nearest unconstraint counterfactual in dark green.

The nearest counterfactual ignoring the constraints are indicated in Table 4.3.

Table 4.3: The data point and the value of its nearest unconstraint counterfactual

Variable	Target data point	Unconstraint counterfactual
Pregnancies	4	3.05
Glucose	173	158
SkinThickness	14	11.45
Insulin	168	135
BMI	29.7	28.8
DiabetesPedigreeFunction	0.361	0.304
Age	33	29

A better counterfactual is one that meets the constraint that require its values to be actionable, feasible and plausible. For the Pima dataset, the main constraints on actionability are that a patient's number of Pregnancies and Age cannot decrease. Plausibility is ensured by requiring the counterfactuals to fall within the convex hull of the training data, while feasibility requires

that variable values remain within the range observed in the training dataset. These restrictions may result in some of the local decision boundaries shown on the biplot containing no valid counterfactual values.

To generate a counterfactual that meets these constraints, while still selecting the nearest possible counterfactual using the biplot method, the projection matrix is recalculated for all possible two-dimensional biplots formed from pairs of components. Specifically, for p variables, this requires creating:

$$^{p!}/_{2!\,(p-2)!}$$

different combinations of the components, where

$$\boldsymbol{M}_i = \boldsymbol{M}\boldsymbol{J}^*$$

$$\boldsymbol{J}^*: p \times 2 \text{ and}$$

$$i \in \{1, \dots, {}^{p!}/_{2!\,(p-2)!}\}.$$

Here $\boldsymbol{J}^*$ consists of zeros except for ones in the rows corresponding to the selected components. For example, if $p = 3$ and the first and third components are selected, then:

$$\boldsymbol{J}^*: 3 \times 2 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

The transformation matrix is then rotated to ensure that the biplot plane passes through the target data point, producing a new biplot with transformation matrix $\boldsymbol{M}_i^{\text{new}}$. Let $\boldsymbol{z}_i$ represent the projection of the target data point $\boldsymbol{x}$ and $\boldsymbol{z}_{i,cf}$ the corresponding counterfactual value identified in the biplot of the i'th combination of the components of the transformation matrix. The counterfactual must be selected from a local decision boundary that is within the convex hull of the training data. The counterfactual in the original feature space is then given by $\widehat{\boldsymbol{x}}_{i,cf} = \boldsymbol{z}_{i,cf}(\boldsymbol{M}_i^{new})^-$. The nearest counterfactual $\widehat{\boldsymbol{x}}_{cf}$ is then selected as the counterfactual that corresponds to the shortest change vector $\boldsymbol{d}_i$ measured as the Mahalanobis distance of the change in the original p-dimensional space:

$$\boldsymbol{d}_i = (\boldsymbol{z}_{i,cf} - \boldsymbol{z}_i)(\boldsymbol{M}_i^{\text{new}})^- \text{ and}$$

$$\widehat{\boldsymbol{x}}_{cf}: \min_i(\boldsymbol{d}_i' \boldsymbol{W}^{-1} \boldsymbol{d}_i).$$

$\boldsymbol{W}$ is the total within-group SSP, as discussed in section 3.4.

The code required to calculate the biplot with different eigenvector can be found in Appendix A under A7: Biplot Boundary Search with different Eigenvalues.

Not all of the i biplots generated will yield counterfactuals that meet the constraints. If none of the biplots can generate a solution, it is assumed that no constraint counterfactual exists for this data point.

For a model with seven variables, this procedure yields 21 different biplots (all possible two-component combinations). These resulted in 11 constraint counterfactual values. In this example, the nearest counterfactual was created from the biplot generated by components numbers 1 and 7. This is shown in Figure 4.11 with the selected counterfactual highlighted in green.

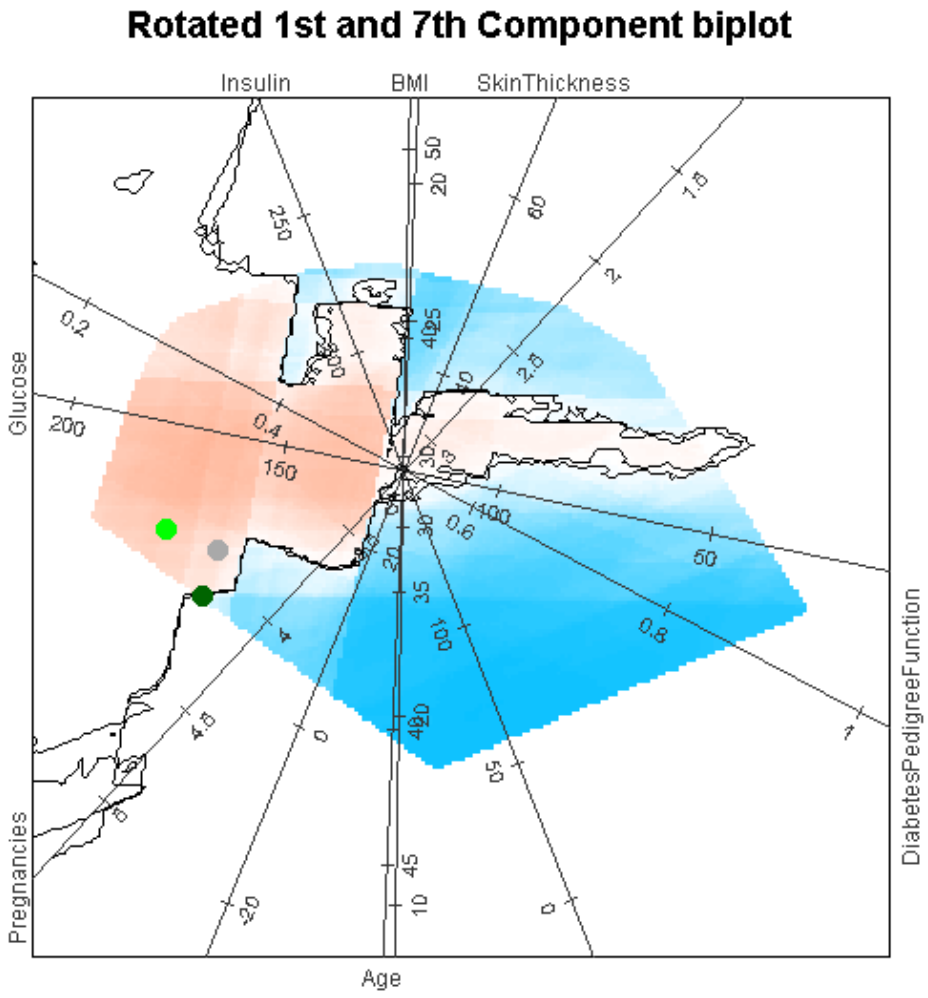


Figure 4.11: RC biplot generated by rotating the first and seventh components of the CVA biplot to pass through the data point and identifying the nearest constraint

counterfactual. The counterfactual is within the convex hull of the training data. A sparser (and therefore nearer) solution is indicated in grey.

The nearest counterfactuals that meet the constraints are indicated in Table 4.4.

Table 4.4: Target data point variable and unconstraint and constraint counterfactual value solutions

Variable	Target data point	Unconstraint counterfactual	Constraint counterfactual
Pregnancies	4	3.05	4.11
Glucose	173	158.5	162
SkinThickness	14	11.56	8.92
Insulin	168	135.39	142.5
BMI	29.7	28.83	26.22
DiabetesPedigreeFunction	0.361	0.31	0.42
Age	33	29	35.38

The unconstrained counterfactual requires a reduction in both the number of Pregnancies and the patient's Age, which are not actionable variables.

The constrained counterfactual is both actionable and feasible, but it implies that the number of Pregnancies increases to 4.11 and Age to 35.38. Importantly, the biplot cannot identify counterfactual solutions that are precisely whole numbers. This is required to ensure both Pregnancies and Age remain unchanged. This limitation, and how it may be addressed, is discussed in the next section.

Once the counterfactual is identified, Shapley values can quantify the contribution of each variable to the prediction of the model and help answer the local interpretation questions listed at the start of this section.

4.3.2 Shapley values

In the original discussion in section 2.9.1, it was discussed how Shapley values explain the prediction via the difference between the prediction $y = f(x)$ and the global average prediction, with:

$$f(x_i) = \theta_0 + \sum_j^M \theta_j \text{ where } \theta_0 = E[f(x)].$$

If the counterfactual value is known, then this expression can be changed to:

$$f(x_{cf}) = f(x_i) + \sum_j^M \theta_j \text{ where } f(x_{cf}) \sim b.$$

Equation 8: Shapley values for change from data point to counterfactual

Therefore, the Shapley value θ_j calculates the exact contribution from each of the $\boldsymbol{M}$ variables in $\boldsymbol{x}$ as the data point changes from the current value to its counterfactual value. The sum of the Shapley values for all features is equal to the difference between the prediction $f(x)$ and $f(x_{cf})$, where the impact on the prediction is fairly distributed among the input features. It remains unchanged regardless of the order in which the variables are changed.

The Shapley value θ_j gives the impact of the change in the j'th variable $\boldsymbol{x}_{cf,j} - \boldsymbol{x}_{i,j}$. If $f(x_i) < b$ then $f(\boldsymbol{x}_{cf}) \geq b$ and $\sum_j^M \theta_j > 0$. When $f(x_i) > b$ then $f(\boldsymbol{x}_{cf}) \leq b$ and $\sum_j^M \theta_j < 0$. The values of the individual θ_j can be positive or negative. Therefore, depending on whether $f(x_i)$ is greater or lesser that b, the changes in the variable values moving you closer to or further from the boundary. If the individual θ_j moves you closer to the boundary, then the change in the j'th variable $\boldsymbol{x}_{cf,j} - \boldsymbol{x}_{i,j}$ supports the change. If the individual θ_j moves you further from the boundary, then the change in the j'th variable $\boldsymbol{x}_{cf,j} - \boldsymbol{x}_{i,j}$ contradicts the change. This can be summarised as follows:

- If $f(x_i) < b$ and $\theta_j > 0$, then the change in $\boldsymbol{x}_{cf} - \boldsymbol{x}_i$ supports the change towards the decision boundary. When $\theta_j < 0$, the change in the variable contradicts the change towards the decision boundary.
- If $f(x_i) > b$, then $\theta_j > 0$ contradicts the change and $\theta_j < 0$ supports the change towards the decision boundary.

Removing the contribution of the contradicting variables reduces the change needed to reach the decision boundary. Therefore, only changing the variables with combined Shapley values equal to the change needed for $f(x) \rightarrow b$ reduces the distance to the decision boundary and the number of variables that need to be changed. This will improve the approximation of the counterfactual value provided by the biplot and give a sparser explanation of the change necessary to reach the decision boundary. This explanation may be less plausible given the distribution of the data.

The Shapley value θ_j can be plotted with the variables ordered from largest to smallest supporting variables, and then from smallest to largest contradicting variables. The length of each bar indicates the impact on the prediction outcome caused by the movement towards

the counterfactual. The change in the variable vector $\boldsymbol{x}_{cf} - \boldsymbol{x}_j$ is indicated next to each of the variable values on the y-axis of the plot.

The Shapley value is calculated by first determining every possible combination of the order of the change in the $p = 7$ variables. This is equal to $p! = 7! = 5040$ different combinations. For each of these orders, the p variable values are changed from the data point value to that of the counterfactual, and the change in the prediction value is measured after each variable change. Therefore, $p! * p$ changes and the prediction value calculated at each step is recorded. The Shapley score is the average contribution to the change from each variable across all possible combinations of orders. As the contribution is calculated in each order possible, it removes the impact of the order in the calculation of the contribution.

The code required to calculate the Shapley values can be found in Appendix A under A1: Shapley value calculation and A8: Local Interpretations with Shapley values. The Shapley values for the example data point are visualised in Figure 4.12.

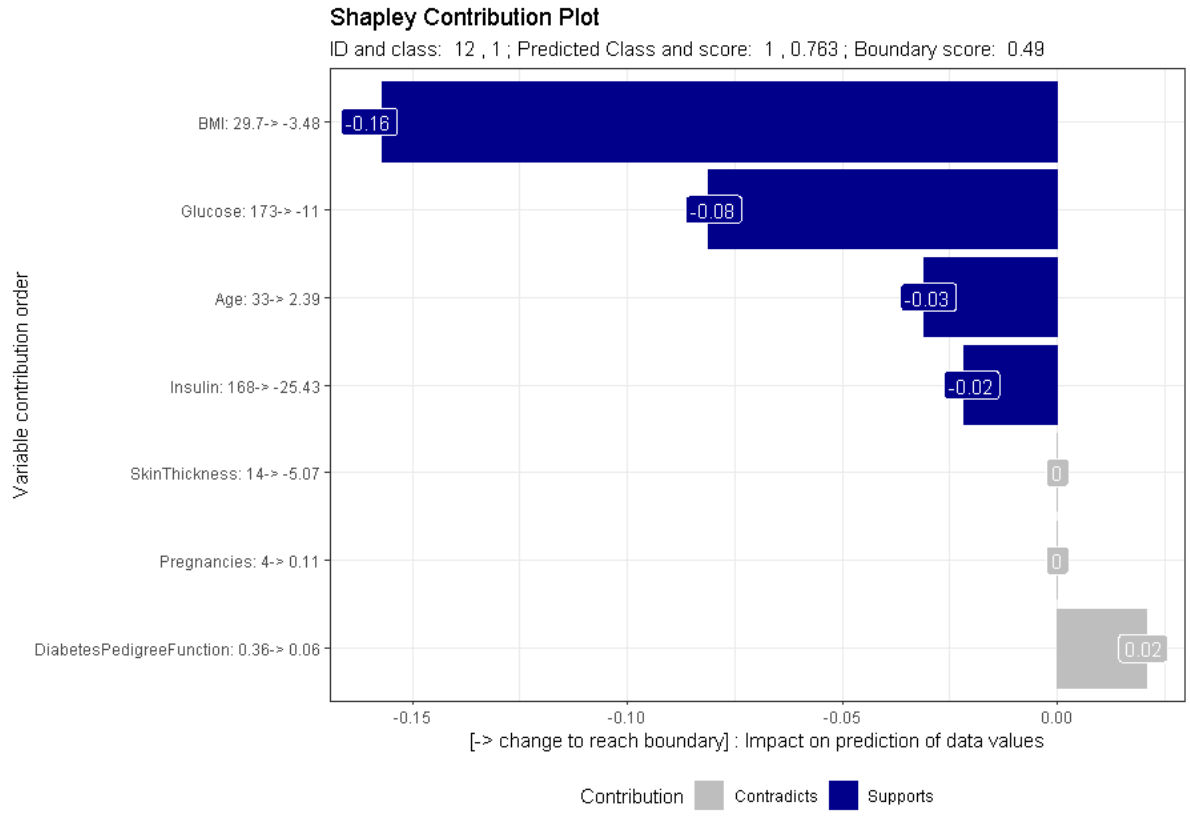


Figure 4.12: Shapley values for the target data indicate the contribution of each variable to the prediction outcome

4.3.3 Local questions answered

The Shapley values indicate how much each variable contributes to the prediction, as well as whether the change in its value supports or contradicts the adjustment needed to cross the decision boundary and alter the prediction outcome. Keeping values that contradict the desired class as unchanged helps identify a sparser and closer counterfactual.

In this case, the patient's BMI is the most significant contributor to the diabetes prediction. Reducing BMI from 29.7 to 26.2 has the largest effect on lowering the prediction score. On its own, however, this change is insufficient to reverse the outcome. It has to be combined with change in Glucose, Insulin and Age.

While Insulin shows a substantial shift from the original value to the counterfactual, its marginal contribution to reducing the prediction is small. The counterfactual adjustments for Pregnancies, SkinThickness and DiabetesPedigreeFunction contradict the desired outcome and can therefore be excluded to yield a sparser counterfactual.

Consequently, changing BMI, Glucose, Insulin and Age shifts the classification from diabetic to non-diabetic. The number of Pregnancies can remain unchanged. This counterfactual is actionable, plausible and feasible. The sparser counterfactual solution is also nearer to the data point, as indicated in Figure 4.11 in grey. However, the value is not on the biplot plane and hence is located in the incorrect area of the biplot when projected onto the plane. This solution is part of the decision boundaries in the full-dimensional space.

This interpretation is consistent with the direction of change visualised on the biplot in Figure 4.11, where the movement from the original data point to the counterfactual occurs primarily along the BMI, Insulin and Glucose axes. The minimal effect of SkinThickness and Pregnancies is also consistent with their axes being almost parallel to the nearest local decision boundary, which is in line with the global interpretation in Figure 4.5.

The calculation of Shapley values using the biplot counterfactuals differs from the approach used in SHAP in two ways:

- SHAP calculates the Shapley values relative to the expected value $E[f(\boldsymbol{x})]$. This is because the method does not provide a way to calculate the actual counterfactual value. Therefore, SHAP will always approximate the contribution of each variable to the predicted value $f(x_i)$. The biplot contribution calculates the Shapley value towards an actual counterfactual value, even if a closer counterfactual value may be possible.

- SHAP calculates the change from the expected value to the predicted value $f(\boldsymbol{x}_i)$. In most cases, the expected value is set as $\theta_0 = E[f(\boldsymbol{x})] = 0.5$. The biplot counterfactual explanation focuses on the changes needed to change from the predicted value $f(\boldsymbol{x}_i)$ to the counterfactual value $f(x_{cf})$. The SHAP value does not refer to the nearest counterfactual for the target data point, but to a global baseline assuming nothing is known about the data point.

The Shapley explanation provided here performs well on metrics measuring the accuracy of local individual prediction explanations:

- Rotating the data to ensure the data point is on the biplot plane ensures consistency and local fidelity between the biplot visualisation and the decision boundary and counterfactual value.
- The biplot provides stability in the interpretation of the predictions. Identical objects have identical explanations, similar objects have similar explanations and non-identical objects do not have identical explanations.
- Redundant variables can be excluded from the explanation by identifying variables that have marginal or contradicting impact on the prediction outcome and keeping their values unchanged in the final counterfactual.

4.4 CONCLUSION TO THE BIPLOT EXPLANATIONS

This chapter demonstrated how biplots can be extended beyond their traditional use in multivariate data visualisation to provide a novel framework for generating and interpreting counterfactual explanations in ML models. By projecting data points and decision boundaries onto a lower-dimensional plane, the method gives both global and local insights into model behaviour, while preserving much of the structure of the original high-dimensional feature space.

At the global level, the method illustrates how variables interact to form decision boundaries, identifies which variables contribute most strongly to predictions and offers robustness metrics based on distances to counterfactuals. This highlights where the underlying model generalises well, where it may overfit and how stable its decision boundaries are. It also enables the construction of surrogate models, providing an interpretable approximation of complex black-box classifiers.

At the local level, the method explains individual predictions by identifying the nearest actionable, feasible and plausible counterfactuals. The decomposition of change vectors using

Shapley values along variable axes quantifies the contribution of each feature and provides clear and intuitive insights into which features must change, by how much and in which direction, in order to alter the prediction. Together, the biplot and counterfactual framework address both the 'why' and the 'what if' aspects of interpretability.

Several limitations were also highlighted. First, dimensionality reduction necessarily results in an approximation of the full decision surface, which may cause local inconsistencies if the target data point is not aligned with the biplot plane. This was addressed through rotation, ensuring local fidelity between the biplot and the decision boundary of the model at the target point. Second, unconstrained counterfactuals may suggest unrealistic or unactionable changes. This was mitigated by restricting counterfactuals to remain within the convex hull of the training data and by enforcing actionability and feasibility constraints. However, this may result in the biplot not being able to identify any counterfactual that meets these requirements. Finally, the method currently applies only to binary classifiers with continuous variable and tabular data. Extensions to multiclass and regression settings are feasible. Including categorical variables will require extending the technique to categorical PCA biplots. The linear variable axes can also be modified to reflect non-linear axes.

Overall, this chapter developed a model-agnostic interpretability framework that balances clarity, efficiency and rigour. The biplot method not only enables visual and intuitive interpretations of the behaviour of the model, but also interpretations that score well on various metrics, including fidelity, completeness, robustness and stability. The interpretations also assist in improving the prediction accuracy of the models.

This study extends beyond the ideas developed by Sohns *et al*. (2023) that were discussed in section 2.9.3, with the following improvements and additional features:

- Calibrated variable axes: Instead of relying on Gabriel's (1971) original variable vectors, this study used calibrated variable axes. This significantly enhances interpretability, as both the data point and its counterfactual can be projected directly onto the axes. Variable values can then be read directly, making it clearer which changes are required to alter a prediction.
- Mahalanobis distance for counterfactual selection: Distances between data points and counterfactuals are measured in the original feature space using Mahalanobis distance, which accounts for correlations between variables. At the same time, the $L2$ distance on the biplot plane is used to ensure consistency with the visual interpretation of the plane. This dual approach balances statistical rigour with interpretability.

- Optimal rotation of the biplot plane: A novel rotation method ensures that the biplot plane passes through the target data point. This guarantees that the biplot interpretation is locally consistent with the prediction of the model, making the explanation stable and ensuring that the change vector points in the correct direction.
- Exploring all combinations of the components of the projection matrix: The method systematically constructs biplots using all pairs of components, thereby approximating different slices of the decision surface. This makes it possible to identify the nearest counterfactual solution across all components and to incorporate actionability, feasibility and plausibility constraints.
- Global and local interpretations via counterfactuals: Local decision boundaries and counterfactuals are used to provide both local and global interpretations. Global interpretations are strengthened by excluding counterfactuals that lie outside the convex hull of the data or in empty regions, ensuring that explanations remain grounded in the observed data distribution.
- Robustness score: A robustness metric is introduced, based on the average distance from data points to their counterfactuals. This enables comparison between models with similar predictive accuracy and helps select the model most resilient to small perturbations in input features.
- Surrogate model for global interpretability: A global surrogate model with high fidelity is constructed using the local decision boundaries. This provides a visual approximation of the underlying model, capturing its global decision logic. In cases where the original model has overfitted, the surrogate model may even yield more accurate predictions by simplifying the decision boundaries to those on the two-dimensional plane.
- Shapley values for local explanations: Shapley values are integrated to provide local-level explanations, quantifying the contribution of each variable to an individual's prediction and identifying which features must change to alter the predicted class.

In summary, the biplot-based framework presented here offers a fast, accurate and intuitive method for generating counterfactuals and producing both global and local model-agnostic interpretations. By combining statistical robustness with visual clarity, the method addresses key limitations of prior approaches and enhances the practical interpretability of ML models.

Chapter 5 gives a practical example to demonstrate how to select a model using the global interpretation methods and interpret individual predictions using the local interpretation methods developed in this chapter.

CHAPTER 5 APPLICATION FOR INTERPRETING LOAN DATA

5.1 INTRODUCTION

The biplot method was applied to model and interpret the loan default data, which can be found on Kaggle at https://github.com/TSMathi/loan_approval_analysis/tree/main.

This chapter follows a similar order to that of Chapter 4. First an ML model was calibrated and the variables selected using the global interpretations provided by the biplot. Then local interpretations were generated for individual data points.

The dataset contained 45 000 entries with 13 input variables that were numeric, integer, binary or categorical variables. The target variable was loan default, with 10 000 entries equal to 1, or as a defaulted loan. The variables are listed in Table 5.1.

The training and testing data were randomly split 80:20. The biplot method developed in this study can only accommodate continuous variables data or binary categorical data types. Therefore, the person_education, person_home_ownership and loan_intent variables were excluded from this analysis. The personal_gender and previous_default variables were converted to 1 and 2 integer values.

Table 5.1: Variables in the loan default dataset

Feature	Data type
person_age	Integer
person_gender	Binary (1 - Female / 2 - Male)
person_education	Categorical (five levels)
person_income	Numeric
person_emp_exp	Integer
person_home_ownership	Categorical (four levels)
loan_amnt	Numeric
loan_intent	Categorical (six levels)
loan_int_rate	Numeric
loan_percent_income	Numeric
cb_person_cred_hist_length	Numeric
credit_score	Integer
previous_defaults	Binary (1 - No / 2 - Yes)
Loan Default	Binary (0/1)

5.2 GLOBAL INTERPRETATION OF THE GBM MODEL

The initial analysis applied a PCA biplot to visualise the training data together with the 10 calibrated variable axes. The PCA biplot is shown in Figure 5.1. The colours represent the outcome of the target variable in the training data, with blue indicating no loan default (a value of 0).

The biplot highlights several outliers along the personal_emp_exp, person_age and cb_ person_cred_hist_length axes. These outliers may bias the model calibration and could therefore be excluded prior to fitting the ML model. These variables are very closely correlated and their axes cannot be distinguished on the biplot.

In this example, all data points lying outside the 90% convex hull of the training data were removed. However, the convex hull fraction was adjusted to balance the trade-off between retaining sufficient data coverage and improving the accuracy of the calibrated prediction model.

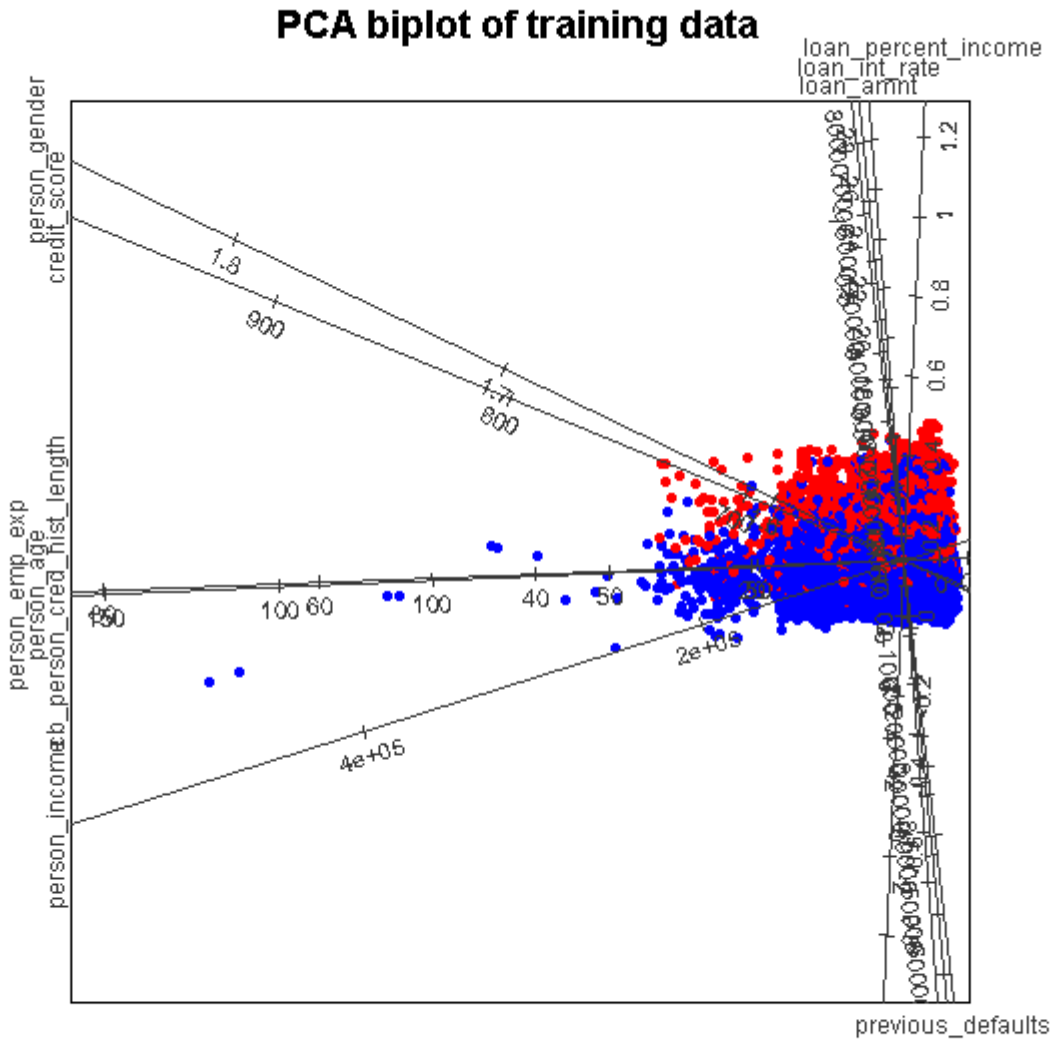


Figure 5.1: PCA biplot of the loan default training data, with the training data indicated in red and blue, for default and no default. It identifies outliers in the data that could be removed to improve the accuracy of the model calibration.

Next, an initial GBM with 1 000 trees was calibrated on the training data after the removal of outliers. A CVA biplot was then constructed, using the prediction outcomes of the model to classify the data into four groups corresponding to true positives, true negatives, false positives and false negatives. The initial CVA biplot is shown in Figure 5.2.

The figure illustrates a clear separation in the data, which is primarily driven by the variable previous_default. When this variable takes the value Yes (or 2), the model correctly predicts non-default in most cases, as reflected by the cluster of blue-coloured points (class 0 or no loan default).

Although this classification could also have been represented with a PCA biplot, the CVA biplot provides stronger separation between the group means by construction, and is therefore more effective in illustrating the role of previous_default in driving prediction outcomes.

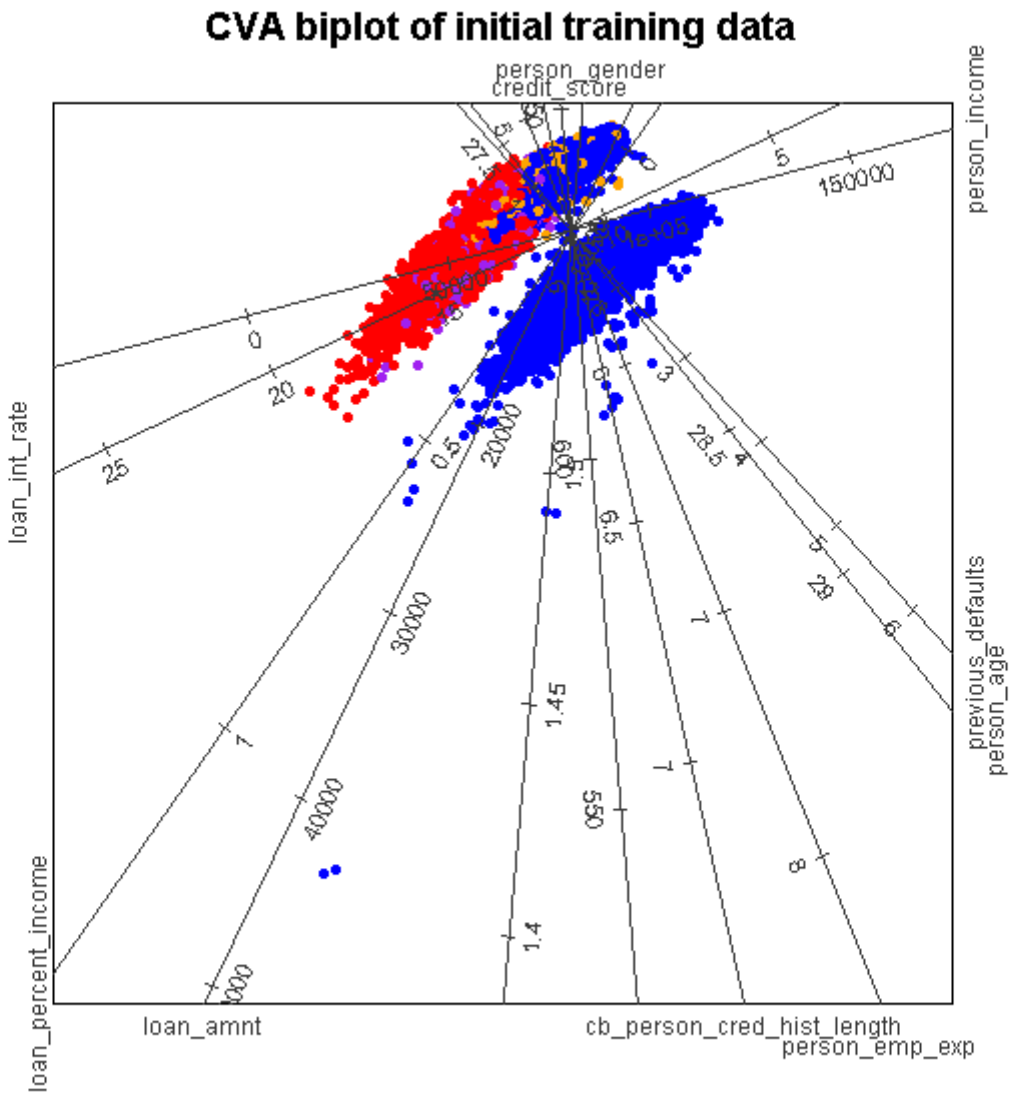


Figure 5.2: Initial CVA biplot of the training data with the data points coloured based on the value of the target variable. It visualises the impact of the previous_defaults variable on the prediction model outcome.

The training dataset initially consisted of approximately 36 000 data points. To reduce the distortion caused by the variable previous_default, both the variable itself and all data points with a previous_default value of 2 were removed. This step reduced the dataset by nearly 50%. In addition, outliers lying outside the 90% convex hull were excluded. These adjustments resulted in a CVA biplot (shown in Figure 5.3) based on the remaining nine variables and ~15 000 training data points.

The number of training data points removed as outliers was determined by the classification results of the model $f(X)$. The model assigned each data point to a group, and these groupings were used to calculate the transformation matrix for the CVA biplot. This, in turn, determined the positioning of the data points on the biplot and highlighted the outliers, which were

removed prior to retraining the model on the filtered dataset. If necessary, this process can be repeated iteratively, with each cycle refining the dataset and the resulting biplot. However, only outliers based on the initial CVA model were removed in this exercise.

The updated biplot provides a clearer view of the decision boundary, separating regions where the model predicts no loan default from those predicting loan default. Some remaining outliers are still visible, but in this example, they were not removed.

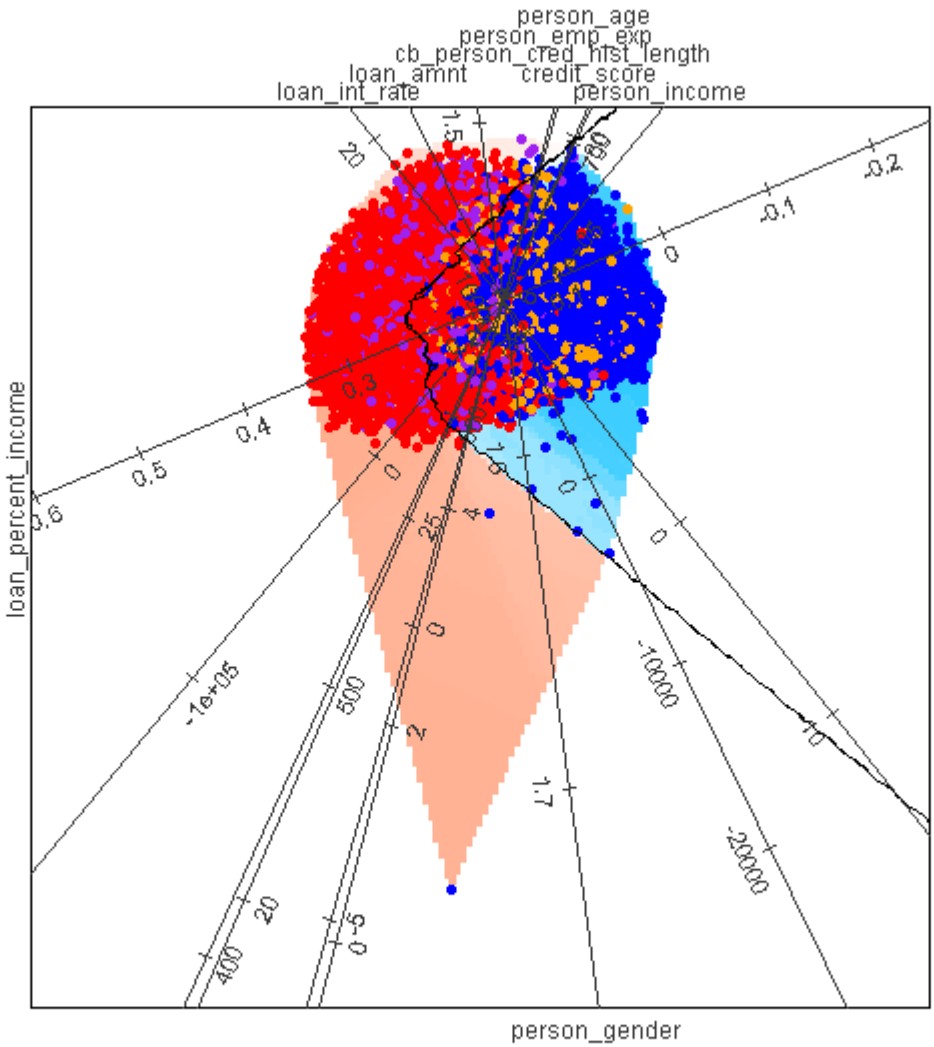


Figure 5.3: CVA biplot of the training data, with the contour lines representing the local decision boundaries used to generate the global counterfactual values

The biplot and its local decision boundaries were used to generate the global counterfactual value for each data point and calculate the distance to the counterfactual along each variable axis. This generated the distance-based variable importance plot shown in Figure 5.4.

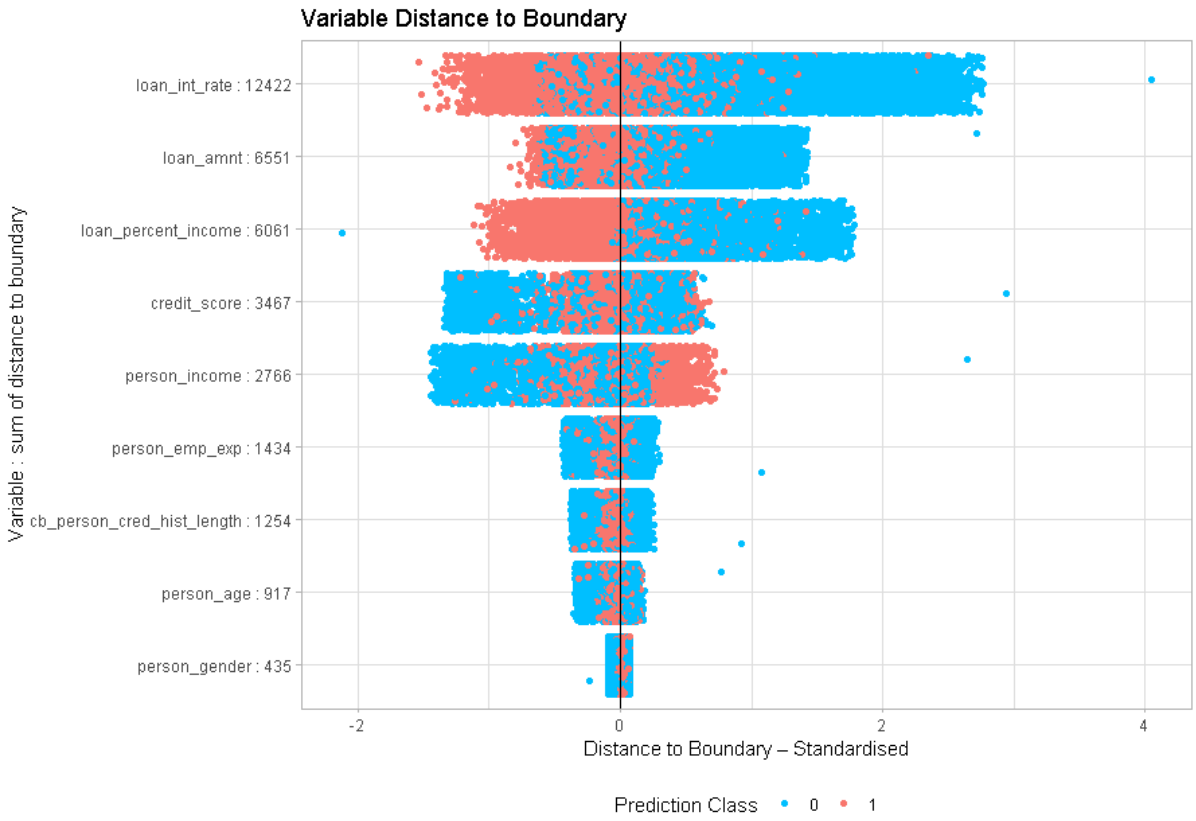


Figure 5.4: Global variable importance plot using the distance to the boundary along each variable axis. The five axes representing the shortest distance to the decision boundary will be removed.

The four variables that represent the shortest average distance for the data to reach the decision boundary are person_gender, cb_person_cred_hist_length, person_emp_exp and person_age. These variables therefore contributed the least to the predictive accuracy of the model and could be removed. In addition, the variable person_income was excluded due to its collinearity with loan_amount and loan_percent_income. As person_income had the lower importance measure, it was removed in constructing the final prediction model.

This recommendation is consistent with the results of the permutation-based variable importance plot (not shown). As the predictive accuracy on the test data remains unchanged irrespective of which of the two is removed, person_income was chosen for exclusion. After recalibrating the GBM model on the training dataset with outliers and low-importance variables removed, a new CVA biplot was generated. The model achieved 80% prediction accuracy on the updated training data.

The prediction accuracy of the GBM on the test dataset with four variables removed and no outlier filtering was 80.6%. The slight improvement in accuracy following outlier removal reflects the benefit of calibration – data points that are trivially easy to classify correctly are

excluded from the training dataset, but remain in the test dataset. The final GBM, calibrated on the refined training dataset and applied to the test data, is illustrated in Figure 5.5.

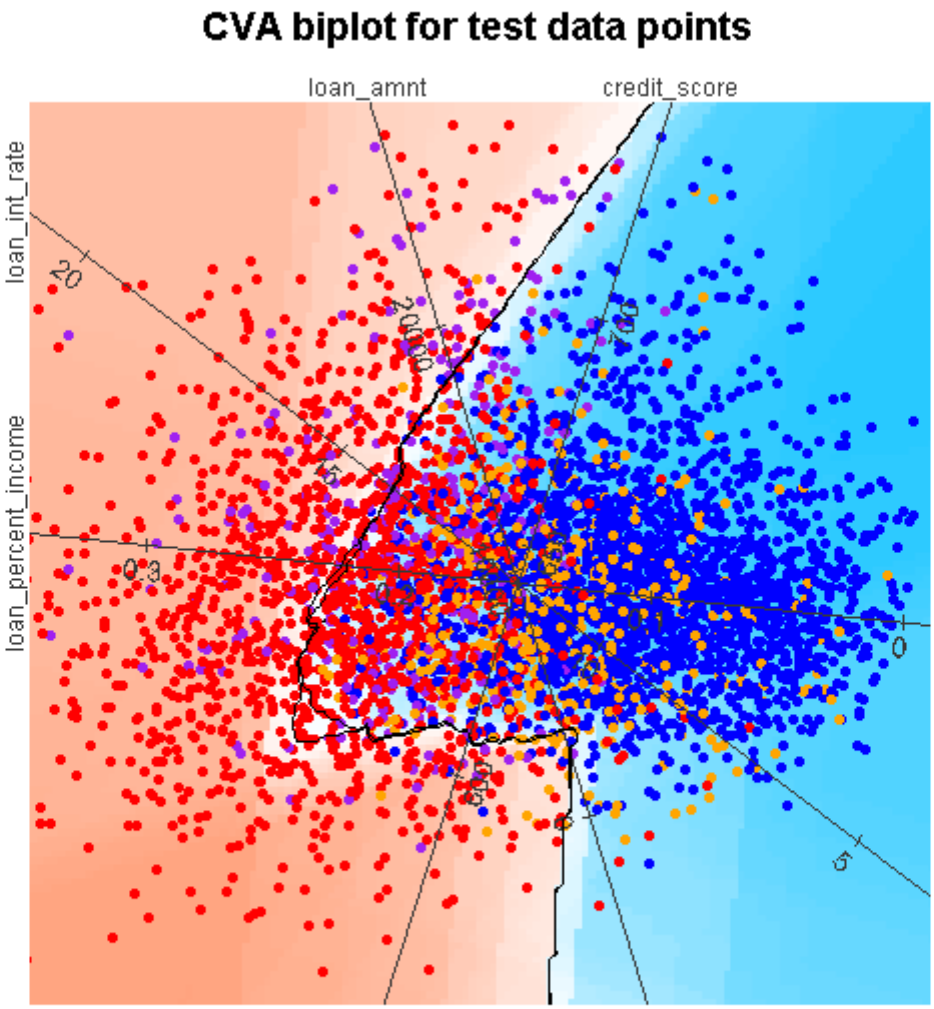


Figure 5.5: CVA biplot of the test data, with both the biplot and the GBM model calibrated on the training data. The colour of the data points reflects their prediction outcome.

The robustness of this final GBM model cannot be compared with earlier versions, as the number of variables had been reduced. Robustness comparisons are only valid across models calibrated on the same feature set.

The biplot can also be used to construct a surrogate global model, shown in Figure 5.6. The surrogate model classifies each data point based on its location on the biplot plane and the class of the nearest local decision boundary. This allows predictions to be interpreted visually, by observing the direction of change required to cross the decision boundary along each variable axis.

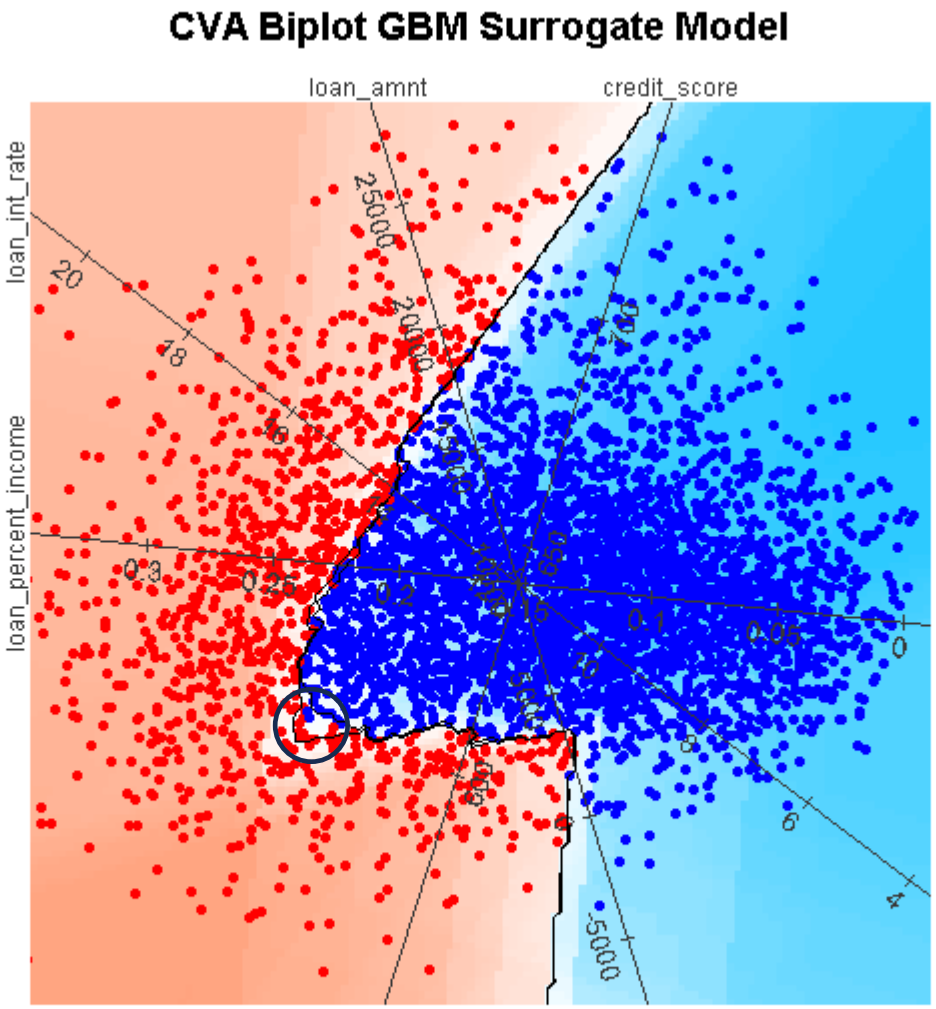


Figure 5.6: The global surrogate model of the GBM model provides an intuitive interpretation of how the predictor variables influence the classification outcome

In the case of the corner in the local decision boundaries indicated by the circle in the bottom left, the gap between $[b \pm \varepsilon]$ is relatively large. The test data points that fall within the gap were classified based the category represented by the nearest contour line.

The fidelity of the surrogate model was evaluated by comparing its accuracy with that of the GBM. In this case, the surrogate model achieved 72.6% accuracy, compared to 80.7% for the GBM.

Despite the reduction in accuracy, the surrogate model offers valuable global interpretability. For instance, it demonstrates that reductions in loan interest rate (loan_int_rate), loan-to-income ratio (loan_percent_income) and loan amount (loan_amnt) generally led to a prediction of non-default, provided the credit score (credit_score) exceeded 610. Below this

threshold, significantly larger reductions in the other variables were needed to maintain a prediction of non-default.

The loan interest rate (loan_int_rate) and loan-to-income ratio (loan_percent_income) variables were the most perpendicular to the largest section of the local decision boundary. Therefore, one would expect that in most individual cases, these two variables would be the most important reason for the prediction outcome. This is confirmed in the next section, where individual explanations are determined.

The analysis also revealed that loan-to-income ratio and credit score were largely uncorrelated, while loan amount and credit score were correlated. Therefore, borrowers with higher credit scores tend to apply for larger loans. In addition, clients with higher loan-to-income ratios typically pay higher interest rates.

These kinds of global insights cannot be obtained from standard interpretation methods discussed in Chapter 2, underscoring the unique value of the biplot-based approach. However, it is important to note that this surrogate model reflects the decision surface along the first two components only. As a result, alternative interpretations may emerge in other projections, and some may contradict the explanations outlined here.

5.3 LOCAL SHAPLEY VALUE INTERPRETATION

After using the global interpretation to select and evaluate the most appropriate GBM model, the CVA biplot can be applied to generate local interpretations and counterfactual explanations that are feasible, plausible and actionable.

To illustrate this, four examples are selected to represent the four types of prediction outcomes produced by the model:

- True positive (red): correctly predicted default
- True negative (blue): correctly predicted non-default
- False positive (purple): incorrectly predicted default
- False negative (orange): incorrectly predicted non-default.

Here, positive refers to loan default. The variable values for the four example data points are shown in Table 5.2.

Table 5.2: Four example data points that represent the four different types of prediction outcomes

	Variable name and value			
Prediction outcome	loan_amnt	loan_int_rate	loan_percent_income	credit_score
Red – true positive	33,000	12.68	0.4	654
Blue – true negative	12,000	12.69	0.05	634
Orange – false negative	25,000	12.61	0.21	625
Purple – false oositive	35,000	7.9	0.25	583

These four points were plotted on the CVA biplot in Figure 5.7, along with their unconstraint counterfactuals.

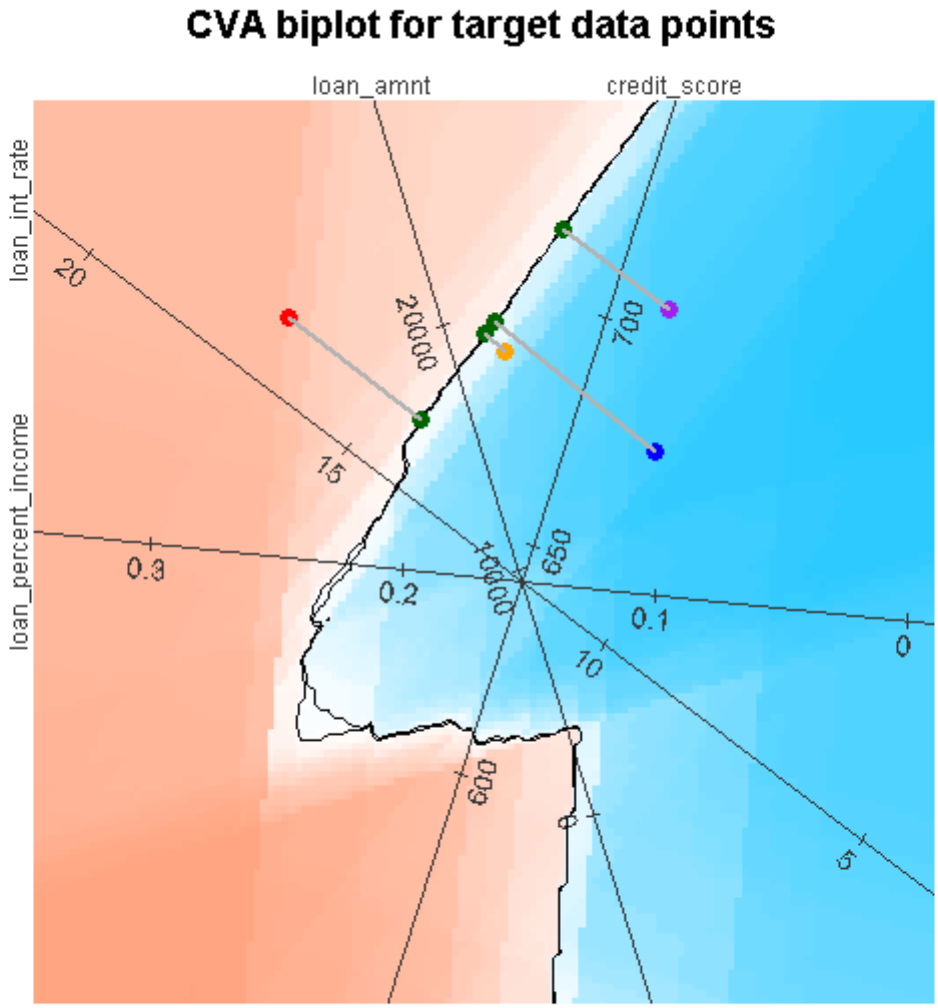


Figure 5.7: Unrotated CVA biplot based on the global optimal GBM model. The four data points are coloured according to prediction outcome, with their nearest counterfactuals indicated in dark green.

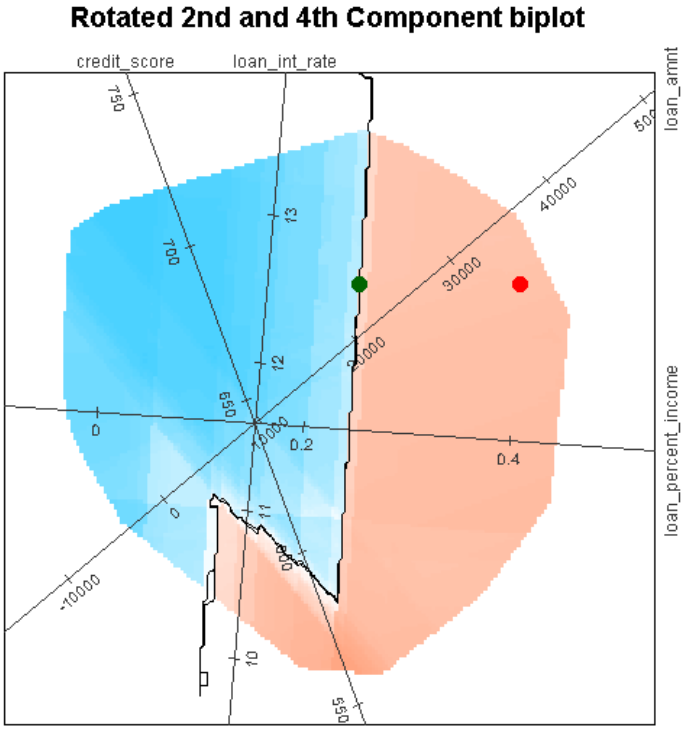
The next step develops local interpretations using constrained counterfactuals. For each of the four examples, the biplot is rotated across all six possible combinations of components, and the nearest counterfactual solution is selected. The actionability constraint requires that certain variables remain fixed – in this case the loan amount applied should not decrease.

Once the nearest valid counterfactual is identified, Shapley values are calculated to provide the individual-level explanation. These values quantify the contribution of each variable to the prediction outcome and clarify which changes are most influential in crossing the decision boundary. In the false positive and false negative cases, this approach also explains why the misclassifications occurred.

The resulting biplots and Shapley plots for the four example points are shown in Figure 5.8, Figure 5.9, Figure 5.10 and Figure 5.11, providing detailed local interpretations that link back to both the counterfactual changes and the structure of the decision boundaries.

5.3.1 Interpreting the true positive prediction

The example and graphs in Figure 5.8 interpret the reasons for the true positive prediction.



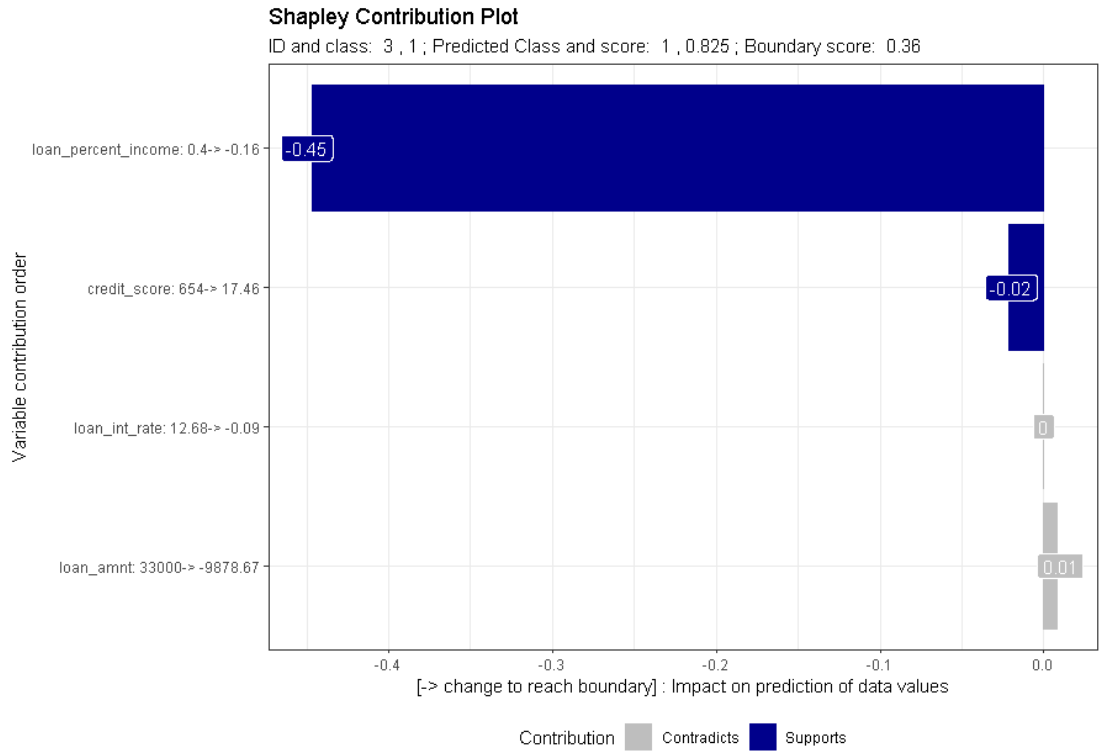


Figure 5.8: Biplot and Shapley plot local interpretation for the red – true positive prediction

The Shapley plot indicates that the main reason for the default prediction and loan decline is the high loan-to-income ratio (loan_percent_income). This needs to reduce from 40% to 24% for the client to qualify for the loan. The biplot method implies a reduction in the loan amount. However, the reduction contradicts the change in the prediction outcome and can be ignored. Therefore, the resulting counterfactual remains actionable. This implies that to retain the same loan amount, but get approval for the loan, personal income needs to increase.

The credit score also contributes to the default prediction. However, a much larger change in the credit score compared to the loan-to-income ratio is required to change the prediction outcome. The change in loan-to-income ratio on its own is sufficient to change the prediction outcome.

This is consistent with the biplot, which shows the loan_percent_income axis being almost perpendicular to a straight decision boundary on the decision surface estimated by the second and fourth components. Therefore, any change in the decision will require a reduction in the loan-to-income ratio.

5.3.2 Interpreting the true negative prediction

The example and graphs in Figure 5.9 interpret the reasons for the true negative prediction.

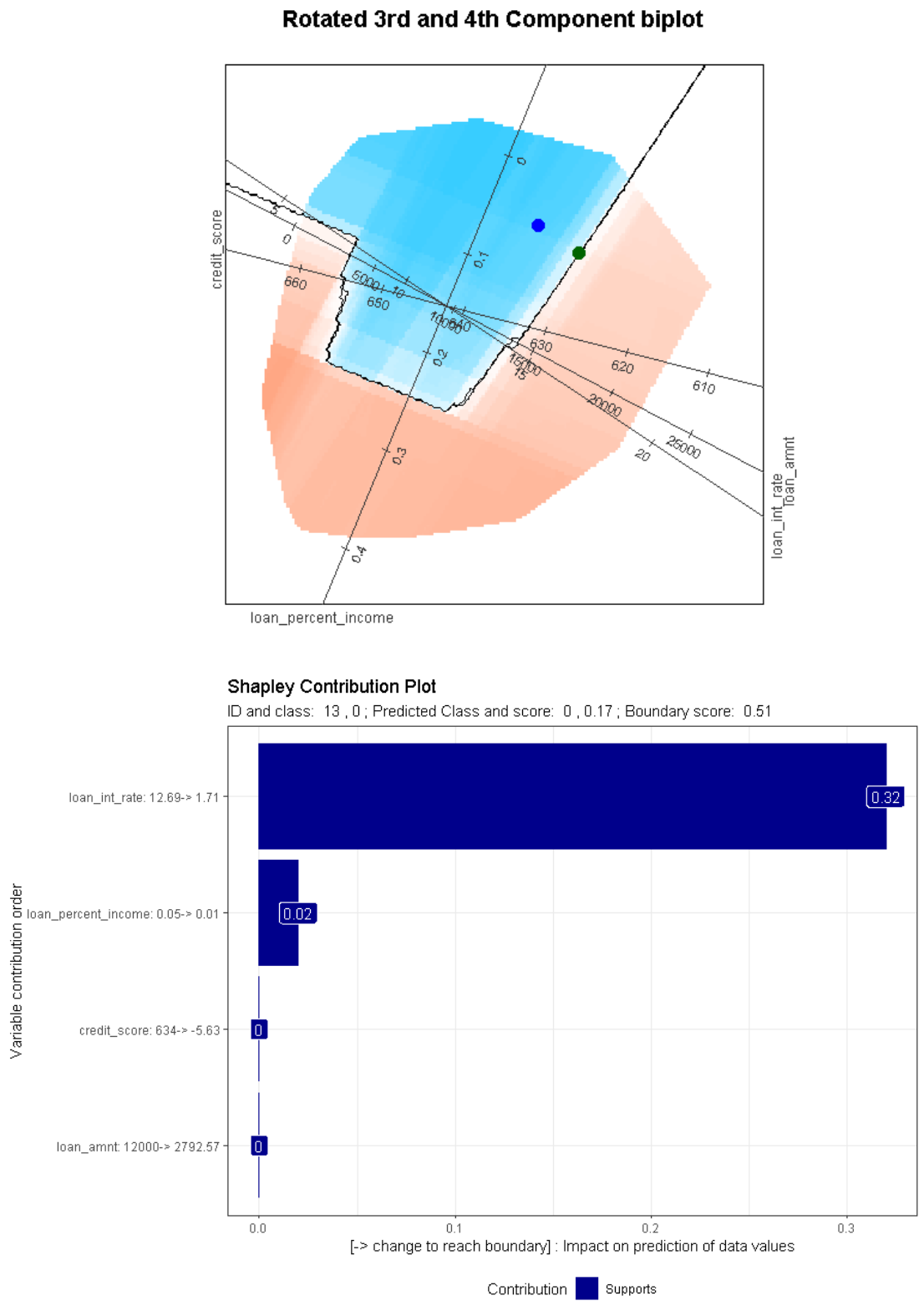


Figure 5.9: Biplot and Shapley plot local interpretation for the blue – true negative prediction

The Shapley plot indicates that there are three reasons for the no-default prediction and loan acceptance. A change in the prediction requires an increase in the loan interest rate of 1.71% and an increase in the loan-to-income ratio of 1%. It is the combination of these two variables

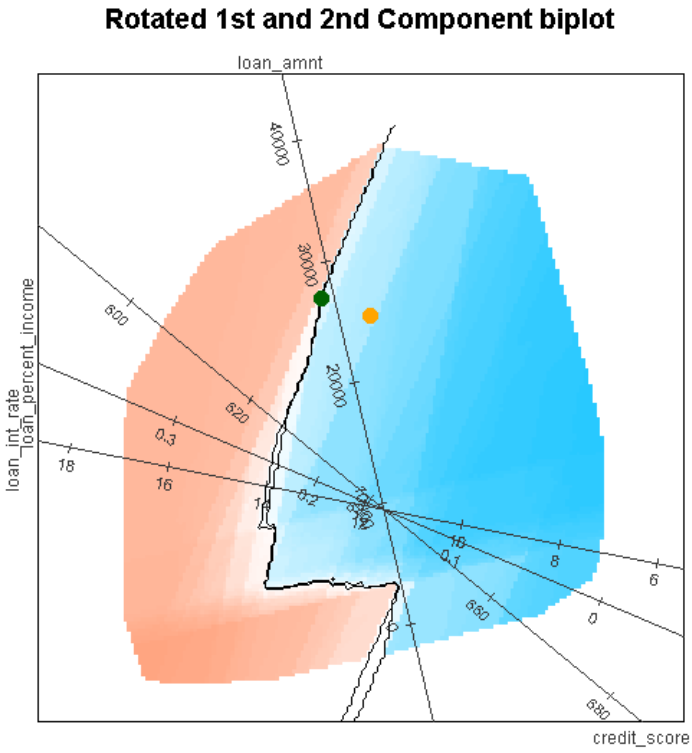
that causes the prediction, and each individual variable would have to be change by a much larger amount to cause the prediction to change. The increase in the loan amount has no impact on the prediction and can be ignored to give an actionable result.

The biplot generated by the first and second component indicates the importance of the loan_percent_income variable, as it is perpendicular to a part of the local decision boundary. However, for this data point it is less significant and contributes only a small part to the no-default prediction, as the axis runs parallel to the nearest section of the local decision boundary. The loan interest rate axis runs the most perpendicular to the nearest local decision boundary, hence it having the greatest relative importance.

If the data point had a higher loan_percent_income value to being with, then this variable would have been closer to the decision boundary along its axis, and the variable would have made a bigger contribution to the prediction outcome.

5.3.3 Interpreting the false positive prediction

The example and graphs in Figure 5.10 interpret the reasons for the false positive prediction.



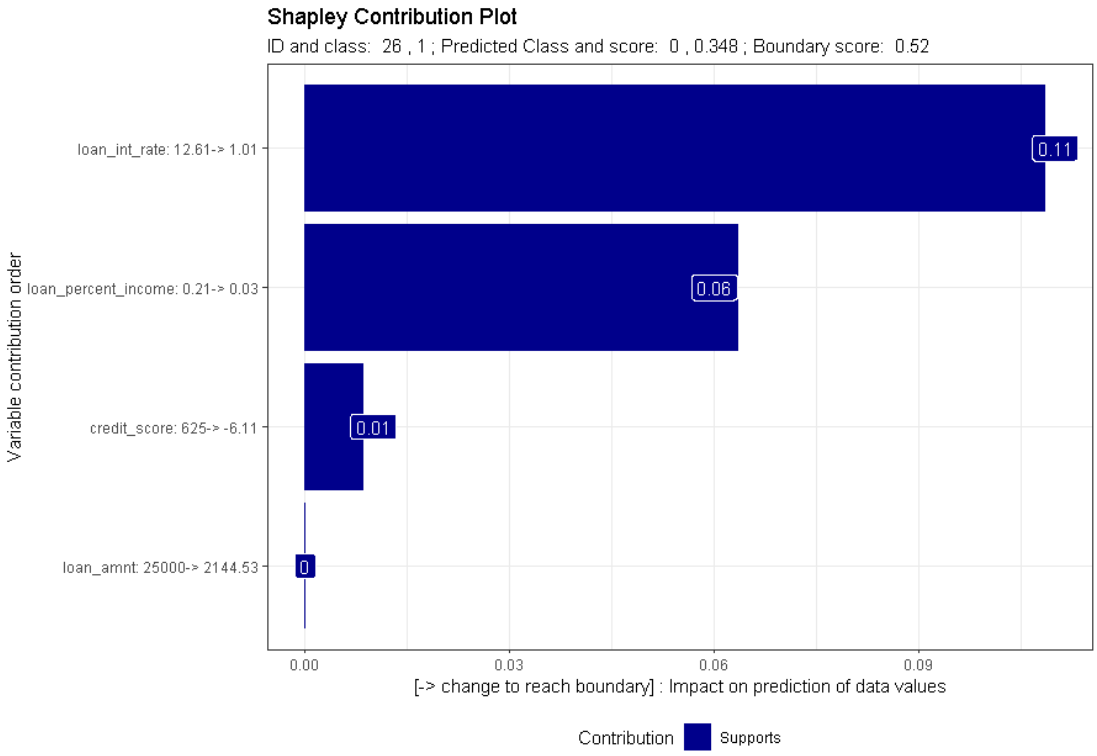


Figure 5.10: Biplot and Shapley plot local interpretation for the orange – false negative prediction

The Shapley plot indicates that there are two reasons for the incorrect no-default prediction and loan acceptance. A change in the prediction requires a combined increase in the loan_int_rate of 1.01% and an increase in the loan_percent_income ratio of 3%. Each change individually will require a much larger increase to change the prediction outcome to a default prediction. It is the low values of these variables for the data point that cause the model to predict a no-default. Higher credit scores and loan amounts will only contribute marginally to changing the outcome and can be ignored.

Therefore, the model behaves consistently for the orange and blue data points. The orange data point is much closer to the decision boundary along the loan_percent_income ratio variable axis than the blue data point. Therefore, for this data point, the loan_percent_income variable contributes more to the prediction outcome. This reduces the change required from the loan interest rate.

This indicates how the biplot explains the multivariate impact of the data on the model predictions and gives insight into the black-box GBM model. Different variables have different implications for the prediction outcome, depending on their current value and their relative value compared to the other variable values. The biplot explanations capture these interactions correctly when providing local interpretations for the predictions.

5.3.4 Interpreting the false negative prediction

The example and graphs in Figure 5.11 interpret the reasons for the false negative prediction.

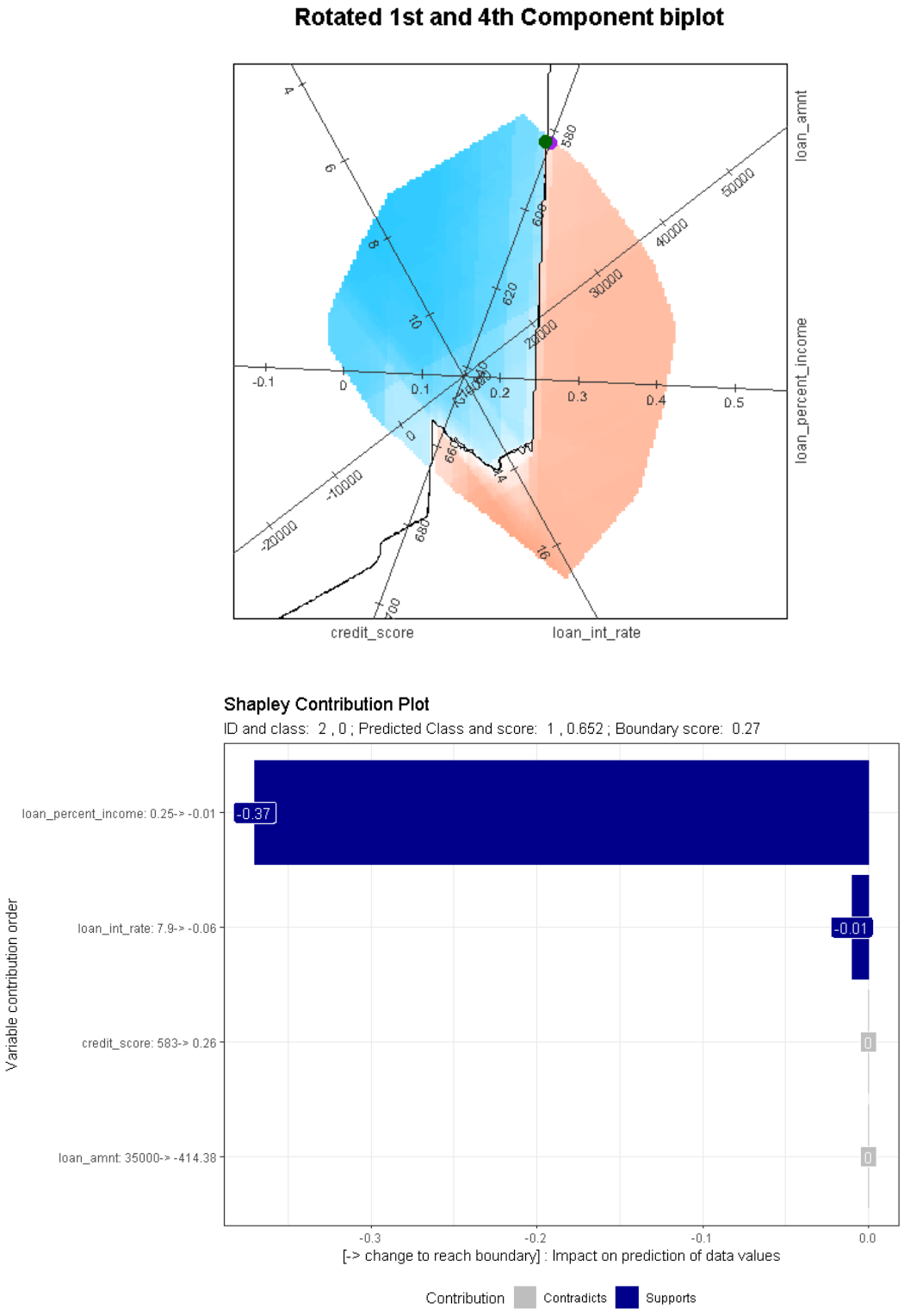


Figure 5.11: Biplot and Shapley plot local interpretation for the purple – false positive prediction

The interpretation for the purple data point implies that the only reason for the default prediction is the high loan-to-income ratio. A small reduction of 1% will change the prediction to no-default. The other changes can be ignored, as they have no impact on the prediction outcome. This is consistent with the biplot interpretation, which indicates that the data point and its counterfactual is very close together and therefore require very little change to change the prediction.

This also indicates why the model made an incorrect prediction. Similar to the true positive prediction, the main driver of default predictions is their relatively high loan-to-income ratios, and reducing this will reduce the prediction likelihood. Data points such as these, which are close to the decision boundary, are more likely to be misclassified. One area of future development proposed is to use the distance to the decision boundary as a measure of uncertainty in the prediction. In this case, the default prediction should have been made with care.

The small change required to change the prediction is also an indication that the model is not robust enough to accommodate these types of cases. If this pattern is seen across many data points, it could indicate the need to calibrate a more robust model.

5.4 CONCLUSION TO THE BIPLOT EXPLANATIONS

This chapter contained a working example of using biplots to explain ML predictions by finding counterfactuals and their corresponding decision boundaries. It provides reasons for individual predictions and answer questions such as “Why was my loan application rejected?” or “What do I need to change for the bank to approve my loan?” Different data points have different reasons, with some requiring more than one variable to change to change the outcome. The variable changes required were also quantified.

The biplot also proved global interpretations of the model, which allowed irrelevant variables and outlier data points that do not contribute the model accuracy to be removed. The simplified model gives more intuitive explanations, but also higher prediction accuracy. A surrogate model can classify new data points with high fidelity while giving very intuitive explanations for the prediction outcome.

The biplot method is model-agnostic, computationally efficient, gives exact Shapley values and give explanations that are actionable, feasible, plausible and sparse. It is also repeatable and can give a diverse set of solutions.

CHAPTER 6 FUTURE AREAS OF RESEARCH

6.1 INTRODUCTION

The biplot visualisation and identification of counterfactuals have further potential applications beyond those discussed in Chapter 4 and Chapter 5. These are discussed in this chapter and can serve as a starting point for further research to improve the accuracy of the biplot method and find new applications of the results.

6.2 GLOBAL SHAPLEY VALUE INTERPRETATION USING ROTATED LOCAL DISTANCE CALCULATIONS

Shapley values are calculated to give local interpretations for individual data points. The primary drawback with using Shapley values is that computation time increases exponentially with the number of variables included. Therefore, calculating the Shapley value becomes computationally unfeasible if it is required for many data points with numerous variables.

To address this, this study initially proposed a distance-based global importance measure: decompose the change vector of each data point to its counterfactual along the variable axes of the biplot, then aggregate the axis-wise distances. While this is efficient and visually aligned with the biplot, distance along an axis is not always a proxy for causal influence. In many cases, a small, well-oriented change in a single variable is sufficient to flip the prediction, even if its geometric component is short.

A more faithful global measure therefore aggregates the Shapley contributions across the dataset. Therefore, an extension of this study is to create a global feature importance metric by computing the Shapley values for a representative set of data points and then summarising them across observations with

$$I_j = mean(|\phi_{ij}|),$$

where ϕ_{ij} is the Shapley contribution of variable j to the prediction for instance i. Optionally, reporting the dispersion (e.g. median, IQR, or bootstrap confidence intervals) can convey the stability and heterogeneity of the variable feature. This yields a model-agnostic global feature importance method that respects interactions and non-linearities, complementing the geometric perspective of the biplot.

However, this will require a significant increase in the necessary computational power and time required to provide solutions. This includes the requirement to rotate the biplot plane for each data point to ensure that the distance to the decision boundary for each data point is correctly approximated by the biplot plane.

6.3 GRAND TOUR OF DATA INSTEAD OF COMPONENT SELECTION

The proposed method of identifying the nearest possible counterfactual requires the consideration of all $^{p!}/_{2!\,(p-2)!}$ combination of component pairs. This can be extended by employing a grand tour (Cook, Buja, Lee & Wickham, 2008) to generate a smooth sequence of orthonormal two-dimensional projections, each yielding a biplot defined by two linear combinations of all the components.

Each tour frame requires the following:

1. Rotating the plane to place the target on the biplot for local fidelity.
2. Estimating the local decision boundaries and identify the nearest constrained counterfactual on the plane.
3. Predicting the value in the original space.
4. Scoring candidates by Mahalanobis distance and retaining the best valid counterfactual across all the frames.

While the grand tour cannot guarantee the globally nearest theoretical counterfactual, it broadens the search beyond fixed pairs, frequently surfacing nearer and more diverse valid counterfactuals. The multiplicity of views also improves outlier detection relative to local decision boundaries and provides richer insight into the decision surface.

6.4 DISTANCE TO DECISION BOUNDARIES AS APPROXIMATION OF PREDICTION PROBABILITIES

This section starts by repeating and then expanding on the ability of an ML model to measure the uncertainty and confidence associated with each prediction, initially discussed in section 2.7.1. It then expands on the ideas to suggest using the distance to the decision boundary as a measure of the confidence of the model in the prediction.

6.4.1 Uncertainty and confidence in predictions

One of the definitions of an interpretation discussed in section 2.7 is the ability to explain the confidence in a prediction. Even though classification models give a prediction value between 0 and 1, this does not necessarily equate to the confidence in the prediction or the likelihood of the data point to be in class 0 or 1. The ability to give prediction likelihoods depends on the model used.

The main drawback of the function-fitting approach is that it does not give a posterior class probability $P(y = k|\boldsymbol{x})$. Without this estimate, the confidence in the prediction cannot be specified. Even methods that do provide the posterior probabilities do not always provide accurate estimates, with naïve Bayes in general pushing predicted values towards 0 and 1 (Niculescu-Mizil & Caruana, 2005). Decision trees assign a prediction of $p = k/n$ as the score of any example assigned to a leaf that contains k positive training examples and n total training examples (Zadrozny & Elkan, 2001).

To maximise the predictions for many practical prediction and decision-making tasks, it is crucial to develop an accurate probabilistic prediction model from data. ML models that are not optimised for obtaining accurate prediction probabilities are referred to as miscalibrated. In general, a set of predictions of a binary outcome is well calibrated if an outcome predicted to occur with probability p do occur about p fraction of the time for each probability p that is predicted (Naeini *et al.*, 2015).

The less certainty there is about the accuracy of a prediction, the less reliably the interpretation of the prediction will be. This in turn will reduce the trust in the model and make the interpretations less useful for helping to understand the nature of the model and the real-world interactions between the features and the output. To increase the trust in the model, the interpretation should quantify the uncertainty of a prediction.

The confidence of a prediction can be enhanced by adding confidence intervals to a prediction. This helps quantify the certainty of a sample being in a certain class. A 95% confidence interval will mean that for 100 such cases, 95 will be in the specified class. Statistical models such as GLMs can give the confidence interval of a prediction, as the link function used assumes a certain distribution for the mean and variance of the prediction. Similar statistics are not available in other black-box models.

For other ML models, the bootstrap method provides a direct computational way of assessing the uncertainty by calculating the $u^*(\boldsymbol{x})$ on say $B = 200$ bootstrap samples and then calculating the 95% pointwise confidence band at each $\boldsymbol{x}$ by finding the 2.5% × 200 = fifth-

largest and smallest values at each $\boldsymbol{x}$ (Hastie *et al.*, 2009). Such explanations, even if no individual reasons for the prediction is provided, will increase the understanding and interpretability of ML models.

6.4.2 Developing an accurate probabilistic prediction model

Two common methods for mapping model predictions to posterior probabilities are Platt calibration and isotonic regression. Unfortunately, these methods are designed for binary classification, and it is not trivial to extend them to multiclass problems (Caruana, 2005).

In the case of support vector machines, the probabilities can be approximated by a sigmoid function using Platt scaling, where:

$$P(y = 1|x) = P_{A,B}(f) = 1/(1 + exp(Af(x) + B),$$

with $f(x)$ the support vector decision function, with a value between -1 and 1, representing the distance from the decision boundary. Various approaches have been suggested on how to calibrate A and B to optimise the estimation of the prediction probabilities (Lin, Lin & Weng, 2007).

The Platt scaling method can similarly be applied to the results of the biplot model. The biplot is a model-agnostic method of measuring the distance to the decision boundary with the vector

$$\boldsymbol{\delta}_i = (\boldsymbol{z}_{cf} - \boldsymbol{z}_i) \; or \; (\boldsymbol{z}_{cf} - \boldsymbol{z}_i)(\boldsymbol{V}_i^{*new})^{-1}$$

used to measure either the Euclidean or the Mahalanobis distance.

Using this distance measure instead of the $f(x)$ allows for the calibration of a posterior class probability $P(y = 1|x)$ for any ML model.

6.5 VISUALISATION AND CLUSTERING OF THE DATA

Another potential global application of the biplot is as a clustering tool, grouping data points based on whether they share the same local decision boundary. These clusters provide an approximation of the groupings identified by the model during training. In a linear model, such as logistic regression, the decision boundary is a hyperplane that separates the data into two classes depending on which side of the plane each point falls.

To identify more than two distinct sub-groups within the data, a non-linear model is needed. Such models have more degrees of freedom to find the optimal boundaries and capture more

complex patterns that separate the data more effectively. These boundaries divide the feature space into regions that minimise a loss function, with each region assigned a class label (e.g. 0 or 1 in a binary classification problem). Even within the same class, the model may identify several distinct sub-groups. In doing so, it provides an efficient method for clustering the data.

While the biplot offers only a two-dimensional approximation of these boundaries and sub-groups, it still serves as a useful visual tool to support interpretation and clustering in line with the model's structure. This approach expands on the MDS method discussed in section 2.9.3, where clusters are formed using k-means clustering based on the proximity of data points in the two-dimensional space generated through classical scaling of the standardised data.

The biplot provides a similar visual representation of the data, but the clustering is guided by the local decision boundaries of the underlying ML model, as visualised on the biplot. Unlike k-means, the number of clusters does not need to be pre-specified. Instead, the clusters emerge naturally from the decision boundaries, with each region enclosed by a boundary typically corresponding to a distinct class or subgroup. If the local decision boundary and the convex hull of the data enclose a region, all data points within that area will be classified in the same way.

Figure 6.1 compares the classical scaling of the iris dataset with k-means clustering (using $k = 3$) to the PCA biplot clustering derived from three different ML models: GLM, GAM and SVM. Although the spatial layout of the data remains the same across both methods, the resulting cluster assignments differ considerably, highlighting the impact of model-based decision boundaries on the interpretation of group structure in the data.

Although the iris data contained three classes of flowers, the ML models are trained on two classes, with versicolor set as class 1 and virginica and setosa as class 0.

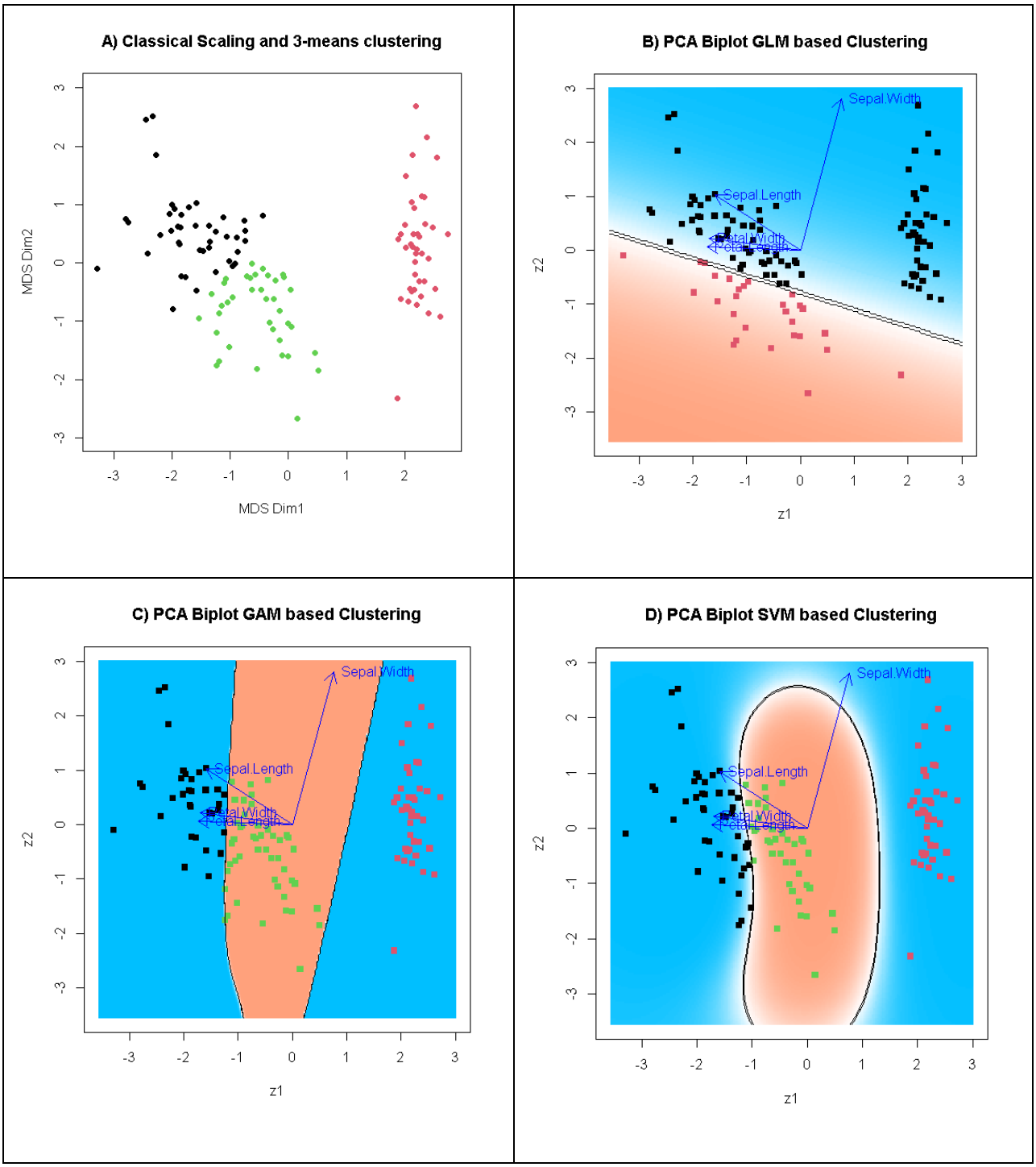


Figure 6.1: Comparison of the MDS with nearest neighbour clustering vs. PCA biplots with various model-based clustering

Illustration A shows the classical scaling MDS results, with the three-means clustering based on the proximity of the data points in the two-dimensional space. Illustrations B, C and D give the same data points but with different clustering reflecting the different local decision boundaries created by the GLM, GAM and SVM models. The GLM model only creates two data clusters.

Neither the k-means clustering nor the ML model had access to the underlying three-class classification of the data. The SVM and GAM models correctly classified 92% of the data points compared to the true underlying class. The three-means model classified only 87% correctly. The GLM model could not identify three different classes of data.

CVA biplots and their local decision boundaries will also cluster the data in the same way. However, the spatial position of the data will be different from that shown by the classical scaling MDS method.

6.6 GENERALISATION OF THE METHODOLOGY

The main restriction to the proposed biplot method for generating counterfactuals is that it is limited to binary classification models built on continuous variable data represented in a tabular format. Future research is required to extend the solution to the following areas:

- regression tasks;
- multi-class classifications;
- categoricial input variables; and
- unstructured data domains such as text, imagery or sequential signals.

Some of these problems have already been discussed in sections 3.9.1 and 4.4.

Regression tasks can be addressed by setting the boundary threshold b equal to the target regression value. The counterfactual will then indicate the change required to increase the regression outcome to above or below this target threshold.

Multi-class classification problems can be addressed by identifying the target category towards which a change is required as class 1 and combining the current category with all the other possible categories as class 0. Then the biplot solution will indicate the change required for an individual data point to reach or exit the target category.

Categorical input variables can be incorporated through either the use of categorical biplots, or through the one-hot encoding of the input variables.

The structure of the code and the output generated will need to be updated to ensure the biplots provide actionable, plausible and feasible solutions. However, these changes are technically achievable within the framework provided.

Significant additional work will be required to apply the biplot method to unstructured data, or for applications such as generating explanations for the predictions made by large-language models.

REFERENCE LIST

Aas, K., Jullum, M. & Løland, A. 2021. Explaining individual predictions when features are dependent: More accurate approximations to Shapley values. *Artificial Intelligence*, 298:1-28. https://doi.org/10.1016/j.artint.2021.103502

Alvarez-Melis, D. & Jaakkola, T.S. 2018. On the robustness of interpretability methods. *arXiv*. Available: http://arxiv.org/abs/1806.08049 [Accessed 6 June 2021].

Apley, D.W. & Zhu, J. 2020. Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society. Series B: Statistical Methodology*, 82(4):1059–1086. https://www.jstor.org/stable/26937906

Argouarc'h, E., Desbouvries, F., Kawasaki, E. & Barat, E. 2024. Generative vs. discriminative modeling under the lens of uncertainty quantification. *arXiv*. Available: https://arxiv.org/abs/2406.09172 [Accessed 30 April 2024].

Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., Garcia, S., Gil-Lopez, S., Molina, D., Benjamins, R., Chatila, R. & Herrera, F. 2020. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115. https://doi.org/10.1016/j.inffus.2019.12.012

Breiman, L. 2001a. Random forests. *International Journal of Advanced Computer Science and Applications*, 45(1):5–32. https://doi.org/10.1023/a:1010933404324

Breiman, L. 2001b. Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.

Cook, D., Buja, A., Lee, E.-K. & Wickham, H. 2008. Grand tours, projection pursuit guided tours, and manual controls. In C.-H. Chen, W. Härdle & A. Unwin (eds.). *Handbook of data visualization* (Vol. 4). Berlin: Springer, 295–314. https://doi.org/10.1007/978-3-540-33037-0

Cox, T. & Cox, M. 2008. Multidimensional scaling. In C.-H. Chen, W. Härdle & A. Unwin (eds.). *Handbook of data visualization* (Vol. 4). Berlin: Springer, 315–347. https://doi.org/10.1007/978-3-540-33037-0_14

Dandl, S., Molnar, C., Binder, M. & Bischl, B. 2020. Multi-objective counterfactual explanations. *Lecture Notes in Computer Science*, 12269(1):448–469. https://doi.org/10.48550/arXiv.2004.11165

Dhurandhar, A., Pedapati, T., Balakrishnan, A., Chen, P.-Y., Shanmugam, K. & Puri, R. 2024. Model agnostic contrastive explanations for classification models. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 14:789–798.

Doshi-Velez, F. & Kim, B. 2017. Towards a rigorous science of interpretable machine learning. *arXiv*. Available: https://arxiv.org/abs/1702.08608 [Accessed 30 June 2021].

Eckart, C. & Young, G. 1936. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218.

European Union. 2016. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj [Accessed 30 May 2025].

Fisher, R.A. 1936. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.

Friedman, J.H. 2001. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 5:1189–1232.

Gabriel, K.R. 1971. The biplot graphic display of matrices with application to principal component analysis. *Biometrika*, 58(3):453–467.

Giorgi, F., Veglianti, F., Silvestri, F. & Tolomei, G. 2025. Generalizability through explainability: Countering overfitting with counterfactual examples. *arXiv*. Available: http://arxiv.org/abs/2502.09193 [Accessed 30 May 2025].

Goldstein, A., Kapelner, A., Bleich, J. & Pitkin, E. 2014. Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65. https://doi.org/10.48550/arXiv.1309.6392

Gower, J.C. & Hand, D.J. 1996. *Biplots*. London: Chapman & Hall.

Gower, J.C., Lubbe, S. & Le Roux, N. 2011. *Understanding biplots*. Chichester: Wiley.

Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Pedreschi, D. & Giannotti, F. 2018. A survey of methods for explaining black box models. *Association for Computing Machinery*, 51(5):1-42. https://doi.org/10.1145/323600

Hastie, T., Tibshirani, R. & Friedman, J.H. 2009. *The elements of statistical learning: Data mining, inference and prediction* (2nd ed.). New York, NY: Springer. https://doi.org/10.1007/978-0-387-84858-7

Holzinger, A., Goebel, R., Fong, R., Moon, T., Müller, K.-R. & Samek, W. 2020. *xxAI – Beyond explainable AI*. International workshop, held in conjunction with ICML 2020. Vienna: Springer. https://doi.org/10.1007/978-3-031-04083-2

Honegger, M.R. 2018. Shedding light on black box machine learning algorithms: Development of an axiomatic framework to assess the quality of methods that explain individual predictions. Master's thesis, Karlsruhe Institute of Technology.

Hotelling, H. 1933. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441. https://doi.org/10.1037/h0071325

Kaufman, L. & Rousseeuw, P.J. 1987. *Clustering by means of medoids*. Department of Mathematics, KU Leuven. Available: https://wis.kuleuven.be/stat/robust/papers/publications-1987/kaufmanrousseeuw-clusteringbymedoids-l1norm-1987.pdf [Accessed 30 May 2025].

Kessy, A., Lewin, A. & Strimmer, K. 2018. Optimal whitening and decorrelation. *The American Statistician*, 72(4):309–314. https://doi.org/10.1080/00031305.2016.1277159

Kim, B. & Austin, U.T. 2016. Examples are not enough, learn to criticize! criticism for interpretability. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS'16)*. Red Hook, NY: Curran, 2288–2296.

Krause, J., Perer, A. & Ng, K. 2016. Interacting with predictions: Visual inspection of black-box machine learning models. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems.* New York, NY: ACM, 5686–5697. https://doi.org/10.1145/2858036.2858529

Laugel, T., Lesot, M.J., Marsala, C., Renard, X. & Detyniecki, M. 2018. Comparison-based inverse classification for interpretability in machine learning. *Communications in Computer and Information Science*, 853:100–111. https://doi.org/10.1007/978-3-319-91473-2_9

Laugel, T., Lesot, M.J., Marsala, C., Renard, X. & Detyniecki, M. 2019. The dangers of post-hoc interpretability: Unjustified counterfactual explanations. In *Proceedings of the Twenty-eighth International Joint Conference on Artificial Intelligence*. Macao: AAAI Press, 2801–2807. https://dl.acm.org/doi/10.5555/3367243.3367428

Le Roux, N.J. & Gardner-Lubbe, S. 2024. A two-groups canonical variate analysis biplot for an optimal display of both means and cases. *Advances in Data Analysis and Classification*, 19:721–748. https://doi.org/10.1007/s11634-024-00593-7

Lim, B.Y. 2012. Improving understanding and trust with intelligibility in context-aware applications. PhD dissertation, Carnegie Mellon University.

Lin, H.-T., Lin, C.-J. & Weng, R.C. 2007. A note on Platt's probabilistic outputs for support vector machines. *Machine Learning*, 68:267–276. https://doi.org/10.1007/s10994-007-5018-6

Lipovetsky, S. & Conklin, M. 2001. Analysis of regression in game theory approach. *Applied Stochastic Models in Business and Industry*, 17(4):319–330.

Lipton, Z.C. 2018. The mythos of model interpretability. *Communications of the ACM*, 61(10):35–43. https://doi.org/10.1145/3233231

Lubbe, S., Le Roux, N., Nienkemper-Swanepoel, J., Ganey, R., Buys, R., Adams, Z. & Manefeldt, P. 2025. MuViSU/*biplotEZ.* Available: https://github.com/muvisu/biplotEZ [Accessed 25 May 2025].

Lundberg, S.M. & Lee, S. 2017. A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems.* Red Hook, NY: Curran, 4768–4777.

Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N. & Lee, S.-I. 2019. Explainable AI for trees: From local explanations to global understanding. *arXiv*. Available: http://arxiv.org/abs/1905.04610 [Accessed 15 January 2021].

Lundberg, S.M. & Lee, S.-I. 2018. Consistent feature attribution for tree ensembles. *arXiv*. Available: http://arxiv.org/abs/1706.06060 [Accessed 15 January 2021].

Mazzine, R. & Martens, D. 2021. A framework and benchmarking study for counterfactual generating methods on tabular data. *arXiv.* Available: https://doi.org/10.48550/arXiv.2107.04680 [Accessed 13 September 2025].

Miller, T. 2017. Explanation in artificial intelligence: Insights from the social sciences. *arXiv*. Available: https://arxiv.org/abs/1706.07269 [Accessed 13 September 2025].

Molnar, C. 2020. *Interpretable machine learning*. Available: https://christophm.github.io/interpretable-ml-book/ [Accessed 1 May 2024].

Mothilal, R.K., Sharma, A. & Tan, C. 2020. Explaining machine learning classifiers through diverse counterfactual explanations. In *FAT* 2020 – Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency.* New York, NY: ACM, 607–617. https://doi.org/10.1145/3351095.3372850

Naeini, P., Cooper, G.F. & Hauskrecht, M. 2015. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the Twenty-ninth AAAI Conference on Artificial Intelligence*, 29(1):2901–2907. https://doi.org/10.1609/aaai.v29i1.9602

Ng, A.Y. & Jordan, M.I. 2001. On discriminative vs generative classifiers: A comparison of logistic regression and naive Bayes. In *NIPS'01: Proceedings of the 15th International Conference on Neural Information Processing Systems: Natural and Synthetic.* Cambridge, MA: MIT Press, 841–848.

Niculescu-Mizil, A. & Caruana, R. 2005. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning.* New York, NY: ACM, 625–632. https://doi.org/10.1145/1102351.1102430

Poyiadzi, R., Sokol, K., Santos-Rodriguez, R., De Bie, T. & Flach, P. 2020. FACE: Feasible and actionable counterfactual explanations. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES 2020).* New York, NY: ACM, 344–350. https://doi.org/10.1145/3375627.3375850

R Core Team. 2025. *R: A language and environment for statistical computing*. Vienna: R Foundation for Statistical Computing.

Republic of South Africa. 2013. Protection of Personal Information Act 4 of 2013. *Government Gazette*, 37067, 26 November. Pretoria: Government Printer.

Ribeiro, M.T., Singh, S. & Guestrin, C. 2016. "Why should I trust you?" Explaining the predictions of any classifier. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Vol. 13–17). New York, NY: ACM, 1135–1144. https://doi.org/10.1145/2939672.2939778

Rodrigues, F.C.M., Espadoto, M., Hirata, R. & Telea, A.C. 2019. Constructing and visualizing high-quality classifier decision boundary maps. *Information,* 10(9): art. 280. https://doi.org/10.3390/info10090280

Rubinstein, Y.D. & Hastie, T. 1997. *Discriminative versus informative learning*. AAAI. Available: https://hastie.su.domains/Papers/kdd97.pdf [Accessed 1 May 2024].

Rudin, C., Chen, C., Chen, Z., Huang, H., Semenova, L. & Zhong, C. 2021. Interpretable machine learning: Fundamental principles and 10 grand challenges. *arXiv*. Available: http://arxiv.org/abs/2103.11251 [Accessed 21 October 2022].

Schulz, L.O., Bennett, P.H., Ravussin, E., Kidd, J.R., Kidd, K.K., Esparza, J. & Valencia, M.E. 2006. Effects of traditional and western environments on prevalence of type 2 diabetes in Pima Indians in Mexico and the US. *Diabetes Care*, 29(8):1866–1871.

Shapley, L. 1952. *A value for N-person games*. RAND Corporation. Available: https://www.rand.org/content/dam/rand/pubs/papers/2021/P295.pdf [Accessed 1 May 2024].

Sharma, C., Sharma, S., Sharma, K., Sethi, G.K. & Chen, H.Y. 2024. Exploring explainable AI: A bibliometric analysis. *Discover Applied Sciences*, 6: art. 615. https://doi.org/10.1007/s42452-024-06324-z

Sharma, S., Henderson, J. & Ghosh, J. 2019. CERTIFAI: A common framework to provide explanations and analyse the fairness and robustness of black-box models. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES'20).* New York, NY: Association for Computing Machinery, 166–172. https://doi.org/10.1145/3375627.3375812

Sohns, J.T., Garth, C. & Leitte, H. 2023. Decision boundary visualization for counterfactual reasoning. *Computer Graphics Forum*, 42(1):7–20. https://doi.org/10.1111/cgf.14650

Strumbelj, E. & Kononenko, I. 2010. An efficient explanation of individual classifications using game theory. *Journal of Machine Learning Research*, 11:1–18. https://doi.org/10.1145/2858036.2858529

Tolomei, G., Silvestri, F., Haines, A. & Lalmas, M. 2017. Interpretable predictions of tree-based ensembles via actionable feature tweaking. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2017)* (Part F1296). New York, NY: ACM, 465–474. https://doi.org/10.1145/3097983.3098039

Ustun, B., Spangher, A. & Liu, Y. 2019. Actionable recourse in linear classification. In *Proceedings of the 2019 Conference on Fairness, Accountability, and Transparency (FAT* 2019).* New York, NY: ACM, 10–19. https://doi.org/10.1145/3287560.3287566

Van Looveren, A. & Klaise, J. 2019. Interpretable counterfactual explanations guided by prototypes. *arXiv*. Available: http://arxiv.org/abs/1907.02584 [Accessed 22 October 2021].

Vapnik, V. & Izmailov, R. 2020. Complete statistical theory of learning: Learning using statistical invariants. In *Proceedings of the Ninth Symposium on Conformal and Probabilistic Prediction and Applications (PMLR),* 128(4):4–40.

Vapnik, V.N. 1999. An overview of statistical learning theory. *IEEE Transactions on Neural Networks*, 10(5):988–999. https://doi.org/10.1109/72.788640

Vapnik, V.N. & Chervonenkis, A.Ya. 1971. On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability and Its Applications*, XVI(2):264–280.

Vilone, G. & Longo, L. 2021. Notions of explainability and evaluation approaches for explainable artificial intelligence. *Information Fusion*, 76:89–106. https://doi.org/10.1016/j.inffus.2021.05.009

Wachter, S., Mittelstadt, B. & Russell, C. 2017. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harvard Journal of Law & Technology*, 31(2):841–887.

White, A. & D'Avila Garcez, A. 2020. Measurable counterfactual local explanations for any classifier. *arXiv*. Available: https://arxiv.org/abs/1908.03020 [Accessed 18 March 2025].

Wielopolski, P., Furman, O., Stefanowski, J. & Zięba, M. 2024. Unifying perspectives: Plausible counterfactual explanations on global, group-wise, and local levels. *arXiv*. Available: http://arxiv.org/abs/2405.17642 [Accessed 18 March 2025].

Williams, J.N., Katakkar, A., Heidari, H. & Kolter, J.Z. 2024. Rethinking distance metrics for counterfactual explainability. *arXiv*. Available: http://arxiv.org/abs/2410.14522 [Accessed 21 March 2025].

Zadrozny, B. & Elkan, C. 2001. Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. In *Proceedings of the Eighteenth International Conference on Machine Learning (ICML 2001).* San Francisco, CA: Morgan Kaufmann, 609–616.

Zou, H. & Hastie, T. 2005. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society. Series B: Statistical Methodology*, 67(5):301–320. https://www.jstor.org/stable/3647580

APPENDIX A: CODE

```
################################################################
# Section 1: Biplot-based Model Utilities
# Description:
#   Utilities for model prediction wrappers, data preparation for
#   biplot visualizations (PCA/CVA), grid coloring and contouring on the
#   biplot plane, plotting helpers, indicator matrices, and Shapley-value
#   approximations for local feature contributions.
#
# Notes:
#   * Function parameters are documented using roxygen-style comments.
################################################################

##############################
# 1) Libraries
##############################
{
  library(MASS)
  library(tidyverse)
  library(pdist)
  library(dplyr)
  library(ggplot2)
  library(reshape2)
  library(grid)

  library(e1071)
  library(gbm)
  library(mgcv)
  library(rpart)
  library(MLmetrics)

  ## Shapley
  library(dplyr)
  library(combinat)

  # for coordinates in biplot LDB
  library(sp)
  library(devtools)

  library(biplotEZ)
}

##############################
# 2) SVM Probability Wrapper
##############################
```

```
#' Predict class-1 probabilities using an e1071 SVM model
#'
#' @param model_use Model object (e1071::svm) trained with probability=TRUE
#' @param data Newdata for prediction (data.frame)
#'
#' @return Numeric vector of class-1 probabilities
svm_pred <- function(model_use = model_use, data){
  prob_heading <- colnames(attr(predict(model_use, newdata = data, probability = TRUE), "probabilities"))
  prob_nr <- which(prob_heading == "1")[1]
  preds <- attr(predict(model_use, newdata = data, probability = TRUE), "probabilities")[, prob_nr]
  return(preds)
}


########################################
# 3) Generic Prediction Function Wrapper
########################################
#' Standardize prediction calls across model families
#'
#' Applies rounding and supports different model families.
#'
#' @param model_use Model object
#' @param model_select Character flag identifying model family
#'   (e.g., "NNET", "RForrest", "LDA", "SVM").
#' @param rounding Integer number of decimal places (floor-based rounding)
#' @param new_data Data frame for prediction
#'
#' @return Numeric vector of predicted probabilities/scores
pred_function <-  function(model_use,
                           model_select,
                           rounding = 2,
                           new_data) {

  m_options <- c("NNET", "RForrest", "LDA", "SVM")

  if(!(model_select %in% c(m_options)))
    pred_value <- floor(predict(model_use, newdata = new_data, type = "response") * (10^rounding)) /
(10^rounding)

  if(model_select == "NNET")
    pred_value <- floor(predict(model_use, newdata = new_data) * (10^rounding)) / (10^rounding)

  if(model_select == "RForrest")
    pred_value <- floor(predict(model_use, newdata = new_data)[, 2] * (10^rounding)) / (10^rounding)

  if(model_select == "LDA")
    pred_value <- floor(predict(model_use, newdata = new_data)$posterior[, 2] * (10^rounding)) /
(10^rounding)
```

```
 if(model_select == "SVM")
   pred_value <- floor(svm_pred(model_use, new_data) * (10^rounding)) / (10^rounding)

 return(pred_value)
}


############################################################
# 4) Data Preparation for Model Prediction & Evaluation
############################################################
#' Prepare dataset for prediction, evaluation, and optional polygon filtering
#'
#' Adds predictions, confusion labels, and plotting attributes.
#'
#' @param model_use Model object
#' @param data_use Data frame to process
#' @param num_vars Number of feature columns (assumed to be the first num_vars)
#' @param cutoff Classification cutoff (default 0.5)
#' @param model_select Character indicating model family (see pred_function)
#' @param target_class Target class index for LDA 1-vs-all reduction (default 2)
#' @param rounding Rounding for prediction values (default 2)
#' @param proj Projection indices (for biplot; default pc.values)
#' @param CVA_method Logical; whether CVA is used (affects scaling)
#' @param standardise Logical; standardize features for PCA (default standardise_use)
#' @param polygon Optional SpatialPolygons object to filter points inside
#' @param Xcenter Numeric vector of feature means
#' @param Xsd Numeric vector of feature sds
#' @param V Projection/loadings matrix
#'
#' @return list with element data_use (augmented data.frame)
set_model_data <- function(model_use, data_use, num_vars = num_vars,
                           cutoff = 0.5,
                           model_select = model_select,
                           target_class = 2,
                           rounding = 2,
                           proj = pc.values,
                           CVA_method = CVA_method,
                           standardise = standardise_use, # set to FALSE for CVA plots
                           polygon = NA, # train_polygon
                           Xcenter = X_center,
                           Xsd = X_sd,
                           V = Vmat) {

 # Remove data outside polygon, if provided
 if(is(polygon, "SpatialPolygons"))  {
   sv <- Xsd
   if(CVA_method == TRUE) standardise <- FALSE # set to FALSE if CVA is TRUE
   if(standardise  == FALSE) sv <- FALSE
   X.st <- scale(data_use[, 1:num_vars], center = Xcenter, scale = sv)
```

```
  Z_values <- X.st %*% V[, proj]
  colnames(Z_values) <- c("x", "y")

  # Select Z data points inside hull polygon
  Z_data_points <- as.data.frame(Z_values)
  coordinates(Z_data_points) <- ~x + y
  Z_data_points$inside_polygon <- !is.na(over(Z_data_points, polygon))
  data_use <- data_use[Z_data_points$inside_polygon == TRUE, ]
}

# Get predictions
data_use$pred_value <- pred_function(model_use = model_use,
                             model_select = model_select,
                             rounding = rounding,
                             new_data = data_use)

# Convert LDA multiclass to 0/1 for final comparison; retain original class
if(model_select  == "LDA")
  data_use$class <- as.numeric(ifelse(data_use$class == target_class, 1, 0))

# Round prediction
data_use <- data_use %>% mutate(pred_value = round(pred_value, 6))

# Predicted class based on cut-off value
data_use$pred_use <- as.numeric(ifelse(data_use$pred_value < cutoff, 0, 1))

# Correct and incorrect prediction
data_use <- data_use %>% mutate(pred_correct = if_else(pred_use == class, 1, 0))

# Store original order
data_use <- data_use %>% mutate(Original_order = row_number())

# Colour allocation for confusion categories (for plotting)
data_use <- data_use %>% mutate(
  pred_confusion = case_when(
    data_use[, "class"] == data_use[, "pred_use"] & data_use[, "class"] == 1 ~ 1,   # TP
    data_use[, "class"] == data_use[, "pred_use"] & data_use[, "class"] == 0 ~ 2,   # TN
    data_use[, "class"] != data_use[, "pred_use"] & data_use[, "class"] == 0 ~ 3,   # FP
    TRUE ~ 4                                                    # FN
  )
)

data_use <- data_use %>% mutate(
  pred_col = case_when(
    data_use[, "class"] == data_use[, "pred_use"] & data_use[, "class"] == 1 ~ "red",     # TP
    data_use[, "class"] == data_use[, "pred_use"] & data_use[, "class"] == 0 ~ "blue",    # TN
    data_use[, "class"] != data_use[, "pred_use"] & data_use[, "class"] == 0 ~ "purple",  # FP
    TRUE ~ "orange"                                                 # FN
  )
```

```
 )

 # Ensure different classes have different symbols. Class 1 = triangle
 data_use <- data_use %>% mutate(pch_x = if_else(data_use$class == 0, 16, 17))

 # Print quick metrics
 print("Confusion matrix: ")
 print(ConfusionMatrix(data_use$pred_use, data_use$class))
 print("Mean prediction: ")
 print(1 - mean(data_use$class)) # for comparison against accuracy: lift provided
 print("")
 print("Accuracy: ")
 print(Accuracy(data_use$pred_use, data_use$class))

 return(list(data_use = data_use))
}


##############################################
# 5) Biplot Input Calculation (PCA / CVA)
##############################################
#' Compute PCA/CVA biplot objects and centering/scaling
#'
#' @param X Feature matrix/data.frame (default train_data[, 1:num_vars])
#' @param standardise Logical; standardize features for PCA
#' @param main_heading Title for biplot
#' @param proj Integer vector of eigenvector indices to retain
#' @param CVA_method Logical; use CVA (if FALSE, PCA)
#' @param CVA_classes Factor of class labels for CVA
#'
#' @return list with X, X_center, X_sd, and biplot_out (biplotEZ object)
biplot_input_calc <- function(X = train_data[, 1:num_vars],
                      standardise = standardise_use, # for PCA
                      main_heading = "Test",
                      proj = 1:2,
                      CVA_method = FALSE,
                      CVA_classes = train_data$Original_class) { # default when CVA FALSE

 X_center <- apply(X, 2, "mean")
 X_sd <- apply(X, 2, "sd")

 # PCA
 if (CVA_method == FALSE) {
   create_bp <- biplot(X, scaled = standardise, Title = main_heading) |>
     PCA(e.vects = proj)
 }

 # CVA
 if (CVA_method == TRUE) {
```

```
   create_bp <- biplot(X, scaled = FALSE, Title = main_heading) |>
     CVA(e.vects = proj, classes = CVA_classes, show.class.means = FALSE)
  }

  return(list(
    #X = X,
    X_center = X_center,
    X_sd = X_sd,
    biplot_out = create_bp
  ))
}


############################################################
# 6) Biplot Plane Coloring, Grid, and Contours
############################################################
library(aplpack)
library(sp) # spatial data on the 2-D surface for polygon calculation

#' Create color-coded prediction grid on the biplot plane
#'
#' @param X Data with at least the first num_vars feature columns and plotting columns
#' @param model Model object used for predictions
#' @param varnames Character vector of feature names
#' @param numvars Number of features
#' @param model_select Character indicating model family
#' @param cutoff Decision threshold for contours
#' @param standardise Logical; standardization flag for back-projection
#' @param proj Indices of projection vectors
#' @param V Matrix of loadings (columns are eigenvectors)
#' @param tV Transpose of V
#' @param CVA_method Logical; whether CVA is used
#' @param Xcenter Numeric vector of means
#' @param Xsd Numeric vector of sds
#' @param m Integer grid density (per side)
#' @param polygon Optional SpatialPolygons to restrict grid area
#' @param no_polygon Logical; if TRUE, use full grid as polygon
#' @param calc_hull Logical; if TRUE, remove grid points outside polygon
#' @param calc_ct_in_hull Logical; if TRUE, compute contours with hull edge at cutoff
#' @param outlie Fraction for convex hull trimming (aplpack::plothulls)
#' @param b_margin Band around cutoff for contours
#' @param rounding Floor-based rounding for prediction scores
#' @param remove_filter Logical; retain for compatibility (not used)
#' @param return_values Logical; return list of grid and metadata
#'
#' @return list with Zgrid, Xgrid, grid.prob, Z.st, pred_col, pred_use, class,
#'   col.value, min_val, max_val, polygon, ct (if return_values=TRUE)
biplot_plane <- function(X = biplot_data,
                         model = model_use, varnames, numvars,
```

```
                model_select = model_select,
                cutoff = 0.5,
                standardise = FALSE, # default is FALSE to include in CVA method
                proj = c(1, 2),
                V = svdx$v, # data from X train to calibrate the biplot
                tV = t(svdx$v),
                CVA_method = FALSE,
                Xcenter = X_center, Xsd = X_sd,
                m = 300,
                polygon = NA,
                no_polygon = FALSE, # X.Poly not currently used, density not specified
                calc_hull = FALSE,
                calc_ct_in_hull = FALSE,
                outlie = 1,
                b_margin = 0.01, # contour lines and margin around cutoff
                rounding = 2, # must be consistent with the b-margin value
                remove_filter = FALSE,
                return_values = TRUE) {

Vr <- V[, proj]
tVr <- tV[proj, ]

{
  sv <- Xsd
  if(CVA_method == TRUE) standardise <- FALSE # set to FALSE if CVA is TRUE
  if(standardise  == FALSE) sv <- FALSE
  X.st <- scale(X[, 1:num_vars], center = Xcenter, scale = sv)
}

# In case of blank, create a plot to ensure new plot steps work correctly
if(dev.cur() == 1)  plot(1, 1, type = "n", asp = TRUE)

## Plot to get the grid area
dev.new()

{
  # First plot to get plot dimensions of x and y axes
  Z.st <- (X.st %*% Vr)
  colnames(Z.st) <- c("x", "y")


  if(is(polygon, "SpatialPolygons")) plot(polygon)
  ## NB: create the biplot
  # First plot to get plot dimensions of x and y axes: use direct, as will be used later
  if(!is(polygon, "SpatialPolygons")) plot(Z.st, asp = 1, ylab = "Y[2]", xlab = "Y[1]")
    # biplot_plot |>
    #   samples(opacity = 0, which = NULL) |>
    #   axes(col = "grey")  |> plot()
```

```
 # Get the plot dimensions
 dev.set(dev.cur())

 minx <- par("usr")[1]
 miny <- par("usr")[3]
 maxx <- par("usr")[2]
 maxy <- par("usr")[4]

 dev.off()
}

## Get the grid points
{
 # min/max values of x and y dimensions, allow square plot later
 min_val <- (min(minx, miny))
 max_val <- (max(maxx, maxy))

 # Grid points in plot area
 yseq <- xseq <- seq(from = min_val, to = max_val, length.out = m)
 Zgrid <- as.matrix(expand.grid(xseq, yseq))
 colnames(Zgrid) <- c("x", "y")

 # Convert back to X dimension space
 Xgrid <- as.data.frame(Zgrid %*% tVr)

 # Return the unstandardised Xgrid
 if(standardise == TRUE) Xgrid <- sweep(Xgrid, 2, Xsd, "*")
 Xgrid <- sweep(Xgrid, 2, Xcenter, "+")
 Xgrid <- as.data.frame(Xgrid)

 # Set feature names
 colnames(Xgrid) <- varnames
}

## Create convex hull (polygon) for the prediction area
# use existing polygon provided unless none provided or no_polygon == TRUE
{
 if(!is(polygon, "SpatialPolygons")) {
  # create polygon if no existing polygon was provided
  dev.new()
  polygon_points <- plothulls(
   x = Z.st[, 1], y = Z.st[, 2], asp = 1,
   xlim = c(min_val, max_val), ylim = c(min_val, max_val),
   fraction = outlie, n.hull = 1, main, add = FALSE, col.hull = 2,
   lty.hull = 1, lwd.hull = 2, density = 1
  )
  dev.off()
  polygon <- SpatialPolygons(list(Polygons(list(Polygon(polygon_points)), ID = "polygon")))
 }
```

```
 if(no_polygon == TRUE){
  # Use entire grid if not limiting to polygon: overwrite the existing polygon
  dev.new()
  polygon_points <- plothulls(
   x = Zgrid[, 1], y = Zgrid[, 2], asp = 1,
   xlim = c(min_val, max_val), ylim = c(min_val, max_val),
   fraction = 1, n.hull = 1, main, add = FALSE, col.hull = 2,
   lty.hull = 1, lwd.hull = 2, density = 1
  )
  dev.off()
  polygon <- SpatialPolygons(list(Polygons(list(Polygon(polygon_points)), ID = "polygon")))
 }

 ## use polygon to set the convex hull filter region: only apply if convex hull is true
 # Select grid points inside hull polygon
 G_data_points <- as.data.frame(Zgrid)
 coordinates(G_data_points) <- ~x + y
 G_data_points$inside_polygon <- !is.na(over(G_data_points, polygon))

 # Remove data points outside the polygon
 Z_data_points <- as.data.frame(Z.st)
 coordinates(Z_data_points) <- ~x + y
 Z_data_points$inside_polygon <- !is.na(over(Z_data_points, polygon))
}

# Score each grid point using the selected model
{
 grid.prob <- pred_function(model_use = model_use,
                            model_select = model_select,
                            rounding = rounding,
                            new_data = Xgrid)
}

# Color palette for grid values
{
 col.vec <- colorRampPalette(c("deepskyblue", "white", "lightsalmon"))(100 + 1)
 col.value <- col.vec[floor(grid.prob * 100) + 1]
}

################
## Contours  ##
################
{
 grid_contour <- matrix(grid.prob, ncol = m, byrow = FALSE)

 if(calc_ct_in_hull == TRUE) {
  # Compute contours in hull; make edge = cutoff to avoid double grouping
  grid_contour[G_data_points$inside_polygon == FALSE] <- cutoff
```

```
   ct <- contourLines(xseq, yseq, grid_contour, levels = c(cutoff - b_margin, cutoff, cutoff + b_margin))
   ct <- ct[sapply(ct, function(cl) cl$level != cut_off)]
  }

  if(calc_ct_in_hull == FALSE)
   ct <- contourLines(xseq, yseq, grid_contour, levels = c(cutoff - b_margin, cutoff + b_margin))
}

################################################
# Remove grid points outside the polygon
################################################

## use convex hull to remove point, based on polygon: No polygon implies no convex hull filter even if true

pred_col <- X$pred_col
pred_use <- X$pred_use
class <- X$class

if(calc_hull == TRUE) {
  grid.prob <-  grid.prob[G_data_points$inside_polygon == TRUE]
  col.value <-  col.value[G_data_points$inside_polygon == TRUE]
  Zgrid <- Zgrid[G_data_points$inside_polygon == TRUE, ]
  Xgrid <- Xgrid[G_data_points$inside_polygon == TRUE, ]

  Z.st <- Z.st[Z_data_points$inside_polygon == TRUE, ]
  pred_col <- pred_col[Z_data_points$inside_polygon == TRUE]
  pred_use <- pred_use[Z_data_points$inside_polygon == TRUE]
  class <- class[Z_data_points$inside_polygon == TRUE]
}

## base plot
dev.off()

# Retained for compatibility
if(remove_filter == TRUE) {
  col.value <- col.value[rows_to_exclude]
  grid.prob <- grid.prob[rows_to_exclude]
  Xgrid <- Xgrid[rows_to_exclude, ]
  Zgrid <- Zgrid[rows_to_exclude, ]
}

if(return_values == TRUE)
  return(list(
    Zgrid = Zgrid, Xgrid = Xgrid, grid.prob = grid.prob, Z.st = Z.st,
    pred_col = pred_col, pred_use = pred_use, class = class,
    col.value = col.value,
    min_val = min_val, max_val = max_val, # for 3d plots
    polygon = polygon, ct = ct
  ))
```

```
}


################################################
# 7) Plot Helpers for biplotEZ Grid & Points
################################################
#' Plot biplot grid, samples, axes, and optional contours using biplotEZ
#'
#' @param grid Output from biplot_plane
#' @param k Integer index of variable to offset label
#' @param label_dist Numeric label offset distance
#' @param tdp Index/indices of points to plot (default all)
#' @param V Loadings matrix (passed through)
#' @param no_grid Logical; hide colored grid
#' @param no_points Logical; hide sample points
#' @param no_contour Logical; hide contours
#' @param new_title Optional new biplot title
#' @param ticks_v Number of ticks on axes
#' @param cex_z Point size for samples
#' @param label_dir "Hor" or "Rad" label direction
#' @param tick.label.cex Axis tick label size
#' @param which Variable indices to label
#' @param X.names Custom variable names for axes
#'
#' @return Invisibly returns NULL (plot function)
plot_biplotEZ <- function(grid = biplot_grid,
                          biplot_plot = biplot_input$biplot_out,
                          k = 0,
                          label_dist = 0.5,
                          tdp = NA,
                          no_grid = FALSE,
                          no_points = FALSE,
                          no_contour = FALSE,
                          new_title = NA,
                          ticks_v = 1,
                          cex_z = 0.5,
                          label_dir = "Hor",
                          tick.label.cex = 0.6,
                          which = 1:num_vars,
                          X.names = var_names) {

  ## Set data from the biplot function
  Z.st <- grid$Z.st
  pred_col <- grid$pred_col
  Zgrid <- grid$Zgrid
  grid.value <- grid$col.value
  ct <- grid$ct

  # Update title if provided
```

```
if(!is.na(new_title))
  biplot_plot$Title <- new_title

# Move position of labels
label.line.vec <- rep(0, num_vars)
label.line.vec[k] <- label_dist

# Base plot
biplot_plot |>
  samples(opacity = 0, which = NULL) |>
  axes(col = "grey",
       label.dir = label_dir,
       which = which,
       X.names = X.names,
       tick.label.cex = tick.label.cex,
       ticks = ticks_v,
       label.line = label.line.vec)  |> plot()

## Add the grid/dense map
if(no_grid == FALSE) {
  points(Zgrid, type = "p", col = grid.value, pch = 15, cex = 0.5)
}

# Add points
if(is.na(tdp[1])) tdp <- 1:nrow(Z.st)

if(no_points == FALSE)
  points(x = Z.st[tdp, 1],
         y = Z.st[tdp, 2],
         type = "p",
         col = pred_col[tdp],
         pch = 16, cex = cex_z)

# Add axes on top
biplot_plot |>
  samples(opacity = 0, which = NULL) |>
  axes(col = "grey22",
       label.dir = label_dir,
       which = which,
       X.names = X.names,
       tick.label.cex = tick.label.cex,
       ticks = ticks_v,
       label.line = label.line.vec)  |> plot(add = TRUE)

# Add contour lines
if(no_contour == FALSE) {
  for(i in 1:length(ct)) {
    lines(ct[[i]]$x, ct[[i]]$y)
  }
```

```
 }

 invisible(NULL)
}


############################################################
# 8) Project Points into Biplot Coordinates (PCA/CVA)
############################################################
#' Project observations into biplot space
#'
#' @param X Feature data.frame/matrix
#' @param proj Projection indices (default pc.values)
#' @param standardise Logical; whether to standardize by Xsd
#' @param CVA_method Logical; if TRUE, force standardise = FALSE
#' @param tdp Indices to plot (if plot = TRUE)
#' @param Xcenter Numeric means
#' @param Xsd Numeric sds
#' @param V Loadings/eigenvector matrix
#' @param plot Logical; draw points (base graphics)
#' @param col_x Point color
#' @param pch_x Point pch
#' @param cex_x Point size
#'
#' @return Matrix with columns (x, y) of projected coordinates
point_inter_proj <- function(X, proj = pc.values,
                     standardise = FALSE, CVA_method = FALSE,
                     tdp = NA,
                     Xcenter = X_center, Xsd = X_sd,
                     V = V,
                     plot = FALSE,
                     col_x = 8, pch_x = 16, cex_x = 1.5) {

 # Apply the standardisation of X: class or testing data
 sv <- Xsd
 if(CVA_method == TRUE) standardise <- FALSE # set to FALSE if CVA is TRUE
 if(standardise  == FALSE) sv <- FALSE
 X.st <- scale(X, center = Xcenter, scale = sv)

 # Project to biplot coordinates
 Z_values <- X.st %*% V[, proj]
 colnames(Z_values) <- c("x", "y")

 if(is.na(tdp[1])) tdp <- 1:nrow(Z_values)

 if (plot == TRUE)
   points(x = Z_values[tdp, 1], y = Z_values[tdp, 2], cex = cex_x, col = col_x, pch = pch_x)

 Z_values <- Z_values[tdp,]
```

```
  return(Z_values = Z_values)
}

################################################
# 9) Filter Helpers (Dynamic Range & AND-combine)
################################################
library(dplyr)
library(rlang)
library(mlbench)

#' Get the range of values in each variable in a data.frame
#'
#' @param data data.frame
#'
#' @return List of expressions like var >= min & var <= max
get_variable_ranges <- function(data) {
  vars <- names(data)

  filters <- map(vars, function(var) {
    rng <- range(data[[var]], na.rm = TRUE)
    expr((!!sym(var)) >= !!rng[1] & (!!sym(var)) <= !!rng[2])
  })

  filters
}

#' Evaluate a list of expressions row-wise with AND logic
#'
#' @param data data.frame to filter
#' @param filter_exprs List of expressions (e.g., from get_variable_ranges)
#'
#' @return Logical vector indicating rows that pass all filters
get_filter_logical_vector <- function(data, filter_exprs) {
  # Combine multiple expressions with AND
  row_filter_expr <- reduce(filter_exprs, function(x, y) expr((!!x) & (!!y)))
  # Evaluate the logical expression on the dataset
  eval_tidy(row_filter_expr, data = data)
}
```

A1: Shapley value calculation

```
############################################################
# 10) Shapley Value Calculation (Exact via Permutations)
############################################################
#' Compute exact Shapley values for one or all observations
#'
#' WARNING: Complexity is factorial in the number of variables.
#'
#' @param model_use Model object for prediction
#' @param num_vars Number of variables (columns of test_data used)
#' @param test_data Data frame of observations (X)
#' @param B_boundary Data frame of boundary values (same cols as test_data)
#' @param individual Logical; if TRUE, compute only for tdp
#' @param tdp Integer row index to compute for when individual=TRUE
#' @param lda,rpart_used,svm_u Flags retained for compatibility
#' @param rounding Integer decimals for floor-based rounding
#'
#' @return list with shapley_store (matrix: nrow(test_data) x num_vars)
Run_Shapley <- function(model_use, num_vars, test_data,
                        B_boundary,
                        individual = FALSE, tdp = 1,
                        lda = FALSE, rpart_used = FALSE, svm_u = FALSE,
                        rounding = rounding) {

 n_orders <- factorial(num_vars)
 n_contrib <- matrix(unlist(permn(num_vars)), ncol = num_vars, byrow = TRUE)
 margin_calc <- rep(0, num_vars)
 margin_store <- matrix(0, nrow = n_orders, ncol = num_vars)
 shapley_store <- matrix(0, ncol = num_vars, nrow = nrow(test_data))

 run_s <- 1
 run_e <- nrow(test_data)

 # Single data point
 if(individual  == TRUE) {
  run_s <- tdp
  run_e <- tdp
 }

 for(t in run_s:run_e) {
  start_time = Sys.time()
  print(t)

  start_data <- as.numeric(test_data[t, ])
  new_data <- as.numeric(B_boundary[t, ])

  for(j in 1:n_orders){
```

```
    s_order <- n_contrib[j, ]
    s_contrib <- matrix(0, nrow = num_vars, ncol = num_vars)

    # Build marginal contribution mask for this permutation
    for(i in 1:num_vars){
      s_contrib[, s_order[i]] <- c(rep(0, i - 1), rep(1, num_vars - i + 1))
    }

    # Add opening row of zeros for starting value
    s_contrib <- rbind(c(rep(0, num_vars)), s_contrib)

    # Replace 0/1s with actual values
    s_contrib <- sweep(1 - s_contrib, 2, start_data, FUN = "*") + sweep(s_contrib, 2, new_data, FUN = "*")
    colnames(s_contrib) <- var_names
    s_contrib <- as.data.frame(s_contrib)

    # Predict
    s_value <- pred_function(model_use = model_use,
                             model_select = model_select,
                             rounding = rounding,
                             new_data = s_contrib)

    # Marginal differences
    for(i in 1:num_vars){
      margin_calc[i] <-  s_value[i + 1] - s_value[i]
    }
    margin_store[j, s_order] <- margin_calc
  }

  s_value[-4] - s_value[-1]  # (Retained from original script)

  # Sum marginal contributions per variable across permutations
  shapley_store[t, ] <- apply(margin_store, 2, sum) / n_orders

  end_time = Sys.time()
  print(end_time - start_time)
  print("...")
 }

 return(list(shapley_store = shapley_store))
}

################################################################
# Section 2.1: Data Input Preparation
# Description:
#   Script to prepare the dataset for modeling. It sets up the number of
#   independent variables, ensures class columns are numeric, stores
#   variable names, and labels the columns consistently for downstream
#   analysis.
```

```
#
# Notes:
#   * This script assumes `data` is already loaded in the environment.
################################################################


##############################
# 1) Select dataset and variables to include: names "data"
##############################

# "2 Data File.R"

##############################
# 2) Set Number of Variables
##############################

# Number of independent variables in the dataset
# Example: PIMA = 8; Iris = 4; Projection example = 2
num_vars <- ncol(data) - 1

##############################################
# 3) Prepare Class Columns for Multiclass Use
##############################################

# For multiclass problems:
#   * Keep the original class in column (num_vars+2)
#   * Set the prediction class in column (num_vars+1)
#   * Convert class column to numeric
#   * Assign Original_class for later use
data_transform <- data
data_transform[, num_vars + 1] <- as.numeric(data_transform[, num_vars + 1])
data_transform[, num_vars + 2] <- as.numeric(data_transform[, num_vars + 1]) # store original class

# if LDA keep the multi class classification
if(lda == F) data_transform[,num_vars+1] <- ifelse(data_transform[,num_vars+1]==tdp_class,1,0)
# final comparison will still be two class, so after fitting model, class and prediction will be made 0/1

##############################
# 4) Store Variable Names
##############################
# Capture the feature names for reference
var_names <- colnames(data_transform)[1:num_vars]

##############################
# 5) Rename Columns
##############################
# Assign consistent column names:
#   * Feature columns = var_names
#   * class = model class (numeric)
```

```
#   * Original_class = retained true class
colnames(data_transform) <- c(var_names, "class", "Original_class")

############################################################
# 6) data pre-process and split in test and train
############################################################

## first standardise the data: will need to convert final answer back
if(first_standardise  == T)  {
  data_sd <- apply(data, 2, "sd")
  data_center <- apply(data, 2, "mean")
  data[,1:num_vars] <- scale(data[,1:num_vars], center = T, scale = T)
}

##Data engineering:
# If you want to apply any data engineering, the procedure must be stored and reversible, to allow return to
original variables
# this is needed to interpret changes needed ito original variables understood by the user
# engineering such as converting categorical to numeric or removing outliers etc can be done not to retain it all
the data
# need to apply it to future test data to be explained
# Therefore, if standardisation is required, it is done later and process of reversing it is in the code
# standardise <- T used later is for visualisation and distance calc, not to improve the model prediction
accuracy needed in data engineering step

## split data into test and train
{
  set.seed(121)
  train_idx <- sample(nrow(data_transform), 4/5* nrow(data_transform))
  train_data <- data_transform[train_idx, ] #train
  test_data <- data_transform[-train_idx, ] #test: data to be explained
}

rm(data_transform)

#########################
####### END #############
#########################


################################################################
# Section 2.2: Model Fitting Utilities
# Description:
#   Requires "train_data" data.frame
#   Fits a selection of classification models (GLM, LDA, SVM, GBM, rpart,
#   GAM, NNET) to the provided training data, based on boolean flags.
#   The chosen model object and its identifier are returned.
#
# Notes:
```

```
#   * Assumes the following objects exist in the environment:
#       - var_names: character vector of feature names (length = num_vars)
#       - num_vars: integer, number of feature variables
#       - lda, svm_u, gbm_used, rpart_used, gam_used, nnet_used: logical flags
################################################################

##############################
# 0) Modeling Hints (Optional)
##############################
# You can specify the model directly and skip the generic parts by setting:
#   model_select <- "<MODEL>"  # e.g., "GLM", "LDA", "SVM", etc.
#   model_use    <- model_<model>


#################################
# 1) Model Fitting Wrapper
#################################
#' Fit a classification model according to selected flags
#'
#' Builds a formula class ~ x1 + x2 + ... and fits the chosen model(s).
#' The last enabled model flag determines the returned model.
#'
#' @param train_data Data frame containing features in columns var_names and
#'   a numeric binary 'class' column.
#'
#' @return list(model_use = <model object>, model_select = <character label>)
model_fitting <- function(train_data = train_data) {

 # Build formula: class ~ x1 + x2 + ...
 formula_data <- paste("class ~ ", var_names[1])
 for (i in 2:num_vars) {
  formula_data <- paste(formula_data, " + ", var_names[i])
 }
 formula_data <- as.formula(formula_data)

 # Base model: GLM (logistic)
 if (glm_use == T) {
  model_select <- "GLM"
  model_glm <- glm(formula_data, data = train_data, family = binomial(link = "logit"))
  model_use <- model_glm
 }

 # LDA (if enabled)
 if (lda == T) {
  model_lda <- lda(formula_data, data = train_data)
  model_select <- "LDA"
  # model_qda <- qda(formula_data, data = train_data)
 }
```

```
# Other models when LDA flag is FALSE
if (lda == F) {

 # SVM (if enabled)
 # Example alt options (commented): kernel='linear', scale=FALSE
 if (svm_u == T)  {
  model_svm <- svm(formula_data, data = train_data, probability = TRUE, type = "C-classification")
  model_select <- "SVM"
 }

 # GBM (if enabled)
 if (gbm_used == T) {
  model_gbm <- gbm(
   formula_data,
   data = train_data,
   distribution = "bernoulli",
   shrinkage = 0.005,
   n.minobsinnode = 10,
   n.trees = 1000,
   interaction.depth = 3
  )
  model_select <- "GBM"
 }

 # rpart (if enabled)
 if (rpart_used == T) {
  train_data_rpart <- train_data
  train_data_rpart[, num_vars + 1] <- as.factor(train_data_rpart[, num_vars + 1])
  model_rpart <- rpart(formula_data, data = train_data, method = "class", model = T, minsplit = 5)
  model_select <- "RForrest"
 }

 # GAM (if enabled)
 if (gam_used == T) {
  formula_data_gam <- paste("class ~ s(", var_names[1], ")")
  for (i in 2:num_vars) {
   formula_data_gam <- paste(formula_data_gam, " + s(", var_names[i], ")")
  }
  formula_data_gam <- as.formula(formula_data_gam)
  model_gam <- gam(formula_data_gam, data = train_data, family = binomial(link = "logit"))
  model_select <- "GAM"
 }

 # NNET (if enabled)
 if (nnet_used == T) {
  library(nnet)
  model_nn <- nnet(
   formula_data,
   data = train_data,
```

```
      size = c(20),
      decay = 0.001,
      maxit = 1000,
      linout = FALSE,  # Binary classification (no linear output)
      trace = FALSE
    )
    model_select <- "NNET"
  }
 }

 # Select final model object according to flags
 if (lda == T)            model_use <- model_lda   # or model_qda
 if (gam_used == T)       model_use <- model_gam
 if (gbm_used == T)       model_use <- model_gbm
 if (rpart_used == T)     model_use <- model_rpart
 if (svm_u == T)          model_use <- model_svm
 if (nnet_used == T)      model_use <- model_nn

 return(list(
  model_use   = model_use,
  model_select = model_select
 ))
}

##########################
####### END ##############
##########################
```

A2: Optimal Rotation of a Biplot

```
################################################################
# Section 2.3: Optimal Rotation of a Biplot (V → Vrho)
# Description:
#   Computes a rotation matrix A that aligns 2D biplot plane
#   spanned by `proj` (typically PCs 1 & 2) with a selected target point (tdp)
#   Returns the rotated loading matrices Vrho and tVrho for downstream use.
#
# Notes:

################################################################

####################################
####### Apply rho to projection 3d
####################################

#' Rotate the loading matrix so a target point lies on the biplot plane
#'
#' Constructs a 3-row matrix Y consisting of (-x_tdp, 0, +x_tdp) in standardized
#' feature space, then solves for a rotation A via SVD to map from full space
#' to the reduced (proj) space. Returns Vrho = V %*% t(A) and its inverse.
#'
#' @param X Data frame/matrix with features in the first num_vars columns
#' @param tdp Integer index of target data point for alignment
#' @param standardise Logical; whether to standardize using Xcenter/Xsd
#' @param Xcenter Numeric vector of feature means
#' @param Xsd Numeric vector of feature sds
#' @param CVA_method Logical; if TRUE forces standardise = FALSE
#' @param proj Integer vector of projection indices (e.g., c(1,2))
#' @param V Loading/eigenvector matrix (columns correspond to components)
#'
#' @return list(Vrho = rotated loadings, tVrho = inverse, A = rotation matrix)
Biplot_rotation <- function(X = biplot_data,
                    # Vmat = Vmat,
                    tdp = 1,
                    standardise = standardise_use, # set to false for CVA plots
                    Xcenter = X_center,
                    Xsd = X_sd,
                    CVA_method = CVA_method,
                    proj = pc.values,
                    V = V) {

  # Select scaling based on PCA/CVA choice
  sv <- Xsd
  if(CVA_method == T) standardise  <- F # set to false if CVA is true
  if(standardise  == F) sv = F

  # Standardize input (applies scaling to X; outputs scaled coordinates)
```

```
 X.st <- scale(X[, 1:num_vars], center = Xcenter, scale = sv)

 # Construct Y with three rows: -x_tdp, 0, +x_tdp
 Y <- as.matrix(rbind(-X.st[tdp,],
                rep(0, num_vars),
                X.st[tdp,]))

 # Extract loadings for full and reduced (proj) spaces
 V  <- V          # full dimensions
 Vr <- V[, proj]   # reduced biplot dimensions

 # Map Y to full PCA/CVA space and to reduced space
 YV  <- Y %*% V
 r   <- 2                    # target biplot dimensionality
 YVr <- Y %*% Vr               # reduced-space coords
 if(num_vars > r) YVr <- cbind(YVr, matrix(0, ncol = num_vars - r, nrow = 3))

 # Solve for rotation A using SVD of cross-covariance
 svdyx <- svd(t(YV) %*% YVr)
 A     <- svdyx$v %*% t(svdyx$u)

 # Rho diagnostic (kept for reference with original code)
 rho <- sum(diag(YVr %*% A %*% t(YV))) / sum(diag(t(YVr) %*% (YVr)))

 # Apply rotation to V and compute inverse
 Vrho  <- V %*% t(A)         # rotated loading matrix
 tVrho <- solve(Vrho)

 # Optionally bypass PCA/CVA (identity) if requested upstream
 if(no_pca  == T) Vrho <- diag(num_vars)

 return(list(Vrho = Vrho,
          tVrho = tVrho,
          A = A))
}

##########################
####### END ##############
##########################
```

A3: Biplot Boundary Search

```
################################################################
# Section 2.4: Biplot Boundary Search
# Description:
#   Formatting and safety guards
#   Uses pdist-based full distance matrices
################################################################

Biplot_boundary_search <- function(
  X = biplot_grid,
  tdp = NA,
  model_select = model_select,
  standardise = FALSE,              # default FALSE to include CVA method
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = FALSE,
  proj = c(1, 2),
  V = V,                            # from training data to calibrate biplot
  tV = tV,
  set_filters = NA,
  train_ranges = NA
)

## Testing data
# X = biplot_grid_Test
# model_select = model_select
# tdp = 28          # Use NA to search all points
# standardise = standardise_use
# Xcenter = X_center
# Xsd = X_sd
# CVA_method = CVA_method
# proj = pc.values
# V = V
# tV = tV
# set_filters = set_filters
# train_ranges = train_ranges

{
  ## --------------------------- Inputs unpack ---------------------------- ##
  if (is.na(tdp[1])) tdp <- seq_len(nrow(X$Z.st))
  ct           <- X$ct
  data_polygon  <- X$polygon
  Z.st         <- matrix(X$Z.st[tdp, ], ncol = 2)      # ensure 2-col matrix
  biplot_pred_use <- X$pred_use[tdp]

  ## -------------------- 1.1 Build boundaries & types -------------------- ##
  nr_boundaries     <- length(ct)
  z_boundaries_list <- vector("list", nr_boundaries)
```

```
z_boundary_type   <- numeric(nr_boundaries)

for (i in seq_len(nr_boundaries)) {
  boundary_vals <- as.matrix(cbind(ct[[i]]$x, ct[[i]]$y))
  colnames(boundary_vals) <- c("x", "y")
  z_boundaries_list[[i]] <- boundary_vals #matrix(boundary_vals, ncol = 2)
  z_boundary_type[i]     <- ct[[i]]$level
}

## ---------------- 1.2 Retain boundaries near the data ---------------- ##
boundary_used <- integer(0)
counter <- 0L

for (i in seq_len(nr_boundaries)) {
  #i<-2
  # Keep only contour points inside the polygon of data
  data_points <- as.data.frame(z_boundaries_list[[i]])
  coordinates(data_points) <- ~ x + y
  data_points$inside_polygon <- !is.na(over(data_points, data_polygon))
  z_boundaries_list[[i]] <- matrix(z_boundaries_list[[i]][data_points$inside_polygon, ],
                                   ncol = 2, byrow = FALSE)

  # Detect closed contour
  circle_contour <- 0L
  if (length(z_boundaries_list[[i]]) > 0L) {
    circle_contour <- ifelse(
      z_boundaries_list[[i]][1, 1] == z_boundaries_list[[i]][nrow(z_boundaries_list[[i]]), 1] &&
      z_boundaries_list[[i]][1, 2] == z_boundaries_list[[i]][nrow(z_boundaries_list[[i]]), 2],
      1L, 0L
    )
  }

  # If closed, check whether it surrounds data
  interior_data <- 0L
  if (length(z_boundaries_list[[i]]) > 0L && circle_contour == 1L) {
    interior_data <- sum(point.in.polygon(Z.st[, 1], Z.st[, 2],
                                          z_boundaries_list[[i]][, 1], z_boundaries_list[[i]][, 2]))
  }

  # Keep if not a circle, or is a circle with interior data
  if ((circle_contour == 0L || interior_data > 0L) && length(z_boundaries_list[[i]]) > 0L) {
    counter <- counter + 1L
    boundary_used[counter] <- i
  }
}

## ---------------- 1.3 Rebuild with retained boundaries ---------------- ##
if (!is.null(boundary_used)) nr_boundaries <- length(boundary_used)
if (is.null(boundary_used))  nr_boundaries <- length(ct)
```

```
if (is.null(boundary_used))  boundary_used <- seq_len(nr_boundaries)

z_boundaries_list <- vector("list", nr_boundaries)
z_boundary_type   <- numeric(nr_boundaries)

for (i in seq_len(nr_boundaries)) {
 j <- boundary_used[i]
 boundary_vals <- cbind(ct[[j]]$x, ct[[j]]$y)
 colnames(boundary_vals) <- c("x", "y")

 # Reduced set: only values within polygon
 data_points <- as.data.frame(boundary_vals)
 coordinates(data_points) <- ~ x + y
 data_points$inside_polygon <- !is.na(over(data_points, data_polygon))
 z_boundaries_list[[i]] <- matrix(boundary_vals[data_points$inside_polygon, ], ncol = 2)

 z_boundary_type[i] <- ct[[j]]$level
}

## -- 1.4 Remove contour points on the wrong side of decision boundary -- ##
for (i in seq_len(nr_boundaries)) {
 # Back to original X-space
 B_boundary_x <- z_boundaries_list[[i]] %*% tV[proj, , drop = FALSE]

 # Standardisation handling
 if (isTRUE(CVA_method)) standardise <- FALSE
 if (isTRUE(standardise)) B_boundary_x <- sweep(B_boundary_x, 2, Xsd, "*")
 B_boundary_x <- sweep(B_boundary_x, 2, Xcenter, "+")
 B_boundary_x <- as.data.frame(B_boundary_x)
 colnames(B_boundary_x) <- var_names

 # Feasibility filters
 rows_to_exclude1 <- get_filter_logical_vector(B_boundary_x, set_filters)
 rows_to_exclude2 <- get_filter_logical_vector(B_boundary_x, train_ranges)
 rows_to_exclude  <- rows_to_exclude1 * rows_to_exclude2

 # Score
 B_boundary_x_pred <- pred_function(
  model_use = model_use,
  model_select = model_select,
  rounding = rounding,
  new_data = B_boundary_x
 )

 # Keep only contour points that yield correct side of cutoff
 if (z_boundary_type[i] < cut_off) {
  correct_pred <- ifelse((B_boundary_x_pred < cut_off) * rows_to_exclude, 1L, 0L)
 } else {
  correct_pred <- ifelse((B_boundary_x_pred >= cut_off) * rows_to_exclude, 1L, 0L)
```

```
  }
  z_boundaries_list[[i]] <- z_boundaries_list[[i]][correct_pred != 0L, , drop = FALSE]
}

## ------------------ 2. Distance & local decision boundaries ---------- ##

# 2.1 Distances from each point to each boundary set (full pdist as original)
point.dist.matrix <- vector("list", nr_boundaries)
for (i in seq_len(nr_boundaries)) {
  point.dist.matrix[[i]] <- as.matrix(pdist(Z.st, z_boundaries_list[[i]]))
}

# For each boundary set, closest boundary point to each Z.st row
B_boundary_z <- vector("list", nr_boundaries)
bb_position  <- vector("list", nr_boundaries)
for (i in seq_len(nr_boundaries)) {
  if (nrow(point.dist.matrix[[i]]) > 0L) {
    bb_position[[i]] <- apply(point.dist.matrix[[i]], 1, "which.min")
    B_boundary_z[[i]] <- z_boundaries_list[[i]][bb_position[[i]], , drop = FALSE]
  } else {
    bb_position[[i]] <- integer(0)
    B_boundary_z[[i]] <- matrix(numeric(0), ncol = 2)
  }
}

# 2.2 Choose the closest across boundary sets; separate by opposite class
z_boundary_row <- matrix(0, ncol = 2, nrow = nr_boundaries)
boundary_class_0 <- z_boundary_type <  cut_off
boundary_class_1 <- z_boundary_type >= cut_off

boundary_set_nr   <- integer(nrow(Z.st))
boundary_set_nr_0 <- integer(nrow(Z.st))
boundary_set_nr_1 <- integer(nrow(Z.st))

B_boundary_z_value_0 <- matrix(0, nrow = nrow(Z.st), ncol = 2)
B_boundary_z_value_1 <- matrix(0, nrow = nrow(Z.st), ncol = 2)

group1 <- (biplot_pred_use == 1)
group0 <- (biplot_pred_use == 0)

position_store <- matrix(0L, ncol = 2, nrow = nrow(Z.st))

for (j in seq_len(nrow(Z.st))) {
  for (i in seq_len(nr_boundaries)) {
    if (length(B_boundary_z[[i]]) == 0L) {
      z_boundary_row[i, ] <- c(1e8, 1e8)  # large value to ignore empties
    } else {
      z_boundary_row[i, ] <- matrix(B_boundary_z[[i]], ncol = 2)[j, ]
    }
```

```
  }

  boundary_set_nr[j]   <- apply(as.matrix(pdist(Z.st[j, ], z_boundary_row)), 1, "which.min")
  boundary_set_nr_0[j] <- apply(as.matrix(pdist(Z.st[j, ], z_boundary_row[boundary_class_1, , drop = FALSE])),
1, "which.min")
  boundary_set_nr_1[j] <- apply(as.matrix(pdist(Z.st[j, ], z_boundary_row[boundary_class_0, , drop = FALSE])),
1, "which.min")

  position_0 <- sum(boundary_class_0) + boundary_set_nr_0[j]
  position_1 <- boundary_set_nr_1[j]

  B_boundary_z_value_0[j, ] <- if (length(B_boundary_z[[position_0]]) == 0L) {
   c(NA_real_, NA_real_)
  } else {
   matrix(B_boundary_z[[position_0]], ncol = 2)[j, ]
  }

  B_boundary_z_value_1[j, ] <- if (length(B_boundary_z[[position_1]]) == 0L) {
   c(NA_real_, NA_real_)
  } else {
   matrix(B_boundary_z[[position_1]], ncol = 2)[j, ]
  }

  position_store[j, 1] <- position_0
  position_store[j, 2] <- position_1
 }

 # Only counterfactual from the opposite side
 B_boundary_z_value_0[group1, ] <- c(0, 0)
 B_boundary_z_value_1[group0, ] <- c(0, 0)
 B_boundary_z_value <- B_boundary_z_value_0 + B_boundary_z_value_1

 ## ---------- 3. Back-project & score (X) ---------- ##
 B_boundary_x_value <- B_boundary_z_value %*% tV[proj, , drop = FALSE]
 if (isTRUE(CVA_method)) standardise <- FALSE
 if (isTRUE(standardise)) B_boundary_x_value <- sweep(B_boundary_x_value, 2, Xsd, "*")
 B_boundary_x_value <- sweep(B_boundary_x_value, 2, Xcenter, "+")
 B_boundary_x_value <- as.data.frame(B_boundary_x_value)
 colnames(B_boundary_x_value) <- var_names

 B_boundary_x_pred <- pred_function(
  model_use = model_use,
  model_select = model_select,
  rounding = rounding,
  new_data = B_boundary_x_value
 )

 return(list(
  B_boundary_z_value = B_boundary_z_value,
```

```
    B_boundary_x_value = B_boundary_x_value,
    B_boundary_x_pred  = B_boundary_x_pred,
    z_boundaries_list  = z_boundaries_list,
    z_boundary_type    = z_boundary_type,
    nr_boundaries      = nr_boundaries,
    bb_position        = bb_position,
    position_store     = position_store
  ))
}

##########################
####### END ##############
##########################
```

A4: Biplot Search of local decision boundaries

```
################################################################
# Main section: Biplot Search of local decision boundaries
# Description:
#   Main driver for loading trainig data, setting global flags, fitting/using a
#   selected model, set tthe g and V functions, constructing the biplot (PCA/CVA) and
#   generating the prediction grid and contours, projecting points

#
# Notes:
#   * Requires the helper scripts and RData files referenced via source()/load().
#   * Assumes dependent scripts define functions used here (see sources).
################################################################

################################################################
# Step 0: Load functions and set model parameters
################################################################
rm(list = ls())
source("1 Functions for biplot.R")

{
 # Global switches
 first_standardise <- FALSE       # If FALSE: modeling uses standardized data in helper scripts
 no_polygon <- FALSE              # If TRUE: do not limit to convex hull polygon
 train_polygon <- NA
 no_pca <- FALSE                  # If TRUE: return rotation = identity (original space)
 cut_off <- 0.5                   # Classification threshold

 # Biplot method and standardization
 CVA_method <- F
 standardise_use <- TRUE
 if (CVA_method == TRUE) standardise_use <- FALSE

 pc.values <- c(1, 2)
 rounding <- 2
 m_grid <- 200                    # Grid density (reduce for heavy models)

 # LDB filter defaults
 set_filters <- list(TRUE)

 # Whether to re-run with polygon filters later
 Polygon_filter <- FALSE
}

################################################################
# Step 1 and 2: Load data and Fit model f(x) (or load pre-fit model)
################################################################
```

```
# Select the MLM type used
# to name the plot in graphs later automatically: Set default GLM
{
 glm_use <- T
 lda <- F
 svm_u <- F
 gam_used <- F
 gbm_used <- F
 rpart_used <- F
 nnet_used <- F
}

# load / create the "data" data.frame - including the test data, saved as "model_data"
# Open "2 Data File.R" and select the correct data: create "data" data.frame

# prepare the data file for use by MLM and Biplots
source("2.1 Data Setup.R")        # Updates data object with var_names, num_vars, etc.

# fit the MLM model , or load existing
{
 source("2.2 Model_use fitting.R") # run model, Requires "train_data" data.frame

 if(Polygon_filter == F) model_results <- model_fitting(train_data = train_data)
 if(Polygon_filter == T) model_results <- model_fitting(train_data =   biplot_data[,1:(num_vars+2)])

 # Allocate results, or load exising model
 model_select <- model_results$model_select
 model_use <- model_results$model_use
 summary(model_use)
}

# load functions
source("2.3 Optimal Rotation.R")   # Provides Biplot_rotation
source("2.4 Biplot Boundary Search.R")  # Provides Biplot_boundary_search

############################################################
# Step 3.1: Select train/test data and set feasibility filters
############################################################

{
 # find the ranges
 if (Polygon_filter == FALSE) train_ranges <- get_variable_ranges(train_data[, 1:num_vars])
 if (Polygon_filter == TRUE) train_ranges <- get_variable_ranges(biplot_data[, 1:num_vars])

 # filter by ranges: Row-selection logical vector: a
 if (Polygon_filter == FALSE) rows_to_exclude <- get_filter_logical_vector(train_data[, 1:num_vars],
train_ranges)
 if (Polygon_filter == TRUE) rows_to_exclude <- get_filter_logical_vector(biplot_data[, 1:num_vars],
train_ranges)
```

```
}

# set start data
{
 if (Polygon_filter == FALSE) start_data <- train_data[rows_to_exclude, ]
 if (Polygon_filter == TRUE) start_data <- biplot_data[, c(1:(num_vars + 2))]
}

#inspect the data
str(start_data)

#################################################################
# Step 3.2: Create data input for biplot: Determine point colours (classification results)
#################################################################
# Option 1: Re-run with polygon filters
#Polygon_filter <- TRUE

biplot_data <- set_model_data(
 model_use,
 data_use = start_data,
 num_vars = num_vars,
 cutoff = cut_off,
 rounding = 3,
 target_class = tdp_class,
 model_select = model_select,
 polygon = train_polygon,
 proj = pc.values,
 CVA_method = CVA_method,
 standardise = standardise_use
)$data_use

# set colours to target data values for PCA
# biplot_data$pred_col <- if_else(biplot_data$class == 1, "red", "blue")

#################################################################
# Step 4: Get projection matrix V (PCA/CVA) and standardisation function g
#################################################################
{
 biplot_input <- biplot_input_calc(
  X = biplot_data[, c(1:num_vars)],
  standardise = standardise_use,
  CVA_method = CVA_method,
  main_heading = paste0(ifelse(CVA_method == TRUE, "CVA", "PCA"),
                        " Biplot based ", model_select),
  CVA_classes = as.factor(biplot_data[, "pred_col"])   # Allow 4 classes for CVA biplot
 )

 V <- Vmat <- biplot_input$biplot_out$Lmat
 tV <- tVmat <- solve(Vmat)
```

```
  X_center <- biplot_input$X_center
  X_sd <- biplot_input$X_sd
  Wmat <- biplot_input$biplot_out$Wmat
  biplot_ax.one.unit <- biplot_input$biplot_out$ax.one.unit

}

#############################################################
# Steps 5–9 (Section 1): Build initial biplot grid & contours
#############################################################
rounding <- 2
outlie <- ifelse(Polygon_filter == TRUE, 1, 0.9)

biplot_grid <- biplot_plane(
  X = biplot_data,
  model_use,
  var_names,
  numvars = num_vars,
  model_select = model_select,
  cutoff = cut_off,
  standardise = standardise_use,
  Xcenter = X_center,
  Xsd = X_sd,
  CVA_method = CVA_method,
  proj = pc.values,
  V = V,
  tV = tV,
  polygon = NA,          # Could also use biplot_train$polygon
  no_polygon = FALSE,
  calc_hull = FALSE,#TRUE,
  calc_ct_in_hull = FALSE,
  outlie = outlie,
  m = m_grid,
  b_margin = 1 / (10^rounding),
  rounding = rounding
)

# Keep polygon for later searches if needed
if (Polygon_filter == FALSE) train_polygon <- biplot_grid$polygon
if (Polygon_filter == TRUE) final_polygon <- biplot_grid$polygon

#############################################################
# Step 9 (Part 2): Plot grid & points
#############################################################
var_names

plot_biplotEZ(
  grid = biplot_grid,
```

```
  # Label offsets (tune as needed)

  #PIMA PCA
  # k = c(8, 4, 5, 6, 3),
  # label_dist = c(0, 1, 0, 0.5,0.5),
  # which = c(4,5),
  # k = c(1, 2),
  # label_dist = c(0.5, 0),

  #Loans PCA
  # k = c(6,7,4,1,2),
  # label_dist = c(0.5, 1, 1,0.5,0.5),

  #Loans initial CVA
  # k = c(6,4,1),
  # label_dist = c(0.5, 1,0.5,1),

  #Loans updated CVA
  # k = c(3,5,1,4,8,9),
  # label_dist = c(0, 0.5,2,1.5,1,0.5),

  tick.label.cex = 0.75,

  # points and features
  cex_z = 0.7,
  no_grid = F,
  no_points = FALSE,
  no_contour = F,
  label_dir = "Paral",
  ticks_v = 3,
  new_title = "CVA biplot of filtered training data"
)

### Investigate the plot

################################################################
# Steps 10–11: Run CF search (unconstrained or constrained)
################################################################

biplot_search <- Biplot_boundary_search(
  X = biplot_grid,
  model_select = model_select,
  tdp = tdp,          # Use NA to search all points
  standardise = standardise_use,
  Xcenter = X_center,
  Xsd = X_sd,
  CVA_method = CVA_method,
  proj = pc.values,
  V = V,
```

```
  tV = tV,
  set_filters = set_filters,
  train_ranges = train_ranges
)

# Plot TDP or CF
ztdp <- point_inter_proj(
  #X = biplot_data[, 1:num_vars],
  X = biplot_search$B_boundary_x_value,
  V = V,
  #tdp = 1,
  plot = TRUE,
  #col_x = "green",
  col_x = biplot_data$pred_col,
  cex_x = 1.00,
  standardise = standardise_use,
  CVA_method = CVA_method
)


################################################################
# Section 3.2 : Biplot plot and search with Test data
# Description:
#   Continues from file 3.1, with the test data now generated
#   Generates biplot_dataT and biplot_grid_Test, which can be used later
# Notes:
#   * Requires the helper scripts and RData files referenced via source()/load().
################################################################

################################################################
# Restart Step 3: Use TEST data
################################################################

  model_data <- test_data

{
  # Ensure TEST data is within TRAIN ranges
  rows_to_exclude <- get_filter_logical_vector(model_data, train_ranges)
  model_data <- model_data[rows_to_exclude, ]

  # Create biplot-ready test data, limited by training polygon if provided
  biplot_dataT <- set_model_data(
    model_use,
    data_use = model_data,
    num_vars = num_vars,
    cutoff = cut_off,
    rounding = 3,
    target_class = tdp_class,
    model_select = model_select,
```

```
   polygon = NA, #train_polygon,         # If provided; NA by default, can also use final_polygon at end of training
process
   proj = pc.values,
   CVA_method = CVA_method,
   standardise = standardise_use
  )$data_use
}

#############################################################
# Step 4 (optional): Recompute V from TEST (if desired)
#############################################################
# If you want a new V/tV for TEST, re-run biplot_input_calc here.
# This section is commented out to preserve the original flow.
# {
#   biplot_input <- biplot_input_calc(
#     X = biplot_dataT[, c(1:num_vars)],
#     standardise = standardise_use,
#     CVA_method = CVA_method,
#     main_heading = "Create Biplot using projection matrix V2",
#     CVA_classes = as.factor(biplot_data[, "pred_col"])
#   )
#   V  <- Vmat <- biplot_input$biplot_out$Lmat
#   tV <- tVmat <- solve(Vmat)
#   X_center <- biplot_input$X_center
#   X_sd <- biplot_input$X_sd
#   X.st <- scale(biplot_input$X, X_center, X_sd)
#   Wmat <- biplot_input$biplot_out$Wmat
#   biplot_ax.one.unit <- biplot_input$biplot_out$ax.one.unit
# }

#############################################################
# Step 4 (part 2): Select rotation and compute Vrho
#############################################################

# Choose target data point index for rotation (unconstrained solution)
tdp <- 30
biplot_dataT[tdp, ]

pc.values <- c(1, 2)

Rotate <- T
{
  if (Rotate == TRUE) {
    V_rotate <- Biplot_rotation(
      X = biplot_dataT,
      tdp = tdp,
      standardise = standardise_use,
      Xcenter = X_center,
      Xsd = X_sd,
```

```
    CVA_method = CVA_method,
    proj = pc.values,
    V = Vmat
   )

   V <- V_rotate$Vrho
   tV <- V_rotate$tVrho

   # update the biplot for the new projection matrix
   {
    Vr <- V[,pc.values]
    tVr <- tV[pc.values,]

    # Minv <- solve(out$Lmat)[e.vects, , drop = F]
    # bp$ax.one.unit <- 1/(diag(t(Minv) %*% Minv)) * t(Minv)
    # PCA: ax.one.unit <- 1/(diag(Vr %*% t(Vr))) * Vr

    biplot_input$biplot_out$ax.one.unit <- 1/(diag(t(tVr) %*% tVr)) * t(tVr)
    biplot_input$biplot_out$Lmat <- V
    biplot_input$biplot_out$Vr <- Vr
    biplot_input$biplot_out$Z <- biplot_input$biplot_out$X %*% biplot_input$biplot_out$Vr
   }
  }

  if (Rotate == FALSE) {
   V <- Vmat
   Vr <- V[,pc.values]
   tV <- tVmat

   biplot_input$biplot_out$ax.one.unit <- biplot_ax.one.unit
   biplot_input$biplot_out$Lmat <- Vmat
   biplot_input$biplot_out$Vr <- Vr
   biplot_input$biplot_out$Z <- biplot_input$biplot_out$X %*% biplot_input$biplot_out$Vr
  }
}

#################################################################
# Step 5-9 (rotated): Build grid & contours for rotated V
#################################################################
pc.values <- c(1, 2)

biplot_grid_Test <- biplot_plane(
  X = biplot_dataT,
  model_use,
  var_names,
  numvars = num_vars,
  model_select = model_select,
  cutoff = cut_off,
  standardise = standardise_use,
```

```
  Xcenter = X_center,
  Xsd = X_sd,
  CVA_method = CVA_method,
  proj = pc.values,
  V = V,
  tV = tV,
  polygon = NA,#train_polygon,     # Limit test to training area if desired: not applicable after a rotation
  no_polygon = F,          # Override polygon (use full area)
  calc_hull = F,          # Irrelevant when no_polygon = TRUE: Need to be TRUE for Biplot CF search
  calc_ct_in_hull = FALSE,
  m = m_grid,
  rounding = 2,
  b_margin = 1 / (10^2)
)

#################################################################
# Step 9 (plot rotated grid & points)
#################################################################
var_names

plot_biplotEZ(
  grid = biplot_grid_Test,
  biplot_plot = biplot_input$biplot_out,

   #k = c(7,4,5,6),
   #label_dist = c(0.5,1,0.5,1),
   tick.label.cex = 0.85,

   tdp = tdp,
  cex_z = 0.65,

  #
  no_grid = T,
  no_contour = T,
  no_points = T,
  label_dir = "Paral",
  ticks_v = 4,
  #new_title = "",
  new_title = "Rotated Component biplot"
)

# Plot Ttdp# Plot TDP in rotated space
ztdp <- point_inter_proj(
  X = biplot_dataT[, 1:num_vars],
  V = V,
  tdp = tdp,
  plot = TRUE,
  col_x = "green",
  #col_x = biplot_dataT$pred_col,
```

```
  cex_x = 1.25,
  standardise = standardise_use,
  CVA_method = CVA_method
)

#############################################################
# Steps 10–11: Run CF search (unconstrained or constrained)
#############################################################
set_filters <- list(TRUE)

# Example constrained filters (commented):
# set_filters <- list(
#   expr(Age >= 33  & Age <= 61),
#   expr(Pregnancies >= 4 & Pregnancies <= 21)
# )

biplot_search <- Biplot_boundary_search(
  X = biplot_grid_Test,
  model_select = model_select,
  tdp = tdp,          # Use NA to search all points
  standardise = standardise_use,
  Xcenter = X_center,
  Xsd = X_sd,
  CVA_method = CVA_method,
  proj = pc.values,
  V = V,
  tV = tV,
  set_filters = set_filters,
  train_ranges = train_ranges
)

# Plot the CF solution
plot_search <- point_inter_proj(
  X = biplot_search$B_boundary_x_value,
  V = V,
#  tdp = 1,
  plot = TRUE,
  col_x = "darkgreen",
  cex_x = 1.25,
  standardise = standardise_use,
  CVA_method = CVA_method
)

## connect tdp to cf
# segments(ztdp[,1], ztdp[,2], plot_search[,1], plot_search[,2], col = "grey70",lwd = 2)


# Inspect results
biplot_dataT[tdp, ]
```

```
biplot_search$B_boundary_x_value[1,]

#################################################################
# End of Main Section
#################################################################
```

A5: Global Interpretations of Robustness and Variable Importance

```
################################################################
# Section 4.1: Global Interpretations – Robust & VIP
# Description:
#   Plot global variable importance-like diagnostics based on
#   distances to the decision boundary on the biplot plane.
#   Guards against undefined vars
#
# Notes:
#   * Assumes objects from the main driver exist (e.g., Vmat, tVmat,
#     biplot_grid, train_ranges, model_use, model_select, etc.).

################################################################

#####################################
############  Plot change  #########
#####################################

## Use unrotated biplots with no additional filters
## The biplot polygon should come from TRAIN to avoid implausible CFs.
## biplot_plane already provides a polygon; use no_polygon = FALSE and calc_hull = TRUE.

# Filters: keep structure but allow easy extension
set_filters <- list(TRUE)

# Main training biplot: no rotation, select grid to search
{
  V <- Vmat
  tV <- tVmat
  biplot_input$biplot_out$ax.one.unit <- biplot_ax.one.unit
  pc.values <- c(1, 2)
  biplot_grid_search <- biplot_grid # Set the biplot to search
}

# Run Boundary Search on the existing (training) grid
{
  biplot_search <- Biplot_boundary_search(
    X = biplot_grid_search, # No require biplot_data, only the results of the biplot_grid
    model_select = model_select,
    standardise = standardise_use,   # FALSE automatically if CVA_method == TRUE
    Xcenter = X_center,
    Xsd = X_sd,
    CVA_method = CVA_method,
    proj = pc.values,
    V = V,
    tV = tV,
    set_filters = set_filters,
    train_ranges = train_ranges
```

```
 )
}

########################################
## VIP-style plot (distance to boundary)
########################################

{
 # Color semantics for points (predicted class)
 class_data <- biplot_grid_search$pred_use
 # Alternative options:
 # class_data <- biplot_data[ , "pred_col"]
 # class_data <- biplot_grid$class


 # Distances are measured on the biplot plane, not in original X-space
 # Vector from each selected point to its nearest boundary point in Z-space
 Z_st_subset <- as.matrix(biplot_grid_search$Z.st) #data points
 Bz_subset   <- as.matrix(biplot_search$B_boundary_z_value) # counterfactuals
 vec_to_boundary_Z <- Bz_subset - Z_st_subset

 # Map to X-space using the inverse loading rows for the plotted components
 # (2×p block of tV)
 tVr <- tV[pc.values,]
 vec_to_boundary <- vec_to_boundary_Z %*% tVr
 colnames(vec_to_boundary) <- var_names
 vec_to_boundary <- as.matrix(vec_to_boundary)

 # If PCA was standardised, convert back by multiplying by X_sd
 if (isTRUE(standardise_use)) {
  vec_to_boundary <- sweep(vec_to_boundary, 2, X_sd, "*")
 }

 # Standardise per-variable for comparability (like z-scores by sd)
 vec_to_boundary_sd <- sweep(vec_to_boundary, 2, X_sd, "/")

 # Combine with class labels for plotting
 boundary_vec_class <- data.frame(
  class = class_data,
  vec_to_boundary_sd,
  check.names = FALSE
 )

 # Aggregate absolute distances per variable across all selected points
 sum_of_distance <- colSums(abs(vec_to_boundary_sd), na.rm = TRUE)

 # Rename columns to include totals (rounded)
 distance_names <- paste0(colnames(boundary_vec_class)[-1],
                          " : ", round(sum_of_distance, 0))
```

```
 colnames(boundary_vec_class)[-1] <- distance_names

 # Order variables by total absolute distance
 ord <- order(sum_of_distance)
 boundary_vec_class <- boundary_vec_class[, c(1, ord + 1L)]

 # Convert class for ggplot colouring (keep 0/1 semantics)
 boundary_vec_class[["class"]] <- factor(boundary_vec_class[["class"]] + 1,
                                         labels = c("0", "1"))

 # Long format for ggplot
 dfp1 <- reshape2::melt(boundary_vec_class,
                        id.vars = c("class"),
                        variable.name = "Variable")
 dfp1$value <- round(as.numeric(dfp1$value), 2)

 # Plot
 distance_plot <- ggplot(data = dfp1, aes(x = value)) +
   geom_jitter(aes(y = Variable, colour = class), size = 1.5) +
   labs(
     x = "Distance to Boundary – Standardised",
     y = "Variable : sum of distance to boundary",
     title = "Variable Distance to Boundary"
   ) +
   scale_color_manual(values = c("0" = "deepskyblue", "1" = "#F8766D")) +
   geom_vline(xintercept = 0) +
   labs(color = "Prediction Class") +
   theme_light() +
   theme(legend.position = "bottom")
}

print(distance_plot)
print(sum_of_distance)

################################################
#### Robustness (total distance) ###############
################################################

# Total absolute distance across variables (rough global robustness)
sum(sum_of_distance, na.rm = TRUE)

# Compare to model-based VIP
library(vip)
vip(model_use)

#####################################
##########  END  ####################
#####################################
```

A6: Surrogate Models

```
################################################################
# Section 4.2: Surrogate models
# Description:
#   Surrogate classification via biplot contour grouping.
################################################################

###################################
####### option 1  #################
###################################

# Select nearest boundary set number, then determine that contour value's class
# (Assumes objects from prior steps exist in the environment.)
# - Expects that 4.2.1 Create Grouping has constructed biplot_grouping with fields:
#   Z.st, ct, polygon, pred_use, class, etc.

# !!!! NB !!!! set the biplot_grid to search
biplot_grid_search <- biplot_grid_Test # Set the biplot to search
biplot_data_surrogate <- biplot_dataT # Set the data points to search

source("4.2.1 Create Grouping.R")

{
  # Contour value and class/colour based on cutoff
  ct_value <- z_boundary_type[boundary_set_nr]
  B_boundary_class <- ifelse(ct_value < cut_off, 0, 1)       # classify by score
  B_boundary_col   <- ifelse(ct_value < cut_off, "blue", "red")  # colour by classification
  # pch_z <- as.matrix(if_else(biplot_data$class == 0, 16, 17))  # shape by actual class (optional)

  # Surrogate model accuracy:
  # Compare surrogate class vs. biplot_grouping predictions
  ConfusionMatrix(B_boundary_class, biplot_grouping$pred_use)
  Accuracy(B_boundary_class, biplot_grouping$pred_use)

  # biplot_grouping may filter rows; compare with its retained data/class as well
  ConfusionMatrix(B_boundary_class, biplot_grouping$class)
  Accuracy(B_boundary_class, biplot_grouping$class)
}


# Update the colours on the grouping grid based on surrogate class
biplot_grouping$pred_col <- B_boundary_col

# Plot (kept as in original; tweak aesthetic params upstream if needed)
plot_biplotEZ(
  grid = biplot_grouping,
  # k = c(1, 4),
  # label_dist = c(0.5),
```

```
  cex_z = 0.65,
  no_points = FALSE,
  tick.label.cex = 0.9,
  label_dir = "Paral",
  ticks_v = 5,
  new_title = "CVA Biplot GBM Surrogate Model"

)

###################################
####### option 2  ################
###################################

# Allocate class based on prediction score of closest grid point:
# - Compute distance from each Z.st point to each grid point
# - Select gridscore of the closest grid point
# - If gridscore > cutoff -> class 1, else class 0
# (Implementation not changed here; see original notes.)


#################################################################
# Script: 4.2.1 Create Grouping
# Description:
#   Build the biplot grid for surrogate grouping and prepare inputs
#   for assigning points to the nearest contour (ct).
#
# Notes:
#   * Uses training-data V (Vmat/tVmat), not any rotated V.
#   * Key difference for grouping: calc_ct_in_hull = TRUE and no_polygon = TRUE.
#################################################################

# --- Rounding for contour grid ----------------------------------------
rounding_ct <- rounding

# --- Build grouping grid (biplot_grouping) ----------------------------------
#   Based on training data biplot_data
biplot_grouping <- biplot_plane(
  X = biplot_data_surrogate,
  model_use, var_names, numvars = num_vars, cutoff = cut_off,
  model_select = model_select,
  # Standardisation handling (FALSE automatically if CVA_method == TRUE)
  standardise = standardise_use,
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = CVA_method,
  proj = pc.values,
  # Use starting V from training (never the rotated values)
  V = Vmat, tV = tVmat,
  # Ignore any pre-existing polygon; create a new one for grouping
```

```
  # no_polygon must be TRUE for grouping; calc_hull must be TRUE
  no_polygon = TRUE,
  calc_hull = TRUE,
  # NB: outer ct hull value used to group by ct value
  calc_ct_in_hull = TRUE,   # Set TRUE for biplot grouping, together with no_polygon = TRUE
  # outlie = 0.9,          # (optional) keep as-is, commented in original
  m = m_grid,
  b_margin = 1/(10^rounding_ct), rounding = rounding_ct,
  return_values = TRUE
)

# --- Prepare inputs for grouping by contours ------------------------------
{
  # 1) Data to classify based on the nearest ct-line
  #    (X.st not needed here; we take the 2D Z.st from biplot_grouping)
  Z.st <- biplot_grouping$Z.st

  # 2) Contours and polygon from training grid
  ct <- biplot_grouping$ct
  data_polygon <- biplot_grouping$polygon  # select CT within the polygon

  # 3) Get grouping based on contours (produces boundary_set_nr, boundary_used)
  source("4.2.2 Biplot Grouping CT.R")

  # --- Optional: derive surrogate class from contour levels ----------------
  #     boundary_class aligns each retained boundary to class 0/1 via cutoff,
  #     then indexes by boundary_set_nr for point-level labels.
  boundary_class <- c()
  for (i in 1:length(boundary_used)) boundary_class[i] <- ct[[boundary_used[i]]]$level
  boundary_class <- ifelse(boundary_class > cut_off, 1, 0)
  boundary_class <- boundary_class[boundary_set_nr]

  # (Optional) Alternative subclass grouping ideas kept from original:
  # original_class <- unique(train_data$Original_class)
  # nr_original_class <- length(original_class)
  # new_class <- original_class
  # for (i in 1:nr_original_class) {
  #   j <- train_data$Original_class == original_class[i]
  #   new_class[j] <- boundary_set_nr[j] + original_class[i] * nr_original_class
  # }
}

#####################################################################
#### Plotting of grouping results based on boundary_set_nr   ####
#####################################################################
# (Plotting is done in the surrogate script; this file only prepares inputs.)

#############################################################
# Script: 4.2.2 Biplot grouping with contour
```

```
# Description:
#   Assign each projected point (rows of Z.st) to the nearest contour
#   (ct level) inside the training-data polygon. This is a grouping
#   helper used by the surrogate workflow
#
# Inputs expected in the environment:
#   - Z.st         : n x 2 matrix of biplot coordinates (from biplot_grouping$Z.st)
#   - ct           : list of contour sets as returned by biplot_plane(... )$ct
#   - data_polygon  : SpatialPolygons object (from biplot_grouping$polygon)
#
# Outputs placed in the environment:
#   - boundary_set_nr : integer vector (length nrow(Z.st)) indicating the
#                       closest contour set index for each point
#   - boundary_used   : integer vector of indices of contour sets retained
#
# Notes vs. boundary-search code:
#   * No removal of "incorrect" predictions (no CF enforcing).
#   * No reordering to force counterfactual pairing.
################################################################

# ----------------------------- 1) Build boundaries ------------------------
nr_boundaries    <- length(ct)
z_boundaries_list <- vector("list", nr_boundaries)
z_boundary_type   <- numeric(nr_boundaries)   # Stores ct level (e.g., 0.495 or 0.505)

for (i in seq_len(nr_boundaries)) {
  boundary_vals <- cbind(ct[[i]]$x, ct[[i]]$y)      # two-column (x,y)
  colnames(boundary_vals) <- c("x", "y")
  z_boundaries_list[[i]] <- boundary_vals
  z_boundary_type[i]     <- ct[[i]]$level
}

# ---------------- 1.2) Retain boundaries that cover/contain data ---------
boundary_used <- integer(0)
counter <- 0L

for (i in seq_len(nr_boundaries)) {
  # Keep only contour points inside polygon
  data_points <- as.data.frame(z_boundaries_list[[i]])
  coordinates(data_points) <- ~ x + y
  data_points$inside_polygon <- !is.na(over(data_points, data_polygon))
  z_boundaries_list[[i]] <- as.matrix(z_boundaries_list[[i]][data_points$inside_polygon, , drop = FALSE])

  # Detect closed contour
  circle_contour <- 0L
  if (length(z_boundaries_list[[i]]) > 0L) {
    circle_contour <- ifelse(
      z_boundaries_list[[i]][1, 1] == z_boundaries_list[[i]][nrow(z_boundaries_list[[i]]), 1] &
      z_boundaries_list[[i]][1, 2] == z_boundaries_list[[i]][nrow(z_boundaries_list[[i]]), 2],
```

```
    1L, 0L
   )
  }

  # If closed, check whether it surrounds (any) data
  interior_data <- 0L
  if (length(z_boundaries_list[[i]]) > 0L && circle_contour == 1L) {
   interior_data <- sum(point.in.polygon(Z.st[, 1], Z.st[, 2],
                                         z_boundaries_list[[i]][, 1], z_boundaries_list[[i]][, 2]))
  }

  # Keep if not a circle, or a circle that encloses data
  if ((circle_contour == 0L || interior_data > 0L) && length(z_boundaries_list[[i]]) > 0L) {
   counter <- counter + 1L
   boundary_used[counter] <- i
  }
 }

 # ---------------- 1.3) Rebuild with retained boundaries only -------------
 if (!is.null(boundary_used)) {
  nr_boundaries <- length(boundary_used)
 } else {
  nr_boundaries <- length(ct)
  boundary_used <- seq_len(nr_boundaries)
 }

 z_boundaries_list <- vector("list", nr_boundaries)
 z_boundary_type   <- numeric(nr_boundaries)

 for (i in seq_len(nr_boundaries)) {
  j <- boundary_used[i]
  boundary_vals <- cbind(ct[[j]]$x, ct[[j]]$y)
  colnames(boundary_vals) <- c("x", "y")

  data_points <- as.data.frame(boundary_vals)
  coordinates(data_points) <- ~ x + y
  data_points$inside_polygon <- !is.na(over(data_points, data_polygon))
  z_boundaries_list[[i]] <- as.matrix(boundary_vals[data_points$inside_polygon, , drop = FALSE])

  z_boundary_type[i] <- ct[[j]]$level
 }

 # ---------------- 2) Distances & nearest-per-boundary ---------------------
 # For each boundary set i, compute distances from each Z.st row to all
 # contour points in that set; then select the nearest point index.
 point.dist.matrix <- vector("list", nr_boundaries)
 for (i in seq_len(nr_boundaries)) {
  point.dist.matrix[[i]] <- as.matrix(pdist(Z.st, z_boundaries_list[[i]]))
 }
```

```
nearest_idx <- vector("list", nr_boundaries)
for (i in seq_len(nr_boundaries)) {
  if (nrow(point.dist.matrix[[i]]) > 0L) {
    nearest_idx[[i]] <- apply(point.dist.matrix[[i]], 1, which.min)
  } else {
    nearest_idx[[i]] <- integer(0)
  }
}

# ---------------- 3) Select the closest boundary set per point -----------
z_boundary_row   <- matrix(0, ncol = 2, nrow = nr_boundaries)
boundary_set_nr  <- integer(nrow(Z.st))

for (j in seq_len(nrow(Z.st))) {
  for (i in seq_len(nr_boundaries)) {
    if (length(nearest_idx[[i]]) == 0L) {
      z_boundary_row[i, ] <- c(1e8, 1e8)  # ignore empty sets
    } else {
      z_boundary_row[i, ] <- z_boundaries_list[[i]][nearest_idx[[i]][j], , drop = FALSE]
    }
  }
  # Choose boundary set whose nearest point is closest to Z.st[j, ]
  boundary_set_nr[j] <- apply(as.matrix(pdist(Z.st[j, , drop = FALSE], z_boundary_row)), 1, which.min)
}

# (Outputs in environment): boundary_set_nr, boundary_used
```

A7: Biplot Boundary Search with different Eigenvalues

```
############################################################
# Section 4.3: Different eigenvalues
# Description:
#   Explore counterfactuals across all 2D eigenvector (PC) combinations.
#   For each PC pair, rotate onto the plane, generate a grid, run the
#   boundary search for the target data point, and record distances.
#   Select the nearest pair according to Mahalanobis-like distance.
#
# Notes:
#   * Logic unchanged from the original script, only formatting,
#     comments, and safety guards added for clarity.
#   * Expects objects from previous steps: Vmat, tVmat, model_use,
#     model_select, biplot_data, Wmat, etc.
############################################################

################################
# 0.1) Setup: target data point #
################################

tdp <- c(3, 13, 26, 2)[4]   # choose a test data point index
tdp <- 12               # PIMA

biplot_dataT[tdp, ]         # inspect chosen row

# Test + training data (filter by training convex hull)
rotate_data <- rbind(biplot_dataT[tdp, ], biplot_data)

#################
# 0.2) Filters #
#################

set_filters <- list(TRUE)   # default

# Examples retained from original:
# set_filters <- list(expr(Sepal.Width > 3))

# set_filters <- list(expr(Age >= 33 & Age <= 61),
#                     expr(Pregnancies >= 4 & Pregnancies <= 21))

# set_filters <- list(expr(loan_amnt >= 33000))

#############################
# 1) Prep structures, defs #
#############################

{
  # All 2D PC combinations
```

```
  n_contrib <- t(combn(num_vars, 2))
  n_pcs <- nrow(n_contrib)

  # Storage
  pc.cf    <- as.data.frame(matrix(0, ncol = num_vars, nrow = n_pcs, dimnames = list(NULL, var_names)))
  pc.cfz   <- matrix(0, ncol = 2, nrow = n_pcs)
  pc.z     <- matrix(0, ncol = 2, nrow = n_pcs)
  pc.distz <- matrix(0, ncol = 1, nrow = n_pcs)
  pc.distz_eucl <- matrix(0, ncol = 1, nrow = n_pcs)

  rounding <- 2
  Rotate <- TRUE

  Wmat_inv <- solve(Wmat)
}

##############################################
# 2) Iterate over all 2D eigenvector pairs  #
##############################################

for (i in seq_len(n_pcs)) {
  cat("PC pair index:", i, "\n")
  pc.vects <- n_contrib[i, ]

  tryCatch({
    ##############################
    # 2.1) Rotation to new plane
    ##############################
    V_rotate <- Biplot_rotation(
      X = rotate_data,
      tdp = 1,
      standardise = standardise_use,
      Xcenter = X_center, Xsd = X_sd,
      CVA_method = CVA_method,
      proj = pc.vects,
      V = Vmat
    )

    V <- V_rotate$Vrho
    tV <- V_rotate$tVrho

    # Use base indices for 2D block projection
    pc.vects <- c(1, 2)
    # update the biplot for the new projection matrix
    {
      Vr <- V[,pc.vects , drop = FALSE]
      tVr <- tV[pc.vects, , drop = FALSE]

      biplot_input$biplot_out$ax.one.unit <- 1/(diag(t(tVr) %*% tVr)) * t(tVr)
```

```
  biplot_input$biplot_out$Lmat<- V
  biplot_input$biplot_out$Vr <- Vr
  biplot_input$biplot_out$Z <- biplot_input$biplot_out$X %*% biplot_input$biplot_out$Vr
}
########################################
# 2.2) Build biplot grid for rotation #
########################################
biplot_grid_rotate <- biplot_plane(
  X = rotate_data,
  model_use, var_names, numvars = num_vars,
  model_select = model_select,
  cutoff = cut_off,

  standardise = standardise_use,
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = CVA_method,

  proj = pc.vects,
  V = V, tV = tV,

  polygon = NA,
  no_polygon = FALSE, # F = create a polygon , with includes the training data based on rotated data
  calc_hull = TRUE, # T = only select boundaries within the polygon
  calc_ct_in_hull = FALSE,
  outlie = 1,

  m = m_grid,
  b_margin = 1 / (10^rounding), rounding = rounding
)

####################################
# 2.3) Boundary search in Z-space  #
####################################
biplot_search <- Biplot_boundary_search(
  biplot_grid_rotate,
  model_select = model_select,
  tdp = 1,

  standardise = standardise_use,
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = CVA_method,

  proj = pc.vects,
  V = V, tV = tV,

  set_filters = set_filters,
  train_ranges = train_ranges
)
```

```
  ##############################################################
  # 2.4) Save CF results for this pair (X-space and Z-space) #
  ##############################################################
  pc.cf[i, ]  <- as.numeric(biplot_search$B_boundary_x_value)
  pc.cfz[i, ] <- as.numeric(biplot_search$B_boundary_z_value)
  pc.z[i, ]   <- as.numeric(biplot_grid_rotate$Z.st[1, ])

  # Distance vector in Z-space
  vec_to_boundary_Z <- as.matrix(biplot_search$B_boundary_z_value - biplot_grid_rotate$Z.st[1, , drop =
FALSE])
  vec_to_boundary   <- vec_to_boundary_Z %*% tVr

  # Mahalanobis-like distance
  pc.distz[i] <- (as.matrix(vec_to_boundary, nrow = 1)) %*% Wmat_inv %*% t(as.matrix(vec_to_boundary,
nrow = 1))
  # pc.distz_eucl[i] <- apply((vec_to_boundary)^2, 1, "sum")
 }, error = function(e) {
  message("Skipping index ", i, " due to error: ", e$message)
 })
}

#########################
# 3) Collate and select #
#########################

{
 pc.distz_row <- cbind(seq_len(n_pcs), pc.distz)
 pc.distz_row <- pc.distz_row[pc.distz_row[, 2] != 0, , drop = FALSE]

 nearest_pc <- ifelse(nrow(pc.distz_row) == 1,
                      pc.distz_row[which.min(pc.distz_row[, 2])],
                      pc.distz_row[which.min(pc.distz_row[, 2]), 1])

 boundary <- pc.cf[nearest_pc, ]
 pc.rotate <- n_contrib[nearest_pc, ]
 print(pc.rotate)
}

#########################
# 4) Apply final rotation #
#########################

{
 V_rotate <- Biplot_rotation(
  X = rotate_data,
  tdp = 1,
  standardise = standardise_use,
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = CVA_method,
```

```
   proj = pc.rotate,
   V = Vmat
  )

  V <- V_rotate$Vrho
  tV <- V_rotate$tVrho

}

################################
# 5) Final biplot grid & plot #
################################

# Use base indices for 2D block projection
pc.values <- c(1, 2)

   # update the biplot for the new projection matrix
   {
    Vr <- V[,pc.vects , drop = FALSE]
    tVr <- tV[pc.vects, , drop = FALSE]

    biplot_input$biplot_out$ax.one.unit <- 1/(diag(t(tVr) %*% tVr)) * t(tVr)
    biplot_input$biplot_out$Lmat<- V
    biplot_input$biplot_out$Vr <- Vr
    biplot_input$biplot_out$Z <- biplot_input$biplot_out$X %*% biplot_input$biplot_out$Vr
   }


biplot_grid_final <- biplot_plane(
  X = rotate_data,
  model_use, var_names, numvars = num_vars,
  model_select = model_select,
  cutoff = cut_off,

  standardise = standardise_use,
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = CVA_method,

  proj = pc.values,
  V = V, tV = tV,

  polygon = NA,
  no_polygon = FALSE,
  calc_hull = TRUE,
  calc_ct_in_hull = FALSE,
  outlie = 1,

  m = m_grid,
  b_margin = 1 / (10^rounding), rounding = rounding
```

```
)

var_names

plot_biplotEZ(
  grid = biplot_grid_final,
  biplot_plot = biplot_input$biplot_out,
  #k = c(2),
  #label_dist = c(0.5),
  tdp = 1,
  cex_z = 1.5,
  tick.label.cex = 0.7,
  label_dir = "Paral",
  ticks_v = 4,
  new_title = "Rotated 1st and 4th Component biplot"
)

point_inter_proj(
  X = rotate_data[, 1:num_vars],
  V = V,
  tdp = 1,
  plot = TRUE,
  proj = pc.values,
  col_x = "green",
  cex_x = 1.5,
  standardise = standardise_use,
  CVA_method = CVA_method
)

########################################
# 6) Final boundary search on final grid #
########################################

biplot_search <- Biplot_boundary_search(
  biplot_grid_final,
  model_select = model_select,
  tdp = 1,

  standardise = standardise_use,
  Xcenter = X_center, Xsd = X_sd,
  CVA_method = CVA_method,

  proj = pc.values,
  V = V, tV = tV,

  set_filters = set_filters,
  train_ranges = train_ranges
)
```

```
point_inter_proj(
  X = biplot_search$B_boundary_x_value,
  V = V,
  plot = TRUE,
  proj = pc.values,
  col_x = "darkgreen",
  cex_x = 1.5,
  standardise = standardise_use,
  CVA_method = CVA_method
)

# Inspect
rotate_data[1, 1:num_vars]
biplot_search$B_boundary_x_value
pc.cf[nearest_pc, ]

#####################
###         END       ###
#####################
```

A8: Local Interpretations with Shapley values

```
############################################################
# Section 4.4: Local interpretation – Shapley values
# Description:
#   Compute and visualise per-variable Shapley contributions for a single
#   target point, relative to its nearest decision-boundary counterfactual.
############################################################

##############################
# 1) Select target and CF   #
##############################

#tdp <- 1

{
  # Target row (features only)
  pred_data <- biplot_dataT[tdp, 1:num_vars]
  pred_info <- biplot_dataT[tdp, ]

  # Choose counterfactual boundary in X-space
  #  - single-point search:
  boundary <- biplot_search$B_boundary_x_value[1, 1:num_vars]
  #  - direct projection alternative:
  # boundary <- proj_boundary
  #  - multi-point search alternative:
  # boundary <- pc.cf[nearest_pc, ]

  # Difference vector (X): boundary - data
  data_to_boundary <- as.numeric(boundary - pred_data)
}

############################################
# 2) Run Shapley and collect contributions  #
############################################

{
  shapley_store <- Run_Shapley(
    model_use, num_vars,
    test_data = pred_data, B_boundary = boundary,   # B_boundary_x also acceptable
    individual = TRUE, tdp = 1,                  # can pass specific SPD index here
    svm_u = svm_u, lda = lda, rpart_used = rpart_used,
    rounding = rounding
  )$shapley_store

  # Single-point (sdp) contribution row
  shapley_cause <- shapley_store[1, ]
}
```

```
##############################################
# 3) Predictions and ordering of variables  #
##############################################

{
 # Current point prediction
 pred_value <- pred_function(
  model_use = model_use,
  model_select = model_select,
  rounding = rounding,
  new_data = pred_data
 )

 # Boundary point prediction
 pred_boundary <- pred_function(
  model_use = model_use,
  model_select = model_select,
  rounding = rounding,
  new_data = boundary
 )

 # Class and order: determines decreasing order for plotting
 pred_use <- as.numeric(ifelse(pred_value >= cut_off, 1, 0))    # predicted class (0/1)
 dec_order <- ifelse(pred_use == 1, TRUE, FALSE)               # TRUE => decreasing
 dist_order <- order(shapley_cause, decreasing = dec_order)     # if class 1, negatives come first
}

##############################################
# 4) Assemble plotting data (no logic change)
##############################################

{
 # Base frame: data value, delta to boundary, and shapley contribution
 df <- as.data.frame(cbind(as.numeric(pred_data), data_to_boundary, shapley_cause))
 df <- df %>% dplyr::rename(pred_data = V1)

 # Standardised distance option (kept as in original; default FALSE)
 df$vec_cause <- df$data_to_boundary
 std_dist <- FALSE
 if (std_dist == TRUE) {
  df$vec_cause <- df$data_to_boundary / X_sd
 }

 # Add names and flags
 df <- cbind(varnames = var_names, dist_order, df)
 df <- df %>% dplyr::mutate(pred_use = pred_use)

 # Label whether each variable supports or contradicts the current class
 df <- df %>% dplyr::mutate(
```

```
  Contribute = dplyr::case_when(
    pred_use == 0 & shapley_cause >= 0 ~ "Supports",     # in class 0, increasing causes supports
    pred_use == 0 & shapley_cause <  0 ~ "Contradicts",
    pred_use == 1 & shapley_cause >= 0 ~ "Contradicts",  # in class 1, increasing causes contradicts
    pred_use == 1 & shapley_cause <  0 ~ "Supports",
    TRUE ~ "Green"
  )
 )

 # Apply order
 df <- df[dist_order, , drop = FALSE]
 df <- df %>% dplyr::mutate(varnames = factor(varnames, levels = varnames))  # keep order after sorting

 # Pretty y-axis labels (value -> delta)
 df <- df %>% dplyr::mutate(
  varnames_p = paste0(varnames, ": ", round(pred_data, 2), "-> ", round(data_to_boundary, 2))
 )
 df <- df %>% dplyr::mutate(varnames_p = factor(varnames_p, levels = varnames_p))
}

########################
# 5) Build the graphic #
########################

{
 # Method 1 (length by change) retained but commented in original
 # ggplot1 <- ggplot(df, aes(fill = Contribute, colour = Contribute, x = vec_cause, y = varnames_p)) +
 #   scale_fill_manual(values = c("Supports" = "darkblue", "Contradicts" = "grey")) +
 #   scale_color_manual(values = c("Supports" = "darkblue", "Contradicts" = "grey")) +
 #   geom_col() + theme_bw() +
 #   geom_label(aes(label = round(shapley_cause, rounding), x = 0), colour = "white", show.legend = FALSE) +
 #   labs(
 #     x = paste0("Change to Boundary - ", ifelse(std_dist == TRUE, "Standardised", "Original")),
 #     y = "Variable contribution order",
 #     title = "Distance to Boundary: Contribution & order from Shapley value",
 #     subtitle = paste("ID and class: ", tdp, ",", class_data$class[tdp],
 #                      "; Predicted Class and score: ", class_data$pred_use[tdp], ",",
 #                      round(class_data$pred_value[tdp], 3),
 #                      "; Boundary score: ", round(pred_boundary, 3))
 #   )

 # Method 2 (length by probability change) — used
 shapley_plot <- ggplot(df, aes(fill = Contribute, colour = Contribute,
                                x = shapley_cause, y = varnames_p)) +
  scale_fill_manual(name = "Contribution",
                    values = c("Supports" = "darkblue", "Contradicts" = "grey")) +
  scale_color_manual(name = "Contribution",
                     values = c("Supports" = "darkblue", "Contradicts" = "grey")) +
  geom_col() + theme_bw() +
```

```
  geom_label(aes(label = round(shapley_cause, rounding),
                 x = round(shapley_cause, rounding)),
             colour = "white", show.legend = FALSE) +
  theme(legend.position = "bottom") +
  labs(
   x = "[-> change to reach boundary] : Impact on prediction of data values",
   y = "Variable contribution order",
   title = "Shapley Contribution Plot",
   subtitle = paste(
    "ID and class: ", tdp, ",", pred_info$class,
    "; Predicted Class and score: ", pred_info$pred_use, ",", round(pred_info$pred_value, 3),
    "; Boundary score: ", round(pred_boundary, 3)
   )
  )
}

print(shapley_plot)

####################################
# 6) Investigate outcome
####################################

biplot_dataT[tdp,]
biplot_grid_Test$Z.st[tdp,] %*% tV[1:2,] + X_center
Z_st_subset[tdp,] %*% tV[1:2,] + X_center
Bz_subset[tdp,] %*% tV[1:2,] + X_center
vec_to_boundary_Z[tdp,] %*% tVr

summary(train_data)
vec_to_boundary[tdp,]
X_sd
vec_to_boundary_sd[tdp,]
sort(abs(vec_to_boundary_sd[tdp,]))

p <- 3.4 * -2.77662 + 5.4 * 0.13186 + 6.2 * -0.06556 + 7.5234
p <- (3.4-0.59*0) * -2.77662 + (5.4+0.16*1) * 0.13186 + (6.2+0.14*1) * -0.06556 + 7.5234
1/ (1+exp(-p))
summary(model_use)

####################################
# 7) Update a modified data point #
####################################

{
 final_data <- rotate_data[1, 1:num_vars]
 # final_data <- biplot_data[tdp, 1:(num_vars + 2)]   # original alternative

 # Select which factors to change:
 # (Edit indices here as needed)
```

```
  var_names
  change_values <- c(5, 2, 4)
  final_data[change_values] <- boundary[change_values]

  pred_value <- pred_function(
    model_use = model_use,
    model_select = model_select,
    rounding = rounding,
    new_data = final_data
  )

  print(final_data)
  print(pred_value)
}

# plot the updated value

point_inter_proj(
  X = final_data,
  V = V,
  plot = TRUE,
  proj = pc.values,
  col_x = "darkgrey",
  cex_x = 1.5,
  standardise = standardise_use,
  CVA_method = CVA_method
)

##########################
####### END ##############
```